

Книга: MicroPython за VK_RA4M2 чрез демонстрационни примери

Цел

Тази книга е направена върху реално наличния пакет от примерни .py файла за VK_RA4M2.

Тук целта не е само да се изброят файловете.

Целта е да се покрият всички глави, теми и примери от плана, без пропуски и без измислени ресурси.

Затова книгата е организирана в три слоя:

1. Ресурсна карта на борда и порта.
2. Матрица за пълност спрямо учебния план.
3. Подробни глави с кодови откъси и препратки към всички реални сорс файлове.

Как да се ползва книгата

Книгата се чете заедно със сорс файловете в `examples`.

Препоръчителният ритъм е:

1. Прочитате теорията в съответната глава.
2. Стартирате показания минипример.
3. Отваряте пълния `.ру` файл.
4. Преглеждате ресурсния `header` в началото му.
5. Променяте само по една стойност и наблюдавате ефекта.

Когато примерът съдържа предпазен флаг като `DO_WRITE`, `DO_BOOTLOADER`, `DO_LIGHTSLEEP`, `DO_DEEPSLEEP` или `SET_DEMO_TIME`, започвайте с безопасната стойност.

Нива на трудност

Всяка глава е маркирана с ниво, за да улесни навигацията:

Ниво	Глави	Описание
[Начинаещ]	0–14	Първи стъпки, езикови основи, GPIO, ADC, DAC, PWM. Не изисква предишен опит.
[Среден]	15–34	Таймери, прекъсвания, asyncio, серийни интерфейси, памет и организация на проекта.
[Напреднал]	35–56	Сигнална обработка, FSM архитектура, DSP, FFT, WS2812 игри и production firmware.

Съдържание

Тази секция дава бърз преглед на всички части и глави в книгата.

Ресурсна карта на VK_RA4M2

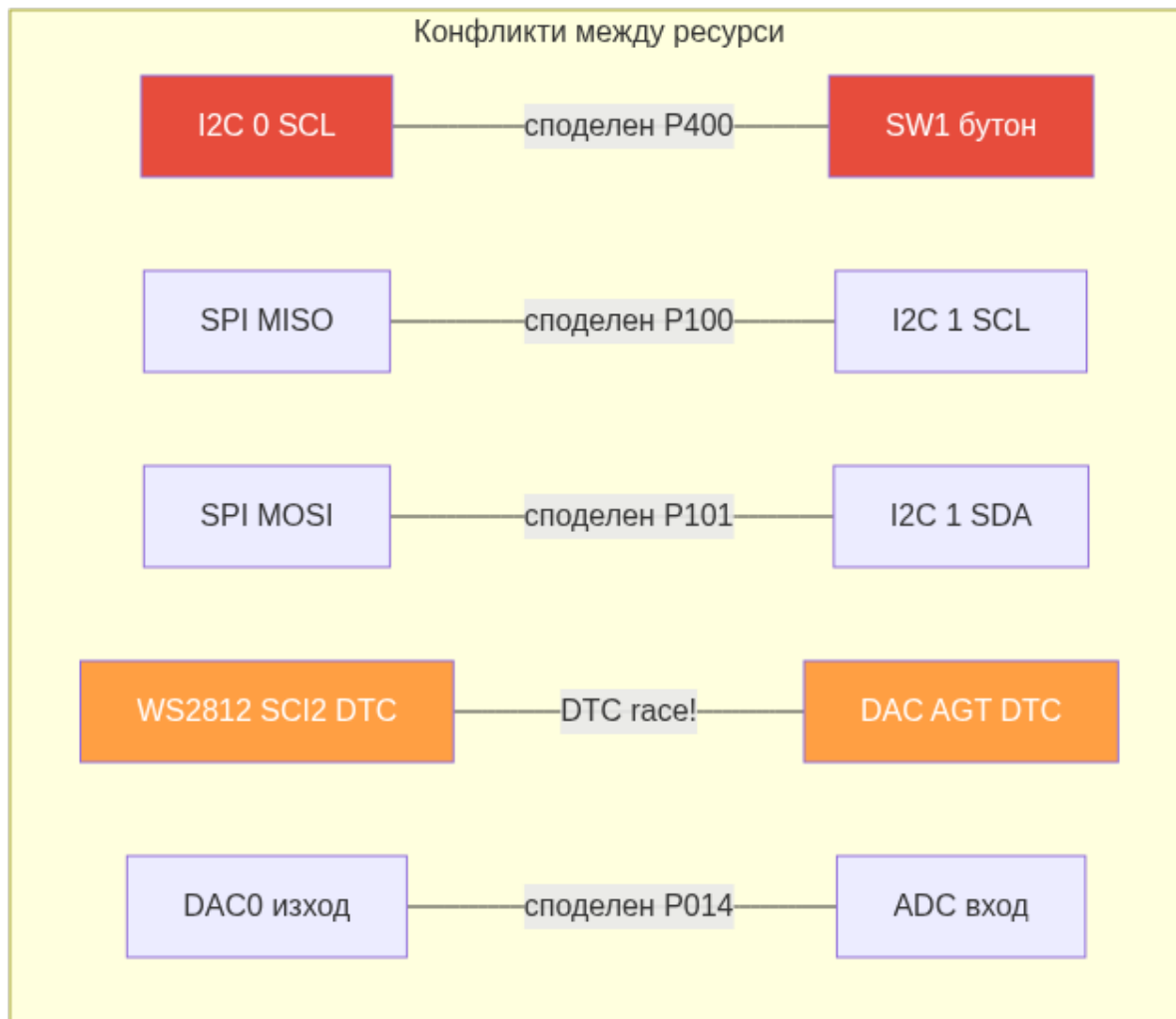
Книгата и примерите приемат следната карта на ресурси:

- LED: 1 брой -> LED1 = P204
- Бутон: 1 брой -> SW1 = P400
- USB device пинове: USBDP = P914, USBDM = P915, USB_VBUS = P407
- ADC външни входове: 13 броя -> P000, P001, P002, P003, P004, P005, P006, P007, P008, P013, P014, P015, P500
- ADC вътрешни източници: 3 броя -> ADC.CORE_TEMP, ADC.CORE_VREF, ADC.VREF
- DAC изходи: 2 броя -> P014, P015
- PWM изходи: 14 броя -> P107, P106, P105, P104, P113, P114, P112, P115, P608, P409, P408, P600, P304, P303
- UART: 4 инстанции -> UART(0), UART(2), UART(7), UART(9)
- I2C master: 2 инстанции -> I2C(0)=P400/P401, I2C(1)=P100/P101
- I2CTarget: 2 инстанции -> I2CTarget(0)=P400/P401, I2CTarget(1)=P100/P101
- SPI: 1 канал -> CS=P103, SCK=P102, MISO=P100, MOSI=P101
- SoftI2C: наличен
- SoftSPI: наличен
- TouchPad входове: 12 броя -> P205, P206, P407, P408, P409, P410, P411, P412, P413, P414, P415, P708
- CTSU специален пин: P207 = TSCAP
- Хардуерни таймери: 6 броя -> Timer(1), Timer(2), Timer(3), Timer(4), Timer(5), Timer(6)
- Софтуерен таймер: Timer(-1)
- RTC: 1 брой
- Вътрешна файлова система: /flash
- renesas.Flash: наличен block device
- dataflash: 8 KB

Важни уточнения:

- SPI тук се разглежда като 1 канал.
- Timer(-1) тук се разглежда като софтуерен таймер.
- SW1 е върху P400, което влиза в конфликт с I2C(0).SCL.
- LED1 е active-low: логическа 0 го светва, логическа 1 го гаси.
- Pin.PULL_UP се ползва в курса; Pin.PULL_DOWN не се разглежда като наличен ресурс в тези уроци.
- lightsleep и deepsleep са показани предпазливо.
- J16 е boot-mode jumper, не е DAC изход.
- DAC timed playback върху P014/P015 поддържа DTC circular, DTC one-shot и DMAC one-shot.
- DAC пинът подава аналогов сигнал, а не мощност към товар. За слушане ползвайте активна колона, усилвател или осцилоскоп, не директно гол говорител.

Таблица на конфликти между ресурси



Конфликти между ресурси

Някои пинове и хардуерни ресурси се споделят. Тази таблица събира всички известни конфликти на едно място:

Ресурс А	Ресурс Б	Споделен пин/хардуер	Последствие
I2C(0).SCL	SW1 (бутон)	P400	Не ползвайте I2C(0) и бутон едновременно. Ползвайте I2C(1).
SPI.MISO	I2C(1).SCL	P100	SPI и I2C(1) не могат едновременно.
SPI.MOSI	I2C(1).SDA	P101	Същото — изберете SPI или I2C(1).
WS2812 SCI2 DTC	DAC AGT DTC	DTC контролер	Race condition при едновременна работа. Спрете DAC преди strip.write().
TouchPad P407	USB_VBUS	P407	TouchPad на P407 може да конфликува с USB детекция.
DAC0	ADC на P015	P014	Пинът е или DAC изход, или ADC вход — не и двете.
DAC1	ADC на P015	P015	Същото за втория DAC канал.

[Pin] Правило: Ако два ресурса споделят пин, конфигурирайте само единия в даден момент. MicroPython няма да ви спре — но резултатът ще бъде непредсказуем.

Пълен каталог

Пълният каталог на всички примери е в края на книгата (секция „Каталог на всички примери“). Всяка тема от учебния план е покрита с поне един .ру файл.

Глава 0: Първо свързване — от кутията до REPL

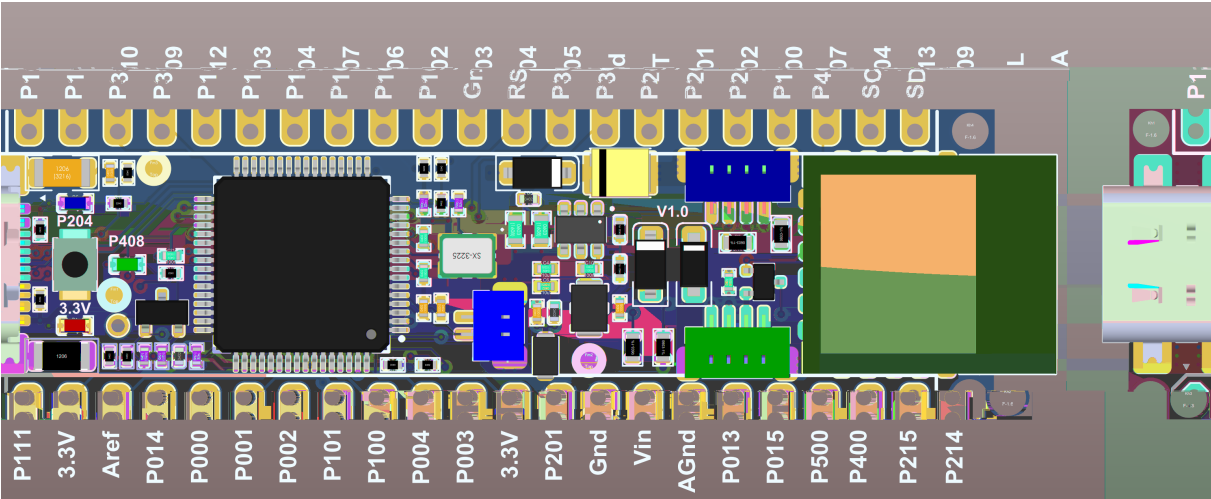
Тази глава е отправната точка. Без нея не можете да стартирате нито един пример от книгата.

Какво е в комплекта

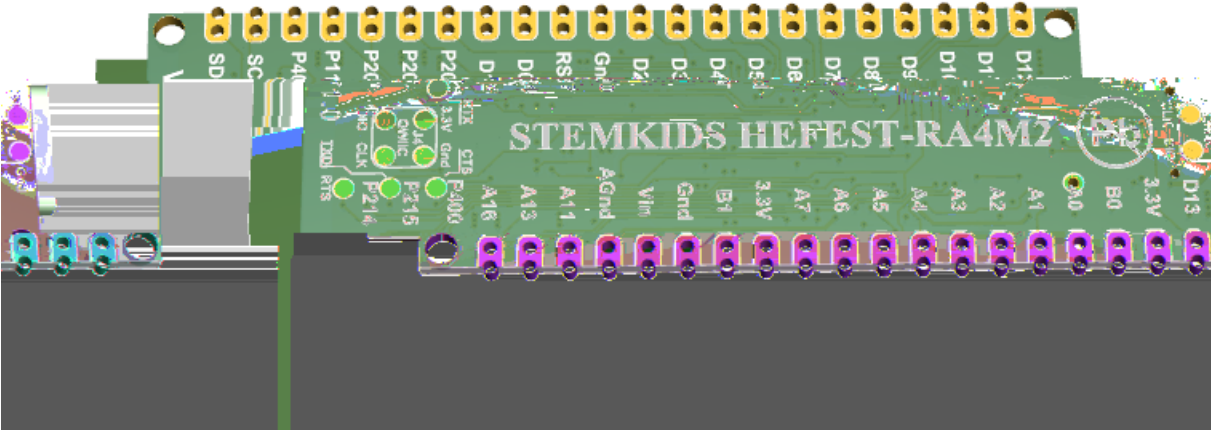
Платката VK_RA4M2 идва в два варианта:

Вариант	Формат	Размер	USB конектор
VK-RA4M2 Nano (HEFEST-RA4M2)	Arduino Nano-подобен	дълга тясна платка	Micro-USB
VK-RA4M2 Femto (VK-RA4M2-16)	Миниатюрен	много компактна	Micro-USB

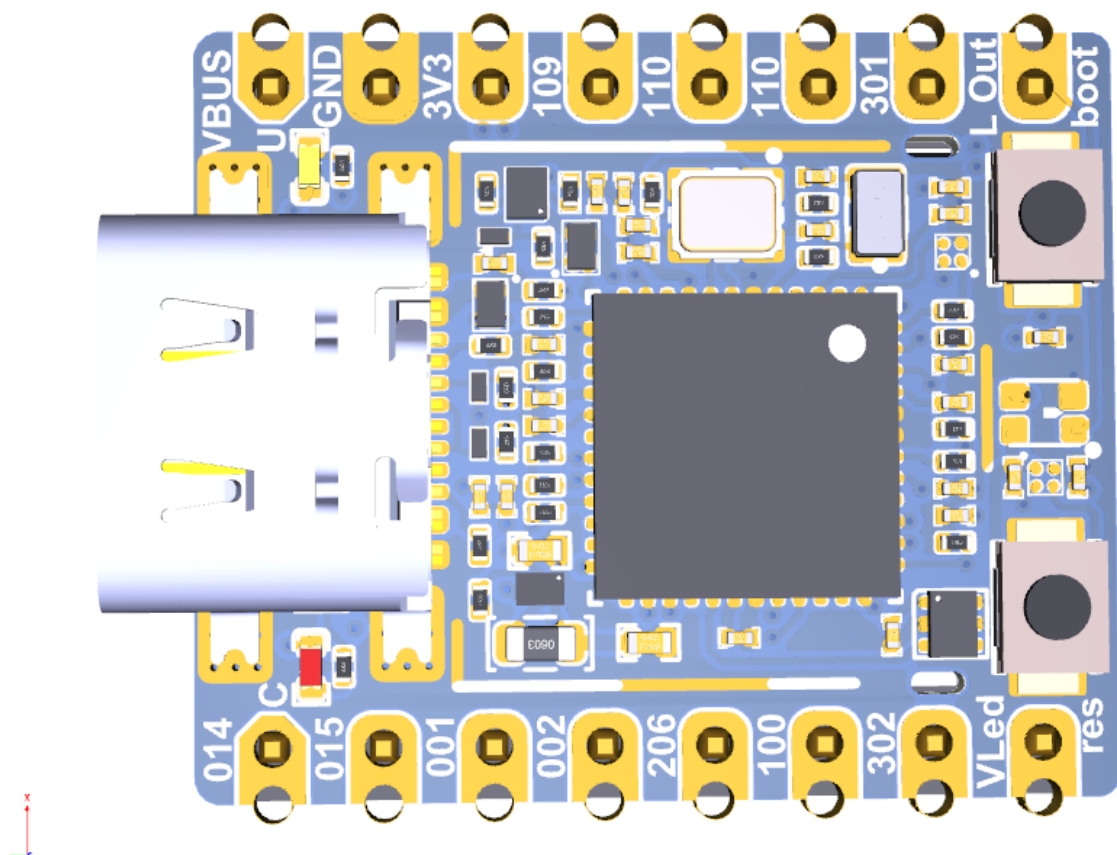
И двата варианта използват **един и същ MCU** — Renesas RA4M2 (Cortex-M33, 100 MHz, 512 KB flash, 128 KB RAM) и се програмират по идентичен начин.



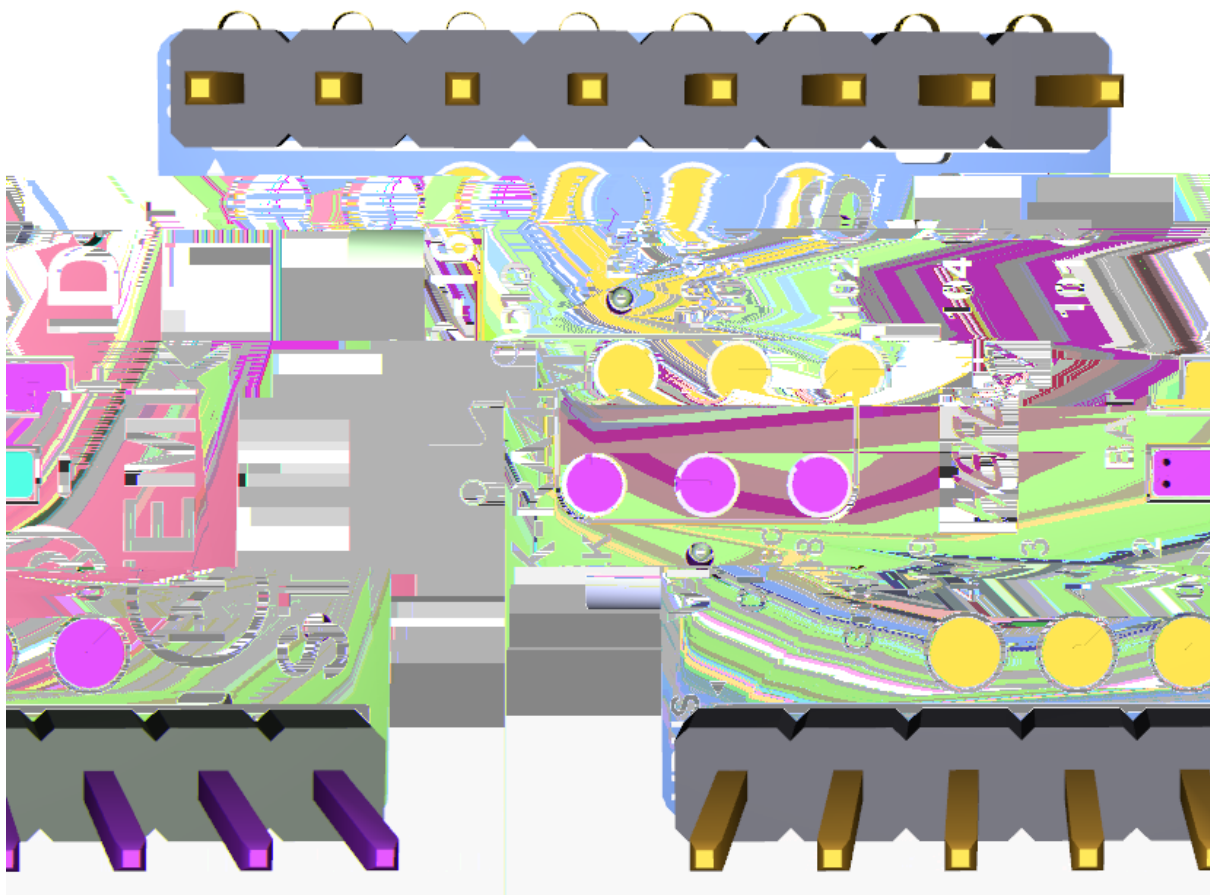
VK-RA4M2 Nano — изглед отпред



VK-RA4M2 Nano — изглед отзад



VK-RA4M2 Femto — изглед отпред



VK-RA4M2 Femto — изглед отзад

Какво ви трябва:

- 1x VK-RA4M2 платка (Nano или Femto)
- 1x Micro-USB кабел (данни, не само зареждане!)
- Компютър с Windows / Linux / macOS

[!] Важно за USB кабела: Някои кабели са само за зареждане и нямат data линии. Ако бордът не се появява — опитайте друг кабел.

Pinout: VK-RA4M2 Nano (HEFEST-RA4M2)

VK-RA4M2 Nano — ключови функции

Горен ред (от USB към антена):

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
P110	P109	P112	P303	P304	P107	P106	P102	P104	P105	Gnd	P301	P302	P200	P207	P204	P113	P409	SCL	SDA

Долен ред (от USB към антена):

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
P111	3.3V	Aref	P014	P000	P001	P002	P101	P004	3.3V	P201	Gnd	Vin	AGnd	P013	P015	P500	P400	P215	P214

Функция	Пин (Nano)	Бележка
LED1	P204	Active-low: value(0) = светва
LED2	P408	Втори вграден LED
SW1 (бутон)	P400	Active-low, споделен с I2C(0).SCL
DAC0	P014	Аналогов изход — аудио
DAC1	P015	Втори DAC канал
I2C(1) SCL	SCL (P100)	Препоръчителен I2C канал
I2C(1) SDA	SDA (P101)	Препоръчителен I2C канал
WS2812 data	P112	SCI2 TX за NRZ протокол
WS2812 power	P500	GPIO за захранване на лентата
ADC входове	P000–P008, P013–P015, P500	13 външни канала
UART(9) TX/RX	P602/P601	Ако е наличен на конектор
USB	Micro-USB	CDC-ACM виртуален COM порт

Pinout: VK-RA4M2 Femto (VK-RA4M2-16)

Femto вариантът има по-малко изведени пинове, но ядрото е същото. Подходящ е за крайни проекти с малък габарит.

Страна	Пинове
Лява	USB, P500, P408, Gnd, 3.3V
Гръб	BAT, P012, P013, P014, 3.3V, Gnd

USB свързване и COM порт



USB свързване — от кабел до REPL

Стъпка 1: Свържете платката с Micro-USB кабел към компютъра.

Стъпка 2: Проверете COM порта:

- **Windows:** Отворете Device Manager → Ports (COM & LPT). Трябва да видите нов COM порт (напр. COM3). Ако не се появява — вижте секцията "Troubleshooting" по-долу.

- **Linux:** `ls /dev/ttyACM*` — трябва да видите `/dev/ttyACM0`.

- **macOS:** `ls /dev/cu.usbmodem*`

[Pin] Бележка: VK-RA4M2 се разпознава като CDC-ACM устройство (виртуален сериен порт). На Windows 10/11 обикновено не са нужни допълнителни драйвери.

Стъпка 3: Ако MicroPython firmware вече е зареден, бордът е готов за REPL.

Зареждане на MicroPython firmware

Ако бордът е нов и няма MicroPython, трябва да заредите firmware:

1. Свалете последния .hex файл за VK_RA4M2 от хранилището на проекта.
2. Използвайте **Renesas Flash Programmer** (безплатен) или **J-Flash Lite**:
 - Свържете борда по USB.
 - Изберете правилния COM порт и MCU (RA4M2).
 - Заредете .hex файла.
 - Натиснете "Start" за програмиране.
3. След успешно зареждане, бордът ще се рестартира и REPL ще бъде достъпен.

Отваряне на REPL

REPL (Read-Eval-Print Loop) е интерактивната конзола на MicroPython. Три начина да я отворите:

Вариант А: Thonny (препоръчителен за начинаещи)

1. Инсталирайте Thonny от thonny.org.
2. Run → Configure interpreter → MicroPython (generic) → изберете COM порта.
3. В долния панел виждате >>> — готово!

Вариант Б: mpremote (препоръчителен за напреднали)

```
pip install mpremote
mpremote connect COM3 repl
```

Виждате >>> — готово.

Вариант В: PuTTY / screen

- **PuTTY (Windows):** Connection type: Serial, Serial line: COM3, Speed: 115200 → Open.
- **screen (Linux/macOS):** `screen /dev/ttyACM0 115200`

Първа проверка

В REPL напишете:

```
>>> print("hello VK_RA4M2!")
hello VK_RA4M2!
>>> from machine import Pin
>>> led = Pin("LED1", Pin.OUT, value=0)    # LED светва (active-low)
>>> led.value(1)                          # LED гасне
```

Ако LED1 светна и после угасна — **бордът работи и е готов за книгата.**

Качване на файл на борда



Workflow — качване, стартиране, спиране

Повечето примери от книгата са .py файлове, които трябва да се копират на /flash:

С mpremote:

```
mpremote connect COM3 fs cp retro_synth.py :/flash/retro_synth.py
```

С Thonny:

1. Отворете файла на компютъра.
2. File → Save copy → MicroPython device → запишете на борда.

Стартиране на файл:

```
>>> import retro_synth          # импортира и изпълнява модула
```

Или директно без копиране:

```
mpremote connect COM3 run dac_retro_sfx.py
```

Reset, Ctrl-C и безопасно спиране

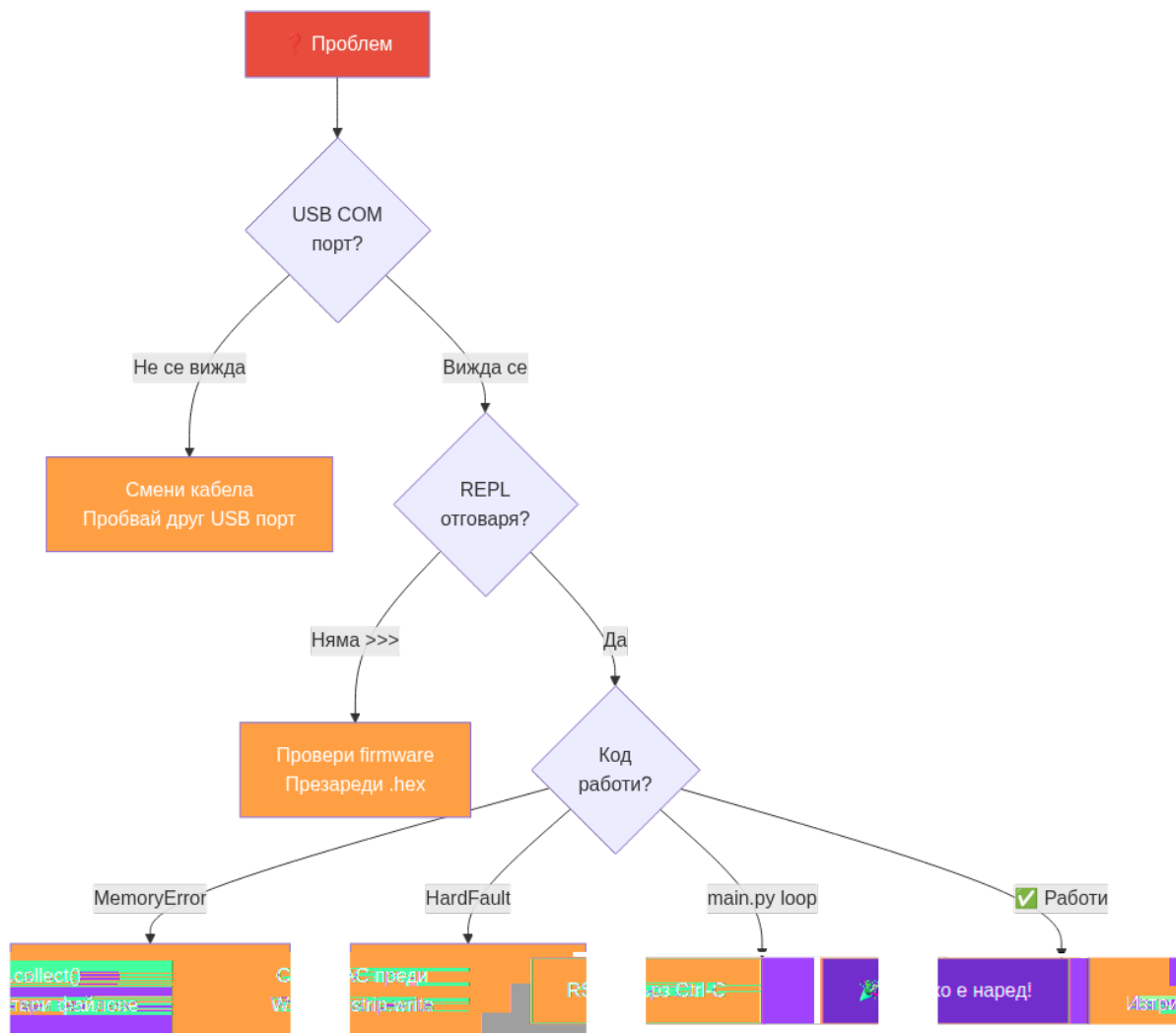
Действие	Как	Кога
Прекъсване на програма	Ctrl-C в REPL	Безкраен цикъл, звук, LED
Мек reset (soft reset)	Ctrl-D в REPL	Рестарт на MicroPython без загуба на файлове
Хардуерен reset	Натиснете бутона RST на борда	При замръзване
Програмен reset	import machine; machine.reset()	От код

[!] Ако `main.py` заседне в безкраен цикъл при boot: 1. Свържете REPL. 2. Натиснете RST бутона на борда. 3. Веднага натискайте Ctrl-C многократно в първите 1-2 секунди — целта е да прекъснете `main.py` преди да стартира напълно. 4. Ако успеете, ще видите `>>>`. Изтрийте или преименувайте проблемния файл: `import os; os.remove("/flash/main.py")`.

Листване на файлове на борда

```
>>> import os
>>> os.listdir("/flash")          # показва всички файлове
>>> os.statvfs("/flash")         # показва свободно място
```

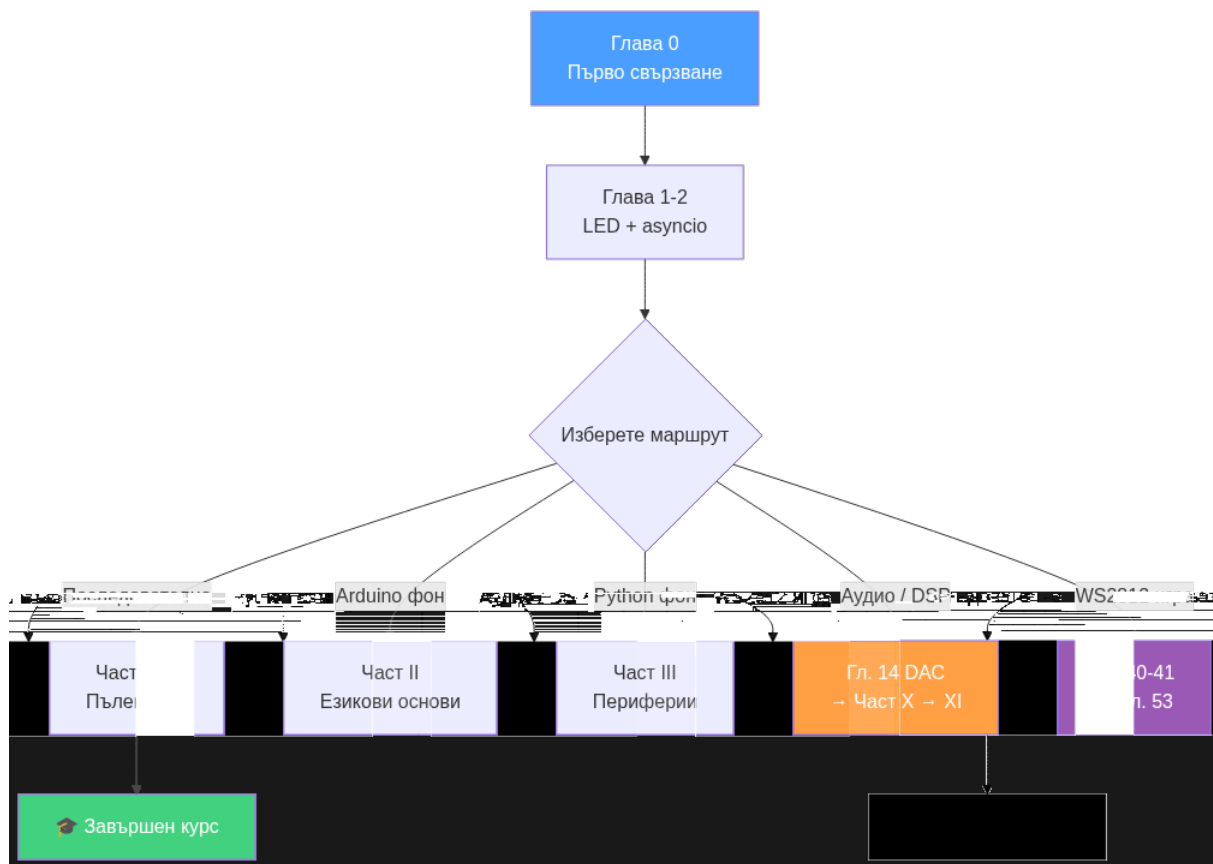
Системен troubleshooting



Troubleshooting flowchart

Симптом	Вероятна причина	Решение
Бордът не се появява на USB	Кабел без data линии	Опитайте друг USB кабел
COM порт се появява и изчезва	Недостатъчно захранване	Ползвайте USB порт директно на компютъра, не хъб
REPL не отговаря (няма >>>)	Firmware не е зареден	Заредете MicroPython .hex файл
>>> се вижда, но команди не работят	Baud rate грешен	Уверете се, че е 115200, 8N1
MemoryError при import	Heap е пълен	import gc; gc.collect() преди import
OSError: [Errno 28] ENOSPC	/flash е пълен	Изтрийте ненужни файлове: os.remove("/flash/old_file.py")
main.py заседна в loop	Безкраен цикъл при boot	RST + бърз Ctrl-C (вж. по-горе)
LED1 не реагира	Грешна active-low логика	value(0) светва, value(1) гасне
HardFault при DAC + WS2812	DTC race condition	Спрете DAC преди strip.write() (вж. Глава 52)

Навигационна карта



Навигационна карта — маршрути по аудитория

Книгата може да се чете последователно, но ето бързи маршрути по интерес:

Ако идвате от...	Препоръчителен старт
Arduino / C embedded	Глава 0 → 1 → Част II (езикови основи) → Част III (периферии)
Python, но не embedded	Глава 0 → 1 → прескочете Част II → Част III → IV
Искате само аудио/DSP	Глава 0 → 14 (DAC) → Част X (елементарен DSP) → Част XI
Искате WS2812 / RGB игра	Глава 0 → 1 → 40–41 (NeoPixel/WS2812) → 53 (RGB_Guardian)
Учебна програма стъпка по стъпка	Глава 0 → 1 → 2 → последователно до края

Книга: MicroPython за VK_RA4M2 чрез демонстрационни примери

Цел

Тази книга е направена върху реално наличния пакет от примерни .py файла за VK_RA4M2.

Тук целта не е само да се изброят файловете.

Целта е да се покрият всички глави, теми и примери от плана, без пропуски и без измислени ресурси.

Затова книгата е организирана в три слоя:

1. Ресурсна карта на борда и порта.
2. Матрица за пълност спрямо учебния план.
3. Подробни глави с кодови откъси и препратки към всички реални сорс файлове.

Как да се ползва книгата

Книгата се чете заедно със сорс файловете в `examples`.

Препоръчителният ритъм е:

1. Прочитате теорията в съответната глава.
2. Стартирате показания минипример.
3. Отваряте пълния `.ру` файл.
4. Преглеждате ресурсния `header` в началото му.
5. Променяте само по една стойност и наблюдавате ефекта.

Когато примерът съдържа предпазен флаг като `DO_WRITE`, `DO_BOOTLOADER`, `DO_LIGHTSLEEP`, `DO_DEEPSLEEP` или `SET_DEMO_TIME`, започвайте с безопасната стойност.

Нива на трудност

Всяка глава е маркирана с ниво, за да улесни навигацията:

Ниво	Глави	Описание
[Начинаещ]	0–14	Първи стъпки, езикови основи, GPIO, ADC, DAC, PWM. Не изисква предишен опит.
[Среден]	15–34	Таймери, прекъсвания, asyncio, серийни интерфейси, памет и организация на проекта.
[Напреднал]	35–56	Сигнална обработка, FSM архитектура, DSP, FFT, WS2812 игри и production firmware.

Съдържание

Тази секция дава бърз преглед на всички части и глави в книгата.

Ресурсна карта на VK_RA4M2

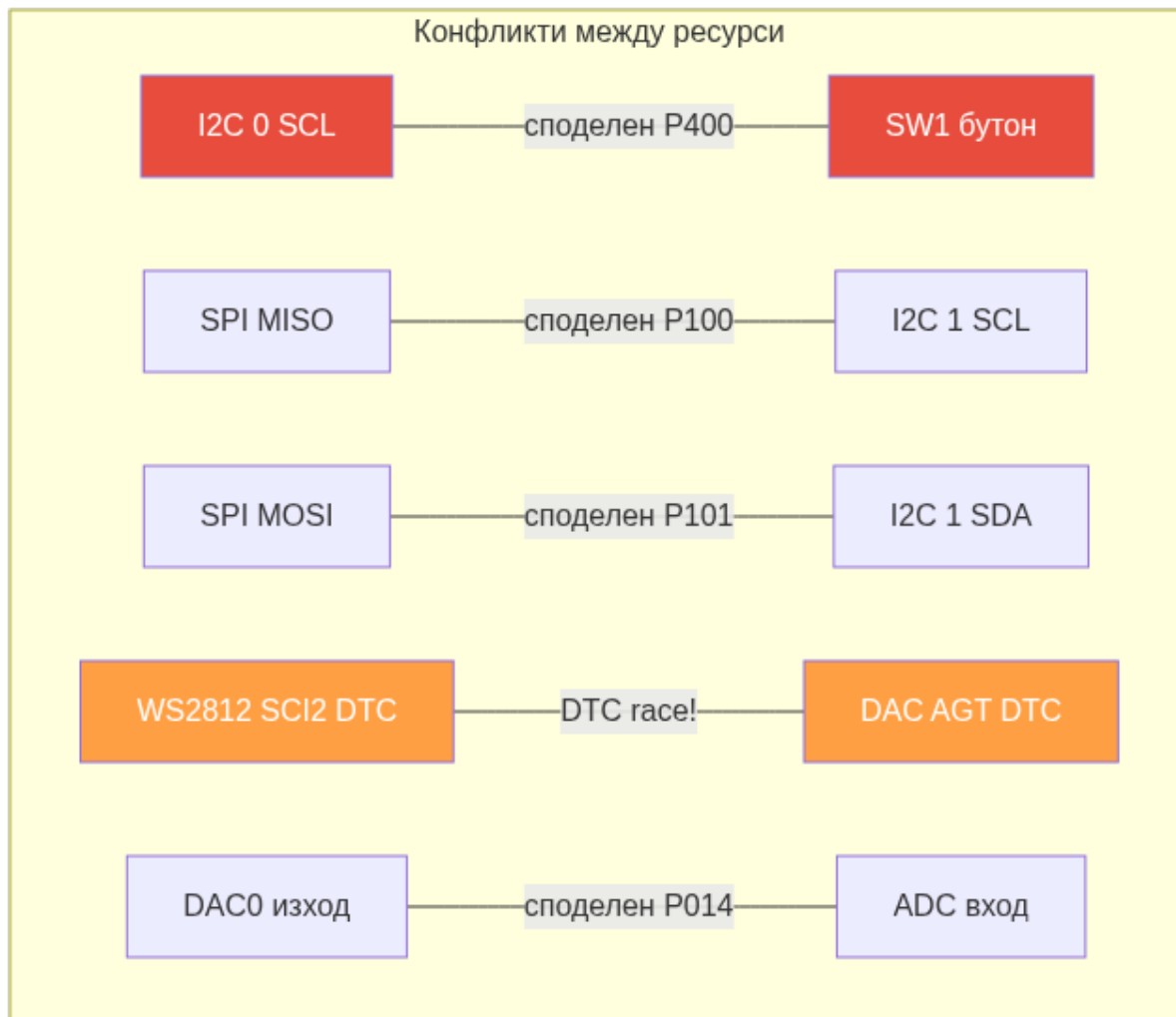
Книгата и примерите приемат следната карта на ресурси:

- LED: 1 брой -> LED1 = P204
- Бутон: 1 брой -> SW1 = P400
- USB device пинове: USBDP = P914, USBDM = P915, USB_VBUS = P407
- ADC външни входове: 13 броя -> P000, P001, P002, P003, P004, P005, P006, P007, P008, P013, P014, P015, P500
- ADC вътрешни източници: 3 броя -> ADC.CORE_TEMP, ADC.CORE_VREF, ADC.VREF
- DAC изходи: 2 броя -> P014, P015
- PWM изходи: 14 броя -> P107, P106, P105, P104, P113, P114, P112, P115, P608, P409, P408, P600, P304, P303
- UART: 4 инстанции -> UART(0), UART(2), UART(7), UART(9)
- I2C master: 2 инстанции -> I2C(0)=P400/P401, I2C(1)=P100/P101
- I2CTarget: 2 инстанции -> I2CTarget(0)=P400/P401, I2CTarget(1)=P100/P101
- SPI: 1 канал -> CS=P103, SCK=P102, MISO=P100, MOSI=P101
- SoftI2C: наличен
- SoftSPI: наличен
- TouchPad входове: 12 броя -> P205, P206, P407, P408, P409, P410, P411, P412, P413, P414, P415, P708
- CTSU специален пин: P207 = TSCAP
- Хардуерни таймери: 6 броя -> Timer(1), Timer(2), Timer(3), Timer(4), Timer(5), Timer(6)
- Софтуерен таймер: Timer(-1)
- RTC: 1 брой
- Вътрешна файлова система: /flash
- renesas.Flash: наличен block device
- dataflash: 8 KB

Важни уточнения:

- SPI тук се разглежда като 1 канал.
- Timer(-1) тук се разглежда като софтуерен таймер.
- SW1 е върху P400, което влиза в конфликт с I2C(0).SCL.
- LED1 е active-low: логическа 0 го светва, логическа 1 го гаси.
- Pin.PULL_UP се ползва в курса; Pin.PULL_DOWN не се разглежда като наличен ресурс в тези уроци.
- lightsleep и deepsleep са показани предпазливо.
- J16 е boot-mode jumper, не е DAC изход.
- DAC timed playback върху P014/P015 поддържа DTC circular, DTC one-shot и DMAC one-shot.
- DAC пинът подава аналогов сигнал, а не мощност към товар. За слушане ползвайте активна колона, усилвател или осцилоскоп, не директно гол говорител.

Таблица на конфликти между ресурси



Конфликти между ресурси

Някои пинове и хардуерни ресурси се споделят. Тази таблица събира всички известни конфликти на едно място:

Ресурс А	Ресурс Б	Споделен пин/хардуер	Последствие
I2C(0).SCL	SW1 (бутон)	P400	Не ползвайте I2C(0) и бутон едновременно. Ползвайте I2C(1).
SPI.MISO	I2C(1).SCL	P100	SPI и I2C(1) не могат едновременно.
SPI.MOSI	I2C(1).SDA	P101	Същото — изберете SPI или I2C(1).
WS2812 SCI2 DTC	DAC AGT DTC	DTC контролер	Race condition при едновременна работа. Спрете DAC преди strip.write().
TouchPad P407	USB_VBUS	P407	TouchPad на P407 може да конфликува с USB детекция.
DAC0	ADC на P015	P014	Пинът е или DAC изход, или ADC вход — не и двете.
DAC1	ADC на P015	P015	Същото за втория DAC канал.

[Pin] Правило: Ако два ресурса споделят пин, конфигурирайте само единия в даден момент. MicroPython няма да ви спре — но резултатът ще бъде непредсказуем.

Пълен каталог

Пълният каталог на всички примери е в края на книгата (секция „Каталог на всички примери“). Всяка тема от учебния план е покрита с поне един .ру файл.

Глава 0: Първо свързване — от кутията до REPL

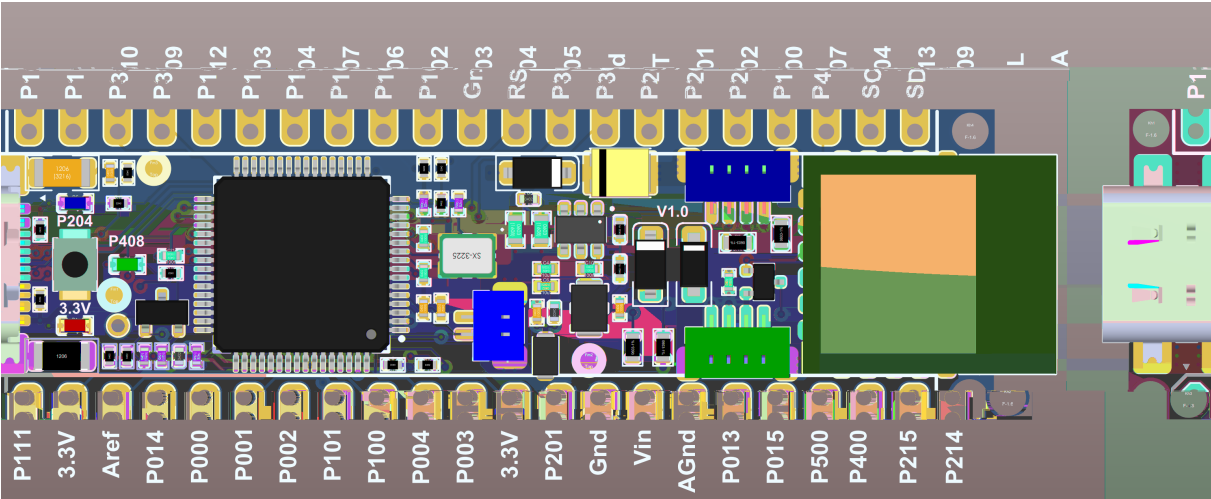
Тази глава е отправната точка. Без нея не можете да стартирате нито един пример от книгата.

Какво е в комплекта

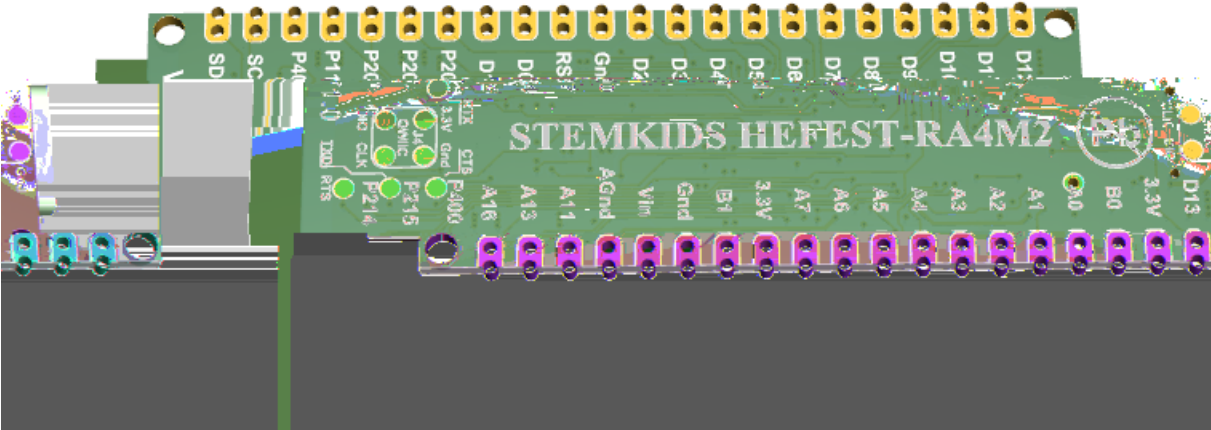
Платката VK_RA4M2 идва в два варианта:

Вариант	Формат	Размер	USB конектор
VK-RA4M2 Nano (HEFEST-RA4M2)	Arduino Nano-подобен	дълга тясна платка	Micro-USB
VK-RA4M2 Femto (VK-RA4M2-16)	Миниатюрен	много компактна	Micro-USB

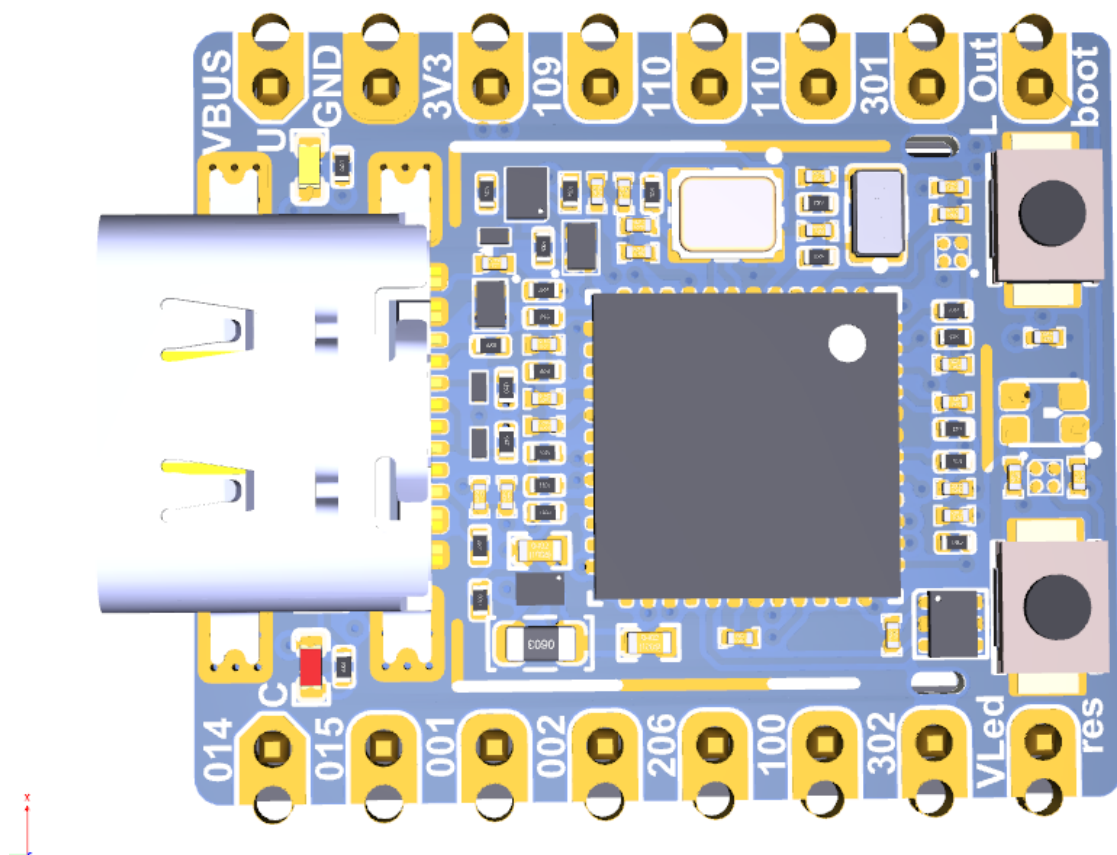
И двата варианта използват **един и същ MCU** — Renesas RA4M2 (Cortex-M33, 100 MHz, 512 KB flash, 128 KB RAM) и се програмират по идентичен начин.



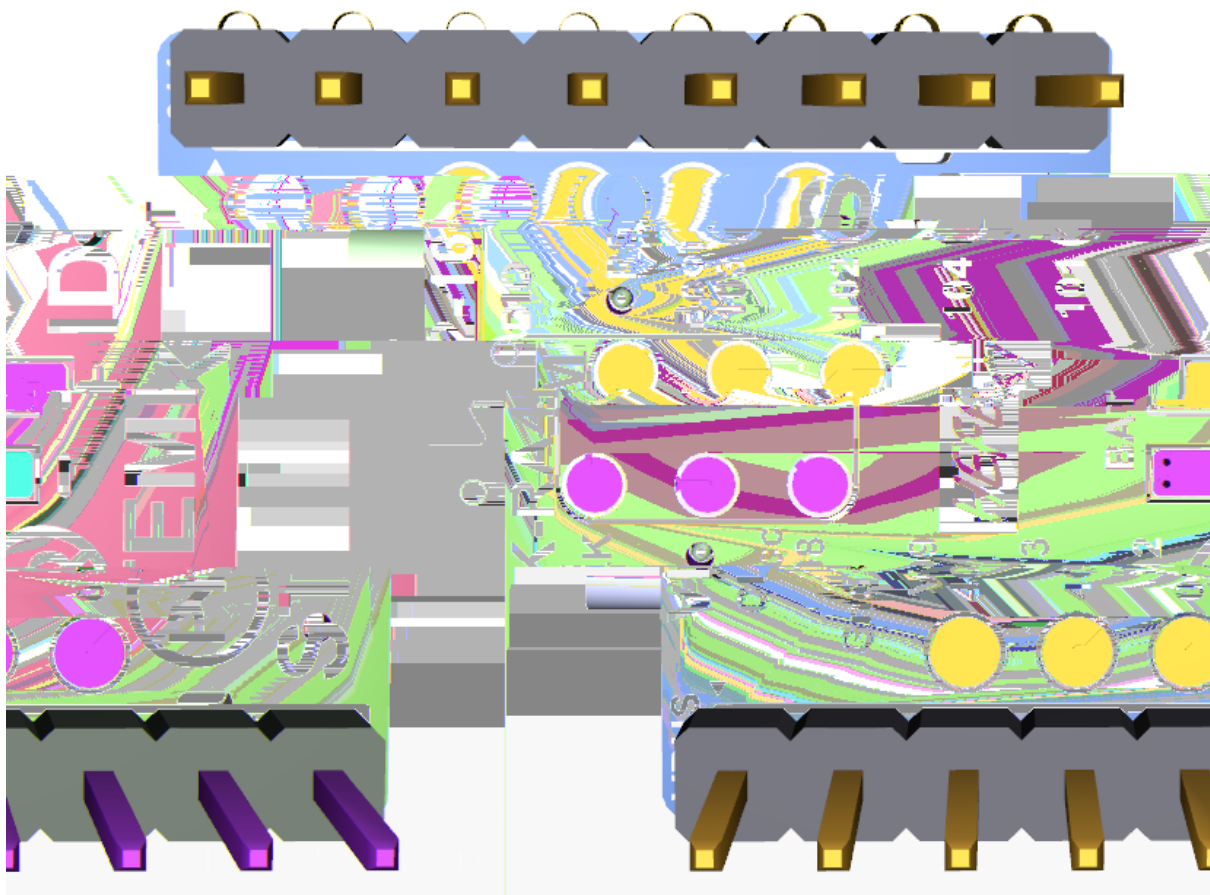
VK-RA4M2 Nano — изглед отпред



VK-RA4M2 Nano — изглед отзад



VK-RA4M2 Femto — изглед отпред



VK-RA4M2 Femto — изглед отзад

Какво ви трябва:

- 1x VK-RA4M2 платка (Nano или Femto)
- 1x Micro-USB кабел (данни, не само зареждане!)
- Компютър с Windows / Linux / macOS

[!] Важно за USB кабели: Някои кабели са само за зареждане и нямат data линии. Ако бордът не се появява — опитайте друг кабел.

Pinout: VK-RA4M2 Nano (HEFEST-RA4M2)

VK-RA4M2 Nano — ключови функции

Горен ред (от USB към антена):

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
P110	P109	P112	P303	P304	P107	P106	P102	P104	P105	Gnd	P301	P302	P200	P207	P204	P113	P409	SCL	SDA

Долен ред (от USB към антена):

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
P111	3.3V	Aref	P014	P000	P001	P002	P101	P004	3.3V	P201	Gnd	Vin	AGnd	P013	P015	P500	P400	P215	P214

Функция	Пин (Nano)	Бележка
LED1	P204	Active-low: value(0) = светва
LED2	P408	Втори вграден LED
SW1 (бутон)	P400	Active-low, споделен с I2C(0).SCL
DAC0	P014	Аналогов изход — аудио
DAC1	P015	Втори DAC канал
I2C(1) SCL	SCL (P100)	Препоръчителен I2C канал
I2C(1) SDA	SDA (P101)	Препоръчителен I2C канал
WS2812 data	P112	SCI2 TX за NRZ протокол
WS2812 power	P500	GPIO за захранване на лентата
ADC входове	P000–P008, P013–P015, P500	13 външни канала
UART(9) TX/RX	P602/P601	Ако е наличен на конектор
USB	Micro-USB	CDC-ACM виртуален COM порт

Pinout: VK-RA4M2 Femto (VK-RA4M2-16)

Femto вариантът има по-малко изведени пинове, но ядрото е същото. Подходящ е за крайни проекти с малък габарит.

Страна	Пинове
Лява	USB, P500, P408, Gnd, 3.3V
Гръб	BAT, P012, P013, P014, 3.3V, Gnd

USB свързване и COM порт



USB свързване — от кабел до REPL

Стъпка 1: Свържете платката с Micro-USB кабел към компютъра.

Стъпка 2: Проверете COM порта:

- **Windows:** Отворете Device Manager → Ports (COM & LPT). Трябва да видите нов COM порт (напр. COM3). Ако не се появява — вижте секцията "Troubleshooting" по-долу.

- **Linux:** `ls /dev/ttyACM*` — трябва да видите `/dev/ttyACM0`.

- **macOS:** `ls /dev/cu.usbmodem*`

[Pin] Бележка: VK-RA4M2 се разпознава като CDC-ACM устройство (виртуален сериен порт). На Windows 10/11 обикновено не са нужни допълнителни драйвери.

Стъпка 3: Ако MicroPython firmware вече е зареден, бордът е готов за REPL.

Зареждане на MicroPython firmware

Ако бордът е нов и няма MicroPython, трябва да заредите firmware:

1. Свалете последния .hex файл за VK_RA4M2 от хранилището на проекта.
2. Използвайте **Renesas Flash Programmer** (безплатен) или **J-Flash Lite**:
 - Свържете борда по USB.
 - Изберете правилния COM порт и MCU (RA4M2).
 - Заредете .hex файла.
 - Натиснете "Start" за програмиране.
3. След успешно зареждане, бордът ще се рестартира и REPL ще бъде достъпен.

Отваряне на REPL

REPL (Read-Eval-Print Loop) е интерактивната конзола на MicroPython. Три начина да я отворите:

Вариант А: Thonny (препоръчителен за начинаещи)

1. Инсталирайте Thonny от thonny.org.
2. Run → Configure interpreter → MicroPython (generic) → изберете COM порта.
3. В долния панел виждате >>> — готово!

Вариант Б: mpremote (препоръчителен за напреднали)

```
pip install mpremote
mpremote connect COM3 repl
```

Виждате >>> — готово.

Вариант В: PuTTY / screen

- **PuTTY (Windows):** Connection type: Serial, Serial line: COM3, Speed: 115200 → Open.
- **screen (Linux/macOS):** `screen /dev/ttyACM0 115200`

Първа проверка

В REPL напишете:

```
>>> print("hello VK_RA4M2!")
hello VK_RA4M2!
>>> from machine import Pin
>>> led = Pin("LED1", Pin.OUT, value=0)    # LED светва (active-low)
>>> led.value(1)                          # LED гасне
```

Ако LED1 светна и после угасна — **бордът работи и е готов за книгата.**

Качване на файл на борда



Workflow — качване, стартиране, спиране

Повечето примери от книгата са .py файлове, които трябва да се копират на /flash:

С mpremote:

```
mpremote connect COM3 fs cp retro_synth.py :/flash/retro_synth.py
```

С Thonny:

1. Отворете файла на компютъра.
2. File → Save copy → MicroPython device → запишете на борда.

Стартиране на файл:

```
>>> import retro_synth          # импортира и изпълнява модула
```

Или директно без копиране:

```
mpremote connect COM3 run dac_retro_sfx.py
```

Reset, Ctrl-C и безопасно спиране

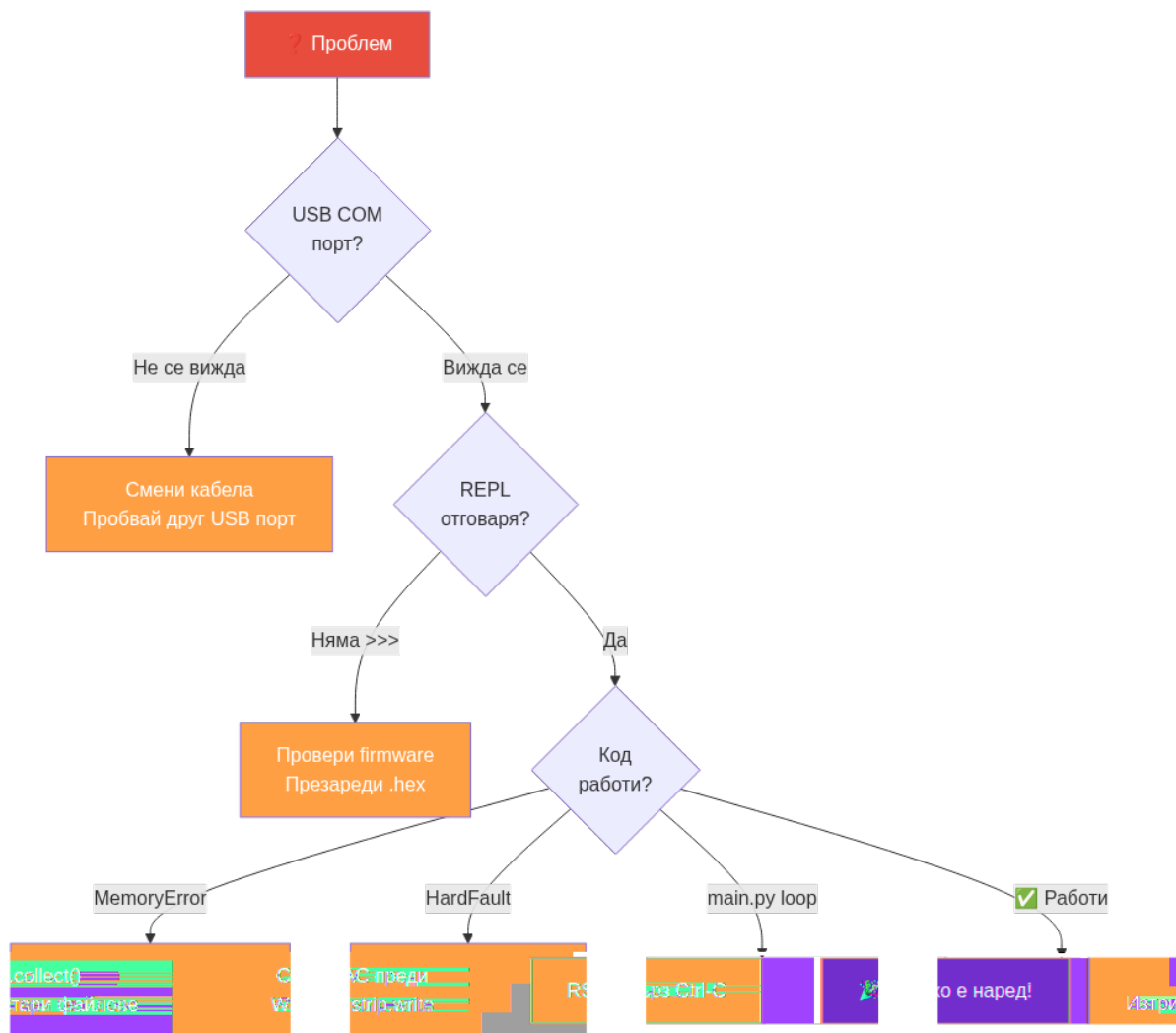
Действие	Как	Кога
Прекъсване на програма	Ctrl-C в REPL	Безкраен цикъл, звук, LED
Мек reset (soft reset)	Ctrl-D в REPL	Рестарт на MicroPython без загуба на файлове
Хардуерен reset	Натиснете бутона RST на борда	При замръзване
Програмен reset	import machine; machine.reset()	От код

[!] Ако `main.py` заседне в безкраен цикъл при boot: 1. Свържете REPL. 2. Натиснете RST бутона на борда. 3. Веднага натискайте Ctrl-C многократно в първите 1-2 секунди — целта е да прекъснете `main.py` преди да стартира напълно. 4. Ако успеете, ще видите `>>>`. Изтрийте или преименувайте проблемния файл: `import os; os.remove("/flash/main.py")`.

Листване на файлове на борда

```
>>> import os
>>> os.listdir("/flash")          # показва всички файлове
>>> os.statvfs("/flash")         # показва свободно място
```

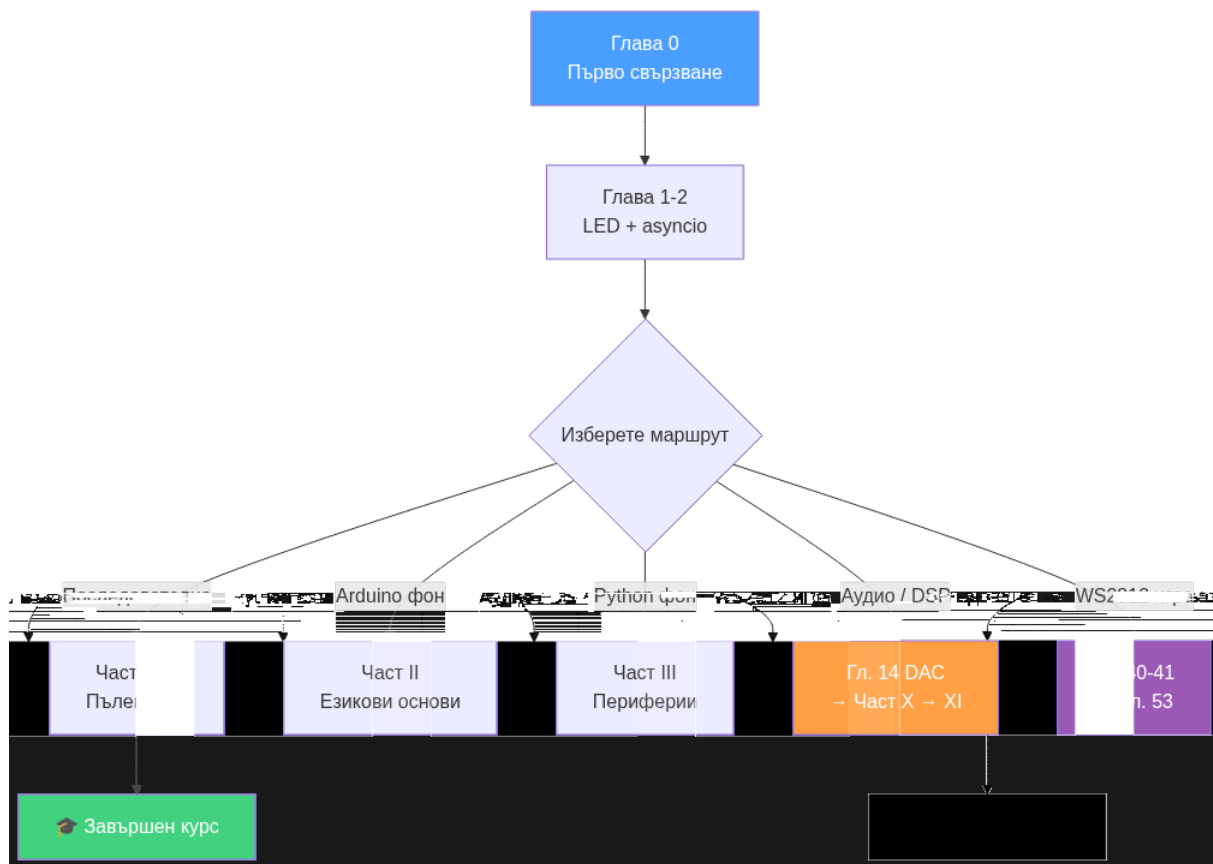
Системен troubleshooting



Troubleshooting flowchart

Симптом	Вероятна причина	Решение
Бордът не се появява на USB	Кабел без data линии	Опитайте друг USB кабел
COM порт се появява и изчезва	Недостатъчно захранване	Ползвайте USB порт директно на компютъра, не хъб
REPL не отговаря (няма >>>)	Firmware не е зареден	Заредете MicroPython .hex файл
>>> се вижда, но команди не работят	Baud rate грешен	Уверете се, че е 115200, 8N1
MemoryError при import	Heap е пълен	import gc; gc.collect() преди import
OSError: [Errno 28] ENOSPC	/flash е пълен	Изтрийте ненужни файлове: os.remove("/flash/old_file.py")
main.py заседна в loop	Безкраен цикъл при boot	RST + бърз Ctrl-C (вж. по-горе)
LED1 не реагира	Грешна active-low логика	value(0) светва, value(1) гасне
HardFault при DAC + WS2812	DTC race condition	Спрате DAC преди strip.write() (вж. Глава 52)

Навигационна карта



Навигационна карта — маршрути по аудитория

Книгата може да се чете последователно, но ето бързи маршрути по интерес:

Ако идвате от...	Препоръчителен старт
Arduino / C embedded	Глава 0 → 1 → Част II (езикови основи) → Част III (периферии)
Python, но не embedded	Глава 0 → 1 → прескочете Част II → Част III → IV
Искате само аудио/DSP	Глава 0 → 14 (DAC) → Част X (елементарен DSP) → Част XI
Искате WS2812 / RGB игра	Глава 0 → 1 → 40–41 (NeoPixel/WS2812) → 53 (RGB_Guardian)
Учебна програма стъпка по стъпка	Глава 0 → 1 → 2 → последователно до края

Глава 0: Първо свързване — от кутията до REPL [Начинаещ]

- Какво е в комплекта
- Pinout: VK-RA4M2 Nano и Femto
- USB свързване и COM порт
- Отваряне на REPL
- Качване на файл на борда
- Reset, Ctrl-C и безопасно спиране
- Системен troubleshooting

Част I: Първи стъпки с борда и с MicroPython

- Глава 1. Първа програма
- Глава 2. Първи асинхронни примери

Част II: Езикови основи на MicroPython

- Глава 3. Променливи и константи
- Глава 4. Оператори, условия и цикли
- Глава 5. Enums в MicroPython
- Глава 6. Масиви и таблици
- Глава 7. Структури в MicroPython
- Глава 8. Указатели и еквивалентите им в MicroPython
- Глава 9. Функции, callbacks и таблици от функции

Част III: Цифрови и аналогови входове и изходи

- Глава 10. Цифров изход
- Глава 11. Цифров вход
- Глава 12. Аналогов вход и ADC
- Глава 13. DAC изход, timed playback и retro synth
- Глава 14. PWM

Част IV: Таймери, прекъсвания и време

- Глава 15. Закъснения, хардуерни таймери и Timer(-1)
- Глава 16. Прекъсвания
- Глава 17. Дебаунс на бутон
- Глава 18. Четене на бутон: polling, IRQ, Timer(-1), debounce и събития
- Глава 19. RTC

Част IVб: asyncio — мост към конкурентно програмиране

(Поставена непосредствено след таймерите, преди серийните интерфейси)

- Глава 20. Основи на asyncio
- Глава 21. asyncio Event синхронизация

Част V: Серийни интерфейси и двоични протоколи

- Глава 22. I2C master и runtime избор на пинове
- Глава 23. SoftI2C и I2CTarget
- Глава 24. Hardware SPI и SoftSPI
- Глава 25. Базова UART комуникация
- Глава 26. Работа с външни модули по интерфейси
- Глава 27. Байтове, bytearray, struct, memoryview и двоични протоколи

Част VI: Системни функции, памет и организация на проекта

- Глава 28. TouchPad: четене, диагностика и cached API
- Глава 29. renesas.Flash, /flash и dataflash
- Глава 30. Управление на паметта в MicroPython
- Глава 31. machine модулът и порт-специфично поведение
- Глава 32. boot.py, main.py и жизнен цикъл на програмата
- Глава 33. Организация на проект: модули, drivers, config и pins.py
- Глава 34. Таблични данни, конфигурации и малки файлови формати

Част VII: Сигнали, измерване, енергия и индикация

- Глава 35. Филтриране, калибриране и хистерезис при входни сигнали
- Глава 36. Измерване и диагностика с осцилоскоп и logic analyzer
- Глава 37. Ниска консумация и събуждане
- Глава 38. Динамична 7-сегментна индикация и матрично сканиране
- Глава 39. NeoPixel
- Глава 40. WS2812 през machine.WS2812 и SCI TX-only

Част VIII: FSM като начин на мислене

- Глава 41. Минимална FSM и таблична FSM
- Глава 42. FSM с timeout
- Глава 43. Conditions vs Events
- Глава 44. Guard функции
- Глава 45. Приоритети и event queue
- Глава 46. Tickless timers и sleep идея

Част IX: Надеждни вградени програми и архитектура на приложението

- Глава 47. Обработка на грешки, recovery и fail-safe поведение
- Глава 48. Watchdog и самовъзстановяване
- Глава 49. Състояние на системата: heartbeat, health checks и debug интерфейс
- Глава 50. Капсуловане на периферии като класове
- Глава 51. От пример към завършено приложение
- Глава 52. RGB_Guardian — казус: игра на WS2812 лента

Част X: Курс по елементарна сигнална обработка

- Урок 1. sample, sample_rate, DC/AC, midpoint=2048
- Урок 2. Генериране на sine, square, triangle, saw
- Урок 3. Signed срещу unsigned, bias и clipping
- Урок 4. Смесване на два тона и headroom
- Урок 5. RMS и power
- Урок 6. FIR low-pass
- Урок 7. IIR biquad low-pass/high-pass
- Урок 8. FFT, Hamming window, magnitude, bands

- Урок 9. `audiomixer` и `block-based mono playback`
- Урок 10. Финален `mini-project`

Част XI: Цифрова обработка на аудио сигнали (DSP)

- Глава 53. `Storm ADC` — DTC-управлявано аудио семплиране
- Глава 54. `FFT` — честотен анализ с `CMSIS-DSP`
- Глава 55. `MSGEQ7` емуляция — 7-лентов спектрален анализатор
- Глава 56. `DSP Synthesis` — Ретро синтезатор архитектура

Част I: Първи стъпки с борда и с MicroPython

Ключови термини: REPL — интерактивна конзола (>>>), в която пишете Python код директно на борда. active-low — LED светва при value(0), не при value(1). asyncio — модул за кооперативна многозадачност: докато една задача чака, друга работи. await — отстъпва управлението на друга задача.

Тази част дава най-краткия вход към борда: първа програма, първи видим резултат и първо усещане за неблокиращо мислене.

Глава 1. Първа програма [Начинаещ]

[Tip] Аналогия: LED1 е като обърнат ключ — при нормален ключ натискаш = светва. Тук value(0) = светва, value(1) = гасне. Платката е проектирана така — запомни го веднъж и всичко следва логично.

LED1 active-low логика:

```
value(1) → GPIO = 3.3V → LED = изгасен  x
value(0) → GPIO = 0V   → LED = светнал   v
```

led.value(...)	Напрежение	LED
1	3.3V	[X] изгасен
0	0V	[Tip] светнал

```
from machine import Pin # Импортираме класа Pin, чрез който управляваме GPIO пиновете.
import time # Импортираме модула time, за да правим закъснения между действията.

led = Pin("LED1", Pin.OUT, value=1) # Създаваме изходен обект за LED1 и започваме с изгасен LED.

for blink_index in range(3): # Обхождаме три последователни мигания с for цикъл.
    led.value(0) # Светваме LED1 като подадем активното за платката ниво.
    time.sleep_ms(300) # Изчакаме 300 милисекунди, за да се вижда светването.
    led.value(1) # Гасим LED1, за да завършим едно мигане.
    time.sleep_ms(300) # Изчакаме още 300 милисекунди преди следващото мигане.
```

[Eye] Какво виждаш: LED1 мига 3 пъти и спира. Ако не мига — провери active-low логиката в таблицата по-горе.

[Lab] Експеримент: Промени 300 на 100 — миганията стават по-бързи. Промени range(3) на range(10) — мига 10 пъти.

[!] Важно: LED1 = P204 на тази платка е active-low. value(1) го ГАСИ, не светва!

[File] Файл: examples/20_language_basics/01_sample_program.py

Резюме

- Показва най-малкия работещ MicroPython пример за VK_RA4M2.
- Ключовата особеност: LED1 е **active-low** — value(0) светва, value(1) гасне.
- Отправна точка за всички следващи практически уроци.

Самопроверка

1. Защо LED1 светва при `value(0)`, а не при `value(1)`?
2. Каква е ролята на `Pin.OUT` и `sleep_ms()` в примера?
3. Коя е най-малката промяна, с която бихте проверили, че разбирате кода?

Упражнения

1. [Lab] Направете различни времена за светнато и изгасено (напр. 100ms свети, 900ms тъмно).

Подсказка: ползвайте два различни `sleep_ms()` — един след `value(0)`, друг след `value(1)`.

2. [Lab] Изнесете броя мигания и закъснението в именувани константи `BLINK_COUNT` и `DELAY_MS`.

Подсказка: `from micropython import const` и дефинирайте `BLINK_COUNT = const(5)` преди цикъла.

3. [Lab] Добавете `print("ON")` и `print("OFF")` и наблюдавайте в конзолата.

Подсказка: поставете `print("ON")` веднага след `led.value(0)`, преди `sleep_ms()`.

Ако не работи

- [!] Симптом: LED не мига изобщо. → Проверете дали `value=1` го инициализира в ИЗГАСЕНО (правилно).
- [!] Симптом: LED свети постоянно. → Вероятно `value(0)` и `value(1)` са разменени — вижте таблицата.

Нататък

→ „Първи асинхронни примери“ — как платката прави две неща едновременно без блокиращ цикъл.

Глава 2. Първи асинхронни примери [Начинаещ]

[Тip] Аналогия: Представете си готвач, който готви две ястия едновременно. Той не чака неподвижно пред едната тенджера — слага я на котлона, поглежда другата, после се връща. `asyncio` работи по същия принцип: докато една задача “чака” (`await`), процесорът върши друга работа.

Блокиращ стил (BAD):

```
blink: вкл 500ms чака [X]
blink: изкл 500ms чака [X]
blink: вкл 500ms чака [X]
(counter НИКОГА не работи!)
```

Asyncio стил (GOOD):

```
t=0ms → blink: вкл LED
t=0ms → counter: started
t=500ms → blink: изкл LED
t=1000ms → counter: 1 секунда
t=1000ms → blink: вкл LED
```

Задача 1: Мигане на LED

```
import asyncio
from machine import Pin

led = Pin("LED1", Pin.OUT, value=1) # active-low: value=1 → изгасен

async def blink_task():
    while True:
        led.value(0)          # [Tip] светваме
        await asyncio.sleep_ms(500) # отстъпваме – друга задача може да работи
        led.value(1)          # [X] гасим
        await asyncio.sleep_ms(500) # пак отстъпваме
```

Задача 2: Броене на секунди

```
async def count_seconds_task():
    seconds = 0
    while True:
        seconds += 1
        print("Изминали секунди:", seconds)
        await asyncio.sleep(1) # чакаме 1 секунда без да блокираме

async def main():
    asyncio.create_task(blink_task()) # стартираме мигането
    asyncio.create_task(count_seconds_task()) # стартираме брояча
    while True:
        await asyncio.sleep(1) # main задачата поддържа event loop-а жив

asyncio.run(main()) # Стартираме event loop-а – без този ред нищо не се случва!
```

[!] Важно: `asyncio.run(main())` е входната точка — без него задачите са само дефиниции, но никога не се изпълняват.

Ключова дума	Смисъл
<code>async def</code>	Дефинира задача, която може да се прекъсва
<code>await</code>	"Чакай, но пусни другите да работят"
<code>asyncio.create_task()</code>	Стартира задача в event loop-а

[Еуе] Какво виждаш: LED1 мига равномерно, а в REPL се извежда брояч на секунди — двете се случват едновременно.

[Lab] Експеримент: Промени `sleep_ms(500)` на `sleep_ms(100)` — LED ще мига 5× по-бързо, докато брояча остава на 1 секунда.

[!] Важно: Ако задача изпълнява дълга работа без `await`, тя блокира всички останали! Дори `time.sleep(1)` вместо `await asyncio.sleep(1)` замразява event loop-а.

Файлове в главата:

- `examples/01_blink_async.py`
- `examples/02_two_tasks.py`

Резюме

- `asyncio` позволява на две задачи да споделят процесора без взаимно блокиране.
- `await` е моментът, в който задачата “отстъпва” и другите могат да работят.
- Без `await` в цикъл — `event loop`-ът замръзва.

Самопроверка

1. Какво прави `await` в асинхронната задача?
2. Защо една `async` задача не бива да работи без да отстъпва управлението?
3. Каква е практическата разлика между две `async` задачи и един блокиращ цикъл?

Упражнения

1. Добавете трета `async` задача с различен период.

Подсказка: дефинирайте `async def new_task():`` и я регистрирайте с `asyncio.create_task()`.

2. Заменете `await asyncio.sleep_ms(500)` с `time.sleep_ms(500)` и наблюдавайте ефекта.

Подсказка: `time.sleep_ms()` е блокиращ — `event loop`-ът замразява и другата задача спира.

3. Добавете брояч, който спира демонстрацията след 10 секунди.

Подсказка: следете `time.ticks_ms()` и извикайте `raise SystemExit` или `asyncio.get_event_loop().stop()` при изтичане.

Ако не работи

- Симптом: виждате поведение само от една задача. Проверка: търсете липсващ `await` или неправилно стартиране с `asyncio.create_task()`.
- Симптом: брояча не се извежда. Проверка: уверете се, че `asyncio.run()` стартира и двете задачи.

Нататък

След тази глава преминете към „Променливи и константи“, за да подредите езиковите елементи, които после ще използвате и в асинхронния код.

[Pin] Бележка: `asyncio` е използван тук неформално, за да покаже идеята. Пълното и систематично покритие на `asyncio` — `create_task`, `gather`, `Event` синхронизация — е в Глави 20–21 (Част IVб).

Част II: Езикови основи на MicroPython

Ключови термини: `int/float` — цяло число / число с десетична точка. `bytearray` — масив от байтове, ефективен за вградени системи. `callback` — функция, подадена като аргумент, извиквана по-късно от системата. `struct` — модул за пакетиране на двоични данни. `id()` — връща уникален адрес на обект в RAM.

Тук се подреждат базовите езикови конструкции, върху които стъпват всички следващи практически глави.

Глава 3. Променливи и константи [Начинаещ]

[Tip] Аналогия: Променлива е кутия с надпис — слагаш нещо вътре, после го сменяш. `const()` е гравирани надпис — не можеш да го промениш след създаването.

Тип	Синтаксис	Може да се промени	Употреба
Променлива	<code>x = 5</code>	[OK] Да	Стойности, които се обновяват
Константа	<code>X = const(5)</code>	[X] Не	Pin номера, delays, прагове
Стринг	<code>NAME = "VK_RA4M2"</code>	[X] (препоръка)	Имена и конфигурация

```
from micropython import const

BOARD_NAME = "VK_RA4M2"      # описателно — може да се промени при нужда
BLINK_COUNT = const(3)       # оптимизирано от компилатора — истинска константа
BLINK_DELAY_MS = const(200)  # 200ms hardcoded — ясно намерение
uses_asyncio = True          # булева — може да варира при runtime
```

[Pin] Защо `const()`? MicroPython заменя `const(3)` директно с числото 3 на всяко място — без *lookup* в речник. Спестява RAM и е по-бързо в *tight loops*.

[Lab] Опитай: Напишете `BLINK_COUNT = const(3)`, после опитайте `BLINK_COUNT = 5` — ще получите грешка. Сравнете с обикновена променлива.

Файл:

- `examples/20_language_basics/02_variables_and_constants.py`

Резюме

- Променлива — кутия, чийто съдържимото се сменя; константа — гравирани надпис.
- `const()` оптимизира кода: числото се инлайн-ва, не се чете от паметта.
- Добра практика: `pin`, `delay` и прагови стойности → `const()` с ГЛАВНИ_БУКВИ.

Самопроверка

1. Каква е разликата между обикновена променлива като `BOARD_NAME` и константа като `BLINK_COUNT = const(3)`?
2. Защо е по-добре важните числови настройки да имат име, вместо да ги пишете като „голи числа“ в кода?
3. Къде в бъдещ хардуерен пример бихте използвали константа за `pin`, `delay` или режим?

Упражнения

1. Изнесете още две стойности от предишния LED пример в константи, например брой мигания и закъснение.
Подсказка: използвайте `from micropython import const` и `BLINK_COUNT = const(3)`.
2. Добавете именувана константа за режим, например `MODE_BLINK = const(1)`, и я използвайте в кратък `if`.
Подсказка: `if mode == MODE_BLINK:` — сравнявайте с константата, а не с числото.
3. Пренапишете малък пример така, че всички константи да са с главни букви, а всички променливи с малки.
Подсказка: конвенцията е `DELAY_MS = const(200)` за константи, `counter = 0` за променливи.

Ако не работи

- Симптом: имената в кода вече не ви подсказват какво правят числата. Проверка: превърнете важните стойности в добре именувани константи.
- Симптом: `const()` не дава очакваната полза. Проверка: използвайте го основно за числови константи и не го бъркайте с обикновена променлива.

Нататък

След тази глава преминете към „Оператори, условия и цикли“, за да започнете да работите не само със стойности, но и с логика и управление на изпълнението.

Глава 4. Оператори, условия и цикли [Начинаещ]

[Tip] Аналогия: Програмата е като рецепта. Аритметиката пресмята (добавяме 2 лъжици). Условието решава (ако тестото е твърде рядко → добавете брашно). Цикълът повтаря (разбивайте 5 минути).

Аритметични оператори

Оператор	Действие	Пример	Резултат
+	Събиране	7 + 3	10
-	Изваждане	7 - 3	4
*	Умножение	7 * 3	21
/	Деление (реално)	7 / 3	2.333...
//	Цяло деление	7 // 3	2
%	Остатък (модул)	7 % 3	1
**	Степенуване	2 ** 10	1024

```
adc_raw = 3000
adc_max = 4095

voltage = adc_raw * 3.3 / adc_max      # 2.42 V — реално деление
percent = adc_raw * 100 // adc_max     # 73 — цяло деление, без дробна част
remainder = adc_raw % 256              # 184 — остатък, полезен за кръгови буфери
power_of_two = 1 << 12                 # 4096 — но << е битов, не аритметичен (вж. по-долу)
```

[Pin] Кога какво: / дава float (бавно на MCU). // дава int (бързо). В embedded код предпочитайте // освен ако наистина трябва дробна част.

Оператори за сравнение

Оператор	Действие	Пример	Резултат
==	Равно	7 == 7	True
!=	Различно	7 != 3	True
<	По-малко	3 < 7	True
>	По-голямо	7 > 3	True
<=	По-малко или равно	3 <= 3	True
>=	По-голямо или равно	7 >= 10	False

```
temperature = 42
threshold_low = 10
threshold_high = 40

is_hot = temperature > threshold_high      # True – над горния праг
is_cold = temperature < threshold_low      # False – не е под долния
is_exact = temperature == 42               # True – точно 42
is_in_range = (temperature >= threshold_low) and (temperature <= threshold_high) # False – 42 > 40
```

[!] **Внимание:** = е присвояване (a = 5). == е сравнение (a == 5). Объркването им е най-честата грешка при начинаещи.

Логически оператори

Оператор	Действие	Пример	Резултат
and	И двете верни	True and False	False
or	Поне едно вярно	True or False	True
not	Обръщане	not True	False

```
button_pressed = True
light_is_dark = True
door_is_open = False

# Лампата светва ако е тъмно И бутонът е натиснат:
lamp_on = light_is_dark and button_pressed      # True

# Вентилаторът тръгва ако вратата е отворена ИЛИ е горещо:
fan_on = door_is_open or (temperature > 35)     # True (42 > 35)

# Аларма ако нещо НЕ е наред:
alarm = not (lamp_on and fan_on)                # False – всичко е OK
```

[Pin] **Приоритет:** not > and > or. За яснота ползвайте скоби: (a > 0) and (b < 10) вместо a > 0 and b < 10.

Присвояване с операция

Оператор	Еквивалентно на	Пример
+=	a = a + 1	count += 1
-=	a = a - 1	level -= 10
*=	a = a * 2	amp *= 2
//=	a = a // 2	value //= 2
&=	a = a & mask	flags &= 0x0F
\	=`	`a = a \
^=	a = a ^ mask	flags ^= 0x01
<<=	a = a << n	value <<= 4
>>=	a = a >> n	value >>= 4

```

count = 0
count += 1      # 1 – най-честият случай: брояч
count += 1      # 2

flags = 0b00000000
flags |= 0x80   # 0b10000000 – задаваме бит 7
flags &= ~0x80  # 0b00000000 – изчистваме бит 7

```

Условия: if / elif / else

```

left_value = 7
right_value = 3

if left_value < right_value:
    print("По-малка")
elif left_value == right_value:
    print("Равна")
else:
    print("По-голяма") # ← това се изпълнява

```

Трите форми на цикли:

```

for index in range(3):      # 0, 1, 2 – знаем колко пъти
    print(index)

counter = 0
while counter < 3:          # докато условие е вярно
    print(counter)
    counter += 1

while True:                 # do-while: изпълни, после провери
    print("поне веднъж")
    if True:
        break

```

[!] Важно: `goto` и `switch/case` не съществуват в `MicroPython`. Еквивалентите са `if/elif/else` и таблици от функции (вижте Глава 9).

[Lab] Опитай: Сложете `left_value = right_value = 5` — кой клон се изпълнява? После добавете `print(f"{left_`

value} vs {right_value}”) преди if, за да разберете защо.

Файл:

- examples/20_language_basics/03_operators_conditions_loops.py

Резюме

- if/elif/else — вземете решение; for — повторете N пъти; while — повтаряйте докато условие е вярно.
- while True + break е equivalent на C's do-while.
- Без break или промяна на условието → безкраен цикъл.

Самопроверка

1. Каква е разликата между аритметичен оператор, оператор за сравнение и логически оператор?
2. Защо while True без условие за изход е опасен при учебен пример?
3. Как бихте използвали if/elif/else и цикъл в реален пример с LED, бутон или ADC?

Упражнения

1. Променете началните стойности на left_value и right_value така, че да минете и през трите клона на if/elif/else.

Подсказка: за третия клон (else) задайте равни стойности, например left_value = right_value = 5.

2. Напишете цикъл, който брои от 5 до 1, а после отпечатва START.

Подсказка: for i in range(5, 0, -1): брои в намаляващ ред с стъпка -1.

3. Направете малък пример с while True и break, който спира след определен брой опити.

Подсказка: ползвайте брояч count = 0 и if count >= MAX: break вътре в цикъла.

Ако не работи

- Симптом: програмата влиза в безкраен цикъл. Проверка: уверете се, че броячът наистина се променя и че имате условие за изход.
- Симптом: влиза грешният клон на условието. Проверка: отпечатайте междинните стойности и проверете последователно всяко условие.

Битови оператори и шестнадесетичен запис

[Тip] Аналогия: Ако обикновените оператори (+, -, ==) работят с числа, битовите оператори работят с отделните битове вътре в числото — като превключватели на платка, всеки с отделна роля.

В embedded програмирането битовите оператори се ползват навсякъде: за маски на регистри, за debounce логика, за пакетиране на данни.

```
a = 0b11001010      # Двоичен запис: 202 в десетичен вид
b = 0xFF             # Шестнадесетичен запис: 255

# Основни битови оператори:
print(a & 0x0F)       # AND маска — взима само долните 4 бита → 0b1010 = 10
print(a | 0x01)       # OR — задава бит 0 → 0b11001011 = 203
print(a ^ 0xFF)       # XOR — обръща всички битове → 0b00110101 = 53
print(~a & 0xFF)      # NOT (инверсия) — обръща битовете → 53
print(a << 2)         # Изместване наляво с 2 → умножение по 4
print(a >> 4)         # Изместване надясно с 4 → горните 4 бита → 0b1100 = 12
```

Оператор	Действие	Пример	Резултат
&	AND (маска)	0b1100 & 0b1010	0b1000
\	OR (задаване)	0b1100 \ 0b1010	0b1110
^	XOR (обръщане)	0b1100 ^ 0b1010	0b0110
~	NOT (инверсия)	~0b0000 & 0xFF	0b11111111
<<	Изместване наляво	1 << 3	0b1000 = 8
>>	Изместване надясно	0xFF >> 4	0b1111 = 15

[Pin] Нотация: 0b = двоично, 0x = шестнадесетично. 0xFF = 255 = 0b11111111. Тези записи се срещат навсякъде в embedded кода.

Типичен пример от по-нататъшните глави — debounce на бутон с битов shift:

```
history = ((history << 1) | button.value()) & 0xFF # Измества историята и добавяме новото четене
if history == 0x00: # 8 последователни 0 → стабилно натиснат
    print("PRESS")
```

Нататък

След тази глава преминете към „Enums в MicroPython“, за да видите как състояния и режими могат да станат по-четими чрез именувани стойности.

Глава 5. Enums в MicroPython [Начинаещ]

[Tip] Аналогия: Светофарите биха могли да са 0, 1, 2 — но STATE_RED, STATE_YELLOW, STATE_GREEN е несравнимо по-четимо. Енит е именувана стойност вместо гол номер.

```
Без enum: if state == 2: # Какво означава 2? [?]
С enum:   if state == STATE_BLINK: # [OK] Веднага ясно!
```

```
from micropython import const

STATE_OFF   = const(0) # LED изключен
STATE_ON    = const(1) # LED включен
STATE_BLINK = const(2) # LED мига

# Таблица за четимо описание (полезна при print/debug):
STATE_NAMES = {0: "OFF", 1: "ON", 2: "BLINK"}

state = STATE_BLINK
print("Текущо:", STATE_NAMES[state]) # → "Текущо: BLINK"
```

[Pin] Защо не enum.Enum? MicroPython не включва модула enum от стандартната библиотека. const() е по-лек и по-бърз — достатъчен за embedded.

[Lab] Опитай: Добавете STATE_ERROR = const(3) и ключ в STATE_NAMES. Напишете if/elif за всяко състояние.

Файл:

- examples/20_language_basics/04_enums.py

Резюме

- Enum = именувана константа вместо магическо число в кода.
- В MicroPython: `STATE_X = const(N) + dict` за печат.
- По-четимо, по-лесно за debugging, нулева runtime цена.

Самопроверка

1. Защо в MicroPython често използваме `const()` вместо пълен `enum` тип?
2. Каква полза има кодът, ако пишете `STATE_BLINK`, а не просто 2?
3. В кой реален пример на борд бихте описали режимите като `STATE_OFF`, `STATE_ON`, `STATE_BLINK`?

Упражнения

1. Добавете още две състояния, например `STATE_ERROR` и `STATE_IDLE`, и направете таблица за печат на имената им.

Подсказка: разширете речника `STATE_NAMES = {0: "OFF", ..., 3: "ERROR", 4: "IDLE"}`.

2. Напишете `if/elif/else`, което реагира различно на всяко състояние.

Подсказка: сравнявайте `state == STATE_ERROR` — не `state == 3` — за четимост.

3. Използвайте именувани състояния в минипример за LED: изключен, включен, мигащ.

Подсказка: дефинирайте `STATE_OFF = const(0)`, после присвоете `state = STATE_BLINK` и реагирайте в `if`.

Ако не работи

- Симптом: стойностите на състоянията са объркващи. Проверка: дръжте имената ясни и последователни, а числата стабилни.
- Симптом: при печат виждате числа, но не и смисъл. Проверка: добавете таблица или `dict`, който превежда състоянието до четимо име.

Нататък

След тази глава преминете към „Масиви и таблици“, за да започнете да съхранявате повече данни, а не само единични стойности.

Глава 6. Масиви и таблици [Начинаещ]

[Тър] Аналогия: *list* е кутия с отделения — гъвкава, може да пазиш различни типове. *array* е редица еднотипни чекмеджета — по-компактна, само един тип. *bytearray* е суров буфер — за протоколи и периферии.

Тип	Пример	RAM	Когато
<code>list</code>	<code>[100, 200, 300]</code>	~повече	Гъвкавост, смесени типове
<code>array("H", ...)</code>	<code>array("H", [0, 1000])</code>	~2× по-малко	16-бит числа, PWM duty
<code>bytearray</code>	<code>bytearray([0x10, 0x20])</code>	минимален	SPI/I2C пакети, WS2812

```

from array import array

adc_samples = [1200, 1215, 1198, 1222]          # list – гъвкав
led_table = [[“LED1”, “P204”], [“SW1”, “P400”]]    # вложен list = таблица
pwm_duty_values = array(“H”, [0, 16384, 32768, 65535]) # 16-бит, 2× по-малко RAM
raw_bytes = bytearray([0x10, 0x20, 0x30, 0x40])    # суров буфер за протоколи

```

[Eye] Визуализация на паметта:

```

list [1200, 1215]:  [ptr→1200][ptr→1215]  ← всеки елемент е обект
array(“H”, [1200]): [0x04B0][0x04B0]      ← директно 2 байта на елемент
bytearray([0x10]): [0x10]                 ← 1 байт на елемент

```

[Lab] Опитай: `import micropython; micropython.mem_info()` преди и след създаването на `list` срещу `array` с 100 елемента — вижте разликата в RAM. (Модулът `micropython` е за диагностика — подробно е разгледан в [Глава 30](#).)

Файл:

- `examples/20_language_basics/05_arrays.py`

Резюме

- `list` — гъвкав, но харчи повече RAM.
- `array` — ефективен за числови данни от един тип.
- `bytearray` — суров буфер, задължителен за комуникационни протоколи.

Самопроверка

1. Кога бихте избрали `list`, кога `array`, и кога `bytearray`?
2. Какво печелите, ако съхранявате измервания в таблица, вместо в отделни променливи?
3. Защо `array(“H”, ...)` и `bytearray(...)` са особено полезни при периферии и буфери?

Упражнения

1. Добавете още няколко ADC проби и изчислете минимална, максимална и средна стойност.
Подсказка: `min(samples)`, `max(samples)` и `sum(samples) // len(samples)` върху `list` или `array`.
2. Направете таблица с няколко бордови ресурса, например LED1, SW1, P014, P015, и ги отпечатайте ред по ред.
Подсказка: `for row in table: print(row[0], row[1])` — вложен списък е директна 2D таблица.
3. Създайте `bytearray` за суров пакет от 4 байта и променете два байта „на място”.
Подсказка: `pkt = bytearray(4)` после `pkt[0] = 0xAA` — `bytearray` е *mutable* за директна промяна.

Ако не работи

- Симптом: получавате `IndexError`. Проверка: вижте дали индексът е в допустимите граници на масива или таблицата.
- Симптом: `array` не приема стойностите. Проверка: проверете `type code-a`, например `“H”` за 16-битови беззнакови стойности.

Нататък

След тази глава преминете към „Структури в MicroPython”, където вече не пазим само редици от стойности, а и по-смислени записи с имена на полета.

Глава 7. Структури в MicroPython [Начинаещ]

[Tip] Аналогия: struct от C е като визитна картичка — name, phone, email заедно. В MicroPython имате три варианта в зависимост от нуждата.

Вариант	Синтаксис	Когато
dict	<code>{"name": "LED1", "pin": "P204"}</code>	Гъвкавост, runtime промяна
namedtuple	<code>Record = namedtuple("R", ("name",))</code>	Read-only, достъп по атрибут
Клас	<code>class Sensor: ...</code>	Методи + данни

```
from collections import namedtuple

# namedtuple – компактна read-only структура
SensorRecord = namedtuple("SensorRecord", ("name", "pin", "unit"))
temp = SensorRecord("Temp", "P000", "°C")
print(temp.name, temp.pin)  # достъп по атрибут

# dict – гъвкава структура
light_sensor = {
    "name": "Light",
    "adc_pin": "P000",
    "limits": {"dark": 800, "bright": 3000}  # структура от структури
}

# списък от структури – C-style масив от struct
devices = [
    {"name": "LED1", "pin": "P204", "kind": "output"},
    {"name": "SW1", "pin": "P400", "kind": "input"},
]
for d in devices:
    print(d["name"], "→", d["pin"])
```

[Pin] Съвет: За статични записи (конфигурация при startup) — namedtuple. За записи, които се обновяват — dict. За групиране на поведение — клас.

Файл:

- `examples/20_language_basics/06_structures.py`

Резюме

- C's struct → MicroPython dict, namedtuple или клас.
- namedtuple е read-only и по-компактен; dict е гъвкав.
- Вложени dict = структура от структури (конфигурация с прагове и режими).

Самопроверка

1. Каква е разликата между dict, namedtuple и списък от dict?
2. Кога е по-удобно да пазите данните като „структура от структури“?
3. Как бихте описали с такава структура сензор, пин, прагове и режим на работа?

Упражнения

1. Добавете второ устройство в списъка `devices` и отпечатайте имената и пиновете и на двете.

Подсказка: `devices.append({"name": "SW1", "pin": "P400"})` и итерирайте с `for d in devices:`.

2. Разширете `light_sensor` с още едно вложено поле, например `limits["alarm"]`.

Подсказка: `light_sensor["limits"]["alarm"] = 3500` добавя поле в съществуващ вложен `dict`.

3. Направете структура за бутон с име, `pin`, `active level` и `debounce` време.

Подсказка: `button = {"name": "SW1", "pin": "P400", "active": 0, "debounce_ms": 40}`.

Ако не работи

- Симптом: четете грешно поле. Проверка: различавайте достъп с ключ `record["name"]` от достъп по атрибут при `namedtuple`.
- Симптом: вложените данни се оказват по-дълбоки от очакваното. Проверка: отпечатайте цялата структура и проследете пътя до полето стъпка по стъпка.

Класове — бърз старт

[Tip] Аналогия: Ако `dict` е чантичка с етикети, класът е чантичка с етикети и с вградени инструкции какво да се прави с тях. Класът събира данни (състояние) и действия (методи) на едно място.

От следващите глави ще срещате класове като `RetroSynth`, `TouchPad`, `AudioADC`. Ето минималният модел:

```
from machine import Pin

class StatusLed:
    def __init__(self, pin_name="LED1"):      # Конструктор — извиква се при създаване.
        self.pin = Pin(pin_name, Pin.OUT, value=1) # self.pin = вътрешно състояние на обекта.

    def on(self):                             # Метод — действие върху обекта.
        self.pin.value(0)                    # Active-low: 0 = светва.

    def off(self):                             # Метод — действие върху обекта.
        self.pin.value(1)                    # 1 = гаси.

led = StatusLed("LED1")                      # Създаваме обект (инстанция на класа).
led.on()                                     # Извикваме метод — LED светва.
led.off()                                    # LED гасне.
```

Елемент	Какво е	Пример
<code>class StatusLed:</code>	Дефиниция на клас (шаблон)	Описва какво е <code>StatusLed</code>
<code>__init__(self, ...)</code>	Конструктор — стартира се при <code>StatusLed()</code>	Настройва пина
<code>self</code>	Препратка към текущия обект	<code>self.pin</code> = пинът на този LED
<code>led.on()</code>	Извикване на метод	Действие върху конкретен обект

[Pin] Бележка: Пълно обяснение на класовете като архитектурен инструмент е в Глава 50 (Капсуловане на периферии). Тук е показан само минимумът, нужен за четене на следващите глави.

Нататък

След тази глава преминете към „Указатели и еквивалентите им в MicroPython“, за да видите как споделяне и промяна на данни се случват без C синтаксис за указатели.

Глава 8. Указатели и еквивалентите им в MicroPython [Начинаещ]

[Тър] Аналогия: Два ключа за една стая — и двата ключа отварят една и съща стая. Ако преместите мебелите с ключ А, ключ Б ще влезе в същата преместена стая. memoryview е като прозорец — виждате стаята и можете да пипате нещата, без да правите копие.

```
alias_list = adc_list:      [100][200][300]
                             ↑      ↑
                             adc_list alias_list ← ЕДИН обект, ДВЕ имена

adc_copy = list(adc_list):  [100][200][300] ← отделно копие
                             ↑      ↑
                             adc_list adc_copy   ← ДВА различни обекта
```

```
adc_list = [100, 200, 300]
alias_list = adc_list      # НЕ е копие – второ име към същия обект
alias_list[1] = 999
print(adc_list)            # [100, 999, 300] – промяната се вижда!

tx_buffer = bytearray([1, 2, 3, 4])
tx_view = memoryview(tx_buffer) # прозорец – без копиране
tx_view[2] = 77
print(tx_buffer)           # bytearray(b'\x01\x02\x03\x04') – промяна!
```

[!] Важно: adc_copy = list(adc_list) прави реално копие. adc_alias = adc_list прави второ название към същия обект. Разликата е критична при буфери!

[Lab] Опитай: `print(id(adc_list) == id(alias_list))` → True. После `print(id(adc_list) == id(list(adc_list)))` → False.

Файлове:

- examples/20_language_basics/07_pointers_equivalents.py
- examples/28_machine_misc/02_machine_mem_and_bootloader.py

Резюме

- alias = original → второ имена, **ЕДИН** обект.
- copy = list(original) → **ДВА** обекта.
- memoryview → прозорец без копиране — задължителен за DMA буфери.

Самопроверка

1. Защо `alias_list = adc_list` не прави копие, а второ име към същия обект?
2. Какво печелите от `memoryview`, ако не искате да копирате буфера?
3. Кога ниското ниво през `machine.mem32` е полезно и защо трябва да се ползва внимателно?

Упражнения

1. Направете второ име към един списък и докажете с печат, че промяна през едното има се вижда и през другото.

Подсказка: `'alias = original'` после `'alias[0] = 99'` и `'print(original[0])'` — трябва да е `'99'`.

2. Създайте bytearray, после memoryview към него, и променете няколко байта през изгледа.

Подсказка: `mv = memoryview(buf)` после `mv[1] = 0xFF` — `buf[1]` се изменя без копиране.

3. Обяснете с кратък експеримент разликата между „копие на данни“ и „втори изглед към същите данни“.

Подсказка: сравнете `id(alias) == id(original)` с `id(list(original)) == id(original)` — единият дава `True`.

Ако не работи

- Симптом: променяте буфер, но оригиналът не се изменя. Проверка: вижте дали не сте направили копие, вместо изглед към същите данни.
- Симптом: ниско ниво през `mem32` дава странен резултат. Проверка: работете само ако знаете точния адрес и смисъла на регистъра.

Нататък

След тази глава преминете към „Функции, callbacks и таблици от функции“, където вече самото поведение се подава и избира като данни.

Глава 9. Функции, callbacks и таблици от функции [Начинаещ]

[Tip] Аналогия: Функция е рецепта — подаваш съставки, получаваши ястие. Callback е поръчка до сервитьора: „Извикай готвач А при готовност“ — сервитьорът не знае готвача, само изпълнява. Таблица от функции е меню — всеки номер съответства на различна операция.

```
Директно извикване:  add_values(7, 3)    → 10 веднага
Callback:            run_callback(print, 42) → print(42) = "42"
Function table:      ops[1](5)          → square(5) = 25
```

```
def add_values(a, b):
    return a + b

def run_callback(callback, value):
    callback(value)          # не знаем коя функция – просто я викаме

def square(value):
    return value * value

def double(value):
    return value * 2

# Таблица от функции – C-style function pointer table
operation_table = [square, double]
print(operation_table[0](5)) # square(5) = 25
print(operation_table[1](5)) # double(5) = 10
```

[Pin] В embedded кода: Callbacks се ползват за IRQ handlers, таймерни тикове, и FSM преходи (вижте Глави 42-47). Таблица от функции замества switch-case.

[Lab] Опитай: Подайте `run_callback(print, "hello")` — функцията `print` е callback. После опитайте `run_callback(square, 7)`.

Файл:

- `examples/20_language_basics/08_functions.py`

Резюме

- Функция = рецепта с вход/изход.
- Callback = функция като аргумент — извиква се по-късно.
- Таблица от функции = C function pointer table → избирате операция по индекс.

Самопроверка

1. Каква е разликата между това да извикате функция веднага и това да я подадете като callback?
2. Защо списъкът от функции може да се мисли като MicroPython еквивалент на таблица от function pointers?
3. Къде в бордов код бихте използвали callback или таблица от операции?

Упражнения

1. Добавете в `operation_table` още една функция, например кубирание или удвояване.

Подсказка: `operation_table.append(lambda x: x3)` или дефинирайте `def cube(x): return x3` и добавете.

2. Напишете функция `run_selected(index, value)`, която избира функция от таблицата и я изпълнява.

Подсказка: `return operation_table[index](value)` — достъпвате по индекс и извиквате директно.

3. Направете callback, който отпечатва стойност, а после заменете печата с друга callback функция.

Подсказка: `run_callback(print, 42)` после `run_callback(square, 42)` — функцията се подава като аргумент.

Ако не работи

- Симптом: callback не се изпълнява. Проверка: вижте дали подавате самата функция (`square`), а не резултата от нейното предварително извикване (`square(5)`).
- Симптом: таблицата от функции връща грешен резултат. Проверка: отпечатайте избрания индекс и `operation_table[i].__name__`.

Нататък

След тази глава преминете към „Цифров изход“, за да започнете да прилагате езиковите конструкции върху реален хардуерен ресурс.

Част III: Цифрови и аналогови входове и изходи

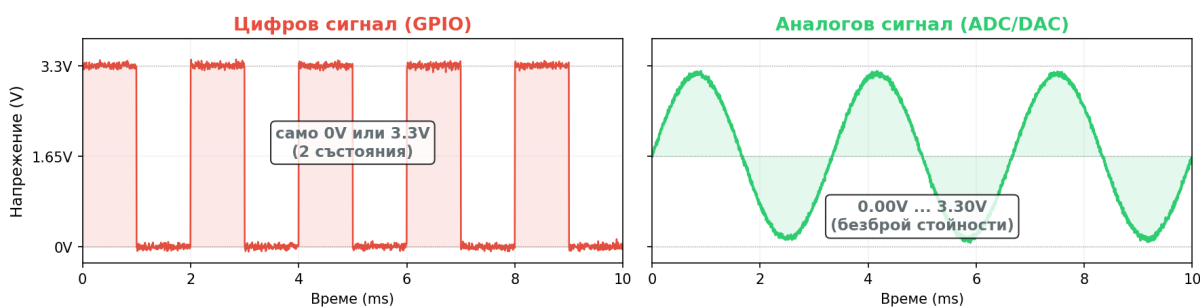
Ключови термини: *GPIO* — пин, конфигуриран като цифров вход или изход (0/1). *ADC* — преобразува аналогово напрежение в число (0–4095 при 12-bit). *DAC* — обратното: число → напрежение. *PWM* — бързо превключване 0/1 с променлив *duty*, имитиращо аналогово ниво. *Vref* — опорно напрежение (3.3V). *wavetable* — масив с предварително изчислена вълнова форма за *DAC*. *DTC* — хардуер, който копира данни без *CPU*.

Вече можем да пишем програми. Сега стъпка по стъпка ще се научим как програмите **обменят информация с околния свят**.

Какво е информация? Всичко, което се променя: натискане на бутон, промяна на температура, настъпване на събитие. Компютрите обработват и обменят два вида информация:

- **Цифрова** — само две състояния: 0 или 1 (изключено/включено, натиснато/отпуснато).
- **Аналогова** — непрекъснат диапазон от стойности: 0.00V ... 3.30V (температура, осветеност, звук).

Два вида информация: цифрова (0/1) и аналогова (непрекъснат диапазон)



Цифров vs аналогов сигнал

Хардуерните блокове за обмен с външния свят:

Блок	Посока	Тип информация	Какво прави	Пример
GPIO изход	Борд → Свят	Цифрова	Подава 0 или 1 на пин	Светване на LED, управление на реле
GPIO вход	Свят → Борд	Цифрова	Чете 0 или 1 от пин	Натискане на бутон, краен изключвател
ADC	Свят → Борд	Аналогова → Цифрова	Преобразува напрежение в число	Температурен сензор, потенциометър
DAC	Борд → Свят	Цифрова → Аналогова	Преобразува число в напрежение	Аудио сигнал, управляващо напрежение
PWM	Борд → Свят	Цифрова (имитира аналогов)	Бързо превключване 0/1 с променлив <i>duty</i> цикъл	Яркост на LED, скорост на мотор

[Pin] Пълната картина: *GPIO* работи с цифрова информация (0/1). *ADC* и *DAC* са мостовете към аналоговия свят — *ADC* чете (аналогов → цифров), *DAC* пише (цифров → аналогов). *PWM* е хибрид — цифров сигнал, който чрез средна стойност имитира аналогово ниво.

В тази част ще видите всичките пет блока — от прост *GPIO* до прецизен *DAC* аудио изход.

Глава 10. Цифров изход [Начинаещ]

В Глава 1 направихте мигането с `Pin.OUT`. Тук разглеждаме по-детайлно GPIO изхода — режими, токови ограничения и множество пинове. Припомняме: LED1 е active-low (вж. Глава 0 за обяснението).

```
from machine import Pin
import time

led = Pin("LED1", Pin.OUT, value=1) # value=1 → изгасен при старт (active-low)
led.value(0)                        # [Tip] светваме
time.sleep_ms(700)
led.value(1)                        # [X] гасим
```

[!] Важно: LED1 = P204, active-low — 0 светва, 1 гасне. GPIO пинът не е мощен захранващ изход — не свързвайте директно двигатели или реле.

[Eye] Какво виждаш: LED1 светва за 700 ms и после угасва.

[Lab] Експеримент: Промени 700 на 100 — мига ли достатъчно бързо да изглежда непрекъснато? (При ~50 ms очите вече не различават мигането.)

Тук се обясняват:

- логическа 0
- логическа 1
- защо при active-low LED те не означават директно „изключено“ и „включено“
- че допустимият ток и товароспособността се гледат по hardware manual и схемата на борда

Файлове:

- examples/21_digital_io/01_digital_output.py
- examples/01_blink_async.py

Резюме

- `Pin.OUT` прави пина управляван изход, а `value(0)` и `value(1)` задават логическото му ниво.
- При LED1 логиката е active-low, затова 0 означава „светни“, а 1 означава „изгаси“.
- Това е първият момент в книгата, в който една единична стойност в кода води до пряк видим ефект на платката.

Самопроверка

1. Защо LED1 светва при `value(0)`, а не при `value(1)`?
2. Каква е ролята на `Pin.OUT` в този пример?
3. Какво бихте променили първо, ако искате LED-ът да мига, вместо само да смени състояние веднъж?

Упражнения

1. Направете функция `blink_n(count, on_ms, off_ms)`, която мига LED1 зададен брой пъти.

Подсказка: `for _ in range(count): led.value(0); sleep_ms(on_ms); led.value(1); sleep_ms(off_ms)`.

2. Добавете `print("LED ON")` и `print("LED OFF")`, за да свържете логиката в кода с физическия ефект.

Подсказка: поставете `print()` веднага след всяко `led.value()`, преди `sleep_ms()`.

3. Направете две различни мигащи последователности: бавна и бърза.

Подсказка: извикайте `blink_n(3, 500, 500)` и после `blink_n(5, 100, 100)` — само параметрите се различават.

Ако не работи

- Симптом: LED1 не реагира. Проверка: използвайте точно `Pin("LED1", Pin.OUT)` и започнете с прост пример без допълнителна логика.
- Симптом: поведението ви изглежда обърнато. Проверка: това е нормално за active-low LED и трябва да се мисли като част от борда, не като грешка в Python.
- Симптом: външен товар не се държи като LED. Проверка: GPIO пинът не е универсален източник на мощност; съобразете се с допустимия ток и товара.

Нататък

След тази глава преминете към „Цифров вход“, за да видите другата половина на GPIO света: не как подаваме ниво, а как го четем.

Глава 11. Цифров вход [Начинаещ]

[Тip] Аналогия: Без PULL_UP входът е като антена без земя — хваща всякакъв шум. PULL_UP е като пружина, която връща бутона в 1 (ненатиснат). Натискате → пружината се победи → 0.

```
Без PULL_UP:  P400 — BTN — GND      → floating = шум x
С PULL_UP:    3.3V —[R]— P400 — BTN — GND
                    |ненатиснат: 1 (pull-up печели)
                    |натиснат:   0 (GND печели)
```

Състояние	button.value()	Смисъл
Бутон пуснат	1	PULL_UP държи входа на 3.3V
Бутон натиснат	0	Пинът се дърпа към GND

```
from machine import Pin

button = Pin("SW1", Pin.IN, Pin.PULL_UP)
print(button.value()) # 1 = пуснат, 0 = натиснат
```

[Еye] Какво виждаш: 1 в REPL когато бутонът е пуснат; 0 когато е натиснат.

[Lab] Опитай: Направете цикъл: `while True: print(button.value())`. Натиснете SW1 — виждате 0.

[!] Важно: SW1 = P400, active-low. Без debounce едно натискане дава много 0-та — нормално е, ще се реши в Глава 17.

Тук се обясняват:

- входно ниво
- активен нисък бутон
- вътрешен PULL_UP
- защо PULL_DOWN не е част от тези уроци за този порт

Файл:

- `examples/21_digital_io/02_digital_input.py`

Резюме

- `Pin.IN` прави пина вход, а `Pin.PULL_UP` му задава стабилно изходно състояние, когато бутонът не е натиснат.
- `SW1` е активен нисък бутон, затова натискането обикновено дава 0, а ненатиснатото състояние дава 1.
- Това е основата за всички следващи теми с бутони, `debounce`, `polling` и прекъсвания.

Самопроверка

1. Защо бутонът се конфигурира с `Pin.PULL_UP`, а не остава `floating`?
2. Какво означава `button.value() == 0` в този конкретен пример със `SW1`?
3. Защо само еднократно `print(button.value())` рядко е достатъчно в реално приложение с бутон?

Упражнения

1. Направете цикъл, който чете бутона периодично и печата `PRESSED` или `RELEASED`.

Подсказка: `if button.value() == 0: print("PRESSED")` в `while True:` с кратко `sleep_ms(20)`.

2. Свържете бутона с `LED1` така, че натискането да включва или изключва светодиода.

Подсказка: `led.value(button.value())` — активно-нисък бутон и активно-нисък `LED` имат съвместима логика.

3. Добавете прост брояч на натисканията и отпечатвайте общия брой след всяка промяна.

Подсказка: следете прехода `last != current` за `0` и увеличавайте брояч само при падащ фронт.

Ако не работи

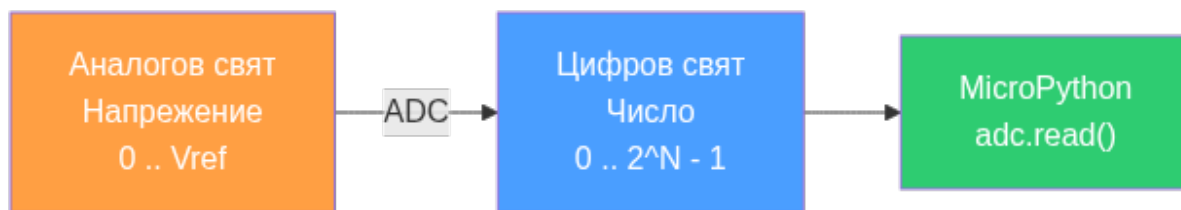
- Симптом: входът се държи нестабилно. Проверка: уверете се, че ползвате `Pin.PULL_UP` и че не оставяте входа `floating`.
- Симптом: логиката е обърната. Проверка: помнете, че `SW1` е `active-low` и натискането дава 0.
- Симптом: натискането се отчита по няколко пъти. Проверка: това е очаквано без `debounce` и ще бъде разгледано в следващите глави.

Нататък

След тази глава преминете към „Аналогов вход и ADC“, за да сравните двоичното четене на GPIO вход със света на непрекъснатите аналогови стойности.

Глава 12. Аналогов вход и ADC [Начинаещ]

ADC (Analog-to-Digital Converter) преобразува информация от **аналоговия** в **цифровия** свят. Реалният свят е аналогов — напрежение, температура, светлина. Процесорът работи само с числа. ADC е мостът между тях.

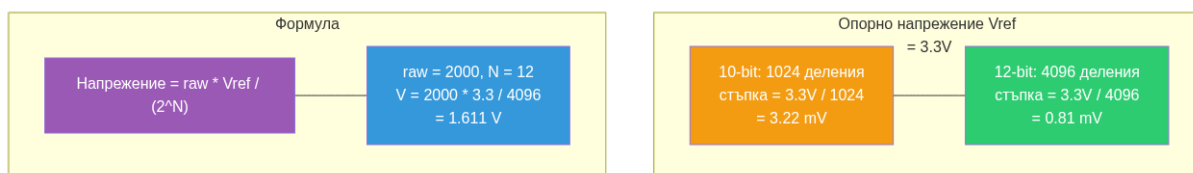


ADC — от аналогов към цифров свят

Опорно напрежение и разрядност

Всяко ADC има два ключови параметъра:

- **Опорно напрежение (Vref)** — определя максималния измерван диапазон. При VK_RA4M2 Vref = **3.3V**.
- **Разрядност (N бита)** — определя на колко деления разделяме диапазона. При N = 10 бита → $2^{10} = 1024$ деления. При N = 12 бита → $2^{12} = 4096$ деления.



ADC — опорно напрежение и разрядност

Формулата

$$\text{Напрежение (V)} = \frac{\text{raw} \times \text{Vref}}{2^N}$$

Където:

- **raw** — измерената стойност от ADC (adc.read()), число от 0 до $2^N - 1$
- **Vref** — опорното напрежение (за VK_RA4M2: 3.3V)
- **2^N** — максималният брой деления (за 12-bit: 4096)

[Pin] *Защо работи: raw / 2^N дава каква част от пълния диапазон е измерена (число от 0.0 до ~1.0). Умножаваме по Vref и получаваме реално напрежение. При raw = 0 → $0 \times 3.3 / 4096 = 0 \text{ V}$ (минимум). При raw = 4095 → $4095 \times 3.3 / 4096 \approx 3.3 \text{ V}$ (максимум — raw се съкращава с 2^N и остава Vref).*

Пример: Vref = 3.3V, разрядност N = 10 бита (1024 деления), adc.read() връща raw = 512:

$$V = 512 \times 3.3 / 1024 = 1.65 \text{ V} \quad (\text{точно средата на диапазона})$$

Пример 2: Същият ADC, raw = 768:

$$V = 768 \times 3.3 / 1024 = 2.475 \text{ V}$$

Пример 3: 12-битов ADC (VK_RA4M2), raw = 2000:

$$V = 2000 \times 3.3 / 4096 = 1.611 \text{ V}$$

[Pin] Стъпка на квантуване — минималната разлика, която ADC може да различи: $\text{стъпка} = V_{\text{ref}} / 2^N$
 За 10-bit: $3.3\text{V} / 1024 = 3.22\text{ mV}$ — не можем да различим 1.000V от 1.002V . За 12-bit: $3.3\text{V} / 4096 = 0.81\text{ mV}$
 — четири пъти по-фино.

Резолуция	Стъпки (2^N)	Стъпка при $V_{\text{ref}}=3.3\text{V}$	Употреба
8-bit	256	~12.9 mV	Бързо, груба оценка
10-bit	1024	~3.2 mV	Баланс бързина/точност
12-bit	4096	~0.81 mV	Прецизно измерване ← VK_RA4M2

```
# Преобразуване на raw ADC стойност в напрежение:
VREF = 3.3
BITS = 12
raw = adc.read() # число 0..4095
voltage = raw * VREF / (1 << BITS) # 1 << 12 = 4096
print(f"{raw} → {voltage:.3f} V") # напр. "2000 → 1.611 V"
```

```
from machine import ADC

temperature_adc = ADC(ADC.CORE_TEMP) # вътрешен сензор — без окабеляване
vref_adc = ADC(ADC.CORE_VREF) # вътрешно референтно напрежение

print(temperature_adc.read()) # число 0-4095 (при 12-bit)
print(vref_adc.read())
```

[Eye] Какво виждаш: Число около 2000-2500 за CORE_TEMP при стайна температура.

[Lab] Опитай: Задръжте пръст върху MCU — CORE_TEMP трябва да се покачи с ~20-50 стъпки.

[!] Важно: *bits*= параметърът е глобален за ADC периферията — не само за конкретен обект. Ако смените резолюцията на един канал, всички канали се засягат.

Важно уточнение за RA4M2:

- Този MCU е с ADC12, тоест хардуерната резолюция тук е 12 bit, а поддържаните режими са 12 / 10 / 8 bit.
- В текущия порт може да се избира режим още при създаване на обекта: `ADC(pin_or_channel, bits=8)`, `ADC(..., bits=10)`, `ADC(..., bits=12)`.
- Настройката за *bits* е глобална за ADC периферията, а не отделна за всеки ADC обект.

Кратък пример:

```
from machine import ADC # Импортираме ADC за избор на резолюция при създаване на обекта.

adc_fast = ADC("P000", bits=8) # Създаваме ADC обект с 8-битова резолюция за бързо четене.
adc_balanced = ADC("P001", bits=10) # Създаваме ADC обект с 10-битова резолюция за баланс бързина/точност.
adc_precise = ADC("P002", bits=12) # Създаваме ADC обект с 12-битова резолюция за максимална точност.

print(adc_fast.read(), adc_balanced.read(), adc_precise.read()) # Четем и трите канала и показваме резулта...
```

Практическа бележка за времето на преобразуване при текущите тактове на VK_RA4M2:

- при минимално валидно семплиране и $\text{ADCLK} = 50\text{ MHz}$ самото A/D преобразуване е приблизително 0.31 us за 8 bit, 0.35 us за 10 bit, 0.39 us за 12 bit

- при по-дълго семплиране, например около `reset/default` настройката на `ADSSTR`, стойностите са приблизително 0.41 us, 0.45 us, 0.49 us
- това е време на хардуерния ADC, а не пълното време на извикването `ADC.read()` от Python
- ако искате да мерите целия път през MicroPython, използвайте `time.ticks_us()` около самото извикване
- за практическо измерване на пълното време на `ADC.read()` виж `examples/23_timing/07_adc_timing_measure.py`, където се мери последователно в 8 / 10 / 12 bit

Наличните външни ADC пинове в този курс са:

P000, P001, P002, P003, P004, P005, P006, P007, P008, P013, P014, P015, P500

Файл:

- `examples/22_analog/01_analog_input_adc.py`
- `examples/23_timing/07_adc_timing_measure.py`

Резюме

- `ADC.read()` връща сурова числова стойност, която представя моментното аналогово ниво на избрания канал.
- Вътрешните канали `ADC.CORE_TEMP` и `ADC.CORE_VREF` са добър старт, защото не изискват външно окабеляване.
- На този порт `bits=8/10/12` се задава при създаване на обекта, но влияе на ADC периферията като цяло, а не само на един обект.
- Времето на самото A/D преобразуване и времето на `ADC.read()` от Python не са едно и също нещо.

Самопроверка

1. Каква е разликата между вътрешен ADC канал като `ADC.CORE_TEMP` и външен пин като P000?
2. Защо параметърът `bits=` трябва да се мисли като настройка на периферията, а не само на единичен обект?
3. Как бихте разграничили „бърз ADC хардуер“ от „по-бавно Python извикване на `ADC.read()`“?

Упражнения

1. Сравнете едно и също входно ниво при `bits=8`, `bits=10` и `bits=12` и запишете как се променя числовият диапазон.

Подсказка: максималната стойност е $2^{\text{bits}} - 1$ — при `bits=8` е 255, при `bits=12` е 4095.

2. Свържете потенциометър или прост делител към външен ADC пин и отпечатвайте стойностите с малко закъснение между четенията.

Подсказка: ``adc = ADC(Pin("P000"))` u `while True: print(adc.read()); sleep_ms(100)`.`

3. Измерете пълното време на `ADC.read()` с `time.ticks_us()` и го сравнете с ориентировъчните хардуерни времена в текста.

Подсказка: ``t = time.ticks_us(); adc.read(); print(time.ticks_diff(time.ticks_us(), t))`.`

Ако не работи

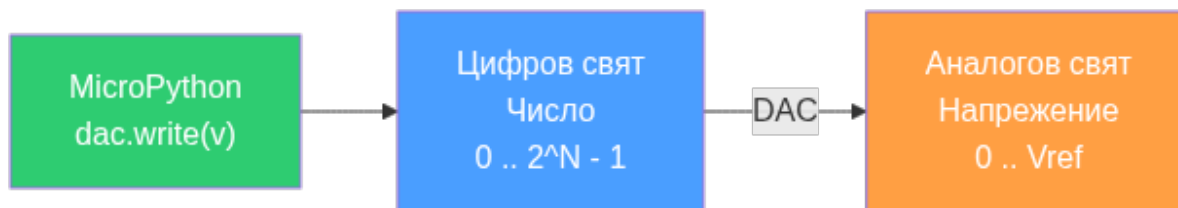
- Симптом: стойностите са почти случайни. Проверка: външният вход може да е floating; дайте му реален източник на напрежение или стабилен делител.
- Симптом: получавате грешка или неочакван резултат от пина. Проверка: вижте дали ползвате валиден ADC пин от изброените в главата.
- Симптом: измереното време е много по-голямо от част от микросекунда. Проверка: това е нормално, защото мерите целия Python път, а не само вътрешното хардуерно преобразуване.

Нататък

След тази глава преминете към „PWM“, за да сравните два различни начина за работа с аналогово поведение: измерване чрез ADC и генериране чрез импулсен изход.

Глава 13. DAC изход, timed playback и retro synth [Начинаещ]

DAC (Digital-to-Analog Converter) е **обратното на ADC** — преобразува число от цифровия свят в напрежение в аналоговия свят. ADC чете, DAC пише.



DAC — от цифров към аналогов свят

Опорно напрежение и разрядност (DAC)

Както при ADC, DAC също има **опорно напрежение (Vref)** и **разрядност (N бита)**:

- **Vref** — максималното напрежение, което DAC може да генерира. За VK_RA4M2: **3.3V**.
- **Разрядност** — на колко нива може да се зададе изходът. При 12-bit: **4096 нива**.

Формулата (обратна на ADC)

$$\text{Напрежение (V)} = \frac{\text{value} \times V_{\text{ref}}}{2^N}$$

Където:

- **value** — числото, което подаваме на DAC (`dac.write(value)`), от 0 до $2^N - 1$
- **Vref** — опорното напрежение (3.3V)
- **2^N** — максималният брой нива (4096 за 12-bit)

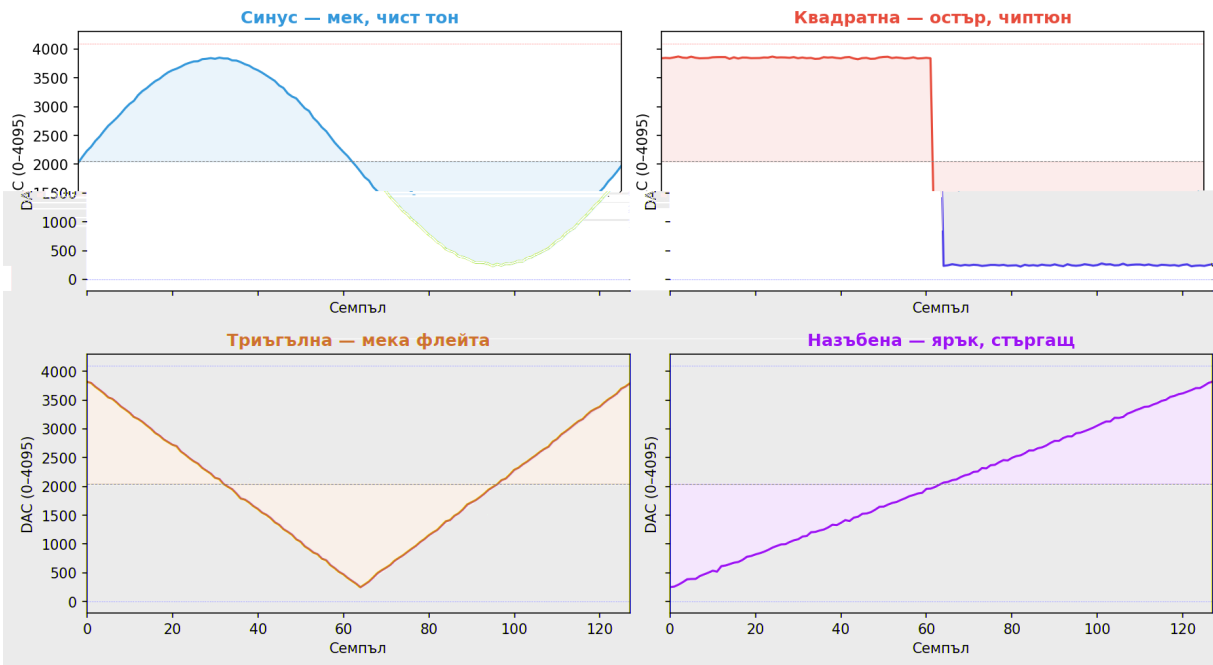
[Pin] Как работи: $\text{value} / 2^N$ дава каква част от пълния диапазон искаме на изхода. Умножаваме по Vref и получаваме реално напрежение. При $\text{value} = 0 \rightarrow 0 \times 3.3 / 4096 = 0V$ (минимум). При $\text{value} = 2048 \rightarrow 2048 \times 3.3 / 4096 = 1.65V$ (средата — "тишина" за аудио). При $\text{value} = 4095 \rightarrow 4095 \times 3.3 / 4096 \approx 3.3V$ (максимум).

value	Напрежение	Смисъл за аудио
0	0.0 V	Минимум — отрицателен връх
2048	1.65 V	Средна точка — тишина (MID)
4095	3.3 V	Максимум — положителен връх

DAC трябва да сменя стойност **56 320 пъти в секунда**. Таймерът (AGT) осигурява точно този такт, а DTC автоматично подава следващото число от масива — без участие на CPU.

Четири вълнови форми от числа

Четири вълнови форми — 128 семпла, 12-bit DAC



Четири вълнови форми — 128 семпла, 12-bit DAC

Графиката показва как изглежда всяка форма на осцилоскоп: DAC стойност (0–4095) по Y, семпл по X. Оцветената зона показва отклонението от средната точка (MID = 2048).

Различните форми дават различен тембър при еднаква честота:

Форма	Звук	Как се изчислява	Хармоници
Синус	Мек, чист тон	<code>math.sin()</code>	Само основният
Квадратна	Остър, "чиптюн"	0 или MAX	Нечетни (1, 3, 5...)
Триъгълна	Мека флейта	Линейна рампа нагоре-надолу	Нечетни, затихващи
Назъбена	Ярък, стъргащ	Рампа + скок	Всички хармоници

Метод	Кога	Кой движи данните
<code>dac.write(v)</code>	Статично ниво	CPU (ред по ред)
<code>dac.write_timed(buf, rate, CIRCULAR, DTC)</code>	Аудио loop до 256 семпла	DTC автоматично
<code>dac.write_timed(buf, rate, CIRCULAR, DMAC)</code>	Аудио loop до 1024 семпла	DMAC автоматично

Hardware DAC pipeline: AGT → DTC → DAC → P014

Hardware DAC pipeline: AGT → DTC → DAC → P014

VK_RA4M2 има два 12-bit DAC канала, които генерират аналогов сигнал директно.

Това е полезно за:

- генериране на аналогови вълни (синус, триъгълна, назъбена)
- ретро звукови ефекти и кратки мелодии
- wavetable цикли
- game UI звуци тип coin, jump, laser, explosion
- тестови сигнали за калибриране

DAC пиновете на VK_RA4M2:

- P014 = DA0 = J19-5 = A4
- P015 = DA1 = J19-6 = A5
- J16 е boot-mode jumper и **не е** DAC изход
- за наблюдение ползвайте осцилоскоп или активна колона, не директно гол говорител

Статичен DAC изход

```

from machine import DAC, Pin # Импортираме DAC и Pin за аналогов изход.

dac = DAC(Pin("P014")) # Създаваме DAC обект върху DA0 (J19-5 / A4).
dac.write(2048) # Записваме средна стойност (2048 от 4095 = ~1.65 V при 3.3 V AVCC0).
dac.write(4095) # Максимална стойност = ~3.3 V.
dac.write(0) # Минимална стойност = 0 V.
  
```

Timed playback — основна идея

Python не подава всеки семпъл ръчно. Вместо това:

- DAC.write_timed(...) конфигурира възпроизвеждането
- AGT таймер дава равномерния sample clock
- DTC или DMAC мести данните от RAM към DAC регистъра
- CPU се ползва за setup и control-rate промени, а не за всеки отделен семпъл

Поддържани комбинации:

	DTC	DMAC
NORMAL (one-shot)	до 65535 семпла	до 65535 семпла
CIRCULAR (безкраен loop)	до 256 семпла	до 1024 семпла

DMAC circular е реализиран чрез хардуерен repeat mode с ISR re-arming — безшевен loop без glitch.

Минимален timed DAC пример:

```
from array import array # Импортираме array за ефективен буфер.
from machine import DAC, Pin # Импортираме DAC и Pin.
import time # Импортираме time за sleep.

wave = array("H", [2048, 3072, 4095, 3072, 2048, 1024, 0, 1024]) # 12-bit DAC стойности за един период.
dac = DAC(Pin("P014")) # Ползваме DA0 на J19-5 / A4.

dac.write_timed(
    wave,
    440 * len(wave), # Един период на 440 Hz = sample rate 3520 Hz.
    mode=DAC.CIRCULAR, # Безкраен loop.
    transfer=DAC.TRANSFER_DTC, # Ползваме DTC (до 256 семпла).
)

time.sleep_ms(500) # Свирим 500 ms.
dac.stop() # Спираме timed playback.
```

Пример: chirp_20k_10k.py — непрекъснат 5 kHz синус (wavetable loop)

[Pin] Бележка за името: Файлът се казва `chirp_20k_10k.py`, но реалният код генерира фиксиран 5 kHz синус с DTC circular, а не честотен sweeper (chirp). Името е наследено от ранна версия. Ако искате истински chirp, вижте `play_sweep()` в `retro_synth.py` (Глава 56).

```

import gc # Импортираме gc за управление на garbage collector.
import math # Импортираме math за sin() функцията.
from array import array # Импортираме array за ефективен буфер с unsigned short стойности.
from machine import DAC, Pin # Импортираме DAC и Pin за аналогов изход.

DAC_PIN = "P014" # Избираме DA0 – J19-5 / A4.
MID = 2048 # Средна точка на 12-bit DAC (0..4095).
AMP = 1800 # Амплитуда на синусоидата.
FREQ = 5000 # Целева честота на синуса: 5 kHz.
TABLE_LEN = 16 # Брой семпли за един период.
SAMPLE_RATE = FREQ * TABLE_LEN # Изчисляваме sample rate: 5000 * 16 = 80 000 Hz.

buf = array("H", [0] * TABLE_LEN) # Създаваме буфер от 16 unsigned short стойности.
for i in range(TABLE_LEN): # Запълваме буфера с един период на синусоидата.
    v = MID + int(AMP * math.sin(2.0 * math.pi * i / TABLE_LEN)) # Изчисляваме DAC стойност.
    if v < 0: # Ограничаваме отдолу до 0.
        v = 0
    elif v > 4095: # Ограничаваме отгоре до 4095.
        v = 4095
    buf[i] = v # Записваме в буфера.

dac = DAC(Pin(DAC_PIN)) # Създаваме DAC обект.
dac.write(MID) # Задаваме начално ниво в средата.

gc.collect() # Принуждаваме garbage collection преди timed playback.
gc.disable() # Забраняваме GC, за да не прекъсва DMA трансфера (вж. бележката по-долу).
print("5 kHz sine, DTC circular (Ctrl-C to stop)...") # Информираме потребителя.
dac.write_timed(buf, SAMPLE_RATE, mode=DAC.CIRCULAR, transfer=DAC.TRANSFER_DTC) # Стартираме безкраен DTC ...

try:
    while True: # Чакаме безкрайно – хардуерът свири самостоятелно.
        pass
except KeyboardInterrupt: # Ctrl-C спира цикъла.
    pass

dac.stop() # Спираме timed playback.
dac.write(MID) # Връщаме изхода в средата.
gc.enable() # Разрешаваме GC отново.
print("Stopped.") # Информираме, че е спрял.

```

Файл: examples/chirp_20k_10k.py

Пример: dac_dual_dtc_dmac.py — двуканален DAC: DTC vs DMAC circular

```

import gc # Управление на garbage collector.
import math # Математически функции за синус.
from array import array # Ефективен буфер за DAC семпли.
from machine import DAC, Pin # DAC и Pin класове.

MID = 2048 # Средна точка на 12-bit DAC.
AMP = 1800 # Амплитуда.
FREQ = 5000 # Целева честота: 5 kHz.
TBL = 32 # Брой семпли за един период – 32.
SR = FREQ * TBL # Sample rate: 5000 * 32 = 160 kHz.

buf = array("H", [0] * TBL) # Буфер за 32 семпла.
for i in range(TBL): # Запълваме с един период синус.
    v = MID + int(AMP * math.sin(2.0 * math.pi * i / TBL)) # Стойност за текущия семпл.
    buf[i] = max(0, min(4095, v)) # Ограничаваме в 0..4095 с max/min.

print("5 kHz sine, {} samples @ {} Hz".format(TBL, SR)) # Отпечатваме параметрите.

dac0 = DAC(Pin("P014")) # DA0 на J19-5 / A4.
dac1 = DAC(Pin("P015")) # DA1 на J19-6 / A5.
dac0.write(MID) # Начално ниво CH0.
dac1.write(MID) # Начално ниво CH1.

gc.collect() # Почистваме паметта.
gc.disable() # Забраняваме GC по време на playback.

print("CH0=DTC circular | CH1=DMAC circular (Ctrl-C to stop)") # Инфо за двата канала.

dac0.write_timed(buf, SR, mode=DAC.CIRCULAR, transfer=DAC.TRANSFER_DTC) # CH0: DTC repeat до 256 семпла, ...
dac1.write_timed(buf, SR, mode=DAC.CIRCULAR, transfer=DAC.TRANSFER_DMAL) # CH1: DMAL repeat до 1024, ISR r...

try:
    while True: # Чакаме – и двата канала свирят самостоятелно.
        pass
except KeyboardInterrupt: # Ctrl-C спира.
    pass

dac0.stop() # Спираме CH0.
dac1.stop() # Спираме CH1.
dac0.write(MID) # Връщаме CH0 в средата.
dac1.write(MID) # Връщаме CH1 в средата.
gc.enable() # Разрешаваме GC.
print("Stopped.") # Готово.

```

Файл: examples/dac_dual_dtc_dmac.py

На осцилоскопа двата канала трябва да са еднакво гладки синуси — без glitch при нито един от тях.

[Pin] *Защо `gc.disable()` преди DMA/DTC playback? MicroPython управлява паметта автоматично чрез Garbage Collector (GC) — периодична процедура, която освобождава неизползвани обекти. Проблемът: GC може да спре CPU за няколко милисекунди, а DTC/DMA очакват процесорът да поднови буфера навреме. Ако GC се включи в точния момент, звукът прекъсва или се появява щракане. Рецептата е: 1. `gc.collect()` — пуснете GC преди критичната секция, за да е heap-ът чист. 2. `gc.disable()` — забранете автоматичния GC. 3. Спирате DMA playback. 4. `gc.enable()` — върнете GC след спиране на playback. Пълното обяснение на gc модула е в Глава 30 (Управление на паметта).*

Пример: retro_synth.py — библиотека за ретро звуци

[Pin] *Какво е RetroSynth — концептуален преглед: RetroSynth е клас (вж. Глава 7 за класове), който скрива цялата DTC/DAC логика зад прост интерфейс: `synth.play_note("A4", 200, "square")` — свири нота La за 200 ms с квадратна вълна. Вътрешно модулет предизчислява wavetable (масив от стойности), подава го на DTC чрез `dac.write_timed()` и спира след зададеното време. Тук го ползваме като черна кутия — пълната архитектура е разгледана в Глава 56.*

Този файл не е самостоятелен пример, а **helper модул**, който се импортира от останалите скриптове. Той капсулова wavetable генерирането и playback логиката в клас RetroSynth.

Основни елементи:

```
MID = 2048 # Средна точка на 12-bit DAC.
DEFAULT_AMP = 1400 # Амплитуда по подразбиране.
DEFAULT_TABLE_LEN = 128 # Семпли за един период.
DEFAULT_NOISE_BURST_LEN = 2048 # Дължина на noise burst буфера.
```

Помощни функции:

- `clamp12(value)` — ограничава стойност в 0..4095.
- `midi_to_freq(midi_note)` — преобразува MIDI номер в Hz по формулата $440 * 2^{((n-69)/12)}$.
- `note_to_freq(note_or_freq)` — приема "A4", "C#5", 440, "1000Hz" и връща честота.
- `semitone_freq(base, semitones)` — транспониране с полутонове.
- `make_square(length, amp, duty_num, duty_den)` — генерира квадратна вълна с избран duty cycle.
- `make_triangle(length, amp)` — генерира триъгълна вълна.
- `make_saw(length, amp)` — генерира назъбена вълна.
- `make_noise_burst(length, amp)` — генерира LFSR шум с линейно затихване.

Клас RetroSynth:

```
synth = RetroSynth("P014") # Създаваме synth обект на DA0.
synth.play_note("A4", 200, "square") # Свири нота A4 за 200 ms, квадратна вълна.
synth.play_sweep("A3", "A6", 300, "saw") # Sweep от A3 до A6 за 300 ms.
synth.play_arpeggio("C4", (0, 4, 7), step_ms=60) # Арпеджо C-E-G.
synth.play_melody([("\E5", 110), ("\B4", 110)]) # Поредица от ноти.
synth.coin() # Звук "монета" (B5 + E6).
synth.jump() # Звук "скок" (sweep нагоре).
synth.laser() # Звук "лазер" (sweep надолу).
synth.explosion() # Звук "експлозия" (noise burst).
synth.stop() # Спира всичко.
```

Файл: examples/retro_synth.py


```

from retro_synth import RetroSynth # Импортираме helper модула.

DAC_PIN = "P014" # DA0 на J19-5 / A4.
synth = RetroSynth(DAC_PIN) # Създаваме synth обект.

MELODY = [ # Дефинираме мелодия като списък от (нота, ms, вълна).
    ("E5", 110, "pulse25"), # Нота E5 за 110 ms с 25% duty pulse.
    ("B4", 110, "pulse25"), # Нота B4 за 110 ms.
    ("C5", 110, "pulse25"), # Нота C5 за 110 ms.
    ("D5", 110, "pulse25"), # Нота D5 за 110 ms.
    ("C5", 110, "pulse25"), # Нота C5 за 110 ms.
    ("B4", 110, "pulse25"), # Нота B4 за 110 ms.
    ("A4", 180, "pulse25"), # Нота A4 за 180 ms (по-дълга).
    (None, 40, "pulse25"), # Пауза 40 ms (None = тишина).
    ("A4", 110, "pulse25"), # Нота A4 за 110 ms.
    ("C5", 110, "pulse25"), # Нота C5 за 110 ms.
    ("E5", 180, "pulse25"), # Нота E5 за 180 ms.
    ("D5", 110, "pulse25"), # Нота D5 за 110 ms.
    ("C5", 110, "pulse25"), # Нота C5 за 110 ms.
    ("B4", 220, "pulse25"), # Нота B4 за 220 ms (финална).
]

def demo(): # Дефинираме демо функцията.
    print("Retro music demo on {} (J19-5 / A4 / DA0)".format(DAC_PIN)) # Печатаме пин информация.
    print("melody -> arpeggio bed -> coin") # Печатаме последователността.
    synth.play_melody(MELODY, gap_ms=12) # Свири мелодията с 12 ms пауза между нотите.
    synth.play_arpeggio("A3", (0, 4, 7, 12), step_ms=55, cycles=6, wave="triangle", gap_ms=5) # Арпеджо A-...
    synth.coin() # Финален "coin" звук.
    synth.stop() # Спираме DAC изхода.
    print("Done.") # Готово.

demo() # Пускаме демото.

```

Файл: examples/dac_retro_music.py

Пример: test_dac_dmac_dtc.py — автоматичен тест на DTC и DMAC режими

Този скрипт тества три режима последователно и проверява дали playback-ът завършва в очаквано време:

- 1. DTC circular** — 128-семплова триъгълна вълна, безкраен loop за 600 ms, после ръчен stop.
- 2. DTC one-shot** — 256-семплов tone burst при 8 kHz, чака да свърши, проверява продължителността.
- 3. DMAC one-shot** — 2048-семплов noise burst при 12 kHz, чака да свърши, проверява продължителността.

Ключова помощна функция:

```

def wait_until_idle(dac, timeout_ms=2500): # Чака DAC да спре да свири.
    start = time.time() # Запомняме началото.
    deadline = time.time() + timeout_ms # Крайно време.
    while dac.playing(): # Докато playback е активен...
        if time.time() - start >= timeout_ms: # Ако таймаутът изтече...
            raise RuntimeError("timeout waiting for timed DAC to stop") # Грешка.
        time.sleep(10) # Чакаме по 10 ms.
    return time.time() - start # Връщаме elapsed ms.

```

При успех печата ALL PASS. При грешка — хвърля RuntimeError с описание кой тест е провалил.

Файл: `examples/test_dac_dmac_dtc.py`

Пример: `dac_firmware_probe.py` — probe за firmware валидация

Подобен на `test_dac_dmac_dtc.py`, но фокусиран върху **firmware debugging**:

- Тества DTC circular, DTC one-shot и DMAC one-shot.
- Печата PASS: за всеки успешен тест.
- При провал — `RuntimeError` с точно описание.
- Напомня, че J16 е boot-mode jumper, не DAC изход.

Файл: `examples/dac_firmware_probe.py`

Пример: `dac_dmac_diag.py` — регистрова диагностика на DMAC

Този скрипт е за **напреднали** — чете директно хардуерните регистри на DMAC, ICU и Bus Error контролера чрез `mem32[]` / `mem8[]`:

```
ICU_BASE = 0x40006000 # Interrupt Controller Unit базов адрес.  
DMAC_BASE = 0x40005000 # DMAC базов адрес.  
DMA_BASE = 0x40005200 # DMA Common Register базов адрес.  
BUS_BASE = 0x40003000 # Bus Error базов адрес.
```

Показва:

- DMAST — дали DMAC е глобално разрешен.
- DMECHR — DMA Error Channel Register (кой канал е дал грешка).
- BUS3ERRSTAT / DMACDTCERRSTAT — Bus и DTC error flags.
- IELSR[46] — ICU Event Link за AGT0_INT.
- DELSR[0..7] — DMA Event Link за всеки DMAC канал.
- DMCRA, DTE, DMREQ, DMSTS за всеки DMAC канал.

Прави три snapshot-а: веднага след start, след 50 ms и след 250 ms, за да видите как регистрите се променят по време на playback.

Файл: `examples/dac_dmac_diag.py`

Нататък

След тази глава преминете към „Закъснения, хардуерни таймери и Timer(-1)“, за да разберете как AGT таймерът задвижва sample clock-а за DAC playback.

Част IV: Таймери, прекъсвания и време

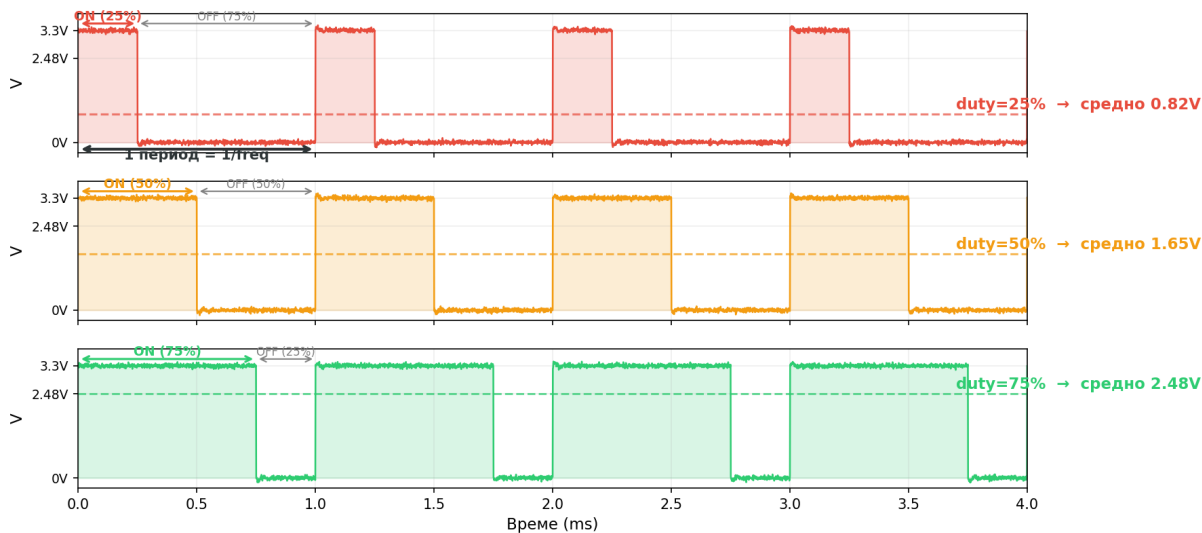
Ключови термини: AGT — хардуерен таймер в RA4M2; `Timer(1)..Timer(6)`. IRQ — хардуерно прекъсване: CPU спира и изпълнява *handler* при събитие. *hard/soft IRQ* — директно в хардуерен контекст (бързо, забранени *print*) vs през *Python scheduler* (по-бавно, безопасно). *bounce* — фалшиви преходи при натискане на механичен бутон. *debounce* — изчакване за стабилен сигнал. RTC — часовник за реално време.

Тази част въвежда времето като реален ресурс: закъснения, таймери, IRQ, *debounce* и RTC.

Глава 14. PWM [Начинаещ]

[Tip] Аналогия: PWM е като светлинен регулатор (*dimmer*) — не намалявате напрежението, а превключвате много бързо вкл/изкл. При 50% *duty* за половин период е вкл, за другата половина е изкл. Очите/двигателят/товарът виждат "средно" ниво.

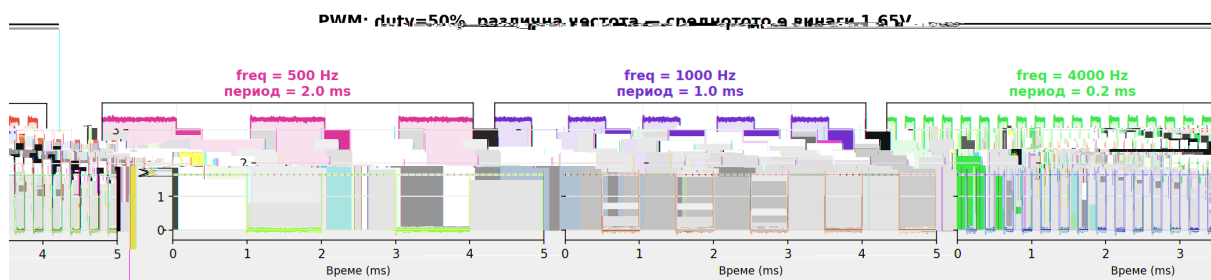
PWM: duty cycle определя средното напрежение
freq = 1000 Hz (период = 1 ms)



PWM duty cycle — 25%, 50%, 75%

Графиката показва три настройки на *duty cycle* при една и съща честота (1 kHz). Колкото по-голям е *duty*, толкова по-високо е средното напрежение:

Параметър	Смисъл	Пример
freq	Колко периода в секунда	1000 Hz = 1ms период
duty	% от периода в ON-фаза	50 = 50% от времето HIGH



PWM при различна честота

При еднакъв duty=50% честотата определя колко пъти в секунда се повтаря ON/OFF цикълът — но средното напрежение остава непроменено (1.65V).

```
from machine import Pin, PWM

pwm = PWM(Pin("P107"), freq=1000, duty=50) # 1kHz, 50% → средно 1.65V
```

[!] Важно: GPT канали — P106 и P107 са на един и същ GPT канал. Ако промените freq на единия, се промени и на другия. duty може да е различен.

```
pwm_a = PWM(Pin("P107"), freq=1000, duty=25)
pwm_b = PWM(Pin("P106"), freq=1000, duty=75) # споделен freq с pwm_a
pwm_a.freq(2000) # ← промяна засяга и pwm_b!
```

[Eye] Какво виждаш: Свържете LED към P107 (с резистор) — яркостта варира при различен duty.

[Lab] Опитай: Сменете duty от 10 до 90 на стъпки от 10 — наблюдавайте как се изменя яркостта или усреденото ниво на товара.

Тук се вижда, че:

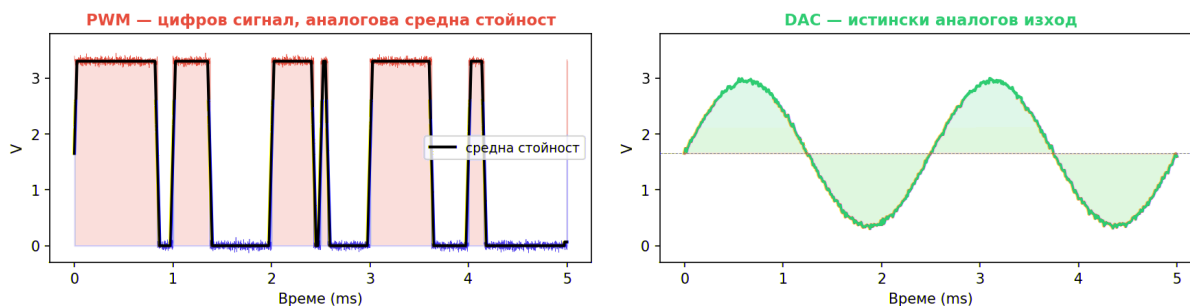
- duty може да е различен
- честотата се споделя от двата изхода на общия GPT канал
- на този порт PWM е върху GPT периферията и общият канал налага обща freq() за A/B изходите
- проверката на реалната форма, фронтове и jitter става с осцилоскоп или логически анализатор, а не само по print от Python

Файлове:

- examples/22_analog/02_pwm_output.py
- examples/03_pwm_ab_same_channel.py

PWM vs DAC — каква е разликата?

PWM имитира аналогово ниво, DAC го генерира директно



PWM vs DAC

PWM **имитира** аналогово ниво чрез бързо превключване — средната стойност съвпада с желаното напрежение, но сигналът реално е цифров (0 или 3.3V). DAC **генерира директно** произволно напрежение между 0 и V_{ref} . За управление на яркост или мотори PWM е напълно достатъчен; за аудио и прецизни аналогови сигнали DAC ([Глава 13](#)) е правилният избор.

[Pin] Филтрация: И при PWM, и при DAC, ако искате чист аналогов сигнал без стъпки и шум, на изхода се слага RC нискочестотен филтър (резистор + кондензатор) — той изглажда цифровите преходи и пропуска само бавната средна стойност.

Резюме

- PWM в този порт стъпва върху GPT, затова честотата е свойство на канала, а не само на отделния пин.
- Двата изхода A/B на един и същ GPT канал могат да имат различен duty, но споделят обща freq().
- PWM е удобен за яркост, управление и сигнални експерименти, но не е същото като истински аналогов DAC изход.
- Ако ви интересуват форма, фронтове и jitter, гледайте сигнала с осцилоскоп или логически анализатор, не само с print().

Самопроверка

1. Защо промяната на `pwm_a.freq()` влияе и на `pwm_b`, ако двата пина са върху общ GPT канал?
2. Каква е практическата разлика между `freq` и `duty`?
3. Защо `print()` не е достатъчен, ако искате да проверите точната форма на PWM сигнала?

Упражнения

1. Променете `duty` през няколко стойности и наблюдавайте как се изменя яркостта или усредненото ниво на товара.

Подсказка: `pwm.duty(d)` за `d` в `[10, 25, 50, 75, 90]` — по-висок `duty` = по-ярко светене.

2. Пуснете двата изхода от общия GPT канал и проверете как общата честота се пази, докато `duty` остава различен.

Подсказка: `pwm_a = PWM(Pin("P107"), freq=1000, duty=25)` и `pwm_b = PWM(Pin("P106"), freq=1000, duty=75)` — `freq` е споделен.

3. Сравнете 500 Hz, 1000 Hz и 2000 Hz и запишете какво се вижда на товар или измервателен инструмент.

Подсказка: `pwm.freq(500)` после `pwm.freq(2000)` — при LED с `duty=50` яркостта остава същата, но звукът при пиезо се мени.

Ако не работи

- Симптом: няма PWM ефект на пина. Проверка: вижте дали пинът е валиден PWM изход за този борд и дали не е в конфликт с друг периферен режим.
- Симптом: вторият канал се „променя сам“. Проверка: ако е върху същия GPT, това е очаквано за `freq()`, защото честотата е общ ресурс.
- Симптом: резултатът изглежда шумен или нестабилен. Проверка: различавайте електрическия шум и товара от реалния PWM timing; проверете и с измервателен инструмент.

Нататък

След тази глава преминете към „Закъснения, хардуерни таймери и `Timer(-1)`“ ([Глава 15](#)), за да видите как AGT таймерът управлява времето в различни режими.

Глава 15. Закъснения, хардуерни таймери и `Timer(-1)` [Среден]

[Tip] Аналогия: `time.sleep_ms(300)` е като да стоите неподвижно пред микровълновата докато свири. `Timer(1)` е като да сложите будилник — върнете се при другите задачи, той ще ви събуди.

Метод	Блокира CPU?	Прецизност	Когато
<code>time.sleep_ms(N)</code>	[OK] Да	~ms	Прости паузи
<code>Timer(1..6) callback</code>	[X] Не	~us	Периодична задача
<code>Timer(-1) callback</code>	[X] Не	~ms	Мек периодичен тик
<code>asyncio.sleep_ms()</code>	[X] Не	~ms	Кооперативен scheduler

Блокиращо закъснение:

```
import time # Импортираме time за блокиращото закъснение.
time.sleep_ms(300) # Това закъснение блокира изпълнението на останалия код.
```

Хардуерен таймер:

```
from machine import Timer # Импортираме Timer за хардуерния таймер.

timer = Timer(1) # Използваме един от наличните хардуерни machine.Timer канали от Timer(1) до Timer(6).
timer.init(freq=4, callback=timer_callback, hard=False) # Стартираме Timer(1) на 4 Hz с callback извън har...
# hard=False (по подразбиране): callback-ът се изпълнява през Python scheduler-а – безопасно за print(), ал...
# hard=True: callback-ът се изпълнява директно в хардуерния IRQ контекст – минимална латентност, но НЕ алок...
# не ползвайте print() и не извиквайте сложни Python обекти вътре. Подходящо само за екстремно кратки и кри...
```

Софтуерен таймер:

```

from machine import Pin, Timer # Импортираме Pin и Timer за демонстрацията на софтуерния таймер.
import time # Импортираме time, за да оставим таймера да работи кратко време.

led = Pin("LED1", Pin.OUT) # Вземаме LED1 като видим индикатор за сработване на callback-а.
soft_timer = Timer(-1) # Създаваме софтуерен таймер чрез идентификатор -1.
soft_state = {"count": 0} # Пазим броя сработвания в mutable структура.
started = False # С този флаг ще следим дали таймерът е стартирал успешно.

def soft_timer_callback(timer_object): # Това е функцията, която ще се извиква при всяко сработване на соф...
    soft_state["count"] = soft_state["count"] + 1 # Увеличаваме брояча при всяко събитие.
    led.value(0 if led.value() else 1) # Превключваме LED, за да виждаме че callback-ът се изпълнява.

if not started: # Ако още не сме стартирали таймера, пробваме втория стил с честота.
    try: # Влизаме в защитен блок за втория опит.
        soft_timer.init(freq=4, callback=soft_timer_callback, hard=False) # Пробваме инициализация с често...
        started = True # Отбелязваме, че вторият стил е успял.
        print("Timer(-1) тръгна с freq API.") # Печатаме кой резервен стил е сработил.
    except Exception as second_error: # Ако и вторият стил не тръгне, само информираме потребителя.
        print("freq API също не тръгна:", second_error) # Показваме причината и за втория неуспех.

```

Тук трябва да остане ясно:

- Timer(1) до Timer(6) са хардуерни
- Timer(-1) е софтуерен
- asyncio е още една неблокираща алтернатива

Практическа бележка за time.ticks_us():

- ползвайте time.ticks_us() и time.ticks_diff() за относителни интервали в микросекунди
- в този порт time.ticks_us() идва от хардуерно поддържан mp_hal_ticks_us(), а не от измислен Python брояч
- това е много полезно за сравнение на latency и ADC.read() време, но не е заместител на осцилоскоп при

хардуерна прецизност

Файлове:

- examples/23_timing/01_delays_and_timers.py
- examples/23_timing/04_software_timer_minus1.py
- examples/23_timing/05_timer_compare_hw_sw.py
- examples/23_timing/07_adc_timing_measure.py
- examples/23_timing/13_hardware_timer_periodic_oneshot.py
- examples/23_timing/14_hardware_timer_output_compare.py
- examples/23_timing/15_hardware_timer_input_capture.py
- examples/23_timing/16_hardware_timer_event_count.py
- examples/23_timing/18_clock_source_proof.py — чете хардуерни SYSTEM/PLL/OSCSF регистри и доказва

дали 16 MHz кристалът е активен clock source

• examples/23_timing/19_timer6_agt5_proof.py — конфигурира Timer(6)/AGT5 на 1000 Hz, декодира tick clock от регистрите, мери реална callback честота с DWT cycle counter, тогъл P302 за осцилоскоп

• examples/23_timing/20_p003_p004_timer_proof.py — сравнява soft Timer(-1) vs hard Timer(1) чрез тогъл на P100 и P302 при 50 Hz; scope верификация

• examples/test_p302_toggle.py — изолиран тест: Timer(6) hard=True и Timer(5) hard=False, ~1000 callbacks за 500 ms на P302

Подглава 15.1. Хардуерни таймери

След базовата идея за Timer(1) и Timer(-1) е важно да се види какво реално може хардуерният таймер на този порт.

В текущия firmware за VK_RA4M2:

- Timer(1) до Timer(6) са хардуерни AGT таймери
- Timer(-1) е софтуерен таймер върху системния tick
- хардуерните таймери поддържат Timer.PERIODIC и Timer.ONE_SHOT
- налични са freq(), period() и counter()
- налични са output compare, input capture и event count
- при event count няма channel callback; ако искате callback на всеки N импулса, ползвате базовия Timer.callback() и задавате period=N

Базов пример за periodic и one-shot:

```
from machine import Timer # Импортираме Timer, за да използваме хардуерните таймери.

periodic_timer = Timer(1) # Вземаме Timer(1) за периодичен режим.
oneshot_timer = Timer(2) # Вземаме Timer(2) за еднократен режим.

periodic_timer.init(mode=Timer.PERIODIC, freq=2, callback=periodic_callback, hard=False) # Пускаме периоди...
print(periodic_timer.period()) # Показваме периода в сурови AGT counts.
print(periodic_timer.counter()) # Показваме текущия суров counter.

oneshot_timer.init(mode=Timer.ONE_SHOT, freq=2, callback=oneshot_callback, hard=False) # Пускаме еднократе...
```

Това се вижда подробно в:

- examples/23_timing/13_hardware_timer_periodic_oneshot.py

Пример за output compare callback:

```
from machine import Timer # Импортираме Timer, защото output compare работи през хардуерния таймер.

timer = Timer(1) # Използваме Timer(1) като базов таймер за compare събитията.
timer.init(mode=Timer.PERIODIC, freq=10, callback=cycle_callback, hard=False) # Пускаме бавен периодичен т...

period_counts = timer.period() # Вземаме периода в сурови counts.
channel_a = timer.channel(1, mode=Timer.OC, compare=period_counts // 4, callback=compare_a_callback) # Нас...
channel_b = timer.channel(2, mode=Timer.OC, compare=(period_counts * 3) // 4, callback=compare_b_callback) ...
```

Това се вижда подробно в:

- examples/23_timing/14_hardware_timer_output_compare.py

Пример за input capture:

```
from machine import Pin, PWM, Timer # Импортираме Pin, PWM и Timer за генерация и измерване на тестов сигнал.

signal_pwm = PWM(Pin("P107"), freq=1000, duty=50) # Генерираме тестов квадратен сигнал 1 kHz на P107.
timer = Timer(1) # Използваме Timer(1) за input capture измерването.

timer.init(mode=Timer.PERIODIC, freq=1_000_000, hard=False) # Пускаме Timer(1) на 1 MHz, за да е 1 count п...
period_channel = timer.channel(0, mode=Timer.IC, pin=Pin("P100"), measure=Timer.IC_PERIOD, edge=Timer.RISIN...
```

Това се вижда подробно в:

- `examples/23_timing/15_hardware_timer_input_capture.py`

Пример за event count и callback на всеки N импулса:

```
from machine import Pin, PWM, Timer # Импортираме Pin, PWM и Timer за броене на входни импулси.

signal_pwm = PWM(Pin("P107"), freq=200, duty=50) # Генерираме входен тестов сигнал 200 Hz на P107.
timer = Timer(1) # Използваме Timer(1) като хардуерен брояч на събития.

timer.init(mode=Timer.PERIODIC, period=20, callback=every_n_events_callback, hard=False) # Искаме callback...
counter_channel = timer.channel(0, mode=Timer.IC, pin=Pin("P100"), measure=Timer.IC_EVENT_COUNT, edge=Timer...
print(counter_channel.capture()) # Четем частичния брой импулси в текущия прозорец.
```

Тук е важната идея:

- `channel callback` за `Timer.IC_EVENT_COUNT` няма
- `capture()` се ползва за `polling` на частичния брой
- базовият `Timer.callback()` се ползва когато искате реакция на всеки N входни импулса

Това се вижда подробно в:

- `examples/23_timing/16_hardware_timer_event_count.py`

Подглава 15.2. Практически checklist за AGT тестове

Тази подглава е кратък runtime checklist за проверка на новия хардуерен timer слой директно на борда.

Ако искаш отделен файл за работа до платката, използвай и:

- `examples/AGT_TEST_CHECKLIST.md`

Преди старта:

- качи актуалния firmware на борда
- качи актуалната директория `examples/` на `/flash/examples`
- отвори REPL, за да виждаш `print()` резултатите
- ако ще гледаш физически изходи, подготви логически анализатор или осцилоскоп

Бърз ред на тестовите:

1. Стартирай `examples/23_timing/13_hardware_timer_periodic_oneshot.py`
2. Стартирай `examples/23_timing/14_hardware_timer_output_compare.py`
3. Свържи P107 -> P100 с джъмпер
4. Стартирай `examples/23_timing/15_hardware_timer_input_capture.py`
5. Остави същия джъмпер P107 -> P100
6. Стартирай `examples/23_timing/16_hardware_timer_event_count.py`

Тест 1: periodic и one-shot

- `Timer(1)` трябва да сработва многократно
- `Timer(2)` трябва да сработи само веднъж
- `Timer(1).counter()` трябва да се променя, докато таймерът работи
- проблем е ако one-shot не дойде или ако дойде повече от веднъж

Тест 2: output compare callback

- `Cycle-end callback count` трябва да расте
- `Compare A callback count` трябва да расте
- `Compare B callback count` трябва да расте
- проблем е ако `cycle callback` работи, а `compare callback`-ите стоят на 0

Опционален хардуерен тест за output compare:

- за `Timer(1)` можеш да добавиш `pin=Pin("P500")` на `channel(1)`
- за `Timer(1)` можеш да добавиш `pin=Pin("P501")` на `channel(2)`
- на осцилоскоп очаквай събития около 1/4 и 3/4 от периода

Тест 3: input capture

- нужен е джъмпер P107 -> P100
- при PWM 1 kHz / 50% duty очаквай Измерен период около 1000
- при същия сигнал очаквай Измерена high ширина около 500
- проблем е ако стойностите са 0, силно шумни или high ширината е по-голяма от периода

Тест 4: event count

- нужен е джъмпер P107 -> P100
- входният PWM е 200 Hz
- прозорецът е 20 импулса
- за около 1.1 s очаквай приблизително 10 или 11 завършени прозореца
- Частичен брой в текущия прозорец трябва да е между 0 и 19
- Приблизителен общ брой импулси трябва да е близо до 220

Тук е важно:

- при `Timer.IC_EVENT_COUNT` няма `channel callback`
- `capture()` се ползва за `rolling` на частичния брой
- базовият `Timer.callback()` се ползва когато искаш реакция на всеки N импулса

Ако нещо не работи:

- провери първо джъмпера P107 -> P100
- провери дали не е останал стар обект PWM или Timer от предишен тест
- извикай `deinit()` на използваните обекти и повтори теста
- ако `periodic` работи, но `compare` не работи, проблемът е в `compare IRQ path-a`
- ако `compare` работи, но `capture` и `event count` не работят, проблемът е в `AGT input pin` или `input mode path-a`

Минимален smoke test:

1. `examples/23_timing/13_hardware_timer_periodic_oneshot.py`
2. `examples/23_timing/15_hardware_timer_input_capture.py` с джъмпер P107 -> P100
3. `examples/23_timing/16_hardware_timer_event_count.py` със същия джъмпер

Ако тези три минат, базовият AGT слой е в добро състояние.

Резюме

- `time.sleep_ms()` е най-простият инструмент, но блокира изпълнението на всичко останало.
- Хардуерният `Timer(n)` е подходящ, когато искате периодично действие да върви независимо от основния код.
- `Timer(-1)` е полезен като софтуерен таймер, но трябва да се мисли различно от истински хардуерен таймер.
- Тази глава полага основата за следващите теми с прекъсвания, `debounce` и събитийна логика.

Самопроверка

1. Каква е практическата разлика между `time.sleep_ms()`, `Timer(n)` и `Timer(-1)`?
2. Кога е по-подходящо да ползвате хардуерен таймер вместо цикъл със `sleep_ms()`?
3. Защо дълъг блокиращ код в `callback` или основния цикъл разваля времевото поведение?

Упражнения

1. Реализирайте едно и също мигане на LED1 веднъж със `sleep_ms()`, веднъж с хардуерен таймер и сравнете поведението.

Подсказка: при `Timer(1)` ползвайте `timer.init(freq=2, mode=Timer.PERIODIC, callback=...)` — `callback` сменя LED без блокиране.

2. Променете периода на таймера от 4 Hz на 2 Hz и 8 Hz и запишете какво се променя.

Подсказка: `timer.init(freq=8, ...)` — по-висок `freq` = по-бързо мигане, задаването е в Hz.

3. Използвайте `examples/23_timing/05_timer_compare_hw_sw.py` и сравнете усещането за точност между `Timer(1)` и `Timer(-1)`.

Подсказка: `Timer(1)` е хардуерен (AGT) с прецизност $\sim \mu s$, `Timer(-1)` е софтуерен с точност $\sim ms$.

Ако не работи

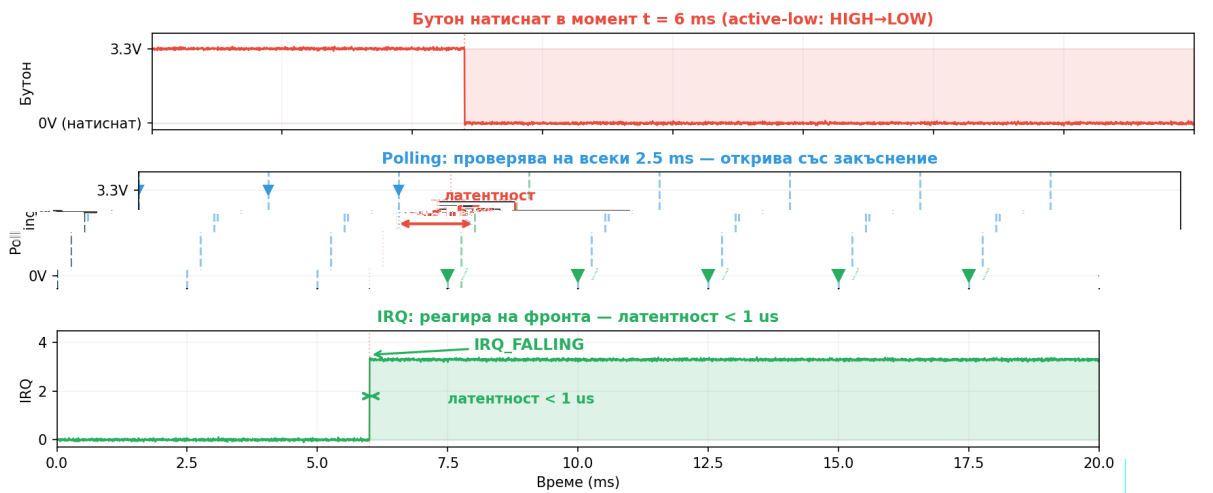
- Симптом: таймерът сработва с грешна честота. Проверка: вижте дали мислите в `freq` или в милисекунди и не смесвате единиците.
- Симптом: всичко „замръзва“ по време на демонстрацията. Проверка: вероятно сте оставили твърде дълъг блокиращ код в основния поток или в `callback`-а.

Нататък

След тази глава преминете към „Прекъсвания“, за да надградите таймерното мислене с реакция на външни събития.

Глава 16. Прекъсвания [Среден]

[Тip] Аналогия: `Polling` е да проверявате вратата на всеки 5 секунди — „почука ли?“ `IRQ` е звънецът на вратата — CPU се занимава с нещо друго и реагира само когато събитието настъпи.



Polling vs IRQ — кой реагира по-бързо?

[!] Важно: IRQ handler трябва да е кратък — само флаг или брояч. Тежката логика (print, I2C, файлове) трябва да е извън IRQ в главния цикъл.

Пример:

```
from machine import Pin, disable_irq, enable_irq # Импортираме Pin, disable_irq и enable_irq за демонстрац...

button = Pin("SW1", Pin.IN, Pin.PULL_UP) # Настройваме SW1 като вход с pull-up.

def button_handler(pin_object): # Това е функцията, която ще се вика при прекъсване от бутона.
    print("IRQ") # Печатаме текст при всяко прекъсване.

button.irq(handler=button_handler, trigger=Pin.IRQ_FALLING, hard=False) # Регистрираме обработчик за падащ...
```

Безопасно четене на споделени данни:

```
irq_state = disable_irq() # Временно забраняваме прекъсванията, за да вземем консистентно копие на брояча.
snapshot = irq_state_data["count"] # Копираме брояча, докато прекъсванията са спрени.
enable_irq(irq_state) # Възстановяваме предишното състояние на прекъсванията.
```

Софтуерни (soft) и хардуерни (hard) прекъсвания

MicroPython поддържа два режима на прекъсване, зададени чрез параметъра `hard=`:

Параметър	Контекст на изп.	Какво може	Какво НЕ може
<code>hard=False</code> (soft, по подразбиране)	Външен scheduler	<code>print()</code> , алокация на памет, <code>I2C.readfrom()</code> , всякакъв Python код	Латентност ~50-200 us
<code>hard=True</code> (хардуерно)	Директно в IRQ к	Само: <code>Pin.value()</code> , запис в bytearray, промяна на флаг, <code>mic</code>	<code>print()</code> , алокация, I2C, UART, float — забранени

Soft IRQ (стандартен MicroPython) — безопасен и гъвкав:

```

from machine import Pin

press_count = 0

def on_press(pin):
    global press_count
    press_count += 1
    print(f"Натискане #{press_count}")      # print() е OK в soft IRQ
    # Можете да четете I2C, пишете в UART, алокирате памет

button = Pin("SW1", Pin.IN, Pin.PULL_UP)
button.irq(handler=on_press, trigger=Pin.IRQ_FALLING, hard=False)

```

Hard IRQ (порт-специфичен за VK_RA4M2) — минимална латентност, но строги ограничения:

```

from machine import Pin

irq_flag = bytearray(1)  # pre-allocated – не алокирай вътре в hard IRQ!

def on_press_hard(pin):
    irq_flag[0] = 1      # само запис в съществуващ буфер – OK
    # print() тук → CRASH!
    # list.append() тук → CRASH!
    # I2C.readfrom() тук → CRASH!

button = Pin("SW1", Pin.IN, Pin.PULL_UP)
button.irq(handler=on_press_hard, trigger=Pin.IRQ_FALLING, hard=True)

while True:
    if irq_flag[0]:
        irq_flag[0] = 0
        print("Натиснат (hard IRQ)")      # тежката работа – в main loop

```

*[Pin] Кога какво: Ползвайте `hard=False` за почти всичко. `hard=True` е нужен само когато латентността от ~100 us е твърде голяма — напр. при *high-speed encoder* или *safety stop бутон*.*

Практическа бележка за измерване на latency:

- може да се направи полезна ориентировъчна оценка чрез два свързани GPIO пина и `time.ticks_us()`
- тази стойност включва GPIO пътя, IRQ доставката, soft callback scheduling и Python callback overhead
- затова тя не е равна на „чистата“ силициева interrupt latency от datasheet или осцилоскоп
- примерът `examples/23_timing/08_interrupt_latency_measure.py` иска jumper между изходен и входен пин

Файлове:

- `examples/23_timing/02_interrupts.py`
- `examples/23_timing/08_interrupt_latency_measure.py`
- `examples/28_machine_misc/02_machine_mem_and_bootloader.py`

Резюме

- Прекъсванията позволяват бордът да реагира веднага на събитие, вместо да чака polling цикълът да стигне до него.
- Краткият IRQ handler и безопасното споделяне на данни са по-важни от това „да направите всичко вътре в

прекъсването“.

- Измерването на latency е полезно, но трябва да се разбира като практическа оценка на целия път, не като чиста силициева стойност.

Самопроверка

1. Каква е разликата между polling и IRQ реакция при бутон?
2. Защо е добра практика IRQ handler-ът да е кратък и прост?
3. Кога и защо бихте използвали `disable_irq()` и `enable_irq()` около споделени данни?

Упражнения

1. Направете IRQ обработчик, който не печата директно, а само брой събитията в споделен брояч.

Подсказка: `press_count[0] += 1` в `bytearray(1)` или глобална променлива — в `main loop` четете и печатате.

2. Сравнете визуално или по лог polling четене и IRQ четене на един и същ бутон.

Подсказка: пуснете двата стила последователно и сравнете колко „призрачни“ натискания регистрира всеки без `debounce`.

3. Пуснете latency примера с jumper и запишете ориентировъчната стойност, която виждате на вашия setup.

Подсказка: файлът `23_timing/08_interrupt_latency_measure.py` изисква jumper между изходен и входен пин за затворена верига.

Ако не работи

- Симптом: получавате твърде много IRQ събития от едно натискане. Проверка: това е нормално без `debounce` и ще се реши в следващата глава.

- Симптом: обработчикът се държи странно или забавя системата. Проверка: намалете работата вътре в IRQ и изнесете тежките действия извън него.

Нататък

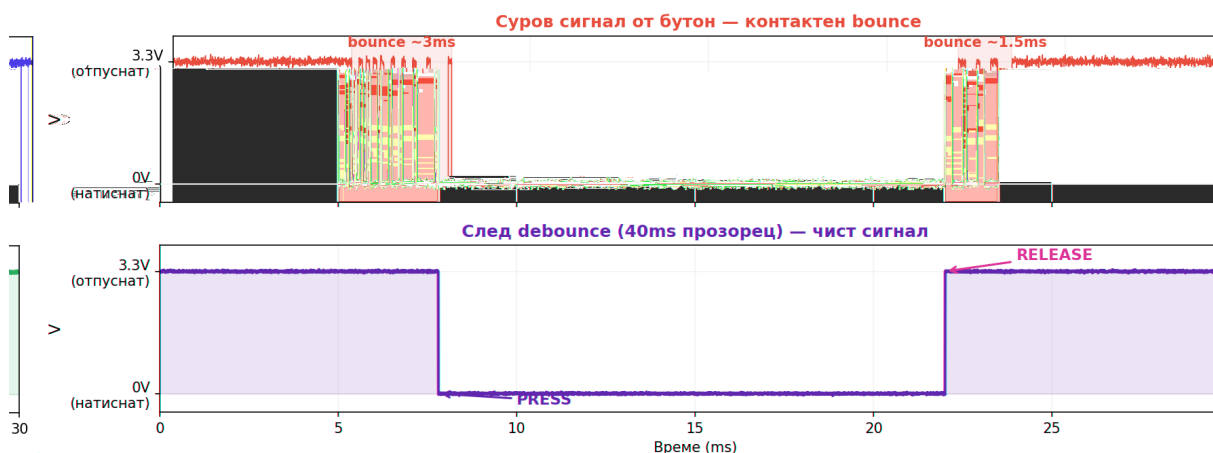
След тази глава преминете към „Дебаунс на бутон“, за да видите как реалният механичен вход се превръща в надеждно събитие.

Глава 17. Дебаунс на бутон [Среден]

В Глава 16 видяхте, че едно натискане на бутон може да генерира множество IRQ заради контактния bounce. Тук решаваме този проблем: превеждаме шумния физически сигнал в единично стабилно събитие.

[Tip] Аналогия: Натиснете физически бутон под микроскоп — контактите подскачат многократно за 1-20 ms преди да се установят стабилно. Без `debounce` едно натискане дава 5-50 импулса. `Debounce` логиката изчаква докато вибрирането спре.

Дебаунс: от шумен механичен сигнал до чисто логическо събитие



Контактен bounce и debounce

Подход	Как работи	Когато
Времеви прозорец	Приема само ако 40ms са изтекли без нов фронт	Прост polling
Стабилни проби	4 поредни еднакви стойности = валидно	asyncio, interrupt

Пример с polling:

```
if current_value != last_stable_value: # Проверяваме дали е настъпила промяна спрямо последното стабилно с...
    now_ms = time.ticks_ms() # Вземаме текущото време само при промяна.
    if time.ticks_diff(now_ms, last_change_ms) >= 40: # Приемаме промяната за валидна само ако е минало по...
        last_change_ms = now_ms # Обновяваме момента на последната приета промяна.
        last_stable_value = current_value # Приемаме новата стойност за стабилна.
```

Пример с asyncio:

```
if stable_value == 0 and stable_count == 4: # Ако видим четири поредни проби с 0, приемаме валидно натискане.
    async_presses += 1 # Увеличаваме броя на валидните асинхронни натискания.
```

Тук има два подхода:

- времеви прозорец в polling цикъл
- последователни стабилни проби в async задача

Файл:

- examples/23_timing/03_button_debounce.py

Резюме

- Механичният бутон не дава чист единичен преход, а кратък шумен пакет от промени.
- Debounce логиката превежда суровото ниво към стабилно натискане и отпускане.
- Времевият прозорец и броят стабилни проби са двата основни инструмента за овладяване на този шум.

Самопроверка

1. Защо един реален бутон има нужда от debounce, въпреки че логиката му изглежда двоична?
2. Каква е разликата между debounce чрез времеви прозорец и debounce чрез поредица стабилни проби?

3. Какво ще стане, ако debounce прозорецът е твърде кратък или твърде дълъг?

Упражнения

1. Променете debounce времето от 40 ms на 10 ms и 80 ms и сравнете стабилността.

Подсказка: `DEBOUNCE_MS = const(10)` — при по-малка стойност ще получите повече призрачни натискания.

2. Добавете печат само при валидно натискане, а не при всяка сурова промяна.

Подсказка: печатайте само когато `stable_value` се промени, а не при всяко четене на `button.value()`.

3. Сравнете polling варианта и `asyncio` варианта върху един и същ бутон.

Подсказка: в `asyncio` варианта заменете `sleep_ms()` с `await asyncio.sleep_ms()` за неблокиращо изчакване.

Ако не работи

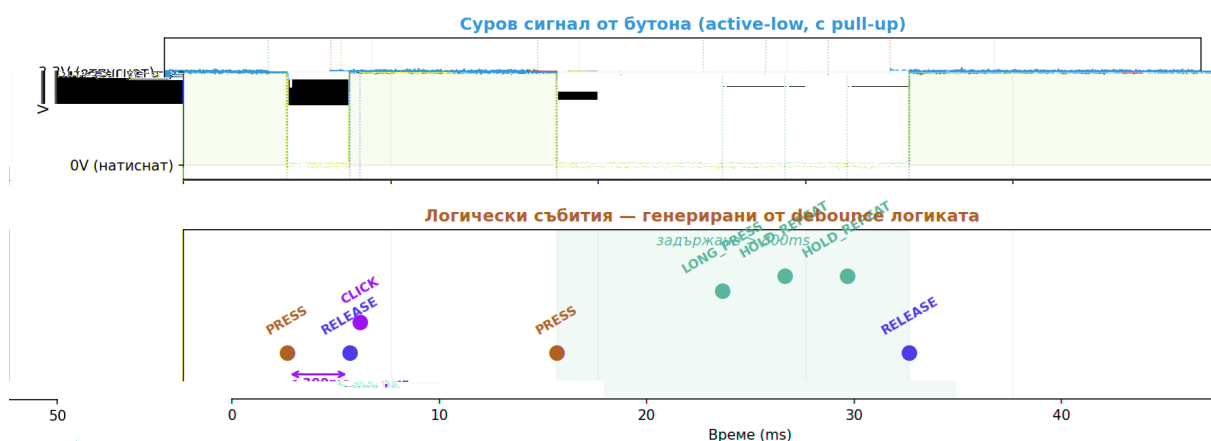
- Симптом: едно натискане се брои по няколко пъти. Проверка: debounce прагът е твърде нисък или пробовземането е твърде грубо.
- Симптом: бутонът започва да реагира мудно. Проверка: debounce прозорецът вероятно е прекалено голям.

Нататък

След тази глава преминете към „Четене на бутон: polling, IRQ, Timer(-1), debounce и събития“, за да сравните няколко практични подхода към един и същ входен проблем.

Глава 18. Четене на бутон: polling, IRQ, Timer(-1), debounce и събития [Среден]

[Тip] Аналогия: Суровото `button.value()` е като суров електрически сигнал — само 0 и 1. Приложението иска смислови събития: "потребителят натисна", "задържа 2 секунди", "двойно натискане". Тази глава е преводачът между суров сигнал и смислово събитие.



От суров сигнал към логически събития

Събитие	Условие
PRESS	Стабилен преход от 1 → 0
RELEASE	Стабилен преход от 0 → 1
CLICK	PRESS + RELEASE в рамките на ~300ms
LONG_PRESS	PRESS задържан > 800ms
HOLD_REPEAT	Повтаряне докато е задържан

Тази глава събира на едно място реалните начини за работа с бутон върху VK_RA4M2.

Суровото четене на SW1 е само началото.

Практическият въпрос е как да преведем шумния active-low вход към стабилни събития като:

- PRESS
- RELEASE
- CLICK
- LONG_PRESS
- HOLD_START
- HOLD_REPEAT

Първата идея е проста: `button.value()` ни дава ниво, но приложението почти винаги иска събития.

Пример с polling и преходи между стабилни състояния:

```
raw_value = button.value() # Четем суровата моментна стойност на бутона.
if raw_value != last_raw_value: # Ако видим ново сурово ниво, започваме нов debounce прозорец.
    last_raw_value = raw_value # Запомняме новата сурова стойност.
    raw_change_ms = now_ms # Запомняме кога е започнала промяната.
if raw_value != stable_value and time.ticks_diff(now_ms, raw_change_ms) >= DEBOUNCE_MS: # Ако новото ниво ...
    stable_value = raw_value # Обновяваме стабилното състояние на бутона.
    if stable_value == 0: # Ако стабилното състояние е 0, имаме валидно натискане.
        print("EVENT: PRESS") # Печатаме събитие за натискане.
    else: # Ако стабилното състояние е 1, имаме валидно отпускане.
        print("EVENT: RELEASE") # Печатаме събитие за отпускане.
```

Тук вече имаме важна промяна в начина на мислене:

- не гледаме „ниво в момента“
- гледаме „последно стабилно състояние“
- генерираме събитие само при стабилен преход

Ако искаме по-бърза реакция от чистото polling, ползваме прекъсване, но не правим тежката логика вътре в IRQ обработчика.

Пример с IRQ_FALLING | IRQ_RISING и отложена обработка:

```
def button_irq_handler(pin_object): # Това е краткият IRQ обработчик за суровите фронтове на бутона.
    irq_state_data["pending"] = 1 # Маркираме, че главният цикъл трябва да довърши логическата обработка.
    irq_state_data["deadline_ms"] = time.ticks_add(time.ticks_ms(), DEBOUNCE_MS) # Отлагаме окончателното ...

if pending and time.ticks_diff(now_ms, deadline_ms) >= 0: # Ако има чакаща работа и debounce прозорецът е ...
    stable_candidate = button.value() # Четем реалното ниво след краткото изчакване.
    if stable_candidate != previous_stable_value: # Ако стабилното ниво реално е ново, генерираме логическ...
        print("EVENT: PRESS" if stable_candidate == 0 else "EVENT: RELEASE") # Печатаме кое логическо съби...
```

Това е по-добрият стил за бутон по прекъсване:

- IRQ само събужда логиката
- debounce се решава извън ISR
- събитието пак е стабилен PRESS или RELEASE, а не суров фронт

Има и по-ниско ниво на работа, когато искаме самото прекъсване да е максимално кратко и да не влиза в Python callback пътя.

Тогава ползваме `Pin.irq(..., fast=True)` и `@micropython.asm_thumb`, а извън IRQ само консумираме кратки събития от shared буфер.

Това е подходящо, когато:

- искаме минимална работа в IRQ
- искаме една малка shared опашка в RAM
- искаме печатът и по-тежката логика да останат извън прекъсването

[!] ИЗВЪН ОСНОВНИЯ КУРС — Напреднал пример: Следващият раздел е опционален и може да бъде прескочен без загуба на последователност. Той изисква познания по ARM Thumb assembly, `ustypes.addressof` и споделена памет между ISR и основен код — теми, които не са покрити другаде в книгата. Той използва външни бутони на P301 и P302 (не вградените SW1 = P400), които трябва да бъдат свързани отделно с pull-up към GND. Ако сте начинаещ — преминете директно към Глава 19.

Пример с два бутонa P301 и P302, fast ASM IRQ, shared bytearray ring buffer и debounce чрез lock + стабилно отпускане:

```

from array import array # Импортираме array, за да държим shared 32-bit стойности в RAM.
from machine import Pin # Импортираме Pin, за да вържем бутоните към external IRQ линиите.
from ctypes import addressof # Импортираме addressof, за да дадем на ASM кода адресите на shared буферите.
import machine # Импортираме machine, за да ползваме disable_irq/enable_irq около consumer логиката.
import micropython # Импортираме micropython, защото fast IRQ handler-ите са @micropython.asm_thumb.
import time # Импортираме time, за да направим release debounce извън IRQ.

events = bytearray(16) # Това е ring buffer-ът за кратките събития от прекъсването.
event_values = array("i", [0] * 16) # Тук пазим snapshot на стойността за всяко събитие.
state = array("i", [0, 0, 0, 0, 0, 0]) # Държим wr_idx, rd_idx, value, lost_events, plus_lock и minus_lock.

queue_addr = addressof(events) # Взимаме адреса на ring buffer-а за директен byte достъп от ASM.
values_addr = addressof(event_values) # Взимаме адреса на snapshot буфера за директен word достъп от ASM.
state_addr = addressof(state) # Взимаме адреса на state блока за директен word достъп от ASM.

exec(build_button_isr_source("plus_isr", +1, 1), globals(), globals()) # Генерираме fast ASM ISR за бутоната...
exec(build_button_isr_source("minus_isr", -1, 2), globals(), globals()) # Генерираме fast ASM ISR за бутон...

plus_button = Pin("P301", Pin.IN, Pin.PULL_UP) # Настройваме P301 като вход с pull-up за бутоната "+".
minus_button = Pin("P302", Pin.IN, Pin.PULL_UP) # Настройваме P302 като вход с pull-up за бутоната "-".

plus_button.irq(trigger=Pin.IRQ_FALLING, handler=plus_isr, fast=True) # Връзваме fast IRQ на падащ фронт з...
minus_button.irq(trigger=Pin.IRQ_FALLING, handler=minus_isr, fast=True) # Връзваме fast IRQ на падащ фронт...

while True: # Главният цикъл остава извън IRQ и само довършва debounce и печата събитията.
    handle_release_debounce() # Re-arm-ваме бутоната едва след стабилно отпускане.
    irq_state = machine.disable_irq() # Кратко спираме IRQ, за да вземем консистентен snapshot на опашката.
    event_code, event_value, lost_events = pop_event() # Вземаме едно събитие и snapshot стойността му от ...
    machine.enable_irq(irq_state) # Връщаме IRQ веднага след кратката критична секция.
    if event_code == 1: # Ако кодът е 1, печатаме събитие за бутоната "+".
        print("PLUS value =", event_value) # Показваме snapshot стойността за натискане на бутоната "+".
    elif event_code == 2: # Ако кодът е 2, печатаме събитие за бутоната "-".
        print("MINUS value =", event_value) # Показваме snapshot стойността за натискане на бутоната "-".
    if lost_events: # Ако ring buffer-ът е бил пълен, показваме, че има изпуснати събития.
        print("Queue overflow, lost events =", lost_events) # Печатаме диагностиката за overflow извън IRQ.
    time.sleep_ms(2) # Оставяме кратък sleep, за да е четим изходът и да работи release debounce логиката.

```

Тук ключовата идея е:

- fast IRQ path не печата и не пипа Python обекти
- ISR прави само три неща: lock, update на стойността, enqueuе на кратко събитие
- debounce не е оставен на механичния бутон, а се довършва извън IRQ чрез lock + стабилно отпускане
- печатът става по snapshot стойността за самото събитие, не по по-късна глобална стойност

Пълният пример е във файла по-долу. Там helper-ът `build_button_isr_source(...)` генерира реалния `@micropython.asm_thumb` код и подава към него адресите на shared буферите.

Файл:

- `examples/23_timing/17_fast_irq_two_buttons_queue.py`

Третият подход е `Timer(-1)` като периодичен sampler.

Той е полезен, когато искаме еднакъв ритъм на пробовземане и по-богати събития за задържане.

Пример със софтуерен `Timer(-1)` и hold логика:

```

soft_timer.init(period=SAMPLE_MS, mode=Timer.PERIODIC, callback=button_sample_callback) # Стартираме Timer...
if timer_state["history"] == 0x00 and timer_state["stable_value"] != 0: # Ако последните осем проби са 0, ...
    timer_state["event_code"] = EVENT_PRESS # Записваме PRESS събитие за основния цикъл.
elif timer_state["stable_value"] == 0 and not timer_state["hold_started"] and time.ticks_diff(now_ms, timer...
    timer_state["event_code"] = EVENT_HOLD_START # Записваме HOLD_START събитие.
elif timer_state["stable_value"] == 0 and timer_state["hold_started"] and time.ticks_diff(now_ms, timer_sta...
    timer_state["event_code"] = EVENT_HOLD_REPEAT # Записваме HOLD_REPEAT събитие.

```

Тук се появяват вече различни класове събития:

- PRESS при стабилен преход 1 -> 0
- RELEASE при стабилен преход 0 -> 1
- CLICK при кратко натискане и отпускане
- LONG_PRESS при достигане на праг по време на задържане
- HOLD_START при началото на продължително задържане
- HOLD_REPEAT при периодически повтаряне, докато бутонът остава натиснат

Четвъртият подход е debounce чрез битово изместване.

Той е компактен и много удобен за периодичен sampler, защото свива последните N проби в една история.

Пример с shift-register debounce:

```

history = ((history << 1) | button.value()) & 0xFF # Премества историята наляво и добавяме новата проба ...
if history == 0x00 and stable_value != 0: # Ако последните осем проби са натиснати, приемаме стабилно PRES...
    print("EVENT: PRESS") # Печатаме събитие за натискане.
elif history == 0xFF and stable_value != 1: # Ако последните осем проби са отпуснати, приемаме стабилно RE...
    print("EVENT: RELEASE") # Печатаме събитие за отпускане.

```

Тук смисълът на history е директен:

- 0x00 означава осем поредни проби с натиснат бутон
- 0xFF означава осем поредни проби с отпуснат бутон
- всички междинни стойности означават „още не е стабилно“

Практически избор на подход:

- ако искате най-простия старт, започнете с polling
- ако искате бързо събуждане на логиката, използвайте IRQ с отложена обработка
- ако искате равномерно пробовземане и hold/repeat семантика, използвайте Timer(-1)
- ако искате компактен debounce филтър, използвайте битово изместване

Файлове:

- examples/23_timing/09_button_polling_events.py
- examples/23_timing/10_button_irq_deferred.py
- examples/23_timing/17_fast_irq_two_buttons_queue.py
- examples/23_timing/11_button_soft_timer_hold.py
- examples/23_timing/12_button_shift_debounce.py
- examples/23_timing/02_interrupts.py
- examples/23_timing/03_button_debounce.py
- examples/23_timing/04_software_timer_minus1.py

Резюме

- Главата събира четенето на бутон като ниво, като стабилен преход и като събитийна система.
- Показани са polling, IRQ с отложена обработка, fast ASM IRQ със shared queue, Timer(-1) sampler и debounce чрез битово изместване.
- Основната идея е, че приложението почти винаги иска PRESS, RELEASE и HOLD събития, а не суров шумен вход.

Самопроверка

1. Каква е разликата между суров фронт от IRQ и стабилно логическо събитие PRESS?
2. Кога Timer(-1) е по-подходящ от чист polling цикъл за бутон?
3. Защо при fast ASM IRQ е по-добре да използваме shared ring buffer, вместо да печатаме директно от ISR?
4. Какво означават 0x00 и 0xFF при debounce чрез битово изместване?

Упражнения

1. Разширете 09_button_polling_events.py, така че след LONG_PRESS да не печата и CLICK при отпускане.
Подсказка: добавете флаг 'long_press_seen' и при 'RELEASE' проверете дали флагът е вдигнат, преди да излъчите 'CLICK'.
2. Добавете CLICK и към 10_button_irq_deferred.py, като използвате продължителността между PRESS и RELEASE.
Подсказка: запазете 'press_time_ms' при 'PRESS' и сравнете с 'time.ticks_diff(now, press_time_ms) < CLICK_MS'.
3. Пуснете 17_fast_irq_two_buttons_queue.py с два бутона на P301 и P302 и измерете колко реда се печатат при едно натискане.
Подсказка: 'lost_events' в изхода показва дали ring buffer-ът е преполнен при бързо натискане.
4. Променете размера на debounce историята в 12_button_shift_debounce.py от 8 на 4 проби и сравнете устойчивостта.
Подсказка: заменете '& 0xFF' с '& 0x0F' и проверявайте за '0x00' (4 нули) вместо за 8.

Ако не работи

- Симптом: получавате много PRESS и RELEASE събития от едно натискане. Проверка: debounce прозорецът или пробовземането са твърде къси.
- Симптом: LONG_PRESS или HOLD_REPEAT не идват. Проверка: вижте дали държите бутона достатъчно дълго и дали прагът за задържане не е твърде голям.

Нататък

След тази глава преминете към „RTC“, за да добавите календарно и часово време към таймерното мислене.

Глава 19. RTC [Среден]

[Tip] Аналогия: time.ticks_ms() е като секундомер — брои от включването. RTC е часовник на стената — знае дата и час, дори след reset (ако е с батерия).

	time.ticks_ms()	RTC
Мери	Интервали от старт	Календарна дата и час
При reset	Нулира се	Запазва се (с батерия)
Употреба	Timing, delays	Timestamps, планиране

Пример:

```
from machine import RTC # Импортираме RTC, за да работим с часовника в реално време.

rtc = RTC() # Създаваме обект за достъп до вградения RTC модул.
print(rtc.info()) # Печатаме диагностичната информация за стартиране на RTC.
print(rtc.calibration()) # Печатаме текущата калибрационна стойност.
print(rtc.datetime()) # Четем текущата дата и час като осемелементен tuple.
```

Тази глава покрива:

- диагностика на RTC
- current datetime tuple
- безопасно задаване на време зад флаг

Файл:

- examples/23_timing/06_rtc_basic.py

Резюме

- RTC е часовникът на борда за календарна дата и час, а не просто брояч на милисекунди.
- `rtc.datetime()` връща структурирано време, което може да се чете, отпечатва и задава внимателно.
- Предпазният флаг за задаване на време е добра практика, защото промяната на часовника е по-чувствителна от обикновен `print()`.

Самопроверка

1. Каква е разликата между RTC и таймер като `Timer(1)`?
2. Какво съдържа tuple-ът от `rtc.datetime()` и защо е полезно да го гледате структуриран?
3. Защо задаването на време трябва да стои зад предпазен флаг в учебен пример?

Упражнения

1. Отпечатвайте `rtc.datetime()` през една секунда няколко пъти и наблюдавайте кои полета се променят.
Подсказка: ``dt = rtc.datetime()`` — ``dt[6]`` е *секундите*, ``dt[5]`` е *минутите*, ``dt[4]`` е *часовете*.
2. Направете четим формат `YYYY-MM-DD HH:MM:SS` от tuple-a на `rtc.datetime()`.
Подсказка: ``"{0}-{02d}-{02d} {02d}:{02d}:{02d}".format(dt[0], dt[1], dt[2], dt[4], dt[5], dt[6])``.
3. Включете задаване на време само през флаг и сравнете изхода преди и след настройката.
Подсказка: ``SET_TIME = False`` — само при ``True`` извиквайте ``rtc.datetime((2024, 1, 1, ...))``.

Ако не работи

- Симптом: стойностите на датата и часа изглеждат странно подредени. Проверка: припомнете си реда на полетата в tuple-a от `rtc.datetime()`.
- Симптом: времето се променя неочаквано по време на тест. Проверка: вижте дали не сте оставили флага за задаване на време включен.

Нататък

След тази глава следват серийните интерфейси — I2C, SPI, UART — с които свързвате реални сензори и модули.

Част IVб: asyncio — мост към конкурентно програмиране

Ключови термини: `event loop` — цикълът, който решава коя задача кога работи. `create_task()` — регистрира нова `async` задача. `Event` — обект за синхронизация: `set()` вдига флаг, `wait()` чака. `gather()` — стартира няколко задачи паралелно.

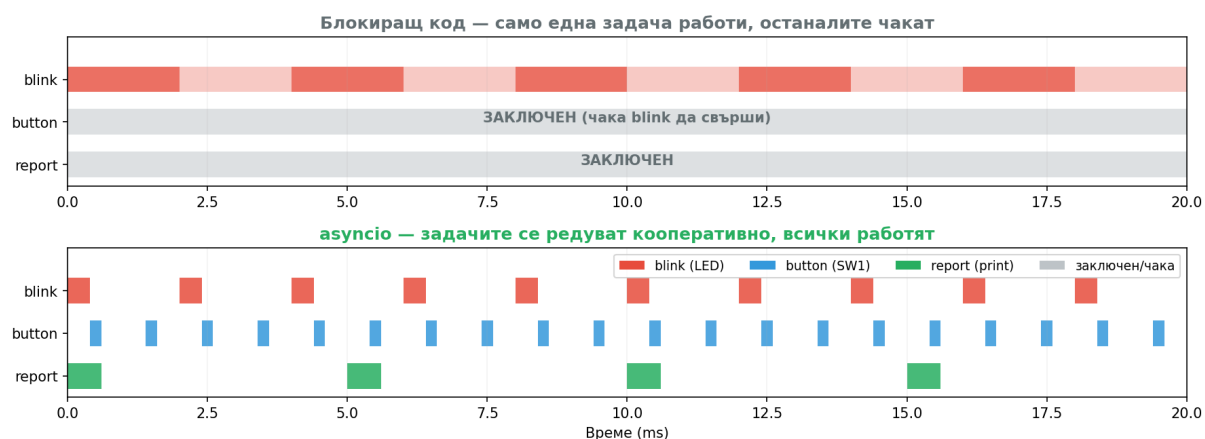
Защо тук? `asyncio` е въведен непосредствено след таймерите и прекъсванията, защото неблокиращото мислене е естествено продължение на `timer/IRQ` логиката. Когато вече знаете как таймерът „пуска“ събитие, `asyncio` е следващата стъпка — кооперативен `scheduler` вместо ръчни `callbacks`.

След като има базово разбиране за време и събития, `asyncio` идва на правилното място като кооперативен модел за едновременност.

Глава 20. Основи на asyncio [Среден]

В Глава 2 видяхте как `blink` и `counter` задачите споделят процесора неформално. Тук подреждаме систематично: `create_task`, `gather`, три задачи едновременно на реален хардуер.

[Тър] Аналогия: Event loop-ът е единствен сервитьор, който обслужва много маси. Всяка маса (задача) казва "чакам поръчката си 500ms" (`await asyncio.sleep_ms(500)`) — сервитьорът отива при друга маса. Никой не чака прав до тезгяха.



Пример:

```
import asyncio # Импортираме asyncio за кооперативна многозадачност.
from machine import Pin # Импортираме Pin за реален хардуерен достъп.

LED_ON = 0 # Active-low: 0 = светва LED1.
LED_OFF = 1 # Active-low: 1 = гаси LED1.

led = Pin("LED1", Pin.OUT, value=LED_OFF) # LED1 = P204, започваме изгасен.
button = Pin("SW1", Pin.IN, Pin.PULL_UP) # SW1 = P400, вътрешен pull-up.

async def blink_task(period_ms): # Асинхронна задача за мигане с зададен период.
    while True: # Безкрайно мигане.
        led.value(LED_ON) # Светваме LED1.
        await asyncio.sleep_ms(period_ms) # Отстъпваме управлението за period_ms.
        led.value(LED_OFF) # Гасим LED1.
        await asyncio.sleep_ms(period_ms) # Отстъпваме управлението отново.
```

Тази глава покрива:

- `create_task` за стартиране на задачи
- `asyncio.gather` за паралелно изпълнение
- три задачи едновременно: мигане, бутон, отчет

Файл:

- `examples/33_asyncio/01_asyncio_basics.py`

Резюме

- Главата систематизира `asyncio` чрез няколко кооперативни задачи.
- Показва се как LED, бутон и отчет могат да живеят като отделни роли в една програма.
- Това е реален практически модел за неблокиращи приложения върху платката.

Самопроверка

1. Какво прави `create_task` спрямо директно извикване на функция?
2. Защо `asyncio` е кооперативен, а не прекъсващ модел?
3. Кога бихте избрали този подход пред много блокиращи цикли?

Упражнения

1. Добавете нова `async` задача, която периодично логва състоянието на приложението.

Подсказка: ``async def log_task(): while True: print("OK"); await asyncio.sleep(1)`` и я регистрирайте с ``create_task()``.

2. Променете периода на мигане според някакво събитие или брояч.

Подсказка: ползвайте глобална ``period_ms`` и я сменяйте от задача за бутон — ``blink_task`` чете ``period_ms`` при всеки цикъл.

3. Спрете една от задачите след `timeout` и проверете дали останалите продължават.

Подсказка: запазете обекта на задачата ``t = asyncio.create_task(...)`` и по-късно извикайте ``t.cancel()``.

Ако не работи

- Симптом: `event loop`-ът „замръзва“. Проверка: някоя задача вероятно съдържа блокиращ код без `await`.
- Симптом: дадена задача сякаш не съществува. Проверка: вижте дали реално е създадена и дали отстъпва управлението.

Нататък

След тази глава преминете към „asyncio Event синхронизация“, за да надградите базовите задачи до координирано асинхронно поведение.

Глава 21. asyncio Event синхронизация [Среден]

[Тip] Аналогия: asyncio.Event е като светлинен сигнал на кръстовище — производителят (бутонът) вдига флага; консуматорът (LED задачата) чака флага и реагира. Без Event двете задачи не знаят за съществуването си.

```
button_producer:  — натискане —> event.set() —>
                                     |
led_consumer:     ————— await event.wait() —> мига LED
                    (кооперативно спи докато чака)
```

Пример:

```
import asyncio # Импортираме asyncio за кооперативна многозадачност.
from machine import Pin # Импортираме Pin за реален хардуерен достъп.

LED_ON = 0 # Active-low: 0 = светва LED1.
LED_OFF = 1 # Active-low: 1 = гаси LED1.

led = Pin("LED1", Pin.OUT, value=LED_OFF) # LED1 = P204, започваме изгасен.
button = Pin("SW1", Pin.IN, Pin.PULL_UP) # SW1 = P400, вътрешен pull-up.
button_event = asyncio.Event() # Създаваме Event обект за синхронизация между задачи.

async def button_producer(): # Тази задача следи SW1 и сигнализира при натискане.
    last_value = 1 # Последна стойност на бутона (ненатиснат).
    current_value = button.value() # Четем SW1.
    if current_value == 0 and last_value == 1: # Нов натиск (falling edge).
        button_event.set() # Сигнализираме Event-a.

async def led_consumer(): # Тази задача чака Event и мига LED1.
    await button_event.wait() # Чакаме Event-a (блокираме кооперативно).
    button_event.clear() # Нулираме Event-a за следващо използване.
```

Тук се вижда:

- asyncio.Event за комуникация между задачи
- производител-консуматор модел
- кооперативно блокиране без busy-wait

Файл:

- examples/33_asyncio/02_asyncio_event_flag.py

Резюме

- Главата показва asyncio.Event като чист механизъм за синхронизация между задачи.
- Моделът producer-consumer прави асинхронната комуникация по-ясна и по-надеждна.
- Това е важна крачка от „много задачи“ към „задачи, които си взаимодействат правилно“.

Самопроверка

1. Каква е ползата от

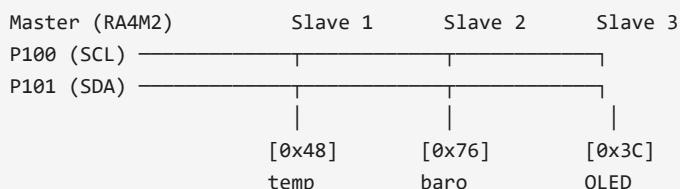
Част V: Сериен интерфейс и двоични протоколи

Ключови термини: *I2C* — двупроводна шина (SCL+SDA), всяко устройство с 7-bit адрес. *SPI* — четирипроводна шина (SCK, MOSI, MISO, CS), по-бърза от *I2C*. *UART* — асинхронна серийна комуникация (TX/RX). *scan()* — *I2C* метод за откриване на устройства. *struct.pack/unpack* — пакетиране/разпакетиране на двоични рамки.

Тук са събрани заедно *I2C*, *SPI*, *UART*, работата с външни модули и базовото мислене в binary frame-ове.

Глава 22. I2C master и runtime избор на пинове [Среден]

[Тър] Аналогия: *I2C* е като обща телефонна линия (2 жици) с многобройни абонати. Всеки абонат (slave) има уникален адрес (7-bit). Master казва "адрес 0x48, дайте данни" — само устройство с адрес 0x48 отговаря.



[!] Конфликт: *I2C(0)* ползва *P400* = *SW1*! Използвайте *I2C(1)* с *P100/P101* за повечето случаи.

Базов пример:

```

from machine import I2C # Импортираме I2C за работа като master върху наличния хардуерен контролер.

i2c = I2C(1, freq=100000) # Създаваме I2C(1) на 100 kHz върху бордовите пинове P100 и P101.
print(i2c.scan()) # Сканираме шината за устройства с 7-битов адрес.
  
```

Runtime пинове:

```

from machine import I2C, Pin # Импортираме I2C и Pin, за да покажем runtime избиране на SCL и SDA.

i2c = I2C(1, freq=400000, scl=Pin("P100"), sda=Pin("P101")) # Създаваме I2C(1) на 400 kHz с изрично зададе...
  
```

Тази глава е важна и заради конфликтите:

- *I2C(0)* ползва *P400/P401*
- *P400* е и *SW1*

Файлове:

- `examples/24_i2c/01_i2c_master_basic.py`
- `examples/24_i2c/02_i2c_runtime_pins.py`

Резюме

- *I2C* е двупроводна шина, в която master-ът управлява обмена, а устройствата отговарят по адрес.
- *scan()* е най-безопасният първи тест, защото проверява дали на шината изобщо има живи устройства.

- При този борд изборът на канал и пинове не е абстрактна подробност, а реален хардуерен конфликт, особено при I2C(0) и SW1.

Самопроверка

1. Каква е ролята на master-а в един I2C обмен?
2. Защо `scan()` е по-добър първи тест от директно четене на регистри от непознат модул?
3. Какъв практически проблем може да възникне, ако използвате I2C(0) и едновременно искате да работите със SW1?

Упражнения

1. Пуснете `scan()` при 100 kHz, после при 400 kHz, и сравнете дали резултатът остава стабилен.
Подсказка: `I2C(1, freq=100000)` и `I2C(1, freq=400000)` — нестабилен сканиран адрес при 400 kHz показва лошо окабеляване.
2. Създайте втори пример с изрично зададени `scl` и `sda` и коментирайте защо това понякога е по-ясно от default конфигурацията.
Подсказка: `I2C(1, freq=400000, scl=Pin("P100"), sda=Pin("P101"))` — изричните пинове правят кода документиращ сам себе си.
3. Направете кратка таблица с ролите SCL, SDA, master, device address.
Подсказка: `SCL` тактува шината (P100), `SDA` пренася данни (P101), `scan()` връща списък от открити адреси.

Ако не работи

- Симптом: няма отговор от шината. Проверка: първо сверете пиновете, адреса/ролята и нужните pull-up или control сигнали.
- Симптом: `scan()` връща празен списък. Проверка: устройството може да не е захранено, да е на грешни пинове или да има конфликт в окабеляването.
- Симптом: получавате нестабилни или безсмислени данни. Проверка: намалете скоростта и проверете окабеляването стъпка по стъпка.

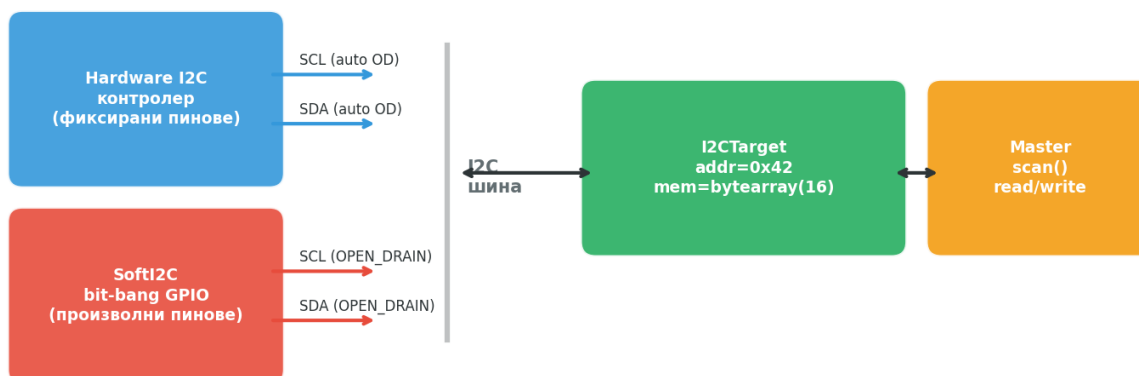
Нататък

След тази глава преминете към „SoftI2C и I2CTarget“, за да сравните хардуерен и софтуерен подход към I2C комуникацията.

Глава 23. SoftI2C и I2CTarget [Среден]

[Pin] `Pin.OPEN_DRAIN` и I2C: I2C протоколът изисква линиите SCL и SDA да са open-drain — всяко устройство може да дръпне линията надолу, но нито едно не я задвижва нагоре активно. Външен pull-up резистор (обикновено 4.7 kΩ към VCC) държи линията високо, когато никой не я дърпа. При SoftI2C трябва изрично да зададете `Pin.OPEN_DRAIN`; при хардуерния I2C портът го прави автоматично.

I2C: Hardware vs SoftI2C + I2CTarget буфер



I2C: Hardware vs SoftI2C + I2CTarget

SoftI2C:

```
from machine import SoftI2C, Pin # Импортираме SoftI2C и Pin за софтуерна I2C шина върху GPIO.

soft_i2c = SoftI2C(scl=Pin("P105", Pin.OPEN_DRAIN), sda=Pin("P104", Pin.OPEN_DRAIN), freq=100000) # Създав...
```

I2CTarget:

```
from machine import I2CTarget, Pin # Импортираме I2CTarget и Pin за работа в target или slave режим.

memory_buffer = bytearray(range(16)) # Подготвяме малък memory-backed буфер, който target-ът ще обслужва.
target = I2CTarget(1, addr=0x42, mem=memory_buffer, mem_addrsize=8, scl=Pin("P100"), sda=Pin("P101")) # Съ...
```

Тук се вижда:

- софтуерна I2C шина върху GPIO
- целево устройство с memory-backed буфер

Файлове:

- examples/24_i2c/03_softi2c_basic.py
- examples/24_i2c/04_i2ctarget_memory.py

Резюме

- SoftI2C показва, че I2C може да се реализира и върху GPIO, когато хардуерният контролер не е най-удобният избор.
- I2CTarget обръща гледната точка: тук бордът вече не е master, а устройство, което отговаря на адрес.
- Memory-backed буферът е добър учебен модел, защото прави протокола видим без сложен външен модул.

Самопроверка

1. Каква е практическата разлика между SoftI2C и хардуерен I2C?
2. Какво означава бордът да работи като I2CTarget, а не като master?
3. Защо memory-backed буферът е удобен начин за първи target пример?

Упражнения

1. Създайте малък буфер от 8 байта и проследете какво съдържание би виждал master-ът.

Подсказка: `memory_buffer = bytearray(range(8))` — master чете `[0, 1, 2, ..., 7]` от адреса на target-a.

2. Променете `addr=0x42` на друга стойност и отбележете как това променя target идентичността.

Подсказка: `I2CTarget(1, addr=0x55, ...)` — `scan()` от master ще покаже `0x55` вместо `0x42`.

3. Сравнете в коментар кога бихте предпочели SoftI2C и кога хардуерен I2C.

Подсказка: `SoftI2C` е по-гъвкав за нестандартни пинове, хардуерен `I2C` е по-бърз и по-надежден при 400 kHz.

Ако не работи

- Симптом: няма отговор от шината. Проверка: първо сверете пиновете, адреса/ролята и нужните pull-up или control сигнали.
- Симптом: target-ът не отговаря. Проверка: вижте дали master и target са на една и съща шина и дали адресът съвпада с очаквания.
- Симптом: получавате нестабилни или безсмислени данни. Проверка: намалете скоростта и проверете окабеляването стъпка по стъпка.

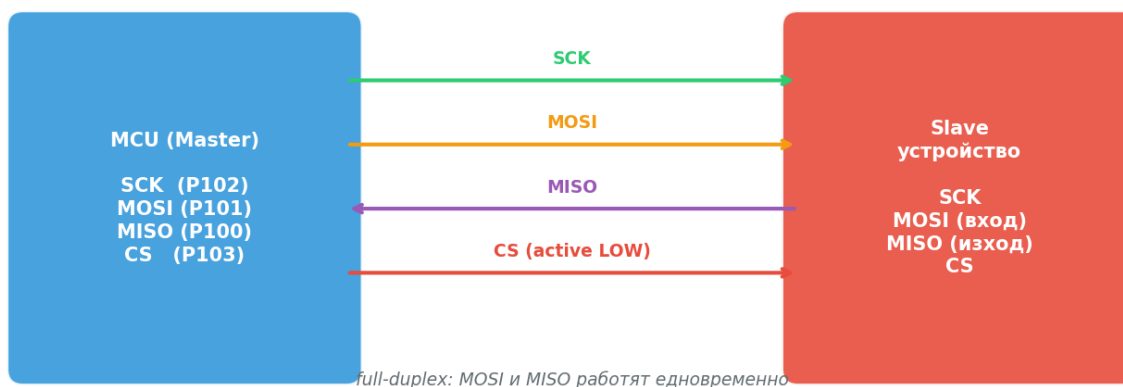
Нататък

След тази глава преминете към „Hardware SPI и SoftSPI“, за да разширите темата от I2C към втори важен сериен интерфейс.

Глава 24. Hardware SPI и SoftSPI [Среден]

[Тip] Аналогия: SPI е като двулентов автопат (MOSI/MISO) с тактов сигнал (SCK). За разлика от I2C, SPI е full-duplex — изпраща и получава едновременно. Всеки slave има отделен CS (Chip Select) пин — не адрес, а физическа линия.

SPI: 4-проводна full-duplex комуникация



SPI: 4-проводна full-duplex комуникация

	I2C	SPI
Жици	2 (SCL+SDA)	4+ (SCK+MOSI+MISO+CS)
Адресиране	7-bit адрес	CS пин
Duplex	Half	Full
Скорост	до ~400kHz	до ~50MHz
Употреба	Сензори, OLED	Flash, ADC, дисплеи

Hardware SPI:

```
from machine import SPI, Pin # Импортираме SPI и Pin за достъп до хардуерния SPI канал.

spi = SPI(0, baudrate=500000, polarity=0, phase=0, bits=8, sck=Pin("P102"), mosi=Pin("P101"), miso=Pin("P10...))
```

SoftSPI:

```
from machine import SoftSPI, Pin # Импортираме SoftSPI и Pin за софтуерен SPI върху GPIO.

soft_spi = SoftSPI(baudrate=100000, polarity=0, phase=0, sck=Pin("P105"), mosi=Pin("P104"), miso=Pin("P106"...))
```

В тази книга SPI остава 1 канал.

Файлове:

- examples/25_spi/01_spi_basic.py
- examples/25_spi/02_softspi_basic.py

Резюме

- SPI е по-прост по идея от I2C, но изисква да сте много ясни за линиите SCK, MOSI, MISO и CS.
- Хардуерният SPI е естественият избор за скорост и стабилност, а SoftSPI е удобен за експерименти и нестандартно окабеляване.
- При SPI режимът (polarity и phase) и физическото loopback или устройство често са първото място, където се появява грешка.

Самопроверка

1. Каква е ролята на MOSI, MISO, SCK и CS в един SPI обмен?
2. Кога е по-разумно да използвате SoftSPI, вместо хардуерен SPI?
3. Защо грешен polarity/phase режим често води до „има трафик, но данните са грешни“?

Упражнения

1. Направете loopback тест, ако setup-ът го позволява, и сравнете изпратените и получените байтове.
Подсказка: свържете `MOSI (P101)` → `MISO (P100)` с жица — изпратеното трябва да се върне непроменено.
2. Променете baudrate и вижте доколко обменът остава стабилен.
Подсказка: `SPI(0, baudrate=1000000, ...)` — намалете скоростта при нестабилни данни на по-дълго окабеляване.
3. Изпробвайте различен polarity/phase режим и наблюдавайте как се променя смисълът на данните.
Подсказка: `polarity=0, phase=0` е SPI Mode 0 — промяната на `phase=1` измества семплирането с половин такт.

Ако не работи

- Симптом: няма отговор от шината. Проверка: първо сверете пиновете, адреса/ролята и нужните pull-up или control сигнали.
- Симптом: няма смислен отговор при loopback или модул. Проверка: вижте дали MOSI и MISO не са разменени и дали CS се управлява правилно.
- Симптом: получавате нестабилни или безсмислени данни. Проверка: намалете скоростта и проверете окабеляването стъпка по стъпка.

Нататък

След тази глава преминете към „Базова UART комуникация“ ([Глава 25](#)), за да видите как SPI мисленето се пренася към точка-до-точка серийна връзка.

Глава 25. Базова UART комуникация [Среден]

[Tip] Аналогия: UART е директна телефонна линия между две устройства — без адреси, без master/slave. TX на едното → RX на другото. Двете страни трябва да са наредени на един и същ baud rate — иначе е като да говорите на различни езици.

RA4M2 (UART9)	Компютър / модул
P602 (TX) →	RX
P601 (RX) ←	TX
GND —	GND

[Pin] За VK_RA4M2: UART(9) с TX=P602, RX=P601 е препоръчителен — другите UART канали споделят пинове с I2C/SPI.

Пример:

```
from machine import UART # Импортираме UART класа за серийна комуникация.
import time # Импортираме time за кратко закъснение.

uart = UART(9, 115200) # Създаваме UART(9) на 115200 baud (TX=P602, RX=P601).
message = "Здравей от VK_RA4M2!\r\n" # Подготвяме съобщение за изпращане.
bytes_sent = uart.write(message) # Изпращаме съобщението по TX пина.

time.sleep_ms(100) # Кратко закъснение, за да пристигнат данни (ако има loopback).
available = uart.any() # Проверяваме дали има налични байтове за четене.
if available > 0: # Ако има данни за четене:
    received = uart.read(available) # Четем наличните байтове от RX буфера.
    print("Получено:", received) # Показваме получените данни.
```

Налични UART канали:

- UART(0) -> TX=P411, RX=P410
- UART(2) -> TX=P302, RX=P301
- UART(7) -> TX=P401, RX=P402, CTS=P403
- UART(9) -> TX=P602, RX=P601, CTS=P603

Тук се учат:

- създаване на UART инстанция
- изпращане и четене на данни
- loopback тест (TX свързан към RX)

Файлове:

- `examples/32_uart/01_uart_basic.py`
- `examples/uart_firmware_probe.py` — UART loopback firmware probe: текстов, бинарен и line-style roundtrip тест на UART(9) при 115200 baud (TX=P602, RX=P601) с pass/fail reporting

Резюме

- Главата въвежда UART като най-простия сериен канал за текст и байтове.
- Показани са базово изпращане, наличие на данни и четене от буфера.
- Loopback тестът е естественият първи sanity check преди по-сложна комуникация.

Самопроверка

1. Каква е ролята на baud rate при UART комуникацията?
2. Защо loopback тестът е добър първи тест на конфигурацията?
3. Каква е практическата разлика между `any()` и `read()`?

Упражнения

1. Изпращайте прост текстов брояч веднъж в секунда по UART.

Подсказка: `uart.write("Count: {}\n".format(i))` в `while True:` с `sleep_ms(1000)` и нарастващ `i`.`

2. Ако имате loopback, отпечатвайте обратно полученото съобщение като echo тест.

Подсказка: след `uart.write(msg)` изчакайте `sleep_ms(10)` и после `if uart.any(): print(uart.read())`.`

3. Добавете функция за изпращане на текстов ред с край `\r\n`.

Подсказка: `def send_line(uart, text): uart.write(text + "\r\n")` — стандартният ред-терминатор за UART терминали.`

Ако не работи

- Симптом: текстът е нечетим. Проверка: baud rate и настройките от двете страни трябва да съвпадат.
- Симптом: няма върнати данни. Проверка: при loopback TX и RX трябва да са свързани по очаквания за теста начин.

Нататък

След тази глава преминете към „Работа с външни модули по интерфейси“, за да надградите голото серийно предаване до реална интеграция на устройства.

Глава 26. Работа с външни модули по интерфейси [Среден]

Тази глава не въвежда нов хардуерен контролер.

[Pin] Бележка: Примерите тук работят с `bytearray`, `bytes`-литерали и бинарни пакети. Ако тези конструкции са нови за вас, вижте следващата Глава 27 за систематичното въведение — после се върнете тук.

Тя събира вече видените I2C, SPI и UART в един практичен модел за мислене.

Когато включвате външен модул, не започвате от „готов driver“, а от четири по-базови въпроса:

1. С какво напрежение и логически нива работи модулът?
2. Кой интерфейс използва: I2C, SPI или UART?
3. Има ли допълнителни control линии като CS, RST, INT, BUSY или EN?
4. Кой е най-малкият безопасен probe, който доказва, че връзката работи?

На VK_RA4M2 има три особено важни практични ограничения:

- I2C(0) дели P400 със SW1
- I2C(1) дели P100/P101 със hardware SPI
- UART(9) е най-удобният отделен канал, когато I2C или SPI вече са заети

Това значи, че изборът на интерфейс не е само логически, а и ресурсен.

Кога кой интерфейс е най-подходящ:

- I2C е удобен за сензори и по-бавни модули с адресирана обща шина
- SPI е удобен за по-бързи модули, при които имате отделен CS и frame-базиран обмен
- UART е удобен за модеми, GPS, Bluetooth и други модули с текстов или байтов поток

Практическият ред на работа почти винаги е един и същ:

1. Свържете GND и проверете напрежението
2. Проверете дали логическите нива на модула са съвместими с борда
3. Свържете сигналните линии на интерфейса
4. Свържете и нужните control линии
5. Стартирайте с минимална скорост и минимален probe
6. Едва след това четете идентификатор, статус или payload

Минимален I2C probe:

```
from machine import I2C # Импортираме I2C за базов probe на външна шина.

i2c = I2C(1, freq=100000) # Създаваме I2C(1) на безопасни 100 kHz.
print(i2c.scan()) # Сканираме шината за адреси на свързани I2C модули.
```

Минимална SPI транзакция:

```
from machine import SPI, Pin # Импортираме SPI и Pin за frame-базиран обмен с външен модул.

cs = Pin("P103", Pin.OUT, value=1) # Държим CS неактивен преди началото на транзакцията.
spi = SPI(0, baudrate=500000, polarity=0, phase=0, bits=8, sck=Pin("P102"), mosi=Pin("P101"), miso=Pin("P10...
cs.value(0) # Активираме target модула чрез CS.
spi.write_readinto(b"\x9F\x00\x00\x00", bytearray(4)) # Изпращаме команда и четем едновременно отговора.
cs.value(1) # Освобождаваме модула след края на frame-a.
```

Минимален UART request/response:

```
from machine import UART # Импортираме UART за сериен обмен с външен модул.

uart = UART(9, 115200) # Създаваме UART(9) с често срещана начална скорост.
uart.write(b"AT\r\n") # Изпращаме примерна текстова команда към модула.
print(uart.read()) # Четем каквото е пристигнало към този момент.
```

Общата идея е проста:

- I2C първо се доказва със `scan()`
- SPI първо се доказва с правилен CS и пълен frame
- UART първо се доказва с правилен baud rate, TX/RX и timeout

След това вече се минава към конкретния регистров модел или към конкретния текстов протокол на модула.

Тук много по-важен от „готовия код“ е datasheet-ът:

- за I2C ви трябва адрес и регистри
- за SPI ви трябва polarity, phase, CS логика и формат на frame-а
- за UART ви трябва baud rate, формат на кадъра и синтаксис на командите

Файлове:

- `examples/34_external_modules/01_i2c_module_probe.py`
- `examples/34_external_modules/02_spi_module_transaction.py`
- `examples/34_external_modules/03_uart_module_request_response.py`

Резюме

- Главата събира I2C, SPI и UART в единен practically-first подход за интеграция на външни модули.
- Фокусът е върху захранване, логически нива, control линии и най-малкия възможен probe.
- На VK_RA4M2 изборът на интерфейс е свързан и с конфликти между пинове и периферии.

Самопроверка

1. Защо при външен модул е грешно да започнете директно от голям driver, без минимален probe?
2. Кога бихте избрали SPI вместо I2C за външен модул?
3. Защо UART(9) често е най-удобният избор, когато вече ползвате I2C(1) или SPI?

Упражнения

1. Направете собствен checklist за свързване на нов външен модул с колони power, pins, protocol, probe.

Подсказка: ``power`` (3.3V/5V?), ``pins`` (SCL/SDA или TX/RX?), ``protocol`` (I2C/SPI/UART?), ``probe`` (`scan` или `AT\r\n`).

2. Разширете `01_i2c_module_probe.py`, така че да чете два различни регистъра, ако адресът е намерен.

Подсказка: след ``scan()`` ползвайте ``i2c.readfrom_mem(addr, reg, 1)`` за всеки от двата регистъра.

3. Добавете timeout и втори опит в `03_uart_module_request_response.py`, преди да обявите връзката за неуспешна.

Подсказка: обвийте ``uart.write()`` и ``uart.read()`` в ``for attempt in range(3):`` с ``sleep_ms(100)`` между опитите.

Ако не работи

- Симптом: модулът мълчи напълно. Проверка: първо вижте GND, захранването и логическите нива, не кода.
- Симптом: има отговор, но той е безсмислен. Проверка: вижте адреса, baud rate, SPI mode, формата на командата и datasheet-а.
- Симптом: модулът тръгва само понякога. Проверка: проверете RST, CS, BUSY, INT и ресурсните конфликти

между пиновете на борда.

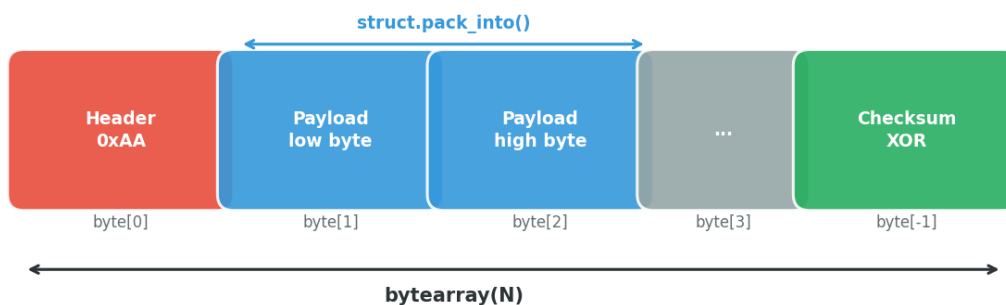
Нататък

След тази глава преминете към „Основи на asyncio“, за да видите как външен модул може да се обслужва неблокиращо и като част от по-голямо приложение.

Глава 27. Байтове, bytearray, struct, memoryview и двоични протоколи [Среден]

Много реални устройства не говорят с удобни Python обекти, а с поредици от байтове.

Двоичен кадър (frame): Header → Payload → Checksum



Двоичен кадър: Header → Payload → Checksum

Това е нормално при:

- UART request/response протоколи
- SPI framed транзакции
- I2C регистрови устройства
- собствени binary packet формати

Минипример:

```
import struct

frame = bytearray(6)
frame[0] = 0xAA
struct.pack_into("<H", frame, 1, 1023)
frame[3] = 7

checksum = 0
for b in memoryview(frame)[: -1]:
    checksum ^= b

frame[-1] = checksum
print(frame)
```

От този пример трябва да останат четири идеи:

- bytearray е променяем буфер от байтове
- struct дава точно binary представяне
- memoryview ви позволява да работите без излишни копия
- checksum или framing логиката трябва да е ясна и повторяема

Свързани файлове:

- `examples/20_language_basics/05_arrays.py`
- `examples/20_language_basics/07_pointers_equivalents.py`
- `examples/24_i2c/04_i2ctarget_memory.py`
- `examples/32_uart/01_uart_basic.py`
- `examples/34_external_modules/02_spi_module_transaction.py`
- `examples/34_external_modules/03_uart_module_request_response.py`

Резюме

- Реалните протоколи често работят с сурови байтове, не с удобни Python структури.
- `bytearray`, `struct` и `memoryview` са основният работен комплект за такива задачи.
- Добре описан пакет е по-важен от „хитър“ код.

Самопроверка

1. Каква е разликата между `bytes` и `bytearray`?
2. Защо `struct.pack_into()` е полезен при работа с фиксирани буфери?
3. Каква е ползата от `memoryview` при по-големи буфери?

Упражнения

1. Направете малък пакет с `header`, `payload length` и `checksum`.

Подсказка: `frame[0] = 0xAA` (header), frame[1] = payload_len`, struct.pack_into("<H", frame, 2, value)` за 16-bit.`

2. Декодирайте обратно пакет чрез `struct.unpack_from()`.

Подсказка: `val, = struct.unpack_from("<H", frame, 2)`` извлича 16-bit стойността от байт 2 на буфера.

3. Добавете прост XOR checksum към UART или SPI тестов пакет.

Подсказка: `cs = 0; for b in memoryview(frame)[-1]: cs ^= b`` — последният байт е `frame[-1] = cs``.

Ако не работи

- Симптом: числата изглеждат „обърнати“. Проверка: вижте дали `endian`-ът е правилен в `struct` формата.
- Симптом: буферът е с грешна дължина. Проверка: опишете кадъра на хартия `byte` по `byte`, преди да кодирате.

Нататък

След като пакетите са ясни, идва естественият въпрос как системата да се държи стабилно, когато комуникацията или периферията се провалят.

Част VI: Системни функции, памет и организация на проекта

Ключови термини: flash — енергонезависима памет (512 KB). RAM — оперативна памет (128 KB), тук живеят буферите и Python heap-ът. GC — автоматично освобождава неизползвана RAM; gc.collect(). boot.py / main.py — файлове, изпълнявани автоматично при старт. tetorview — поглед върху буфер без копиране.

Тази част събира системните теми: TouchPad, flash, управление на паметта, machine, startup и подредба на проекта.

Глава 28. TouchPad: четене, диагностика и cached API [Среден]

Пример:

```
from machine import Pin, TouchPad # Импортираме Pin и TouchPad за работа с CTSU вход.

touch = TouchPad(Pin("P205")) # Създаваме TouchPad върху валиден touch пин P205.
touch.config(500) # Задаваме праг 500 за метода value().
print(touch.read()) # Четем необработената CTSU стойност.
print(touch.value()) # Преобразуваме стойността към 0 или 1 според прага.
print(touch.read_value()) # Взимаме едновременно count, value и последна FSP грешка.
```

Тук се виждат:

- сурова стойност
- логическа стойност по праг
- комбинирано четене

Файл:

- examples/26_touchpad/01_touchpad_basic.py

Резюме

- TouchPad.read() връща сурова CTSU стойност, value() я сравнява с праг и връща 0 или 1, а read_value() дава и двете плюс FSP грешка.
- Прагът в config() е ключов — твърде нисък = фалшиви активации, твърде висок = пропуснати докосвания.
- CTSU е порт-специфичен хардуер на Renesas — стойностите зависят от платката, окабеляването и средата.

Самопроверка

1. Защо суровата стойност от read() варира между четенията, дори без докосване?
2. Каква е ролята на прага в config(500) и как бихте го калибрирали за конкретен touch pad?
3. Каква допълнителна информация дава read_value() спрямо read() + value() поотделно?

Упражнения

1. Прочетете read() 20 пъти в цикъл и изчислете средна стойност и разброс — това е базата за избор на праг.
Подсказка: `samples = [touch.read() for _ in range(20)]` след това `sum(samples)//20` за средна стойност.
2. Свържете value() с LED1: докосване светва, отпускане гаси.
Подсказка: `led.value(0 if touch.value() else 1)` в цикъл — `value()` връща 0 или 1 спрямо прага от `config()`.

3. Опитайте различни touch пинове (P205, P206, P408) и сравнете суровите стойности при докосване.

Подсказка: `TouchPad(Pin("P206")).read()` — различните пинове имат различна чувствителност поради различна капацитивност.

Ако не работи

- Симптом: `read()` винаги връща 0 или много ниска стойност. Проверка: уверете се, че пинът е валиден touch пин и че TSCAP (P207) е свързан правилно.

- Симптом: `value()` винаги е 1 или винаги е 0. Проверка: калибрирайте прага — първо прочетете суровата стойност без докосване и с докосване, после изберете праг между двете.

Диагностика, cached API и асинхронно следене

Базовото `read()` блокира докато CTSU завърши измерването. За по-гладка интеграция с `asyncio` ползваме `cached API`.

Диагностичен пример:

```
TouchPad.sample_rate(20) # Стартираме глобалния cooperative sampler на 20 пълни scan-a в секунда.
touch.start() # Стартираме non-blocking scan, ако не е в ход такъв.
TouchPad.service() # Придвижваме вътрешния sampler една стъпка от VM контекст.
print(touch.ready(), touch.read_cached(), touch.value_cached()) # Показваме cached състоянието.
print(touch.diagnose(8)) # Печатаме резултат от кратък диагностичен цикъл.
```

Асинхронен пример:

```
TouchPad.sample_rate(50) # Стартираме sampler на 50 scan-a в секунда.

async def touch_task(): # Асинхронна задача за следене на TouchPad.
    while True: # Безкрайно следене.
        if tp.ready(): # Проверяваме дали пробата е готова.
            print(tp.read_cached(), tp.value_cached(), tp.age_ms()) # Печатаме cached стойностите.
            await asyncio.sleep_ms(20) # Сканираме на 20 ms.
```

Тази глава е важна, защото показва не само „натиснато/ненатиснато“, а и сервисно поведение на CTSU слоя.

Файлове:

- `examples/26_touchpad/02_touchpad_diagnostics.py`
- `examples/04_touchpad_async.py`

Резюме

- `sample_rate()` стартира cooperative sampler, който управлява CTSU scan-овете автоматично — не е нужен ръчен `read()` на всеки тик.

- `read_cached()` и `value_cached()` връщат последната готова стойност без блокиране — идеални за `asyncio` задачи.

- `diagnose()` е диагностичен инструмент, който показва стабилността на четенията за кратък период — полезен при калибриране и окабеляване.

Самопроверка

1. Каква е разликата между `read()` (блокиращо) и `read_cached()` (non-blocking)?
2. Защо `sample_rate(50)` е по-подходящ за responsive UI, а `sample_rate(5)` — за енергоспестяване?
3. Какво показва `age_ms()` и кога стойността му е знак за проблем?

Упражнения

1. Стартирайте `sample_rate(20)` и логвайте `read_cached()` в asyncio задача на 50ms — наблюдавайте дали стойностите се обновяват навреме.

Подсказка: ``await asyncio.sleep_ms(50)`` между четенията — ``age_ms()`` под 100 ms означава навременно обновяване.

2. Сравнете `diagnose(8)` за два различни пина — кой е по-стабилен и защо?

Подсказка: ``touch.diagnose(8)`` отпечатава 8 последователни четения — по-малкият разброс = по-стабилен пин.

3. Направете asyncio задача, която превключва LED1 при докосване, ползвайки само `value_cached()`.

Подсказка: проследявайте прехода ``prev_val != touch.value_cached()`` и при промяна превключете ``led.value()``.

Ако не работи

- Симптом: `read_cached()` връща остаряла стойност (`age_ms()` расте). Проверка: уверете се, че `TouchPad.service()` се извиква редовно или че `sample_rate()` е стартиран.

- Симптом: `diagnose()` показва голям разброс. Проверка: touch pad-ът може да е шумен поради лошо окабеляване, липсващ TSCAP кондензатор или близък шумен източник.

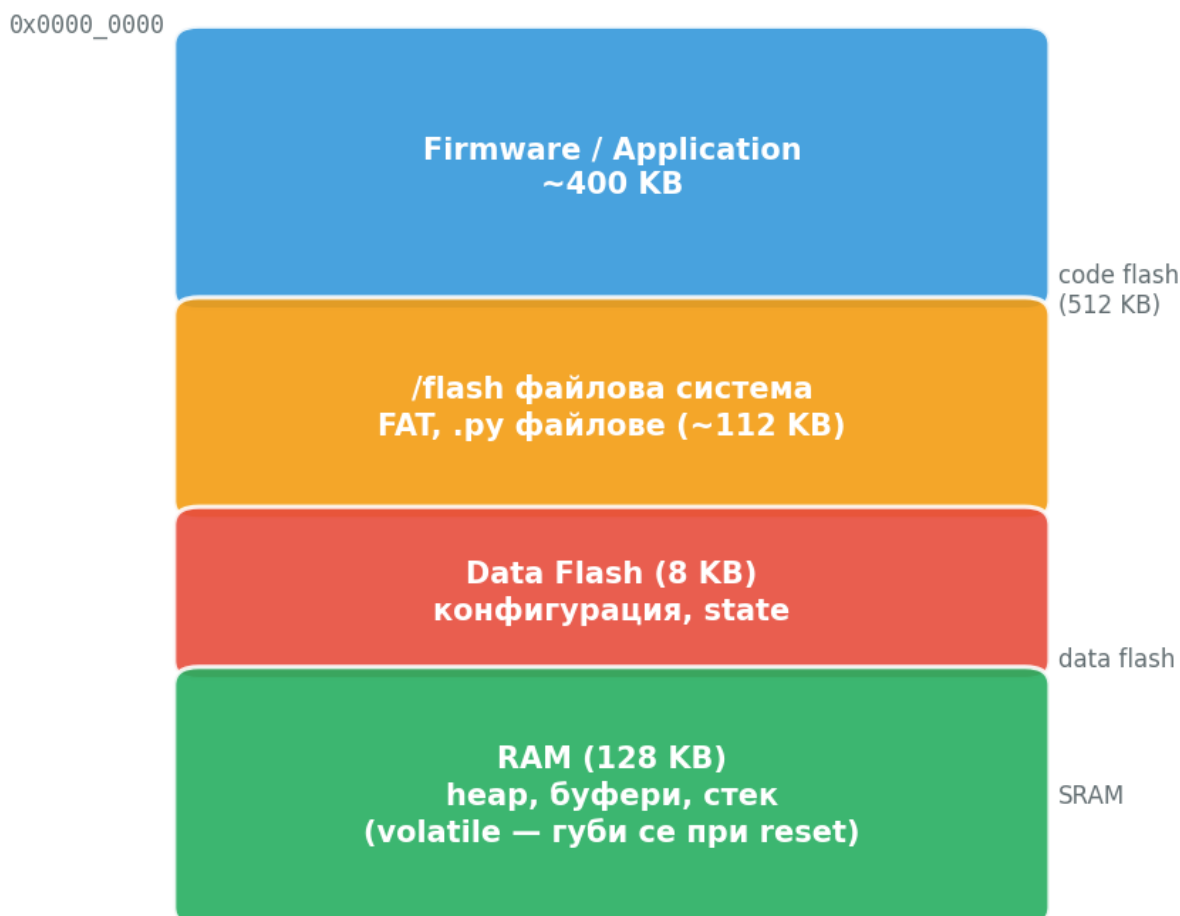
Нататък

След тази глава преминете към „renesas.Flash, /flash и dataflash“, за да видите как бордът пази данни и извън RAM.

Глава 29. renesas.Flash, /flash и dataflash [Среден]

[Тip] Аналогия: RAM е работното бюро — бързо, но при изключване всичко пада. Flash е архивното шкафче — по-бавно, но данните преживяват reset. dataflash е специален бележник (8KB) за конфигурация, независим от файловата система.

VK_RA4M2: Memory Map (RA4M2)



VK_RA4M2: Memory Map

Памет	Размер	Жива след reset?	Употреба
RAM (heap)	128 KB	[X]	Работни данни, буфери
/flash (code flash)	512 KB	[OK]	Програма, .py файлове
dataflash	8 KB	[OK]	Конфигурация, state

[!] **Важно:** Flash има ограничен брой write цикли (~100,000 за code flash). Не пишете при всеки тик на таймера — само при реална промяна на конфигурация.

Пример с renesas.Flash:

```
import os # Импортираме os, за да покажем съдържанието на файловата система /flash.
import renesas # Импортираме renesas модула, който излага Flash block device.

flash = renesas.Flash() # Създаваме block device обект върху вътрешната flash памет.
sector = bytearray(16) # Подготвяме малък буфер за четене на първите байтове от блок 0.
flash.readblocks(0, sector) # Четем първите 16 байта от блок 0 в буфера.
print(os.listdir("/flash")) # Показваме кои файлове има във вътрешната файлова система.
```

Пример с dataflash:

```
import dataflash # Импортираме dataflash модула за достъп до отделната Data Flash област.

print(dataflash.size()) # Печатаме общия размер на Data Flash областта.
print(dataflash.block_size()) # Печатаме размера на erase блока.
print(dataflash.write_size()) # Печатаме минималния размер за запис.
print(dataflash.read(0, 16)) # Четем първите 16 байта от Data Flash.
```

Пример за устойчиво състояние в dataflash и видима индикация с LED1:

```
from machine import Pin # Импортираме Pin за управление на LED1.
import dataflash # Импортираме dataflash за устойчиво съхранение на състояние.

led = Pin("LED1", Pin.OUT, value=1) # Използваме board alias-а LED1, започваме изгасен.
state = dataflash.read(0, 1)[0] # Връщаме стойността на първия байт.

if state == 0x00: # Ако предишният старт е записал 0x00, сега правим erase към 0xFF.
    dataflash.erase_block(0) # Изтриваме блок 0, всички байтове стават 0xFF.
    blinks_per_sec = 2 # Бавно мигане означава erase.
else: # Иначе приемаме, че байтът е 0xFF и записваме 0x00.
    dataflash.write(0, bytes([0x00])) # Записваме 0x00 в първия байт.
    blinks_per_sec = 10 # Бързо мигане означава write.
```

Този пример показва как един байт в dataflash може да пази състояние между два старта на програмата.

За VK_RA4M2 LED индикацията е върху LED1, защото провереният board LED е P204 и е active-low.

При всяко стартиране се редуват две състояния:

- 0xFF след erase_block(0)
- 0x00 след write(0, bytes([0x00]))

Скоростта на мигане показва кой преход е станал на текущия boot.

Тук се разграничават:

- файлова система
- block device
- отделен data flash регион

Проверени размери за текущата конфигурация на VK_RA4M2:

- MCU вариантът на тази платка е с общо 512 KB code flash и 128 KB RAM
- в текущия linker layout 290 KB са отделени за firmware/application region, а 94 KB са отделени като FLASH_FS за /flash
- отделно има 8 KB dataflash
- /flash е върху вътрешния flash block device, монтиран като FAT; block size е 512 bytes
- реално свободното място за файлове е малко по-малко от 94 KB, защото част отива за самата файлова система

Файлове:

- examples/27_storage/01_flash_blockdev.py
- examples/27_storage/02_dataflash_basic.py
- examples/27_storage/03_dataflash_state_led.py

Резюме

- RAM (128 KB) е бърза но `volatile`; `/flash` (94 KB) е файлова система за `.py` файлове; `dataflash` (8 KB) е отделна област за конфигурация.
- Flash има ограничен брой `write` цикли (~100,000) — пишете **само при реална промяна**, не при всеки тик.
- `renesas.Flash` е `block device`, `dataflash` е отделен модул — двете не се бъркат, въпреки че и двете са „flash”.

Самопроверка

1. Каква е практическата разлика между запис в `/flash` (файлова система) и запис в `dataflash` (директен `block access`)?
2. Защо не трябва да записвате в `flash` при всяка итерация на `main loop`?
3. Как примерът с LED индикацията доказва, че `dataflash` е `persistent` между `reset`-и?

Упражнения

1. Запишете брояч в `dataflash`, който се увеличава при всеки `boot` — след 5 рестарта ще видите стойност 5.
Подсказка: четете байт с `dataflash.read(0,1)[0]`, ако е `0xFF` го нулирайте, иначе прочетете и увеличете с 1.
2. Използвайте `os.statvfs("/flash")` за да проверите колко свободно място остава на файловата система.
*Подсказка: `f = os.statvfs("/flash"); free = f[0] f[3] - f[0]` е `block size`, `f[3]` е свободни блокове.**
3. Прочетете `dataflash.read(0, 64)` и отпечатайте байтовете като `hex` — наблюдавайте разликата между `0xFF` (erased) и записани данни.
Подсказка: `"".join("{:02X}".format(b) for b in dataflash.read(0, 64))` дава четим `hex` изход.

Ако не работи

- Симптом: `dataflash.write()` хвърля грешка. Проверка: можете да пишете само в `erase`-нат блок (всички байтове `0xFF`). Извикайте `erase_block()` преди запис.
- Симптом: файл на `/flash` изчезва след `reset`. Проверка: уверете се, че файлът е затворен (`f.close()`) преди `reset` — незатворен файл може да корумпира FAT таблицата.

Нататък

След тази глава преминете към „Управление на паметта в MicroPython“, за да разберете как работата с буфери и памет влияе на реалните приложения.

Глава 30. Управление на паметта в MicroPython [Среден]

[Tip] Аналогия: MicroPython heap е ограничено бюро (128KB RAM). GC (Garbage Collector) е чистачът — пуска се когато бюрото е пълно и изхвърля ненужни неща. В критичен код (DMA, аудио) чистачът не трябва да се намеси — затова `gc.disable()` преди критична секция.

RAM: [heap: 128KB]

GC освобождава тук при `gc.collect()`

`gc.mem_free()` байта оставащи

Команда	Действие
<code>gc.collect()</code>	Пусни GC веднага
<code>gc.mem_free()</code>	Свободни байти
<code>gc.mem_alloc()</code>	Заети байти
<code>gc.disable()</code>	Спри автоматичен GC
<code>micropython.mem_info()</code>	Детайлна карта

[Pin] Връзка: Практическото приложение на `gc.disable()` вече видяхте в Глава 14 (DAC playback). Тук разглеждаме принципа систематично.

Диагностичен пример:

```
import gc # Импортираме gc за контрол и измерване на garbage-collected heap-a.
import micropython # Импортираме micropython за mem_info и други runtime helper-и.

gc.collect() # Пускаме GC в предвидим момент, за да започнем от по-чисто състояние.
print(gc.mem_free(), gc.mem_alloc()) # Печатаме свободния и зетия heap.
micropython.mem_info() # Печатаме обобщена информация за heap-a.
```

Буферен пример:

```
buffer = bytearray(32) # Създаваме един буфер, който ще използваме.
window = memoryview(buffer)[8:12] # Вземаме прозорец към част от буфера без копиране (вж. Глава 8 за основ...
window[0] = 123 # Променяме първия байт през memoryview.
print(list(buffer[8:12])) # Показваме, че оригиналният буфер е променен на място.
```

Тук „управление на паметта“ не означава ръчно местене на адреси както в C, а по-скоро:

- да разбираме кои обекти заемат място в heap-a на MicroPython
- да различаваме RAM heap от stack, firmware и постоянна flash памет
- да ограничаваме излишните алокации в чувствителни по време участъци
- да използваме буфери вместо да създаваме нови обекти във всеки цикъл

Потвърдени факти за текущия VK_RA4M2 порт:

- garbage-collected heap-ът се подава на MicroPython чрез linker символите `_heap_start` и `_heap_end`
- linker-ът заделя отделен фиксиран stack регион от 8 KB
- `/flash` и `dataflash` са постоянна памет за файлове и данни, а не RAM heap за Python обекти

Практични API-та, които реално са налични в този порт:

- `gc.collect()` за принудително стартиране на garbage collector в удобен момент
- `gc.mem_free()` и `gc.mem_alloc()` за ориентир колко heap е свободен и колко е зает
- `gc.threshold()` за по-напреднал контрол кога да се задейства GC
- `micropython.mem_info()` и `micropython.mem_info(1)` за по-подробна картина на heap-a
- `micropython.alloc_emergency_exception_buf()` за изключително важни exception-и в interrupt/critical контекст
- `micropython.heap_lock()` и `micropython.heap_unlock()` за напреднали случаи, когато временно не трябва да се допуска алокация

Практически правила, които най-често дават реален ефект:

- правете `bytearray` веднъж и после го пълнете отново, вместо да създавате нов буфер всеки път
- използвайте `memoryview`, когато искате прозорец към част от буфера без копиране

- ако имате временни големи обекти, освобождавайте ги и извиквайте `gc.collect()` в предвидим момент, а не наред критична операция
- при interrupt-и, callbacks и latency-sensitive код избягвайте нови алокации; подгответе буферите предварително
- не смесвайте `machine.mem8/mem16/mem32` с управлението на Python heap-а; това са различни нива на достъп до паметта

Тази тема се свързва естествено с вече наличните примери:

- `07_pointers_equivalents.py` показва shared reference и memoryview
- `04_i2ctarget_memory.py` показва предварително заделен mutable буфер, подаден към периферен API
- `01_flash_blockdev.py` и `02_dataflash_basic.py` показват, че persistent storage не е същото като heap RAM

Файлове:

- `examples/28_machine_misc/04_memory_management.py`
- `examples/20_language_basics/07_pointers_equivalents.py`
- `examples/24_i2c/04_i2ctarget_memory.py`
- `examples/27_storage/01_flash_blockdev.py`
- `examples/27_storage/02_dataflash_basic.py`

Резюме

- Главата развива темата „Управление на паметта в MicroPython“ около по-чувствителни, по-системни или порт-специфични възможности.
- Тук вече не е достатъчно само да извикате API-то; важно е да разбирате ограничения и режима на работа.
- Това е зоната, в която практическата дисциплина става толкова важна, колкото и самият пример.

Самопроверка

1. Коя е най-важната особеност или ограничение в „Управление на паметта в MicroPython“?
2. Кой ресурс е volatile и кой е persistent в този тип пример?
3. Как бихте измерили или наблюдавали дали примерът се държи стабилно?

Упражнения

1. Добавете диагностичен печат или проста метрика, която да прави състоянието по-видимо.

Подсказка: ``print(gc.mem_free(), gc.mem_alloc())`` преди и след голяма алокация показва ефекта върху heap.

2. Променете един ключов праг, буфер или режим и сравнете поведението преди и след това.

Подсказка: сравнете ``list`` срещу ``array("H", ...)`` за 100 елемента — ``micropython.mem_info()`` показва разликата.

3. Свържете темата с реално действие на LED1, SW1 или малък лог в паметта, ако това е приложимо.

Подсказка: извикайте ``gc.collect()`` и след него ``gc.mem_free()`` — покажете резултата с кратко мигане на LED1.

Ако не работи

- Симптом: поведението е чувствително или шумно. Проверка: търсете грешни прагове, неподходящ режим или неправилни очаквания към хардуера/порта.
- Симптом: резултатът не съвпада с интуицията. Проверка: разграничете RAM, постоянна памет, кеширане и порт-специфични особености.

Нататък

След тази глава преминете към „machine модулът и порт-специфично поведение“, за да слезете още едно ниво по-близо до самия порт и хардуер.

Глава 31. machine модулът и порт-специфично поведение [Среден]

Информационен пример:

```
import machine # Импортираме machine, за да покажем системните функции на порта.

uid = machine.unique_id() # Четем уникалния 128-битов идентификатор на MCU.
uid_hex = "".join("{:02x}".format(byte_value) for byte_value in uid) # Преобразуваме байтовете до четим hex...

print("machine.freq() =", machine.freq()) # Показваме текущата честота на ядрото.
print("machine.unique_id() =", uid_hex) # Печатаме уникалния идентификатор в hex вид.
print("machine.reset_cause() =", machine.reset_cause()) # Печатаме причината за последния reset.
machine.info() # Печатаме подробна системна информация от самия порт.
```

Ниско ниво:

```
irq_state = machine.disable_irq() # Временно забраняваме прекъсванията и пазим предишното състояние.
machine.enable_irq(irq_state) # Възстановяваме прекъсванията веднага след демонстрацията.
print(hasattr(machine, "mem32")) # Показваме дали портът излага 32-битов memory access.
```

Sleep режими:

```
DO_LIGHTSLEEP = False # Оставяме lightsleep изключен по подразбиране.
DO_DEEPSLEEP = False # Оставяме deepsleep изключен по подразбиране.

print("Извикваме кратък machine.lightsleep(20).") # Подготвяме потребителя за кратък sleep.
machine.lightsleep(20) # Извикваме кратък lightsleep за 20 ms.

if DO_LIGHTSLEEP: # Само ако е включено, пробваме по-дълъг lightsleep.
    machine.lightsleep(100) # Извикваме lightsleep за 100 ms.
```

Тази глава е мястото за:

- machine.info
- machine.freq
- machine.unique_id
- machine.reset_cause
- disable_irq и enable_irq
- mem8, mem16, mem32
- предпазен machine.bootloader
- sleep, lightsleep, deepsleep

Практическа бележка за честоти и ток:

- machine.freq() е полезен за бърза проверка на runtime честотата; текущата board конфигурация е 100 MHz системен clock и 50 MHz периферни clock-и за много блокове
- токовите числа в hardware manual-а са за самия MCU при конкретни условия, а не за цялата платка през

USB

- типични стойности от manual-а са около 15.4 mA в normal mode с всички peripheral clocks enabled, 6.1 mA при disabled peripheral clocks и 4.4 mA в sleep mode
- за ниска консумация manual-ът дава ориентир от 0.7 uA за software standby и приблизително 5 ... 16 uA за deep software standby според retained функциите
- по време на 12-bit A/D conversion analog current е типично около 0.8 mA, което е отделна аналогова консумация и не бива да се бърка с общата консумация на платката

Файлове:

- examples/28_machine_misc/01_machine_info_and_id.py
- examples/28_machine_misc/02_machine_mem_and_bootloader.py
- examples/28_machine_misc/03_sleep_modes.py
- examples/23_timing/18_clock_source_proof.py — чете SYSTEM/PLL/OSCSF хардуерни регистри за доказване на clock source (вж. и [Глава 15](#))
- examples/encoder_ok_polling.py — async polling loop за machine.Encoder (X1 mode, filter=4), печата позиция и делта
- examples/test_encoder.py — диагностичен тест за machine.Encoder: ISR counter, ThreadSafeFlag, GTCNT/GTCCRA/GTCCRB/GTST регистри

Резюме

- Тук machine вече не е просто помощен модул, а мястото, където се вижда самоличността и ниското ниво на самия порт.
- machine.info(), freq(), unique_id() и reset_cause() са полезни за диагностика, инвентаризация и проверка на runtime състоянието.
- disable_irq(), mem32, bootloader, lightsleep() и deepsleep() изискват повече дисциплина, защото работят близо до системното поведение на борда.
- Предпазните флагове в примерите не са излишни, а начин да упражнявате чувствителни API-та без излишен риск.

Самопроверка

1. Кога бихте ползвали machine.info() и кога machine.unique_id()?
2. Защо след disable_irq() е важно да възстановите точно предишното състояние с enable_irq(irq_state)?
3. Защо DO_LIGHTSLEEP, DO_DEEPSLEEP и подобни флагове са добра учебна практика при чувствителни операции?

Упражнения

1. Направете кратък „паспорт на борда“, който печата freq, unique_id, reset_cause и няколко реда от machine.info().

Подсказка: `unique_id` с `"".join("{:02x}".format(b) for b in machine.unique_id())`.`

2. Измерете реалното време около machine.lightsleep(20) с time.ticks_ms() и сравнете резултата с подадената стойност.

Подсказка: `t0 = time.ticks_ms(); machine.lightsleep(20); print(time.ticks_diff(time.ticks_ms(), t0))` — обикновено е малко повече от 20.`

3. Оставете рискована операция като bootloader() или по-дълбок sleep зад булев флаг и тествайте първо само безопасния клон.

Подсказка: `DO_BOOTLOADER = False` — само при `True` извикайте `machine.bootloader()`, за да избегнете`

случайно влизане.

Ако не работи

- Симптом: бордът изглежда „замръзва“ или сменя режима неочаквано. Проверка: вижте дали не сте включили bootloader, deepsleep или по-агресивен sleep сценарий.
- Симптом: low-level частта дава странни резултати. Проверка: разграничете нормален Python обект от директен достъп през mem8/mem16/mem32 и проверете дали наистина ви трябва ниско ниво.
- Симптом: след експеримент с прекъсванията поведението става непредвидимо. Проверка: уверете се, че enable_irq() възстановява предишното състояние, а не произволна стойност.

Нататък

След тази глава преминете към „Динамична 7-сегментна индикация и матрично сканиране“, където системното мислене за timing и ресурси вече се превръща в реална периферна организация.

Глава 32. boot.py, main.py и жизнен цикъл на програмата [Среден]

Една от най-големите разлики между „пускам пример“ и „имам устройство“ е жизненият цикъл на програмата.

На практика трябва да си отговорите:

- какво се случва веднага след reset
- кой код е безопасен за много ранно стартиране
- кой код може да блокира
- къде се създават драйверите
- къде се стартира основната логика

Полезен минимален модел е:

```
# boot.py
from machine import Pin

status_led = Pin("LED1", Pin.OUT, value=1)

def early_init():
    status_led.value(0)
    status_led.value(1)

early_init()
```

```
# main.py
import app

app.run()
```

```
# app.py
def run():
    print("Стартира приложението.")
    # Тук вече може да се създадат обекти, задачи и основен loop.
```

Практическото правило е:

- `boot.py` прави само ранна и безопасна инициализация
- `main.py` е тънка входна точка
- реалното приложение живее в отделни модули

Това разделение е важно, защото ако сложите твърде много логика в `boot.py`, диагностицирането става трудно, а `recovery` сценарият става по-рисков.

Свързани файлове:

- `examples/index.py`
- `examples/28_machine_misc/01_machine_info_and_id.py`
- `examples/28_machine_misc/03_sleep_modes.py`

Резюме

- Жизненият цикъл на програмата трябва да е предвидим още от `reset-a`.
- `boot.py` е за ранна инициализация, а не за голяма бизнес логика.
- `main.py` трябва да е проста входна точка към реалните модули на приложението.

Самопроверка

1. Защо не е добра идея целият проект да бъде написан директно в `boot.py`?
2. Какво печелите, ако `main.py` само извика `app.run()`?
3. Кои операции са безопасни рано при стартиране и кои е по-добре да се отложат?

Упражнения

1. Разделете един от по-дългите ви примери на `boot.py`, `main.py` и `app.py`.

Подсказка: в `'boot.py'` само инициализирайте пиновете, в `'main.py'` — `'import app; app.run()'`, а логиката е в `'app.py'`.

2. Добавете проста ранна индикация с `LED1`, която показва, че началната инициализация е приключила.

Подсказка: в края на `'boot.py'` мигнете `LED1` веднъж — `'led.value(0); sleep_ms(100); led.value(1)'`.

3. Направете вариант, в който `main.py` първо печата кратък banner с версия и ID на устройството.

Подсказка: `'print("VK_RA4M2 v1.0 ID:", "".join("{:02x}".format(b) for b in machine.unique_id()))'`.

Ако не работи

- Симптом: бордът влиза в цикъл от рестартирания. Проверка: преместете тежката логика от `boot.py` към `main.py` или `app.py`.
- Симптом: не е ясно къде се чупи програмата. Проверка: добавете кратки диагностични `print()` стъпки в ранния startup.

Нататък

След тази глава вече има смисъл да подредите проекта по файлове и роли, вместо да държите всичко в един дълъг скрипт.

Глава 33. Организация на проект: модули, drivers, config и pins.py [Среден]

След като има ясен жизнен цикъл, следващият въпрос е как да подредите кода така, че да не се превърне в дълъг и крехък файл.

Полезна малка структура е:

```
/flash/  
  boot.py  
  main.py  
  app.py  
  config.py  
  pins.py  
  drivers/  
  services/
```

Идеята е проста:

- pins.py описва board mapping-a
- config.py пази настройки и константи
- drivers/ държи ниското ниво
- services/ държи по-високото поведение
- app.py свързва всичко

Минипример:

```
# pins.py  
LED_STATUS = "LED1"  
BUTTON_OK = "P400"  
DAC_OUT = "P014"
```

```
# config.py  
BUTTON_SAMPLE_MS = 10  
AUDIO_WAVE = "pulse25"
```

```
# app.py  
from machine import Pin  
import pins  
import config  
  
status_led = Pin(pins.LED_STATUS, Pin.OUT, value=1)  
print("Button sample period =", config.BUTTON_SAMPLE_MS)
```

Така пиновете и настройките вече не са „магически числа“, разпръснати по целия проект.

Свързани файлове:

- examples/34_external_modules/01_i2c_module_probe.py
- examples/retro_synth.py

Резюме

- Подреденият проект разделя board mapping, настройки, драйвери и логика.
- pins.py и config.py намаляват риска от скрити зависимости.
- Това е най-лесният преход от демо файл към поддържим firmware проект.

Самопроверка

1. Каква е ролята на pins.py и защо е по-добре от разпръснати pin низове из проекта?
2. Какво трябва да влиза в drivers/ и какво в services/?
3. Защо е полезно app.py да сглобява системата, вместо всичко да се създава навсякъде?

Упражнения

1. Извадете pin имената от един ваш пример в отделен pins.py.

Подсказка: `LED_STATUS = "LED1"` в `pins.py`, после `from pins import LED_STATUS` — само едно място за промяна.

2. Извадете тайминги и прагове в отделен config.py.

Подсказка: `DEBOUNCE_MS = 40` и `BLINK_PERIOD_MS = 500` в `config.py` — импортирайте с `from config import *`

3. Разделете код, който говори директно с хардуер, от код, който взема решения.

Подсказка: driver функция `led_on()` скрива `pin.value(0)`, а application код само извиква `led_on()` — без да знае за пина.

Ако не работи

- Симптом: след смяна на пин нищо не тръгва. Проверка: вижте дали всички места вече ползват pins.py, а не старо твърдо кодирано име.
- Симптом: проектът има дублирани стойности. Проверка: съберете period-и, прагове и pin mapping на едно място.

Нататък

Когато кодът вече е подреден, следващата голяма стъпка е да мислите и в двоични данни, а не само в прости цели числа и низове.

Глава 34. Таблични данни, конфигурации и малки файлови формати [Среден]

Не всяка промяна в проекта трябва да изисква редакция на Python сорса.

Често е по-добре да отделите:

- настройки
- прагове
- preset-и
- мелодии
- таблици за състояния

Прост и полезен формат е JSON:

```
import json

cfg = {
    "button_sample_ms": 10,
    "audio_wave": "triangle",
    "melody_tempo": 120,
}

with open("/flash/config.json", "w") as f:
    json.dump(cfg, f)
```

Това е особено удобно, когато:

- искате различни режими без промяна на кода
- пазите калибрационни стойности
- съхранявате малки preset-и за звук или светлина

Свързани файлове:

- examples/27_storage/01_flash_blockdev.py
- examples/27_storage/02_dataflash_basic.py
- examples/27_storage/03_dataflash_state_led.py
- examples/retro_synth.py

Резюме

- Конфигурацията трябва да е отделена от логиката, когато има шанс да се променя.
- Малки файлови формати като JSON са удобни за настройки и preset-и.
- Табличният подход прави проектите по-гъвкави и по-лесни за поддръжка.

Самопроверка

1. Кои стойности във ваш проект са конфигурация, а не логика?
2. Кога е по-добре да ползвате файл вместо твърдо кодирана константа?
3. Какви данни не е разумно да пазите в прекалено сложен формат на такъв малък проект?

Упражнения

1. Изнесете прагове, delay-и или имена на пинове в JSON файл.

Подсказка: ``import json; f = open("config.json", "w"); json.dump({"delay_ms": 200}, f); f.close()``.

2. Направете два различни preset-а и превключване между тях.

Подсказка: ``presets = {"slow": {"delay": 500}, "fast": {"delay": 100}}; cfg = presets["fast"]``.

3. Запишете калибрационни стойности във flash и ги зареждайте при старт.

Подсказка: при липсващ файл ползвайте ``try/except OSError`` и заредете стойности по подразбиране.

Ако не работи

- Симптом: програмата пада при четене на config. Проверка: добавете разумни стойности по подразбиране при липсващ или повреден файл.
- Симптом: config файлът расте неконтролируемо. Проверка: разделете настройки, telemetry и user data.

Нататък

Остава финалната интеграционна глава: как от поредица примери се стига до завършено приложение.

Част VII: Сигнали, измерване, енергия и индикация

Ключови термини: moving average — филтър, осредняващ последните N стойности; намалява шум. hysteresis — два прага (горен/долен) за стабилно превключване. jitter — вариации в точния момент на периодично събитие. WS2812 — адресируем RGB LED, GRB формат, данни по 1 проводник. deepsleep — режим с минимална консумация.

След като проектът вече общува с реален хардуер, идват филтрирането, измерването, предпазливата работа с low-power режими и визуалните изходи — от класическа 7-сегментна индикация до програмируеми RGB ленти.

Глава 35. Филтриране, калибриране и хистерезис при входни сигнали [Напреднал]

Сензорният вход почти никога не е идеален.

Дори когато `ADC.read()` работи коректно, ще видите:

- шум
- кратки отклонения
- температурен дрейф
- нестабилност около прагове

Полезен модел е комбинация от:

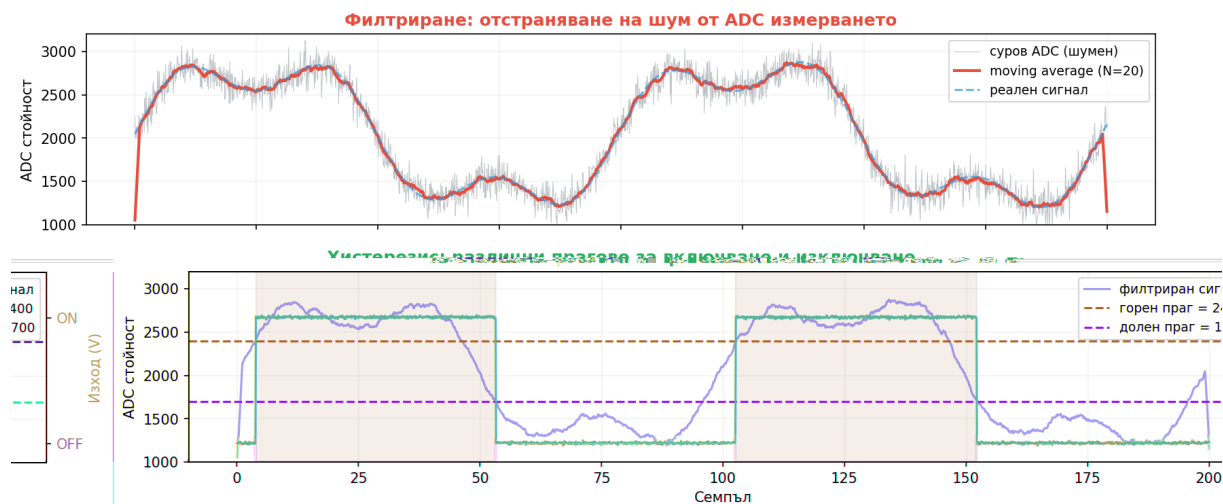
- просто филтриране
- калибрация
- хистерезис

Минипример:

```
def moving_average(samples):
    return sum(samples) // len(samples)

def hysteresis_state(value, state, low_th=1800, high_th=2200):
    if state == 0 and value >= high_th:
        return 1
    if state == 1 and value <= low_th:
        return 0
    return state
```

Глава 35: Филтриране, калибриране и хистерезис



Филтриране, калибриране и хистерезис

Идеята е:

- филтърът намалява шума
- калибрацията превежда raw стойности към полезен диапазон
- хистерезисът спира „тракането“ около границата

Свързани файлове:

- examples/22_analog/01_analog_input_adc.py
- examples/23_timing/07_adc_timing_measure.py
- examples/26_touchpad/01_touchpad_basic.py
- examples/26_touchpad/02_touchpad_diagnostics.py

Резюме

- Чистият входен сигнал рядко идва „готов“ от хардуера.
- Филтрирането, калибрацията и хистерезисът са нормална част от реалното приложение.
- Стабилното решение е по-ценно от максимално чувствителното.

Самопроверка

1. Защо един праг без хистерезис често води до нестабилно поведение?
2. Каква е ролята на moving average филтъра?
3. Каква е разликата между raw стойност и калибрирана стойност?

Упражнения

1. Добавете moving average върху ADC пример.

Подсказка: поддържайте `samples = [adc.read() for _ in range(N)]` и заменяйте най-стария елемент — `samples.pop(0); samples.append(new)`.

2. Добавете двоен праг и сравнете поведението с единичен праг.

Подсказка: ползвайте функцията `hysteresis_state(value, state, low_th=1800, high_th=2200)` от примера.

3. Запишете минимална и максимална измерена стойност и направете проста калибрация към 0..100%.

Подсказка: `pct = (raw - raw_min) * 100 // (raw_max - raw_min)` след определяне на `raw_min` и `raw_max` при старт.*

Ако не работи

- Симптом: стойността „прескача“. Проверка: вижте дали window-ът за филтър не е прекалено малък.
- Симптом: реакцията е прекалено бавна. Проверка: филтърът може да е прекалено тежък за тази задача.

Нататък

Следващата стъпка е да измервате не само логиката, но и реалния електрически сигнал навън.

Глава 36. Измерване и диагностика с осцилоскоп и logic analyzer [Напреднал]

Когато една периферия „уж работи“, но резултатът е съмнителен, софтуерният `print()` често не е достатъчен.

Тогава идват:

- осцилоскопът
- logic analyzer-ът
- измерванията с `ticks_us()`

Един много полезен навик е да отбелязвате важни моменти с `debug pin`:

```
from machine import Pin

probe = Pin("P204", Pin.OUT, value=1)

def mark():
    probe.value(0)
    probe.value(1)
```

Така можете да видите:

- кога влиза callback
- колко често идва периодично събитие
- има ли jitter
- съвпада ли очакваният timing с реалния сигнал

Тази глава е особено важна за:

- PWM
- UART
- SPI
- I2C
- DAC
- debounce и button timing

Свързани файлове:

- `examples/23_timing/07_adc_timing_measure.py`
- `examples/23_timing/08_interrupt_latency_measure.py`
- `examples/23_timing/13_hardware_timer_periodic_oneshot.py`
- `examples/32_uart/01_uart_basic.py`
- `examples/dac_firmware_probe.py`

- `examples/dac_dmac_diag.py`

Резюме

- Истинската диагностика често изисква измерване на пина, не само четене на лог.
- `Debug pin` и `ticks_us()` са мощна комбинация за базова инструментализация.
- Осцилоскопът и `logic analyzer`-ът намаляват догадките и ускоряват `root cause` анализа.

Самопроверка

1. Кога `print()` е недостатъчен за `timing` проблем?
2. Какво печелите от кратък `debug pulse` върху отделен `pin`?
3. Кога е по-подходящ осцилоскоп и кога `logic analyzer`?

Упражнения

1. Добавете `debug pin marker` към един `timer callback`.

Подсказка: `probe.value(0); probe.value(1)` в началото на `callback`-а — осцилоскопът ще покаже кратък `HIGH` импулс.

2. Измерете период и `jitter` на периодичен таймер.

Подсказка: запишете `time.ticks_us()` при всяко сработване и отпечатайте разликата — жито показва `jitter`.

3. Наблюдавайте един `UART frame` или `DAC waveform` на реален инструмент.

Подсказка: свържете `P602 (TX)` към `logic analyzer` — при `115200 baud` всеки бит е около `8.7 μs` широк.

Ако не работи

- Симптом: измерването обърква самия експеримент. Проверка: не печатайте тежки съобщения вътре в чувствителен `callback`.
- Симптом: виждате шумен или странен сигнал. Проверка: проверете общата маса, `probe` настройката и `bandwidth limit`-а на инструмента.

Нататък

След като умеете да измервате и диагностицирате, идва въпросът как да спестите енергия, без да загубите контрол над системата.

Глава 37. Ниска консумация и събуждане [Напреднал]

При много бордови приложения системата не трябва да работи „на пълна газ“ непрекъснато.

Полезната идея е:

- работете кратко и целенасочено
- спете, когато няма работа
- събуждайте се по време или по събитие

Минипример:

```
import machine
import time

print("Подготвяме работа.")
time.sleep_ms(20)
machine.lightsleep(50)
print("Събудихме се.")
```

Практическият ред на мислене е:

- първо овладейте кратък и безопасен `lightsleep`
- после измерете какво се запазва и какво не
- едва след това експериментирайте по-смело с по-дълбоки режими

За `VK_RA4M2` това трябва да се прави предпазливо, защото `lightsleep` и `deepsleep` вече са показани в книгата като чувствителни за допълнителна хардуерна валидация.

Свързани файлове:

- `examples/28_machine_misc/03_sleep_modes.py`
- `examples/23_timing/06_rtc_basic.py`
- `examples/30_fsm/08_tickless_sleep.py`

Резюме

- Ниската консумация е въпрос на архитектура, не само на една функция.
- Започвайте с кратки, контролирани `lightsleep` експерименти.
- Преди по-дълбок сън винаги проверявайте кои ресурси и състояния оцеляват след wake-up.

Самопроверка

1. Защо е по-разумно да започнете с кратък `lightsleep`, а не директно с агресивен `deepsleep`?
2. Каква е ролята на `RTC` в `power-aware` архитектурата?
3. Кои части от приложението трябва да умеят да се инициализират отново след събуждане?

Упражнения

1. Измерете реално колко време трае кратък `lightsleep`.

Подсказка: `t=time.ticks_ms(); machine.lightsleep(50); print(time.ticks_diff(time.ticks_ms(), t))` — сравнете с 50 ms.`

2. Направете приложение, което работи на интервали и спи между тях.

Подсказка: ``do_work(); machine.lightsleep(1000)` в цикъл — `lightsleep(1000)` заменя `sleep_ms(1000)` с по-ниска консумация.`

3. Опишете кои обекти във вашия проект трябва да се създават наново след `reset` или `deep sleep`.

Подсказка: *всичко в RAM (Pin, Timer, I2C) губи контекст след `deepsleep` — само `dataflash` оцелява.*

Ако не работи

- Симптом: след wake-up поведението е различно от очакваното. Проверка: вижте кои обекти разчитат на старо състояние.
- Симптом: режимът е нестабилен. Проверка: върнете се към по-кратък и по-безопасен `lightsleep` сценарий.

Нататък

След тази част вече има добра основа да мислите не само за отделни ресурси, а за цялото приложение като наблюдаема система.

Глава 38. Динамична 7-сегментна индикация и матрично сканиране [Напреднал]

Пример:

```
from machine import Pin # Импортираме Pin, за да покажем базовото сканиране на външен дисплей и бутони.
import time # Импортираме time за краткото динамично опресняване.

segment_pins = [Pin(name, Pin.OUT) for name in ("P105", "P104", "P113", "P114", "P115", "P304", "P303")] #...
digit_pins = [Pin(name, Pin.OUT) for name in ("P608", "P409")] # Това са двата общи извода за две цифри.
patterns = {"1": (0, 1, 1, 0, 0, 0, 0), "2": (1, 1, 0, 1, 1, 0, 1)} # Това са сегментните шаблони за две п...

for refresh_step in range(10): # Повтаряме кратък опреснителен цикъл десет пъти.
    for digit_index, symbol in enumerate(("1", "2")): # Показваме символ 1 на първата цифра и 2 на втората...
        for digit_pin in digit_pins: # Първо изключваме всички цифри, за да няма ghosting.
            digit_pin.off() # Изключваме текущата цифра.
        for segment_pin, level in zip(segment_pins, patterns[symbol]): # Зареждаме нужния сегментен шаблон...
            segment_pin.value(level) # Настройваме сегмента според шаблона.
        digit_pins[digit_index].on() # Включваме само текущата цифра за кратко време.
        time.sleep_ms(5) # Държим цифрата включена кратко, за да се получи динамична индикация.
```

Клавиатурна матрица:

```
row_pins = [Pin(name, Pin.OUT) for name in ("P408", "P600")] # Това са редовете на примерна 2x2 клавиатурн...
col_pins = [Pin(name, Pin.IN, Pin.PULL_UP) for name in ("P301", "P302")] # Това са колоните на примерната ...
pressed_keys = [] # Подготвяме списък за натиснати клавиши от матрицата.

for row_index, row_pin in enumerate(row_pins): # Сканираме всеки ред поотделно.
    for reset_row in row_pins: # Първо изключваме всички редове.
        reset_row.off() # Изключваме реда.
    row_pin.on() # Активираме текущия ред.
    for col_index, col_pin in enumerate(col_pins): # Проверяваме всяка колона в активния ред.
        if col_pin.value() == 0: # Ако колоната падне на 0, приемаме че бутонът е натиснат.
            pressed_keys.append((row_index, col_index)) # Записваме координатите на натиснатия бутон.
```

Този пример изисква външен хардуер, но е много важен за свързване на GPIO, време и логика.

Файл:

- `examples/29_display_scan/01_dynamic_7seg_and_keypad.py`

Резюме

- Главата комбинира GPIO, време и външен хардуер в една по-сложна, но много показателна демонстрация.
- Основната идея е, че мултиплексирането и матричното сканиране разчитат на правилен ред и темпо на опресняване.
- Това е добър пример как софтуерното време пряко влияе на видимия хардуерен резултат.

Самопроверка

1. Защо динамичната индикация не работи добре при твърде бавно опресняване?
2. Каква е идеята зад матричното сканиране на клавиши?
3. Кой проблем ще се появи първо, ако редът на управление на сегменти и цифри е грешен?

Упражнения

1. Променете честотата на опресняване и наблюдавайте кога започва видимо мигане.

Подсказка: намалете `sleep_ms(5)` до `sleep_ms(20)` — при над 20 ms на цифра мигането ще стане видимо.

2. Добавете проста таблица за превод от координати на матрицата към символи.

Подсказка: `keymap = {(0,0): "1", (0,1): "2", (1,0): "3", (1,1): "4"}` и `keymap.get((row, col), "?")`.

3. Използвайте LED1 като диагностичен индикатор при намерен натиснат клавиш.

Подсказка: `led.value(0 if pressed_keys else 1)` след матричното сканиране — светва при всяко открито натискане.

Ако не работи

- Симптом: дисплеят е слаб или премигва. Проверка: периодът на опресняване вероятно е твърде дълъг или кодът блокира между циклите.
- Симптом: разчитането на клавишите е грешно. Проверка: прегледайте съответствието между редове, колони и реалното окабеляване.

Нататък

След тази глава преминете към „Минимална FSM и таблична FSM“, за да превърнете входовете, изходите и времето в по-структурирана логика.

Глава 39. NeoPixel [Напреднал]

NeoPixel е удобен пример за „умен“ външен RGB светодиоден модул.

Той не се управлява като обикновен RGB LED с три PWM канала.

Вместо това се използва един data проводник с точно времедиаграмиране, което MicroPython крие зад модула `neopixel`.

В този курс използваме:

- P500 като задължителен enable pin за модула в този setup
- P112 като DIN data линия
- 1 пиксел за най-простия тест

Това е удобно за демонстрация, но има важно уточнение:

- P500 трябва да бъде включен задължително в този setup
- при по-дълга лента или по-голям ток може да е нужно отделно основно захранване
- при отделно захранване масата пак трябва да е обща с борда

Ключовата идея е, че работата с NeoPixel минава през три стъпки:

1. Включват захранването на модула
2. Записват RGB стойност в буфера
3. Извикват `write()`, за да се изпрати timed bitstream към пиксела

Минимален пример:

```
from machine import Pin # Импортираме Pin за power-enable и data линията.
import neopixel # Импортираме neopixel драйвера за RGB модула.

vcc = Pin("P500", Pin.OUT, value=1) # Включваме P500, защото без него модулт не е активен в този setup.
np = neopixel.NeoPixel(Pin("P112"), 1) # Създаваме NeoPixel обект за един пиксел върху P112.
np[0] = (40, 0, 0) # Зареждаме червено в буфера на първия пиксел.
np.write() # Изпращаме буфера и реално сменяме цвета на модула.
```

Тук има две практични разлики спрямо обикновен GPIO или PWM пример:

- цветът се задава като кортеж (R, G, B)
- промяната не се вижда преди `write()`

Втори важен практически момент е яркостта.

В учебните примери стойностите са ниски, например 40, за да се ограничи токът и да се намали рискът при първи тест.

Трети важен момент е, че NeoPixel е външен модул, така че физическото свързване е толкова важно, колкото и кодът:

- DIN на модула трябва да отиде към P112
- P500 трябва да е включен
- захранването трябва да е стабилно
- ако се ползва отделно основно захранване, масата трябва да е обща с борда

Файлове:

- examples/neopixel.py
- examples/neopixel_test.py

Резюме

- Главата показва NeoPixel като специализиран външен RGB модул с еднопроводен data интерфейс.
- Основният practically-first модел е power, запис в буфера и `write()`.
- Ниската яркост и правилното захранване са критични за безопасен първи тест.

Самопроверка

1. Защо NeoPixel не се управлява като обикновен RGB LED с три отделни PWM канала?
2. Каква е ролята на `write()` след присвояване на `np[0] = (...)`?
3. Защо P500 остава задължителен, дори ако за по-дълга лента добавите отделно основно захранване?

Упражнения

1. Увеличете броя пиксели на 2 или 4 и направете цветово преливане по лентата.

Подсказка: `np = neopixel.NeoPixel(Pin("P112"), 4)` после `np[i] = colors[i % len(colors)]` и `np.write()``.`

2. Добавете функция, която изгасва всички пиксели след определено време без действие.

Подсказка: `for i in range(len(np)): np[i] = (0,0,0)` след `np.write()`` — нулевите RGB стойности изгасват пикселите.`

3. Свържете SW1 и сменяйте цвета на NeoPixel-а при всяко натискане.

Подсказка: ползвайте индекс в масив от цветове `colors = [(40,0,0),(0,40,0),(0,0,40)]` и `idx = (idx+1) % len(colors)`.`

Ако не работи

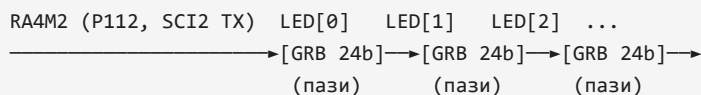
- Симптом: няма никаква светлина. Проверка: вижте дали P500 реално подава захранване и дали DIN е към правилния вход на модула.
- Симптом: цветът е грешен или нестабилен. Проверка: намалете яркостта, скъсете проводниците и проверете общата маса.
- Симптом: единичният модул работи, а лентата не. Проверка: при повече пиксели осигурете отделно основно захранване, но оставете P500 включен.

Нататък

След тази глава прочетете и следващата глава за WS2812 през machine.WS2812, за да видите разликата между софтуерен bitstream и SCI-базиран TX-only подход.

Глава 40. WS2812 през machine.WS2812 и SCI TX-only [Напреднал]

[Тip] Аналогия: WS2812 лентата е като верига от пощенски кутии. Всеки LED-пиксел прочита първите 24 бита от сигнала, запазва ги и предава останалите на следващия. Тоест трябва само един data пин — без адреси, без clock.



Метод	Изисква	Горна граница	Употреба
neopixel + bitstream()	GPIO	много пиксели	Standard MicroPython
machine.WS2812 (SCI TX)	SCI TX пин	лента с DTC трансфер	VK_RA4M2 специфично

[!] DTC конфликт: WS2812 на SCI2 и DAC на AGT споделят DTC. Спрете DAC преди strip.write() (виж Глава 52).

WS2812B е от същото семейство адресируеми RGB светодиоди, но в този порт вече има и втори начин за управление.

Вместо класическия neopixel модул с machine.bitstream(), може да се използва machine.WS2812.

Този драйвер кодира данните през подходящ SCI канал и извежда само data сигнала през избрания TXD/MOSI pin.

Практически това означава:

- към самия светодиоден модул свързвате само DIN към избрания data pin
- не подавате външен SCK и MISO към лентата
- продължавате да работите с буфер и write(), почти както при NeoPixel

От гледна точка на Python кодът е еднопроводен: има само data pin към модула.

От гледна точка на микроконтролера вътрешно се използва SCI хардуерът, така че избраният pin автоматично определя кой SCI блок ще бъде зает.

В текущата работеща реализация timing-ът вече не се прави с по-старото 3-bit кодиране.

Сега драйверът използва:

- вътрешен SCI поток около 2.857143 MHz
- 5-bit кодиране за всеки логически WS2812 бит
- кратък скрит LOW prefix преди payload-a
- достатъчно дълъг LOW suffix за latch/reset
- непрекъснат DTC feed, за да няма байтови дупки в натоварена IRQ среда

Това е важно, защото от Python страна API-то си остава просто:

- пишете RGB стойности в буфера
- извиквате write()
- не мислите за ръчно bit timing управление

От практическа гледна точка е най-добре да започнете с дефолтните параметри на драйвера и да не задавате b audrate, освен ако нямате конкретна експериментална причина.

Поддържаните data pin-ове в този порт са:

- SCI0 -> P101, P205, P411
- SCI1 -> P213, P401, P501
- SCI2 -> P102, P112, P302
- SCI9 -> P109, P203, P409, P602

Това значи, че P112 вече е default pin, но не е единствен възможен.

Важното ограничение е по канали, не само по pin.

Ако изберете pin от SCI2 групата, не трябва едновременно да очаквате:

- machine.WS2812
- UART(2)
- SPI(2) през SCI2

Същата логика важи и за другите групи, ако вече ползвате същия SCI канал за друг интерфейс.

Минимален пример с алтернативен pin:

```
from machine import Pin # Импортираме Pin за захранването и data линията.
from machine import WS2812 # Импортираме специализирания драйвер за WS2812.
import time # Импортираме time за кратко изчакване след включване на захранването.

vcc = Pin("P500", Pin.OUT, value=1) # Включваме P500, защото без него модулет не е активен в този setup.
time.sleep_ms(100) # Изчакваме power-enable линията и модулет да се стабилизира.
strip = WS2812(pixel_count=1, pin=Pin("P109"), channels=3) # Създаваме драйверен обект върху алтернативния...
strip[0] = (40, 0, 0) # Зареждаме червено в буфера на първия пиксел.
strip.write() # Изпращаме буфера към модула през SCI TX-only backend-a.
```

Подобно на NeoPixel, и тук цветът не се вижда преди write().

Разликата е, че тук timing-ът не идва от общия bitstream механизъм, а от специализиран C-side драйвер в порта.

Това е полезно по три причини:

- API-то е ясно вързано към WS2812
- можете да изберете удобен SCI TX/MOSI pin според реалното окабеляване
- логиката за кодиране, непрекъснато предаване и latch остава в C, а не в Python

Важно е и захранването:

- в този setup P500 трябва да бъде включен задължително
- при повече пиксели може да е нужно отделно основно захранване, но P500 остава включен
- масата на захранването и масата на борда трябва да са общи

Практически работен ред е:

1. Започнете с `pixel_count=1`.
2. Проверете последователно `red`, `green`, `blue`, `white`, `off`.
3. Едва след това минете към повече пиксели или матрица.

Важното ограничение е, че този драйвер е порт-специфичен.

Тоест примерите са силно полезни точно за VK_RA4M2, но не бива да очаквате същия `machine.WS2812` API на произволен друг MicroPython порт.

Файлове:

- `examples/ws2812_sci.py`
- `examples/ws2812_sci_test.py`
- `examples/ws2812_sci_chase_145.py`
- `examples/ws2812_sci_p101.py`
- `examples/ws2812_sci_p109.py`
- `examples/ws2812_async_test.py` — верифицира че `WS2812.write_async()` не блокира CPU: тест пин P107

маркира `timing`, `Timer(-1)` нотификация, статистика за `frame rate` и грешки

Резюме

- Главата показва WS2812 като специализиран `machine` клас, реализиран в този порт върху няколко SCI TX/MOSI pin групи.
- За самото свързване към модула е нужен само избраният `data pin`, без външен `clock pin` към светодиода.
- Конфликтите са по SCI канал: не може един и същ SCI блок да се дели едновременно от WS2812 и UART или SPI.

Самопроверка

1. Каква е разликата между `neopixel.NeoPixel` и `machine.WS2812` в този порт?
2. Защо потребителят свързва само DIN -> избрания TXD/MOSI pin, въпреки че вътрешно драйверът използва SCI?
3. Какво общо имат P109 и UART(9) при използване на `machine.WS2812`?

Упражнения

1. Пуснете един и същ пример веднъж с P112, а после с P109, и сравнете нужните промени в Python кода.
Подсказка: само `pin=Pin("P109")` се мени — но `P109` е от `SCI9` групата, не използвайте `UART(9)` едновременно.
2. Променете `pixel_count` на 2 или 4 и направете цветен „марш“ по къса лента.
Подсказка: пишете `(R,G,B)` в `strip[i]` — не забравяйте GRB реда: `(0,40,0)` в буфера дава червено на лентата.
3. Изберете pin от SCI0 или SCI1 групата и проверете кои други интерфейси вече не бива да активирате едновременно.
Подсказка: `P101` е `SCI0` — при `machine.WS2812(pin=Pin("P101"))` не може да ползвате `SPI.MOSI` едновременно.
4. Пуснете `examples/ws2812_sci_chase_145.py` върху 1 + 12*12 светодиода, като държите P500 включен и

осигурете достатъчно основно захранване, и наблюдавайте дали трите цвята се местят равномерно.

Подсказка: осигурете поне 2A основно захранване за 145 пиксела — недостатъчен ток дава нестабилни цветове.

Ако не работи

- Симптом: няма никаква светлина. Проверка: вижте дали P500 реално подава захранване и дали DIN е към правилния вход на модула.
- Симптом: получавате грешка за зает bus. Проверка: избраният SCI канал може вече да е зает от UART или SPI със същия номер.
- Симптом: цветовете са нестабилни при повече пиксели. Проверка: осигурете отделно основно захранване и обща маса, но не изключвайте P500.
- Симптом: няма светлина на матрицата. Проверка: уверете се, че P500 е включен и че масата е обща.
- Симптом: neopixel примерът работи, а този не. Проверка: това е индикация, че точно избраният SCI TX/MOSI path трябва да се валидира на реален хардуер.

Нататък

След тази глава преминете към „Минимална FSM и таблична FSM“, за да видите как входовете, изходите и времето се организират в структурирана логика на състояния.

Част VIII: FSM като начин на мислене

Ключови термини: FSM (Finite State Machine) — краен автомат с дефинирани състояния и преходи. guard — условие за разрешаване на преход. event queue — опашка от чакащи събития. tickless — CPU спи между събития вместо да проверява постоянно.

Тази част преминава от отделни примери към събитийно поведение, състояния и структурирано управление на логиката.

Глава 41. Минимална FSM и таблична FSM [Напреднал]

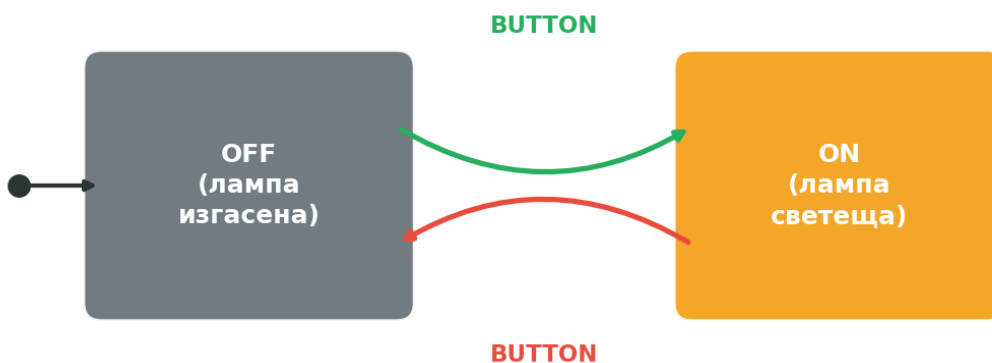
В Глава 9 видяхте как таблица от функции замества switch-case. FSM таблицата е следващата стъпка: ключът вече е двойка (state, event), а стойността е новото state.

[Tip] Аналогия: FSM (Finite State Machine) е като светофар — системата е ВИНАГИ в точно ЕДНО от дефинираните състояния. Получава събитие (BUTTON, TIMEOUT) и преминава в следващо. Не може да е едновременно на ЗЕЛЕНО и ЖЪЛТО.

Светофар FSM:
RED → (TIMER) → GREEN → (TIMER) → YELLOW → (TIMER) → RED
↑
(циклически loop)

Диаграма на лампа FSM (ON/OFF с бутон):

FSM: Лампа ON/OFF — превключване с бутон



FSM: Лампа ON/OFF

Двата стила:

```
# 1. Минимален if/elif (до 3-4 състояния):
if state == "OFF":
    if event == "BUTTON": state = "ON"

# 2. Таблица (5+ състояния, по-четима):
transitions = {
    ("OFF", "BUTTON"): "ON",
    ("ON", "BUTTON"): "OFF",
    ("ON", "TIMEOUT"): "OFF",
}
state = transitions.get((state, event), state)
```

[Pin] Правило: До 3 състояния → if/elif. При 4+ → таблица. При поведение + данни → клас с FSM.

Минимална FSM:

```
STATE_OFF = "OFF" # Това е първото състояние на лампата.
STATE_ON = "ON" # Това е второто състояние на лампата.
EVENT_BUTTON = "BUTTON" # Това е събитието от бутона.

state = STATE_OFF # Започваме с изключена лампа.

def handle_event(current_state, event_name): # Дефинираме преходната функция на машината на състояния.
    if current_state == STATE_OFF and event_name == EVENT_BUTTON: # Ако сме в OFF и бутонът е натиснат, ми...
        return STATE_ON # Връщаме новото състояние ON.
    if current_state == STATE_ON and event_name == EVENT_BUTTON: # Ако сме в ON и бутонът е натиснат, мина...
        return STATE_OFF # Връщаме новото състояние OFF.
    return current_state # Ако няма преход, оставаме в същото състояние.
```

Таблична FSM:

```
state = "OFF" # Започваме в състояние OFF.

transition_table = {("OFF", "BUTTON"): "ON", ("ON", "BUTTON"): "OFF"} # В таблицата описваме преходите кат...

event_sequence = ["BUTTON", "BUTTON", "BUTTON", "BUTTON"] # Симулираме поредица от събития.
```

Графично идеята може да се мисли така:

```
OFF --BUTTON--> ON
ON  --BUTTON--> OFF
```

Файлове:

- examples/30_fsm/01_fsm_lamp_button.py
- examples/30_fsm/02_fsm_table.py
- examples/31_case_study/02_two_fsms.py — декомпозиция на bathroom контрол на две независими FSM (лампа + вентилатор) с общ event sequence и реални LED1/SW1

Резюме

- Главата развива системното мислене чрез темата „Минимална FSM и таблична FSM“.
- Основният акцент е върху разделяне на състояния, събития, условия, приоритети и срокове.

- Това е архитектурен материал: целта е не само да работи, а да остане четим и разширяем.

Самопроверка

1. Коя е ключовата моделна идея в „Минимална FSM и таблична FSM“?
2. Как тази глава прави поведението по-структурирано от чиста поредица от `if` проверки?
3. Кое решение тук помага най-много за бъдещо разширяване на системата?

Упражнения

1. Добавете още едно състояние, събитие, `guard` или правило според темата на главата.

Подсказка: добавете `("ON", "LONG_PRESS"): "BLINK"` в `transition_table` и обработете новото `STATE_BLINK`.

2. Отпечатайте текущото състояние и последното обработено събитие за по-лесно проследяване.

Подсказка: `print("state={} event={}".format(state, event))` след всяко `transitions.get(...)` извикване.

3. Опишете с кратка диаграма или таблица как бихте разширили модела с още един реален вход.

Подсказка: добавете ред в `transition_table` с ключ `("BLINK", "BUTTON")` за нов преход обратно към `OFF`.

Ако не работи

- Симптом: системата влиза в неочаквано състояние. Проверка: търсете липсващ преход, грешен приоритет или слабо дефиниран `event/condition`.

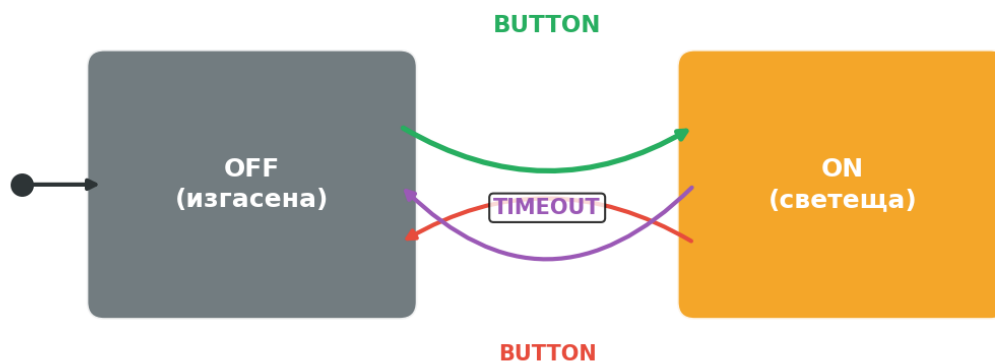
- Симптом: моделът става нечетим. Проверка: върнете се към ясни имена на `states`, `events` и отделни помощни функции или таблици.

Нататък

След тази глава преминете към „FSM с timeout“, за да добавите време като част от самата логика на състоянията.

Глава 42. FSM с timeout [Напреднал]

FSM с timeout: автоматично изгасване



TIMEOUT = ако няма BUTTON за N ms → автоматичен OFF

FSM с timeout

Пример:

```
state = "OFF" # Започваме с изключена лампа.

transition_table = {("OFF", "BUTTON"): "ON", ("ON", "TIMEOUT"): "OFF", ("ON", "BUTTON"): "OFF"} # Описваме...

event_sequence = ["BUTTON", "TIMEOUT", "BUTTON", "BUTTON"] # Симулираме бутон и timeout събития.
```

Тази глава е преходът от „лампа + бутон“ към „лампа + бутон + време“.

Файл:

- `examples/30_fsm/03_fsm_timeout.py`

Резюме

- Добавянето на timeout към FSM преходите позволява системата да реагира на **липсата** на събитие, не само на появата му.
- `time.ticks_ms()` и `time.ticks_diff()` са основните инструменти за проследяване на срокове в callback-free среда.
- Timeout е и предпазен механизъм: ако бутонът „забрави“ да изключи лампата, таймерът я гаси автоматично.

Самопроверка

1. Как timeout се различава от обикновено събитие в таблицата на преходи?
2. Какво се случва, ако потребителят натисне бутон точно в момента на timeout — кой преход печели?
3. Как бихте нулирали timeout-а при повторно натискане на бутона?

Упражнения

1. Добавете състояние `DIMMED` — лампата преминава в намален режим 5 секунди преди да изгасне напълно.
Подсказка: добавете `("ON", "TIMEOUT"): "DIMMED" и `("DIMMED", "TIMEOUT"): "OFF" с различни прагове.
2. Направете timeout-а конфигуриран чрез речник `config = {"timeout_ms": 3000}` и го сменете без рестарт.
Подсказка: `deadline = time.ticks\_add(now, config["timeout\_ms"])` — сменете `config["timeout\_ms"]` по всяко време.
3. Добавете `print()` лог при всяко timeout събитие с `ticks_ms()` стойността за по-лесна диагностика.
Подсказка: `print("TIMEOUT at", time.ticks\_ms())` точно преди `event = "TIMEOUT" се генерира.

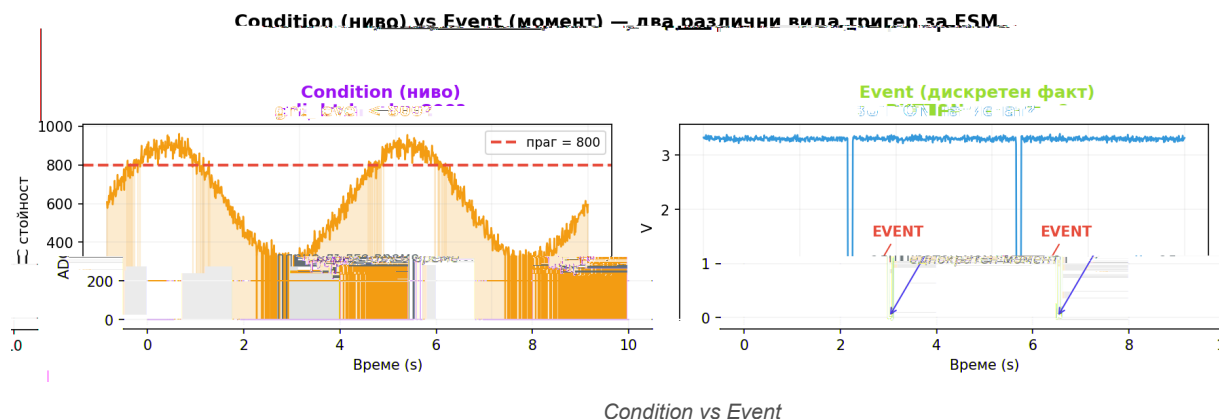
Ако не работи

- Симптом: лампата не изгасва след timeout. Проверка: уверете се, че `TIMEOUT` събитието реално се генерира — проследете `ticks_diff` стойностите.
- Симптом: timeout-ът се задейства прекалено рано. Проверка: проверете дали `ticks_add / ticks_diff` се ползват правилно — обръкването на аргументите е честа грешка.

Нататък

След тази глава преминете към „Conditions vs Events“, за да видите по-ясно кога да мислите в условия и кога в събития.

Глава 43. Conditions vs Events [Напреднал]



Пример:

```
light_level = 600 # Това е моментното ниво от светлинния сензор в симулацията.
dark_threshold = 800 # Това е прагът, под който приемаме че е тъмно.
event_name = "BUTTON" # Това е дискретното събитие от бутон.

condition_is_dark = light_level < dark_threshold # Това е condition, която може да е вярна дълго време.

if condition_is_dark and event_name == "BUTTON": # Комбиниране условие и събитие в общо решение.
    print("Лампата може да се включи, защото е тъмно и има бутонно събитие.") # Печатаме положителния изход.
else: # Ако комбинацията не е изпълнена, не включваме лампата.
    print("Няма достатъчно условия за включване.") # Печатаме отрицателния изход.
```

Тук разграничението е:

- condition може да е вярно дълго време
- event е дискретен факт

Файл:

- examples/30_fsm/04_conditions_vs_events.py

Резюме

- **Condition** е ниво (може да е вярно дълго време): `light_level < 800`. **Event** е дискретен факт: `BUTTON` натиснат.
- FSM решението обикновено комбинира и двете: event задвижва прехода, condition го разрешава или забранява.
- Разграничаването помага да се избегнат грешки като „лампата се включва непрекъснато, докато е тъмно“ (condition третиран като event).

Самопроверка

1. Дайте пример за condition и event от ежедневието, които се комбинират в едно решение.
2. Какво се случва, ако третирате condition като event — какъв бърк може да възникне?
3. Как бихте превърнали condition (`light_level < 800`) в event (`BECAME_DARK`) с edge detection?

Упражнения

1. Добавете втора condition (`temperature > 30`) и комбинирайте двете в общо решение за вентилатор.

Подсказка: ``if is_dark and is_hot and event == "BUTTON":`` — всички условия трябва да са верни едновременно.

2. Реализирайте edge detection: генерирайте event `DARK_ENTERED` само при **прехода** от светло към тъмно, не при всяко четене.

Подсказка: сравнете ``prev_dark != current_dark`` — само при промяна от ``False`` към ``True`` генерирайте събитието.

3. Нарисувайте кратка таблица с три реални входа (бутон, светлинен сензор, PIR) и определете кои са conditions и кои са events.

Подсказка: бутон дава event (натискане), светлинен сензор дава condition (тъмно/светло), PIR дава event (засечено движение).

Ако не работи

- Симптом: действието се повтаря непрекъснато, вместо да се случи веднъж. Проверка: вероятно condition се третира като event — добавете edge detection или флаг `already_handled`.

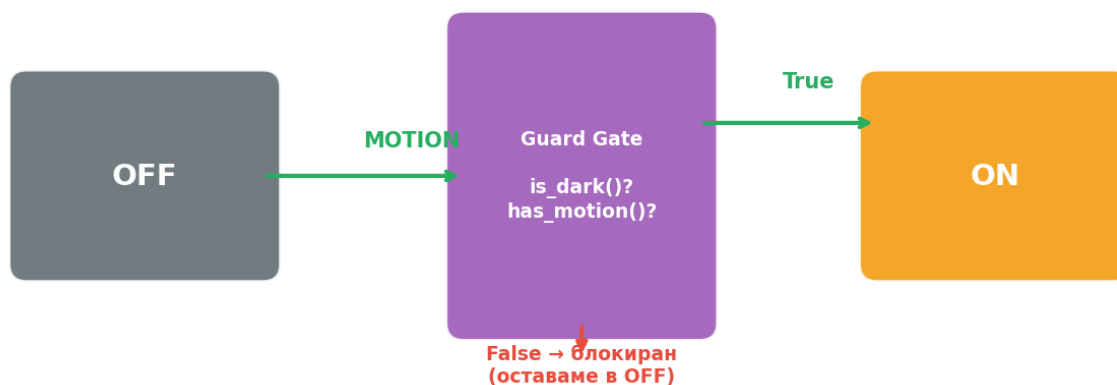
- Симптом: действието никога не се случва. Проверка: вижте дали event-ът и condition-ът се проверяват **едно временно** — ако event-ът се изчиства преди condition-а да е проверен, решението винаги е `False`.

Нататък

След тази глава преминете към „Guard функции“, за да направите преходите между състояния по-умни и по-контролирани.

Глава 44. Guard функции [Напреднал]

Guard функция: портал, който разрешава или блокира прехода



Guard функция като портал

Пример:

```
context = {"light": 500, "pir": True} # Подготвяме контекст със стойности от сензори.
state = "OFF" # Започваме в състояние OFF.
event_name = "MOTION" # Симулираме събитие за засечено движение.

def is_dark(data): # Това е първата guard функция.
    return data["light"] < 800 # Връщаме True само ако е достатъчно тъмно.

def has_motion(data): # Това е втората guard функция.
    return data["pir"] is True # Връщаме True само ако PIR сензорът вижда движение.

if state == "OFF" and event_name == "MOTION" and is_dark(context) and has_motion(context): # Комбинираме е...
    state = "ON" # Минаваме в ON само ако всички проверки са успешни.
```

Тук преходът не зависи само от state и event, а и от допълнителни условия.

Файл:

- examples/30_fsm/05_guard_functions.py

Резюме

- Guard функция е **допълнителна проверка** преди преход: дори state и event да съвпадат, преходът не се случва, ако guard-ът върне False.
- Guard функциите правят FSM таблицата по-чиста: вместо да добавяте нови състояния за всяка комбинация от сензори, добавяте guard към съществуващ преход.
- Guard функциите трябва да са **чисти** (без странични ефекти) — те само четат данни и връщат True/False.

Самопроверка

1. Каква е разликата между guard функция и condition от предишната глава?
2. Какво се случва, ако guard функция има страничен ефект (напр. включва мотор)?
3. Как бихте тествали guard функциите изолирано, без реален хардуер?

Упражнения

1. Добавете трета guard функция is_nighttime(data), която проверява часа от RTC, и я комбинирайте с останалите.

Подсказка: `from machine import RTC; rtc = RTC(); return rtc.datetime()[4] >= 22 or rtc.datetime()[4] < 6`.

2. Направете guard функциите конфигурируеми: прагът на is_dark() да идва от речник config, а не от хардкодирана стойност.

Подсказка: `def is_dark(data, cfg=config): return data["light"] < cfg["dark_threshold"]`.

3. Добавете диагностичен print() в случая, когато event-ът е правилен, но guard-ът е блокирал прехода — това помага при дебъг.

Подсказка: `if event == "MOTION" and not is_dark(ctx): print("Guard blocked: not dark enough")`.

Ако не работи

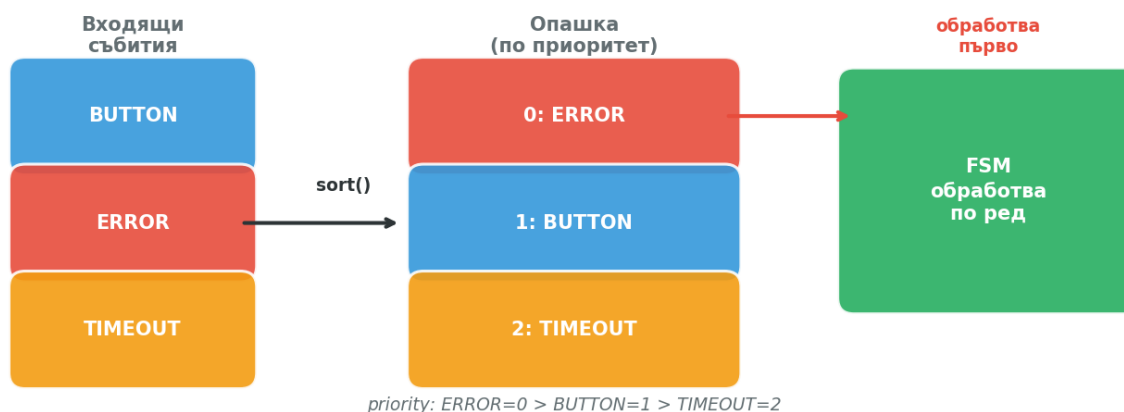
- Симптом: преходът никога не се случва, въпреки че event-ът идва. Проверка: отпечатайте стойностите, които guard функциите виждат — прагът може да е грешен или данните да не се обновяват.
- Симптом: преходът се случва когато не трябва. Проверка: вижте дали guard-ите се комбинират с and (всички трябва да са True) или с or (достатъчен е един).

Нататък

След тази глава преминете към „Приоритети и event queue“, за да организирате събитията, когато вече не са само едно по едно.

Глава 45. Приоритети и event queue [Напреднал]

Event Queue с приоритети: ERROR винаги се обработва пръв



Event Queue с приоритети

Приоритети:

```
pending_events = ["BUTTON", "ERROR", "TIMEOUT"] # Симулираме едновременно чакащи събития.
priority_order = {"ERROR": 0, "BUTTON": 1, "TIMEOUT": 2} # По-малкото число означава по-висок приоритет.

pending_events.sort(key=lambda event_name: priority_order[event_name]) # Подреждаме събитията по приоритет.
```

Event queue:

```
event_queue = [] # Създаваме проста event queue като обикновен Python list.

event_queue.append(("BUTTON", {"source": "SW1"})) # Добавяме събитие от бутон.
event_queue.append(("PIR", {"source": "pir_sensor"})) # Добавяме събитие от PIR сензор.
event_queue.append(("TIMEOUT", {"ms": 30000})) # Добавяме timeout събитие.

while event_queue: # Обработваме опашката, докато има чакащи събития.
    event_name, payload = event_queue.pop(0) # Взимаме най-старото събитие от началото на опашката. Забеле...
    print("Обработваме", event_name, "с payload", payload) # Печатаме кое събитие се обработва сега.
```

Това е мостът между:

- прекъсвания
- таймери
- сензори
- FSM логика

Файлове:

- examples/30_fsm/06_priorities.py
- examples/30_fsm/07_event_queue.py

Резюме

- Когато няколко събития чакат едновременно, **приоритетът** определя кое се обработва първо — `ERROR` преди `BUTTON` преди `TIMEOUT`.
- Event queue (`list` или `collections.deque`) разделя **генерирането** на събитие (в `IRQ` или сензорен `callback`) от **обработката** му (в `main loop`).
- Това е мостът между прекъсвания, таймери, сензори и FSM логиката — всяко от тях пуска събитие в опашката, а FSM ги консумира подредено.

Самопроверка

1. Какво се случва, ако `ERROR` и `BUTTON` пристигнат едновременно, но няма приоритизация?
2. Защо event queue е по-безопасна от директна обработка на събитието в `ISR`?
3. Каква е разликата между `list.pop(0)` и `collections.deque.popleft()` по отношение на производителност?

Упражнения

1. Добавете ново събитие `SENSOR_ALERT` с приоритет между `ERROR` и `BUTTON` и проверете, че подредбата е правилна.

Подсказка: `priority_order["SENSOR_ALERT"] = 0.5` — но речникът изисква цели числа, ползвайте ``1`` и изместете ``BUTTON`` на ``2``.

2. Добавете брояч `dropped_events` за случая, когато опашката е пълна — така ще знаете дали губите данни.

Подсказка: `if len(event_queue) < MAX_QUEUE: event_queue.append(ev) else: dropped_events += 1`.

3. Заменете `list` с `collections.deque()`, `16`) и измерете дали обработката е по-бърза при пълна опашка.

Подсказка: `from collections import deque; q = deque(), 16)` — `q.append(ev)` и `q.popleft()` са $O(1)$.

Ако не работи

- Симптом: събитията се обработват в грешен ред. Проверка: уверете се, че `sort()` се извиква **преди** обработката, и че `priority_order` речникът покрива всички типове събития.
- Симптом: опашката расте безкрайно. Проверка: добавете максимална дължина или изхвърляйте най-старите low-priority събития при препълване.

Нататък

След тази глава преминете към „Tickless timers и sleep идея“, за да видите как FSM мисленето може да се свърже и с икономия на време и енергия.

Глава 46. Tickless timers и sleep идея [Напреднал]

Пример:

```
import machine # Импортираме machine за кратък lightsleep между събитията.
import time    # Импортираме time за работа с ticks_ms и ticks_diff.

tasks = [("lamp_timeout", 120), ("fan_timeout", 450), ("display_refresh", 40)] # Подготвяме три задачи с р...
start_ms = time.ticks_ms() # Запомняме началния момент.
next_deadline_ms = min(deadline for _, deadline in tasks) # Избираме най-близкия срок като tickless цел.
sleep_ms = max(0, next_deadline_ms - time.ticks_diff(time.ticks_ms(), start_ms)) # Изчисляваме колко време...
machine.lightsleep(sleep_ms) # Спим точно до най-близкото планирано събитие с lightsleep.
elapsed_ms = time.ticks_diff(time.ticks_ms(), start_ms) # Изчисляваме изминалото време след съня.
ready_tasks = [task_name for task_name, deadline in tasks if deadline <= elapsed_ms] # Избираме всички зад...
```

Тази глава не разчита на агресивни sleep режими, а показва начина на мислене:

- намираме най-близкия срок
- спим точно дотогава
- обработваме всички изтекли задачи

Файлове:

- examples/30_fsm/08_tickless_sleep.py
- examples/28_machine_misc/03_sleep_modes.py

Резюме

- Вместо да проверяваме всички задачи при всеки тик, **tickless** подходът пресмята кога е най-близкият срок и спи точно дотогава.
- `machine.lightsleep(ms)` намалява консумацията между събитията, без да променя логиката на приложението.
- Моделът „намери най-близкия deadline → спи → обработи всички изтекли задачи“ е основата на всеки ефективен embedded scheduler.

Самопроверка

1. Каква е разликата между `time.sleep_ms()` (блокиращ) и `machine.lightsleep()` (икономичен) за tickless подхода?
2. Какво се случва, ако `lightsleep()` се събуди по-рано от очакваното (напр. от IRQ)?
3. Как бихте обработили случая, когато две задачи имат един и същи deadline?

Упражнения

1. Добавете четвърта задача с deadline 200 ms и проверете, че системата обработва всички задачи в правилния ред.

Подсказка: `tasks.append(("new_task", 200))` — след `lightsleep` проверете `deadline <= elapsed_ms` за всяка задача.

2. Измерете реалната точност на `lightsleep()` — колко милисекунди грешка има при sleep от 100 ms?

Подсказка: `t0 = time.ticks_ms(); machine.lightsleep(100); err = time.ticks_diff(time.ticks_ms(), t0) - 100`.

3. Направете задачите повтарящи се: след обработка, задачата добавя нов deadline `текущо_време + период_ms`.

Подсказка: след обработка `tasks[i] = (name, time.ticks_add(time.ticks_ms(), period_ms))` за рестартиране.

Ако не работи

- Симптом: системата не се събужда навреме. Проверка: `lightsleep()` може да не е точен до милисекунда — добавете малък марж или ползвайте `idle()` вместо `lightsleep()` за по-кратки интервали.
- Симптом: задачи се пропускат. Проверка: след `lightsleep()` проверете **всички** задачи с `deadline <= elapsed`, не само първата.

Нататък

След тази глава преминете към [Част IX](#) — „Обработка на грешки, recovery и fail-safe поведение“ ([Глава 47](#)), за да видите как реалните системи се предпазват от неизбежните хардуерни и софтуерни проблеми.

Част IX: Надеждни вградени програми и архитектура на приложението

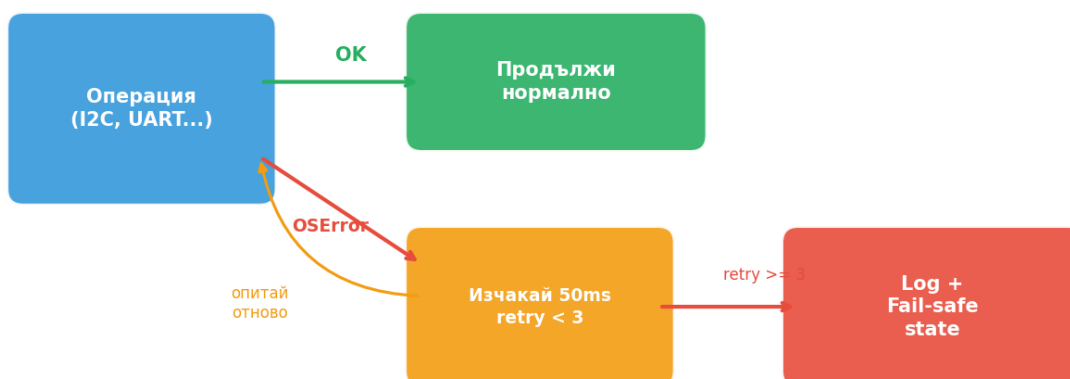
Ключови термини: watchdog — хардуерен таймер, рестартиращ системата при увисване. heartbeat — периодичен сигнал „жив съм“. fail-safe — безопасно състояние при грешка. sentinel — специална стойност (напр. 255), означаваща „невалиден“. капсулиране — обвиване на периферия в клас с чист API.

Този част събира error handling, watchdog мислене, наблюдаемост, класове и общата архитектура на завършения проект. Завършва с RGB_Guardian — цялостен казус, който интегрира всичко научено дотук.

Глава 47. Обработка на грешки, recovery и fail-safe поведение [Напреднал]

[Tip] Аналогия: В реално устройство грешките не са изключение — те са нормалната работа. Кабелът може да изпадне. I2C устройството може да не отговори. Аналоговият вход може да е шумен. Добрата програма очаква и управлява тези случаи, вместо да се срива.

Error handling: retry → fail-safe



Error handling: retry → fail-safe

Без retry: I2C грешка → crash x
С retry: I2C грешка → изчакай 50ms → опитай отново x 3 → ако неуспешно → log + safe state v

Стратегия	Когато	Пример
Retry с delay	Временна bus грешка	I2C, UART timeout
Default стойност	Сензор недостъпен	Последна известна стойност
Fail-safe state	Критична грешка	LED изгасен, мотор спрян
Watchdog reset	Системата е заседнала	WDT.feed() спира → reset

При реално устройство грешките не са изключение, а част от нормалната работа.

Това важи особено за:

- външни модули
- кабели и конектори

- шумни входове
- временни bus грешки
- непълни отговори по UART

Полезен модел е:

```
import sys
import time

def read_with_retry(reader, retries=3, delay_ms=50):
    for attempt in range(retries):
        try:
            return reader()
        except OSError as e:
            sys.print_exception(e) # Печата пълен traceback – стандартен начин за дебъг в MicroPython.
            time.sleep_ms(delay_ms)
    return None
```

[Pin] `sys.print_exception(e)` е стандартният начин в MicroPython да отпечатате пълен traceback на грешка. За разлика от `print(e)`, който показва само съобщението, `sys.print_exception(e)` показва целия стек — незаменимо при дебъг на вградени устройства.

Още по-важно е какво правите след неуспеха:

- записвате последната грешка
- връщате системата в безопасно състояние
- решавате дали да повторите, да деградирате функционално или да спрете

За fail-safe състояние често е по-важно „нищо опасно да не стане“, отколкото „всичко да продължи на всяка цена“.

Свързани файлове:

- examples/34_external_modules/01_i2c_module_probe.py
- examples/34_external_modules/02_spi_module_transaction.py
- examples/34_external_modules/03_uart_module_request_response.py
- examples/27_storage/03_dataflash_state_led.py

Резюме

- Грешките трябва да са очаквана част от проекта, а не странична мисъл.
- Полезните инструменти са retry, timeout, диагностика и fail-safe поведение.
- Recovery логиката трябва да е ясна и ограничена, а не безкрайно повторение без контрол.

Самопроверка

1. Каква е разликата между retry и fail-safe поведение?
2. Защо безкрайните повторения често са лоша стратегия?
3. Какво бихте записали като „последна грешка“, за да помогнете на диагностика?

Упражнения

1. Добавете retry логика към един I2C или UART пример.

Подсказка: обвийте `i2c.readfrom_mem()` в `try/except OSError` с `for attempt in range(3):`.

2. Добавете флаг `last_error` и отпечатване на смислено съобщение.

Подсказка: `'last_error = str(e)'` в `'except'` блока, после `'if last_error: print("ERR:", last_error)'` в главния цикъл.

3. Дефинирайте кое е безопасното състояние за вашия проект при провал на външен модул.

Подсказка: `'def enter_safe_state(): led.value(1); motor_stop(); print("SAFE")'` — извиквайте при неуспешни `retries`.

Ако не работи

- Симптом: проектът „замръзва“ при проблемен модул. Проверка: вижте дали липсва `timeout` или `limit` на повторенията.
- Симптом: след грешка системата остава в хаотично състояние. Проверка: добавете ясен `enter_safe_state()` блок.

Нататък

След тази глава вече има смисъл да обсъдите и `watchdog` идеята, но честно спрямо текущите възможности на порта.

Глава 48. Watchdog и самовъзстановяване [Напреднал]

Watchdog-ът е класически `embedded` инструмент: ако приложението спре да „диша“, системата се рестартира.

Watchdog: `feed()` нулира таймера, без `feed()` → RESET



Watchdog: `feed/reset` цикъл

Важно уточнение за тази книга:

- като хардуерна идея `watchdog` е много ценен
- в текущия `renesas-ra` порт `WDT` не е поддържан

Затова тук темата е учебна и архитектурна, а не готов `runtime` пример за този борд.

Правилният мисловен модел е:

- имате основен „жив“ `loop` или `scheduler`
- имате ясни здравни точки, на които системата доказва, че е работоспособна
- `watchdog` се „храни“ само ако тези точки са минати успешно

Добър преди-`watchdog` pattern е софтуерно здравно следене:

```
import time

last_alive_ms = time.time()

def mark_alive():
    global last_alive_ms
    last_alive_ms = time.time()

def looks_stuck(timeout_ms=2000):
    return time.time() - last_alive_ms > timeout_ms
```

Това не е хардуерен watchdog, но учи на правилната дисциплина:

- къде системата доказва, че е жива
- кога трябва да се мине в safe state
- как да се подготви бъдеща миграция към реален WDT

Свързани файлове:

- examples/01_blink_async.py
- examples/02_two_tasks.py
- examples/33_asyncio/01_asyncio_basics.py

Резюме

- Watchdog е важна embedded концепция, но в текущия renesas-ra порт не е наличен като поддържан runtime ресурс.
- Полезно е още сега да се мисли в health points, stuck detection и controlled recovery.
- Софтуерният health monitor е добра подготовка за бъдещ истински watchdog.

Самопроверка

1. Защо watchdog не трябва да се „храни“ просто по таймер, без реална проверка за здравето на системата?
2. Какво бихте използвали като признак, че приложението е живо?
3. Каква е честната разлика между watchdog концепция и текущата поддръжка на порта?

Упражнения

1. Добавете last_alive_ms логика към едно ваше приложение.

Подсказка: извиквайте mark_alive() в главния цикъл и в looks_stuck(2000) определяйте дали системата е заседнала.

2. Направете LED индикация, която показва „жив“ режим и „грешка“ режим.

Подсказка: бавно мигане (500ms) = жив; бързо мигане (100ms) = грешка — различете по health_ok флаг.

3. Опишете на хартия къде бихте поставили WDT.feed(), ако този ресурс стане наличен по-късно.

Подсказка: WDT.feed() трябва да е след всяка успешна проверка на health point-овете, не просто в main loop.

Ако не работи

- Симптом: системата изглежда „жива“, но не върши полезна работа. Проверка: health point-ът може да е поставен на грешното място.
- Симптом: recovery логиката се активира прекалено често. Проверка: вижте дали timeout-ът е реалистичен за най-бавния ви нормален сценарий.

Нататък

След надеждността на програмата идва следващата голяма тема: как реалният сигнал да стане по-чист и по-стабилен.

Глава 49. Състояние на системата: heartbeat, health checks и debug интерфейс [Напреднал]

Когато проектът порасне, вече не е достатъчно да знаете само дали „нещо свети“.

Трябва да знаете:

- живо ли е приложението
- кога последно е имало грешка
- кой модул изостава
- какво е текущото състояние

Полезен минимален модел е:

```
health = {  
    "loops": 0,  
    "last_error": None,  
    "audio_active": False,  
}  
  
def heartbeat_tick():  
    health["loops"] += 1
```

Това може да се комбинира с:

- LED heartbeat
- периодичен status print
- UART debug команда
- броячи за грешки и retries

Свързани файлове:

- examples/01_blink_async.py
- examples/02_two_tasks.py
- examples/32_uart/01_uart_basic.py
- examples/dac_firmware_probe.py

Резюме

- Системното състояние трябва да бъде наблюдаемо, не скрито.
- Heartbeat, counters и last_error са прости, но много полезни инструменти.
- Диагностика трябва да е част от проекта, а не спешна добавка след проблем.

Самопроверка

1. Какво е heartbeat и какво различава от обикновено мигане?
2. Кои три status полета биха били най-полезни за вашия проект?
3. Защо last_error е по-полезен от просто „Exception occurred“?

Упражнения

1. Добавете health речник към един ваш проект.

Подсказка: ``health = {"loops": 0, "last_error": None}`` и ``health["loops"] += 1`` в главния цикъл.

2. Направете периодичен статус print веднъж на секунда.

Подсказка: ``if time.ticks_diff(time.ticks_ms(), last_print_ms) >= 1000: print(health); last_print_ms = time.ticks_ms()``.

3. Добавете LED шаблон, който показва нормален режим и грешка.

Подсказка: нормален = 1 кратко мигане на 2 секунди; грешка = 3 бързи мигания — различете по ``health["last_error"]``.

Ако не работи

- Симптом: диагностиката пречи на времечувствителна логика. Проверка: преместете тежките съобщения извън callback-и и IRQ.
- Симптом: има твърде много логове. Проверка: ограничете печата по период или по събитие.

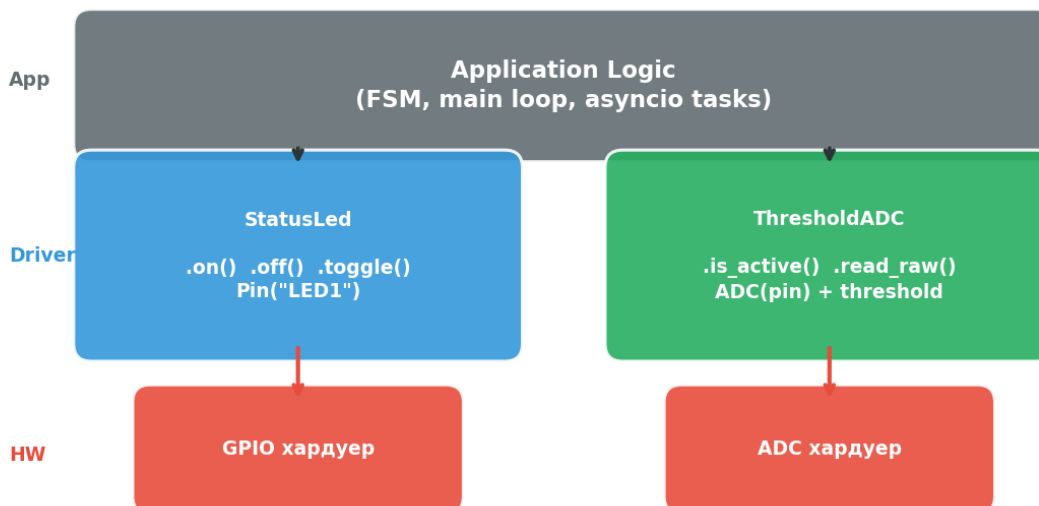
Нататък

След като системата е наблюдаема, идва естествената стъпка да капсулирате отделните ресурси като ясно дефинирани обекти.

Глава 50. Капсуловане на периферии като класове [Напреднал]

В Глава 7 видяхте минималния StatusLed клас. Тук ключовото не е синтаксисът, а защо класът е правилният инструмент за периферен драйвер.

Капсулиране: App → Driver → Hardware



Капсулиране: App → Driver → Hardware

Класът не е само „обектно програмиране заради самото него“.

В бордови проект той е удобен начин да съберете на едно място:

- хардуерен ресурс
- локално състояние
- помощни методи

- ясна граница между driver и application logic

Минипример:

```
from machine import Pin

class StatusLed:
    def __init__(self, pin_name="LED1"):
        self.pin = Pin(pin_name, Pin.OUT, value=1)

    def on(self):
        self.pin.value(0)

    def off(self):
        self.pin.value(1)
```

Тази идея вече се вижда и в `retro_synth.py`: ниското ниво е скрито зад по-чисто API.

Свързани файлове:

- `examples/retro_synth.py`
- `examples/26_touchpad/02_touchpad_diagnostics.py`

Резюме

- Класовете помагат да съберете състояние и поведение на едно място.
- Това прави проектите по-четими, по-лесни за тест и по-лесни за повторна употреба.
- Най-голямата полза е ясната граница между хардуерен детайл и логика на приложението.

Самопроверка

1. Какво печелите, ако даден периферен ресурс е капсулиран в клас?
2. Къде е границата между клас-драйвер и клас-услуга?
3. Защо глобалните променливи стават проблем при по-голям проект?

Упражнения

1. Превърнете един LED или бутон пример в клас.

Подсказка: ``class Button: def __init__(self, pin): self.pin = Pin(pin, Pin.IN, Pin.PULL_UP)`` и метод ``def is_pressed(self): return self.pin.value() == 0``.

2. Превърнете ADC четене с праг в клас с метод `is_active()`.

Подсказка: ``class ThresholdADC: с `self.adc = ADC(Pin(pin))`` и ``def is_active(self): return self.adc.read() > self.threshold``.

3. Изнесете повтаряща се логика от два примера в общ helper клас.

Подсказка: ако два примера използват ``gc.collect()`` + ``print(gc.mem_free())``, изнесете го в ``class MemMonitor: def report(self):``.

Ако не работи

- Симптом: класът изглежда по-сложен от скрипта. Проверка: класът трябва да събира смислено състояние, не просто да обвива един ред код.
- Симптом: логиката е размазана между много обекти. Проверка: разграничете driver слой от application layer.

Нататък

Когато класовете са на място, следващият естествен въпрос е как да пазите настройки, preset-и и таблици извън самия код.

Глава 51. От пример към завършено приложение [Напреднал]

Това е най-важната „мета“ глава в края на книгата.

Тя събира всичко дотук в един практичен ред:

1. Изберете минимален работещ пример.
2. Потвърдете хардуерния ресурс на борда.
3. Изнесете pin mapping и config.
4. Създайте малки драйвери или helper класове.
5. Добавете status и error reporting.
6. Добавете fail-safe и recovery стратегия.
7. Едва след това разширявайте поведението.

Полезен skeleton е:

```
def init_board():
    pass

def init_services():
    pass

def run_forever():
    while True:
        pass

def main():
    init_board()
    init_services()
    run_forever()

main()
```

Това изглежда просто, но точно тук се ражда качественият firmware проект:

- ясна последователност
- ясни модули
- ясни точки за диагностика
- ясни места за бъдещо разширение

Свързани файлове:

- examples/33_asyncio/01_asyncio_basics.py
- examples/retro_synth.py
- examples/dac_retro_music.py
- examples/31_case_study/01_bathroom_control.py — симулация на bathroom light+fan контрол с

PIR/humidity/door сензори и реални LED1/SW1; демонстрира multi-input single-pass decision логика

- examples/31_case_study/02_two_fsms.py — същият казус, декомпозиран в две независими FSM

Резюме

- Завършеното приложение не е „голям пример“, а подредена система.
- Най-стабилният път е да растете от доказан минимален пример към все по-структуриран проект.
- Диагностика, конфигурация и recovery трябва да са част от архитектурата, а не следмисъл.

Самопроверка

1. Кой е най-рисковият преход между „примерът работи“ и „проектът е готов“?
2. Кои три архитектурни стъпки бихте добавили веднага към следващия си проект?
3. Как бихте разбрали, че проектът е вече „подреден“, а не просто „още работещ“?

Упражнения

1. Изберете един пример от книгата и го преработете в малък проект със структура `config`, `pins`, `app`.

Подсказка: ``pins.py`` съдържа ``LED = "LED1"``, ``config.py`` — ``BLINK_MS = 500``, ``app.py`` — логиката, ``main.py`` само ``app.run()``.

2. Добавете `health` състояние, `last_error` и безопасно спиране.

Подсказка: обвийте ``run_forever()`` в ``try/except Exception as e: health["last_error"] = str(e); enter_safe_state()``.

3. Направете кратък release checklist за собственото си приложение.

Подсказка: включете: хардуерна проверка, ``boot.py`` тестване, error logging, watchdog ready, flash space — 5 точки.

Ако не работи

- Симптом: проектът работи само като един голям файл. Проверка: разделете startup, drivers и основна логика.
- Симптом: всяка нова функция чупи старата. Проверка: липсват граници между модулите и стабилни точки за проверка.

Нататък

След тази глава преминете към [Глава 52](#) (RGB_Guardian), за да видите как всички научени техники — FSM, WS2812, asyncio, RetroSynth и буфери без GC — се комбинират в завършена интерактивна игра.

Глава 52. RGB_Guardian — казус: игра на WS2812 лента [Напреднал]

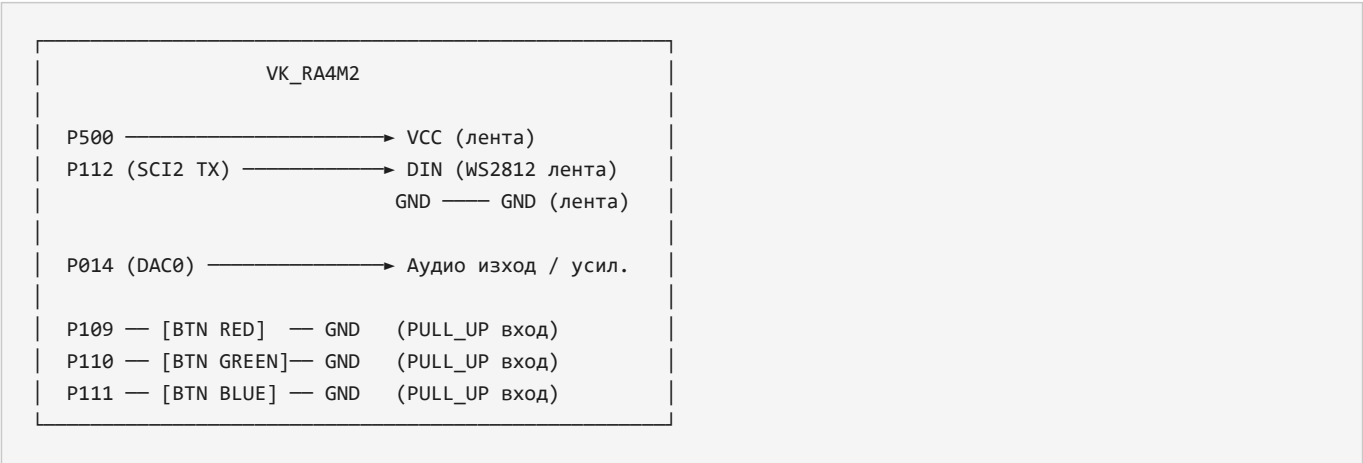
Какво е RGB_Guardian

RGB_Guardian е завършена интерактивна аркадна игра, написана на MicroPython, която работи върху 61-LED WS2812 лента. Играчът стреля оцветени куршуми по врагове с бутони, а RetroSynth произвежда звукови ефекти при всяко действие.

Казусът е умишлено избран, защото събира **наведнъж** всичко от предходните части:

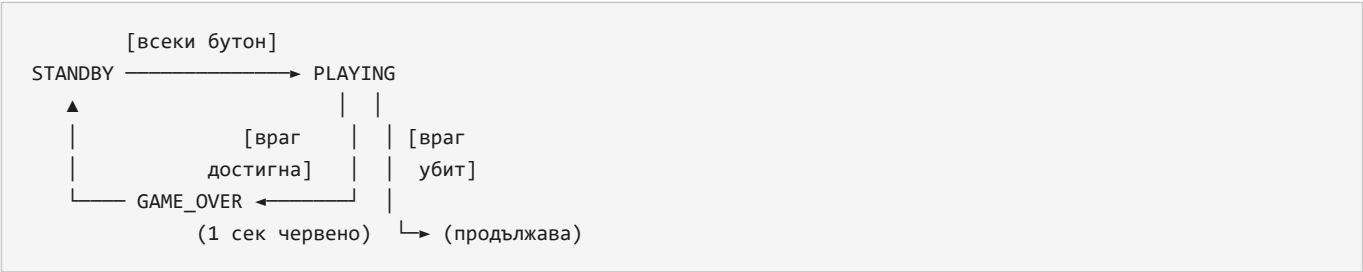
Техника	Откъде идва
machine.WS2812 — SCI TX-only	Глава 40
RetroSynth — DAC/DTC wavetable	Глава 56
FSM (STATE_STANDBY, PLAYING, GAME_OVER)	Глава 41-47
Pre-allocated bytearray пулове	Глава 30
Три бутона + IRQ/polling	Глава 17-18
Asyncio scheduler за render loop	Глава 20-21

Хардуер



Сигнал	Пин	Тип	Описание
WS2812 данни	P112	SCI2 TX	NRZ протокол 800 kHz, DTC трансфер
WS2812 захранване	P500	GPIO OUT	HIGH=вкл, LOW=изкл за пестене на ток
DAC аудио	P014	DAC0	12-бит аналогов изход, AGT тактуване, DTC буфер
Бутон червен	P109	GPIO IN PULL_UP	натиснат = LOW (активно ниско)
Бутон зелен	P110	GPIO IN PULL_UP	натиснат = LOW
Бутон син	P111	GPIO IN PULL_UP	натиснат = LOW

Архитектура на играта



Game loop при всеки тик:

Enemy spawn → Bullet move → Collision check → WS2812 render + RetroSynth SFX
--

[!] Критичен хардуерен детайл: DTC race condition На RA4M2 съществува race condition между DTC каналите на SCI2 (WS2812) и AGT (DAC). Ако strip.write() се извика докато DAC/AGT е активен, може да настъпи HardFault. Решение: функцията push() спира DAC преди всяко strip.write(): `python def push(): synth.stop() # спира DAC/AGT — предотвратява DTC race strip.write() # SCI2 DTC трансфер`

Три слоя памет без runtime allocation:

```
# Pre-allocated bytearray pools – fixed size, zero runtime allocation
b_pos    = bytearray(MAX_BULLETS) # позиция на куршум
b_color  = bytearray(MAX_BULLETS) # цвят: 0=червен 1=зелен 2=син
b_active = bytearray(MAX_BULLETS) # 1=в полет, 0=свободен слот

e_pos    = bytearray(ENEMY_COUNT) # позиция на враг (255=убит)
e_color  = bytearray(ENEMY_COUNT) # цвят на враг
e_active = bytearray(ENEMY_COUNT) # 1=жив, 0=убит/неактивен
```

Структура	Тип	Размер	Алтернативата
b_pos, b_color, b_active	bytearray	10 байта всяка	list от dict ~300+ байта
e_pos, e_color	bytearray	30 байта всяка	list от dict ~600+ байта
_sin_table	bytearray	64 байта	math.sin() всеки кадър
COLORS, C_OFF...	tuple константи	1× при import	нов tuple всеки кадър

Sentinel стойности: e_pos[j] = 255 → мъртъв враг; b_active[i] = 0 → свободен куршумен слот.

Звукови ефекти

Събитие	Метод	Описание
Изстрел	play_sweep("A5", "A6", 40, "square", steps=6)	Кратък лазерен chirp нагоре
Правилен цвят	coin()	Класически ретро монета B5→E6
Грешен цвят	play_sweep("G4", "C3", 80, "saw", steps=8)	Тъжен звук надолу
Game Over	explosion()	LFSR шумов burst

Ключова идея: collision detection без списъци

```
# Проверка за попадение: куршум и враг на един и същи LED
for b in range(MAX_BULLETS):
    if not b_active[b]:
        continue
    for e in range(ENEMY_COUNT):
        if e_active[e] and e_pos[e] == b_pos[b] and e_color[e] == b_color[b]:
            e_active[e] = 0 # враг убит
            b_active[b] = 0 # куршум изчезва
            synth.coin()    # звуков ефект при попадение
```

Цветовото съвпадение е едновременно геймплей механика и учебна демонстрация: само куршум от правилния цвят убива правилния враг.

Файл

- `examples/RGB_Guardian.py`

Ключови зависимости

- `examples/retro_synth.py` — RetroSynth модулът трябва да е на борда
- `machine.WS2812` — специализиран SCI TX-only драйвер (C модул на порта)
- Три бутона на P109, P110, P111 и WS2812 лента на P112

Резюме

- Главата демонстрира как техники от целия курс работят заедно в реална програма.
- Pre-allocated буфери, FSM state машина, hardware WS2812 и DAC аудио действат без конфликти.
- Ключовата архитектурна идея е: никакви списъци, никакви `append/remove` в `game loop` — само `bytearray` индексирани.

Самопроверка

1. Защо играта използва `bytearray` вместо `list` за куршуми и врагове?
2. Кой три FSM състояния управляват потока на играта и как се преминава между тях?
3. Как цветовото съвпадение между куршум и враг служи едновременно като геймплей механика и memory-efficient design?

Упражнения

1. Добавете четвърти цвят (бял) и четвърти бутон — как се мащабира архитектурата?
Подсказка: разширете `'b_color'` и `'e_color'` до 4 стойности и добавете нова колона в `'COLORS'` tuple-a.
2. Добавете „живот“ показател: LED 0 да показва броя останали животни (1-3) в различни цветове.
Подсказка: `'lives_colors' = {3: (0,20,0), 2: (20,20,0), 1: (20,0,0)}` и рендирайте LED 0 от тях преди `'strip.write()'`.
3. Увеличете `ENEMY_COUNT` до `LED_COUNT - 5` и измерете дали `render loop`-ът забавя при пълен екран.
Подсказка: хронометрирайте `'time.ticks_us()'` около `'push()'` — очакван е линеен ръст с броя активни пиксели.

Ако не работи

- Симптом: лентата не светва. Проверка: `PIN_WS_POWER = "P500"` трябва да е HIGH преди WS2812 init.
- Симптом: RetroSynth хвърля `OSError`. Проверка: `retro_synth.py` трябва да е копиран на /flash преди стартиране.
- Симптом: играта замръзва при много врагове. Проверка: намалете `ENEMY_COUNT` или `LED_COUNT`.

Нататък

С RGB_Guardian вече имате не само набор от работещи примери, а и модел как от тях да построите цялостно приложение за реален борд. **Част X** навлиза в специализираните DSP теми.

Каталог на всички примери

Този каталог е финалната проверка, че нито един пример не е пропуснат в книгата.

- `examples/01_blink_async.py` : асинхронно мигане на LED1
- `examples/02_two_tasks.py` : две едновременни `asyncio` задачи
- `examples/03_pwm_ab_same_channel.py` : два PWM изхода върху един GPT канал
- `examples/04_touchpad_async.py` : асинхронно следене на TouchPad с `cached API`
- `examples/dac_dmac_diag.py` : DMAC регистрова диагностика за `timed DAC path-a`
- `examples/dac_firmware_probe.py` : директна проверка на DTC и DMAC `timed DAC firmware path-a`
- `examples/dac_retro_music.py` : кратка мелодия и `arpeggio bed` върху DAC
- `examples/dac_retro_sfx.py` : готов demo за `coin, jump, laser, explosion` върху DAC
- `examples/неопixel.py` : NeoPixel rainbow cycle с P500 захранване и P112 data линия
- `examples/неопixel_test.py` : кратък NeoPixel smoke test с основни цветове и изгасване
- `examples/retro_synth.py` : reusable helper за ноти, sweep, arpeggio и ретро звукови ефекти
- `examples/ws2812_sci.py` : WS2812 rainbow cycle през `machine.WS2812` и `SCI2` върху P112
- `examples/ws2812_sci_test.py` : кратък WS2812 smoke test през `machine.WS2812`
- `examples/ws2812_sci_chase_145.py` : три движещи се цвята върху `1 + 12*12` WS2812 светодиода
- `examples/ws2812_sci_p101.py` : WS2812 rainbow cycle през `machine.WS2812` върху алтернативния `SCI0 pin P101`
- `examples/ws2812_sci_p109.py` : кратък WS2812 smoke test през `machine.WS2812` върху алтернативния `SCI9 pin P109`
- `examples/20_language_basics/01_sample_program.py` : първа примерна програма
- `examples/20_language_basics/02_variables_and_constants.py` : променливи и константи
- `examples/20_language_basics/03_operators_conditions_loops.py` : оператори, условия и цикли
- `examples/20_language_basics/04_enums.py` : enum-подобни стойности с `const()`
- `examples/20_language_basics/05_arrays.py` : списъци, вложени таблици, `array`, `bytearray`
- `examples/20_language_basics/06_structures.py` : `dict`, `namedtuple`, вложени структури
- `examples/20_language_basics/07_pointers_equivalents.py` : `references`, `memoryview`, `machine.mem32`
- `examples/20_language_basics/08_functions.py` : функции, `callbacks` и таблици от функции
- `examples/21_digital_io/01_digital_output.py` : цифров изход с LED1
- `examples/21_digital_io/02_digital_input.py` : цифров вход с SW1
- `examples/22_analog/01_analog_input_adc.py` : ADC вътрешни и външни канали
- `examples/22_analog/02_pwm_output.py` : базов PWM изход
- `examples/23_timing/01_delays_and_timers.py` : блокиращо закъснение, хардуерен таймер, `uasyncio`
- `examples/23_timing/02_interrupts.py` : GPIO прекъсвания и защитено четене
- `examples/23_timing/03_button_debounce.py` : дебаунс с `polling` и с `uasyncio`
- `examples/23_timing/04_software_timer_minus1.py` : софтуерен `Timer(-1)`
- `examples/23_timing/05_timer_compare_hw_sw.py` : сравнение между `Timer(1)` и `Timer(-1)`
- `examples/23_timing/06_rtc_basic.py` : RTC основи
- `examples/23_timing/07_adc_timing_measure.py` : измерване на `ADC.read()` с `time.ticks_us()`
- `examples/23_timing/08_interrupt_latency_measure.py` : ориентировъчно измерване на GPIO interrupt latency
- `examples/23_timing/09_button_polling_events.py` : `polling` четене на бутон със `PRESS`, `RELEASE`, `CLICK` и `LONG_PRESS`
- `examples/23_timing/10_button_irq_deferred.py` : бутон по прекъсване с отложена `debounce` обработка
- `examples/23_timing/11_button_soft_timer_hold.py` : `Timer(-1) sampler` с `PRESS`, `RELEASE`, `HOLD_START` и `HOLD_REPEAT`
- `examples/23_timing/12_button_shift_debounce.py` : `debounce` чрез битово изместване и `shift-register` история

- `examples/23_timing/13_hardware_timer_periodic_oneshot.py` : хардуерен `Timer.PERIODIC`, `Timer.ONE_SHOT`, `period()` и `counter()`

- `examples/23_timing/14_hardware_timer_output_compare.py` : `output compare callback`-и за `channel(1)` и `channel(2)`

- `examples/23_timing/15_hardware_timer_input_capture.py` : `input capture` за период и `pulse width`

- `examples/23_timing/16_hardware_timer_event_count.py` : `event count` и `callback` на всеки `N` импулса чрез

базовия `timer callback`

- `examples/23_timing/17_fast_irq_two_buttons_queue.py` : два бутона с `fast ASM IRQ`, `shared ring buffer` и печат

извън прекъсването

- `examples/AGT_TEST_CHECKLIST.md` : практичен `runtime checklist` за AGT таймерния слой на борда

- `examples/24_i2c/01_i2c_master_basic.py` : базов I2C master

- `examples/24_i2c/02_i2c_runtime_pins.py` : I2C с `runtime` зададени пинове

- `examples/24_i2c/03_softi2c_basic.py` : `SoftI2C`

- `examples/24_i2c/04_i2c_target_memory.py` : `I2CTarget` с `memory-backed` буфер

- `examples/25_spi/01_spi_basic.py` : `hardware SPI`

- `examples/25_spi/02_softspi_basic.py` : `SoftSPI`

- `examples/26_touchpad/01_touchpad_basic.py` : базов `TouchPad`

- `examples/26_touchpad/02_touchpad_diagnostics.py` : диагностика, `tuning` и `cached API`

- `examples/27_storage/01_flash_blockdev.py` : `renesas.Flash` и `/flash`

- `examples/27_storage/02_dataflash_basic.py` : `dataflash` модул

- `examples/27_storage/03_dataflash_state_led.py` : устойчиво състояние в `dataflash` и `LED` индикация

- `examples/28_machine_misc/01_machine_info_and_id.py` : `machine.info`, `freq`, `unique_id`

- `examples/28_machine_misc/02_machine_mem_and_bootloader.py` : `disable_irq`, `enable_irq`, `mem32`, `bootloader`

- `examples/28_machine_misc/03_sleep_modes.py` : `sleep`, `lightsleep`, `deepsleep`

- `examples/28_machine_misc/04_memory_management.py` : `gc`, `micropython.mem_info`, `bytearray`, `memoryview`

- `examples/29_display_scan/01_dynamic_7seg_and_keypad.py` : динамична индикация и матрично сканиране

- `examples/30_fsm/01_fsm_lamp_button.py` : минимална `FSM`

- `examples/30_fsm/02_fsm_table.py` : таблична `FSM`

- `examples/30_fsm/03_fsm_timeout.py` : `FSM` с `timeout`

- `examples/30_fsm/04_conditions_vs_events.py` : `conditions` срещу `events`

- `examples/30_fsm/05_guard_functions.py` : `guard` функции

- `examples/30_fsm/06_priorities.py` : приоритети между събития

- `examples/30_fsm/07_event_queue.py` : `event queue`

- `examples/30_fsm/08_tickless_sleep.py` : `tickless` идея и `sleep`

- `examples/32_uart/01_uart_basic.py` : базова `UART` комуникация и `loopback` тест

- `examples/34_external_modules/01_i2c_module_probe.py` : I2C scan и безопасен `probe` на външен модул

- `examples/34_external_modules/02_spi_module_transaction.py` : `SPI framed transaction` с `CS` и анализ на суровия

отговор

- `examples/34_external_modules/03_uart_module_request_response.py` : `UART request/response` с `timeout` към

външен модул

- `examples/33_asyncio/01_asyncio_basics.py` : `asyncio` основи с три паралелни задачи

- `examples/33_asyncio/02_asyncio_event_flag.py` : `asyncio Event` синхронизация

- `examples/index.py` : скрипт за листване на всички примери на борда

Част X: Курс по елементарна сигнална обработка

Pin] Тази част е навигационен обзор. Пълното съдържание на 12-те урока — с код, диаграми и обяснения — се намира в придружаващия документ: `[dsp_for_kids/vk_ra4m2_signal_processing_course_kids_bg.md` Курсът е завършен — 12 урока (0–11), 19 PNG диаграми, готов код за всеки урок. Тук са включени ключовите визуализации и кодови фрагменти за бърза ориентация.

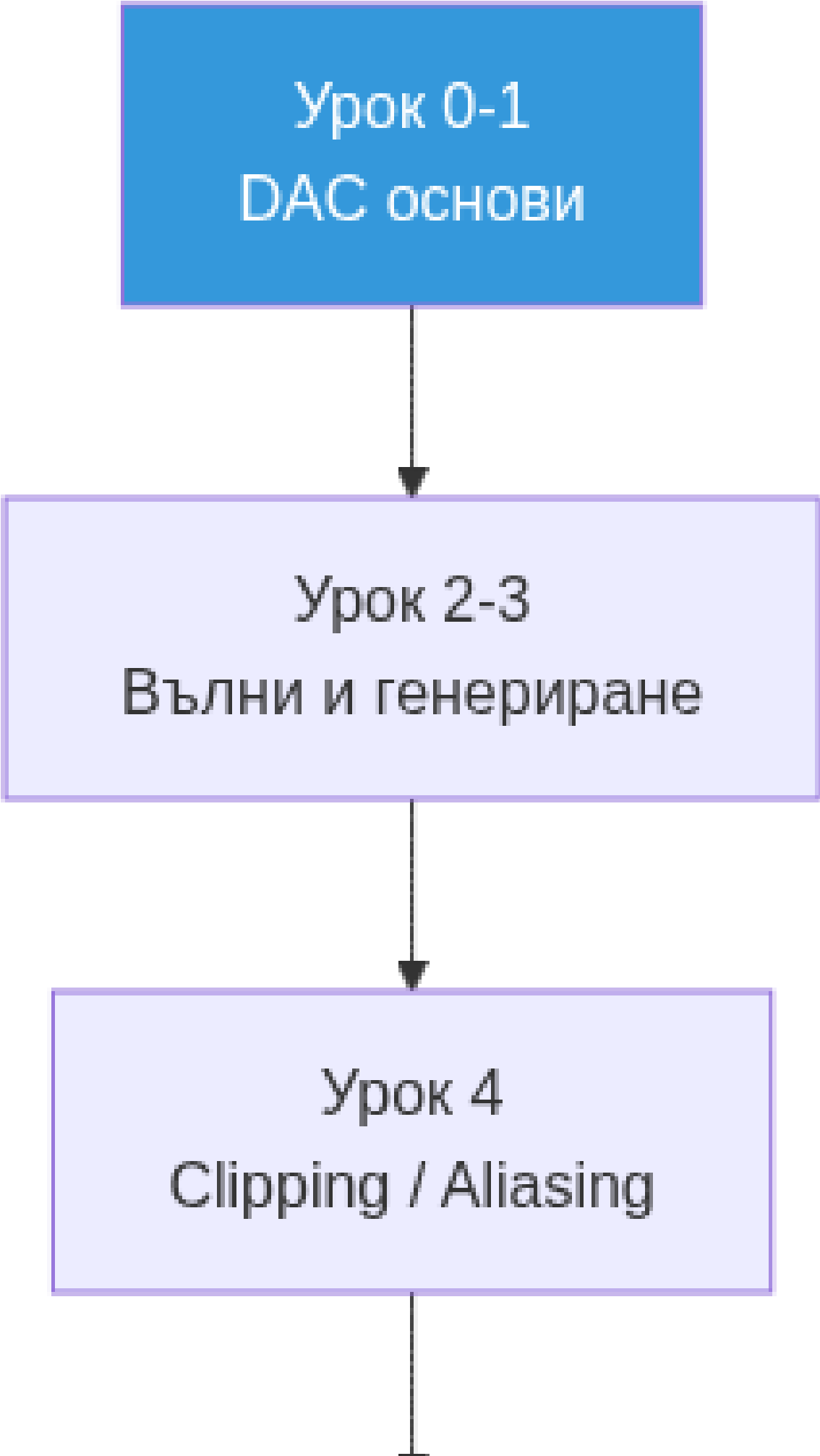
Защо тази част е преди техническата DSP архитектура? Елементарният курс въвежда *sample rate*, *DC/AC*, *clipping*, *wavetable* и *FFT* на достъпно ниво. Тези концепции са необходима основа за разбирането на *Storm ADC*, *CMSIS-DSP* и *MSGEQ7* емуляцията в Част XI.

Тази част изгражда подреден курс върху вече наличните блокове:

Блок	Роля
machine.DAC + DTC/DMAC	Точен аналогов изход към P014
retro_synth.py	Wavetable синтез и SFX
audiomixer	Block-based mono playback и смесване
dsp (CMSIS-DSP)	RMS, power, FIR, IIR, FFT

Учебен маршрут

Урок 0-1
DAS основи



```
graph TD; A[Урок 0-1  
DAS основи] --> B[Урок 2-3  
Вълни и генериране]; B --> C[Урок 4  
Clipping / Aliasing]; C --> D[ ];
```

Урок 2-3
Вълни и генериране

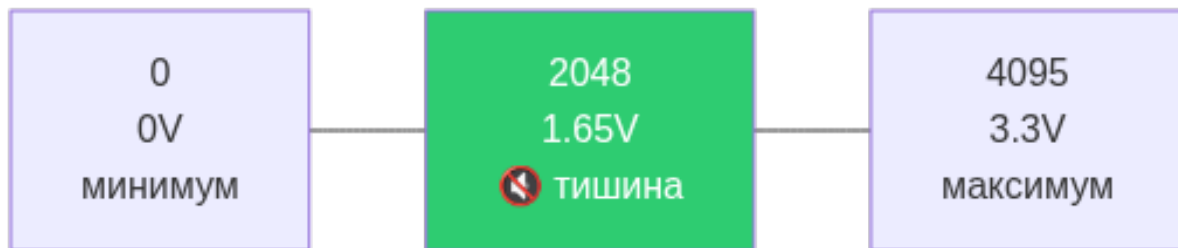
Урок 4
Clipping / Aliasing

Как работи DAC pipeline



DAC pipeline — число → напрежение → звук

Числото от 0 до 4095 се превръща в напрежение от 0V до 3.3V. При тишина подаваме 2048 (средата).



DC/AC обхват: 0 → 2048 → 4095

Как работи курсът

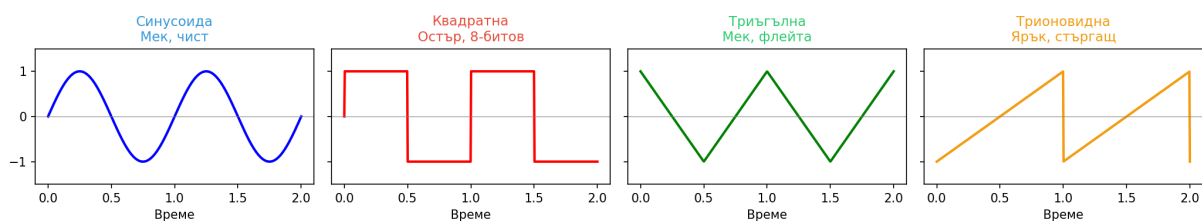
Всеки урок има:

1. **Аналогия** от реалния свят
2. **Код** — готов скрипт за платката
3. **Наблюдение** — какво да чуеш / видиш
4. **Експеримент** — промени 1 нещо и виж разликата

Златно правило: Променяй само по ЕДНО нещо наведнъж.

Вълновите форми

Вълнови форми

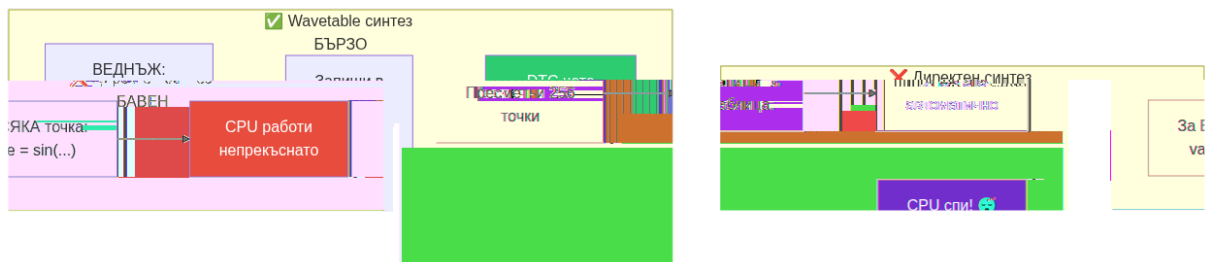


Четири вълнови форми

```

from retro_synth import RetroSynth
synth = RetroSynth("P014")
synth.play_note("A4", 500, "square")    # Квадратна — "чиптюн"
synth.play_note("A4", 500, "triangle")  # Триъгълна — "мека флейта"
synth.play_note("A4", 500, "saw")       # Трионовидна — "остър тембър"
  
```

Wavetable синтез — защо е бързо

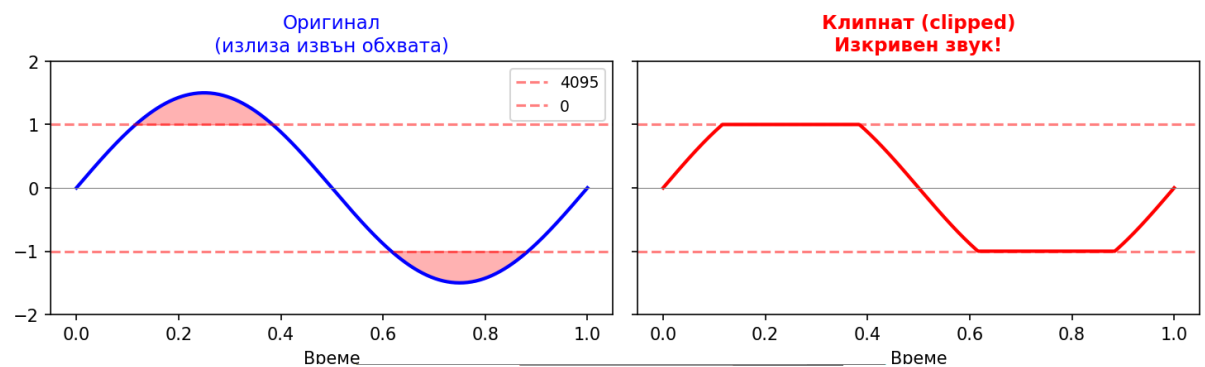


Wavetable синтез — директен vs lookup

Пресметни таблицата **веднъж**, DTC я чете автоматично. CPU спи.

Клипване

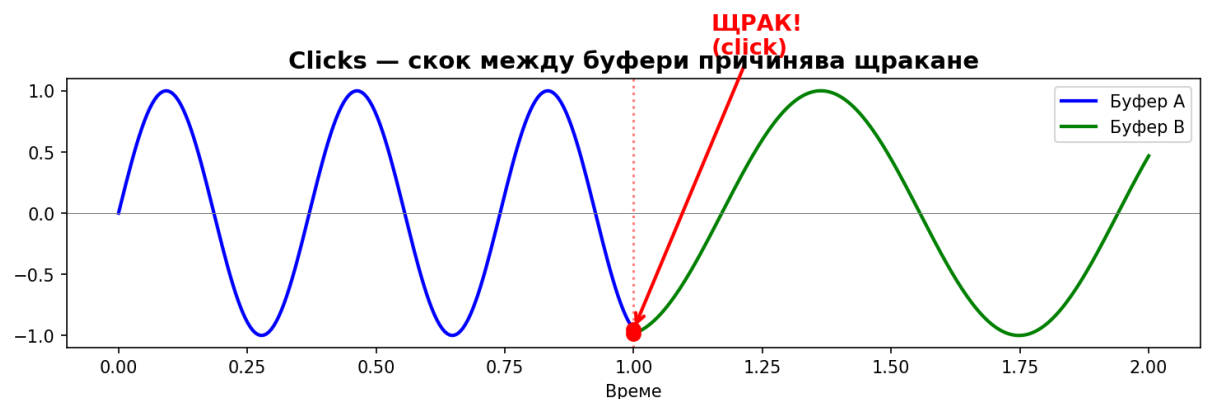
Клипване (Clipping)



Клипване — оригинал vs clipped

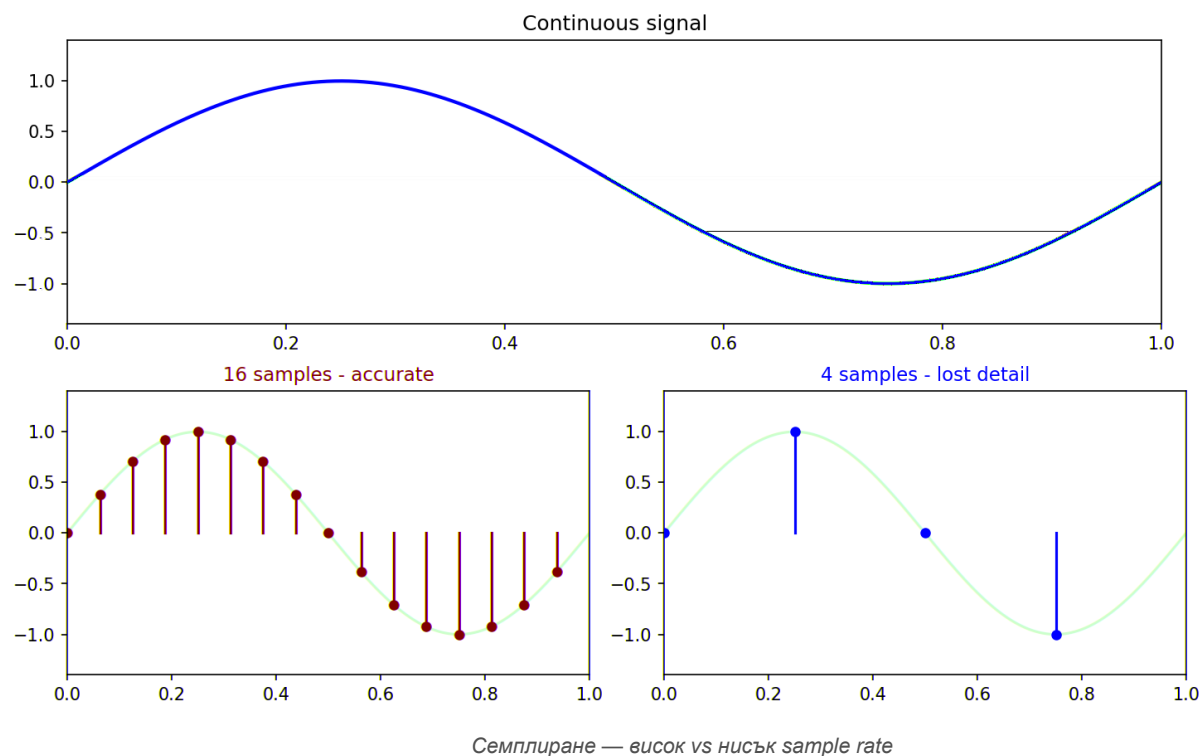
```
def clamp12(v):
    if v < 0: return 0
    if v > 4095: return 4095
    return v
```

Правило: При смесване на N сигнала, намали всеки по $1/N$.

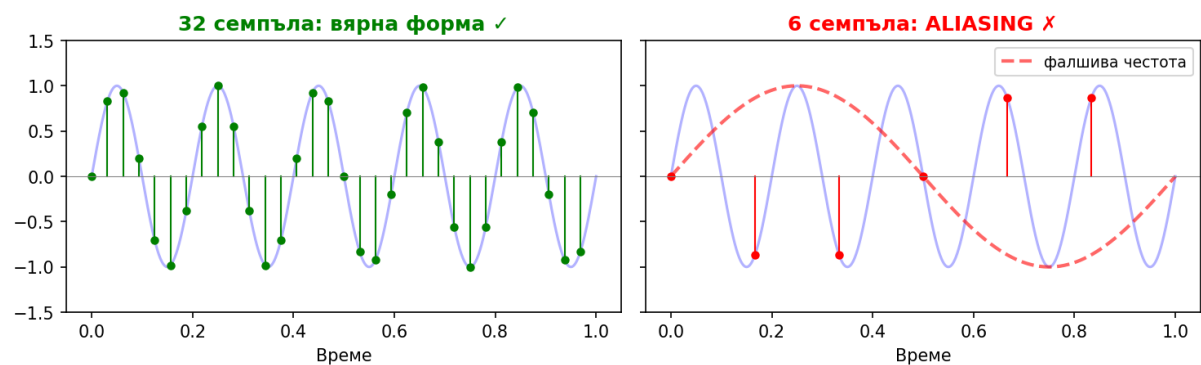


Clicks — скок между буфери

Sampling и Aliasing

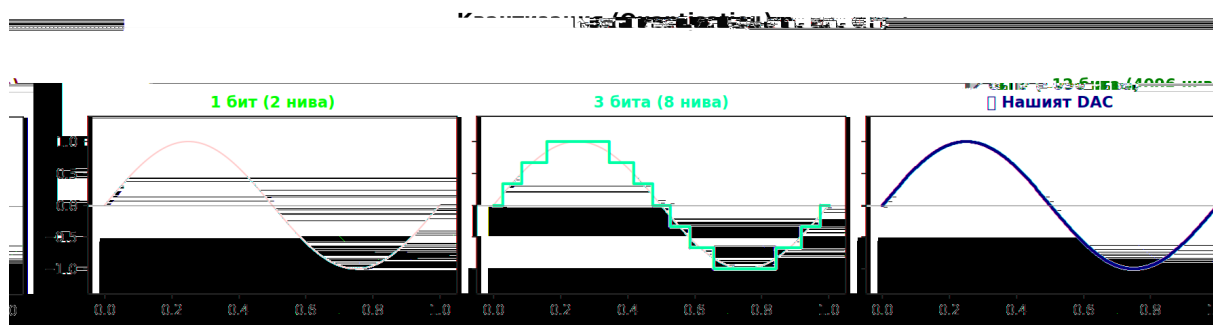


Aliasing — фалшиви честоти при нисък sample rate



Aliasing — достатъчен vs нисък sample rate

Квантизация

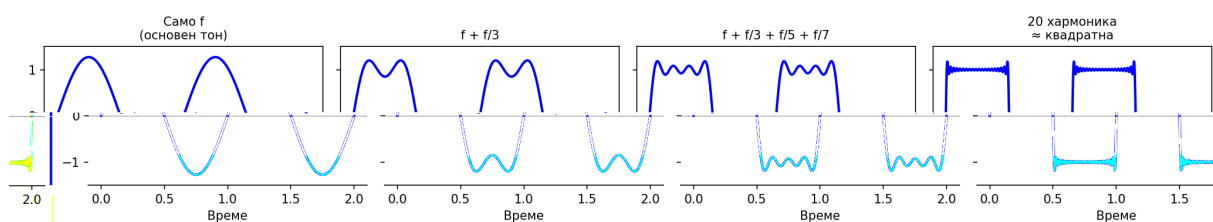


Квантизация — 1/8/12/16 бита

VK_RA4M2 DAC е **12-битов**: стойности 0–4095.

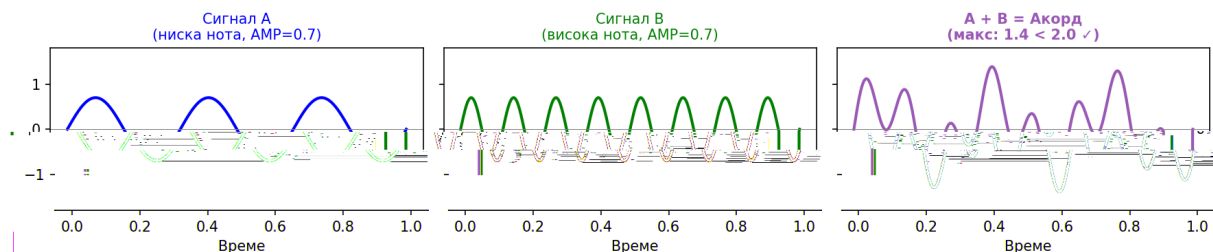
Теоремата на Фурие

Теорема на Фурие — квадратна вълна от синусоиди



Аддитивен синтез — хармоници

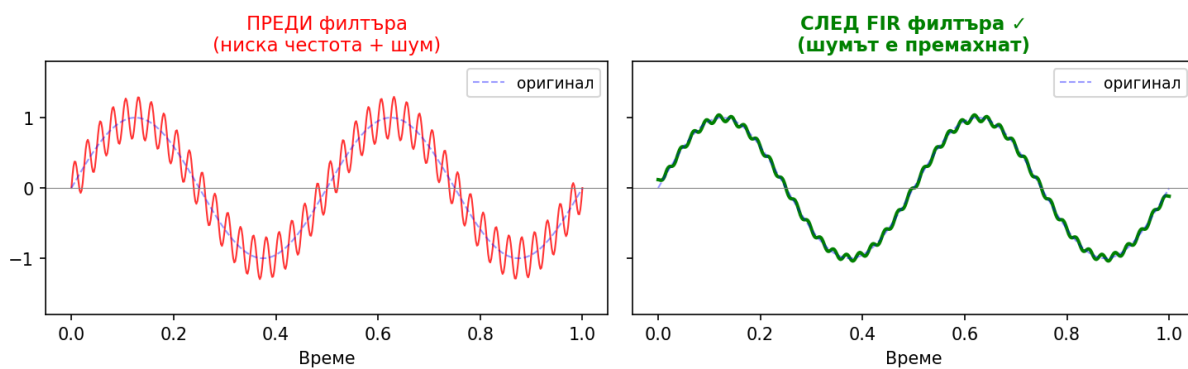
Смесване на два сигнала (адитивен синтез)



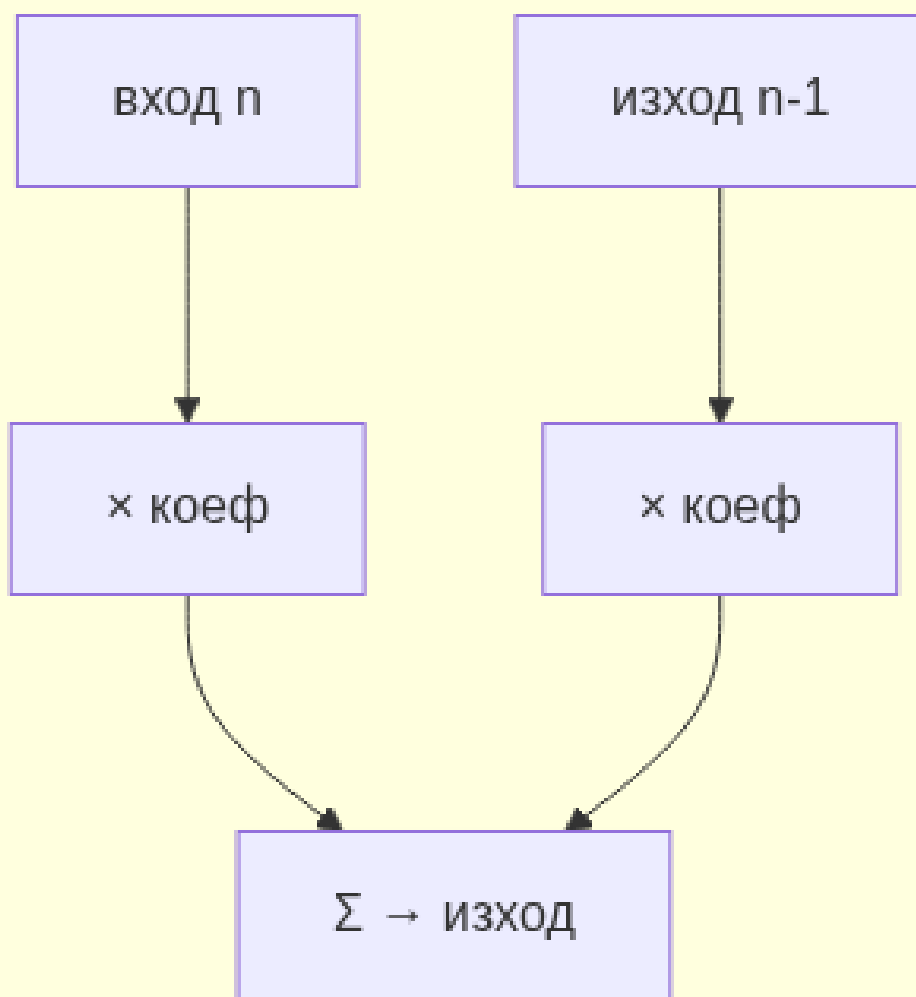
Смесване на два тона

FIR и IIR филтри

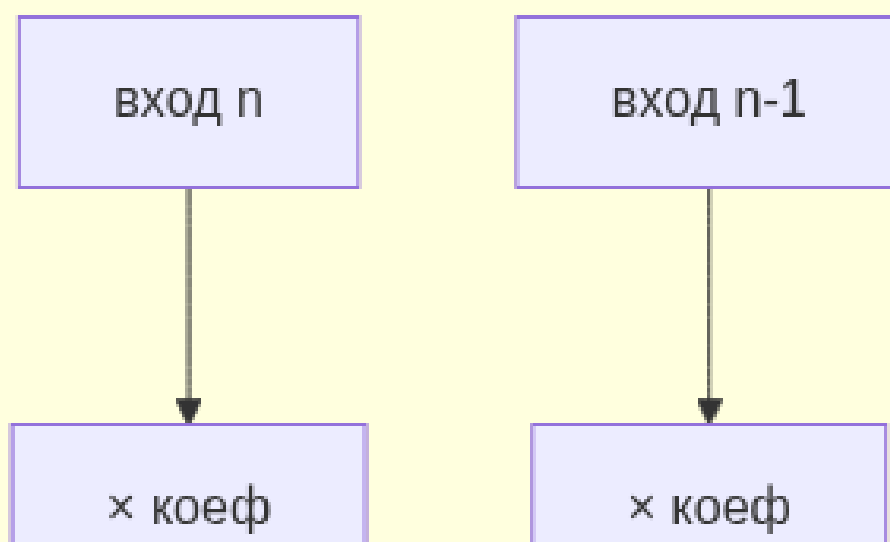
FIR нискочестотен филтър



FIR филтър

IIR с памятью 

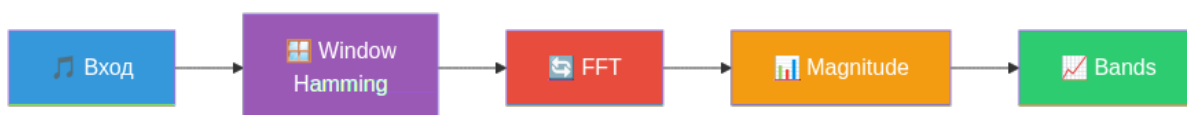
FIR без памяти



FIR vs IIR

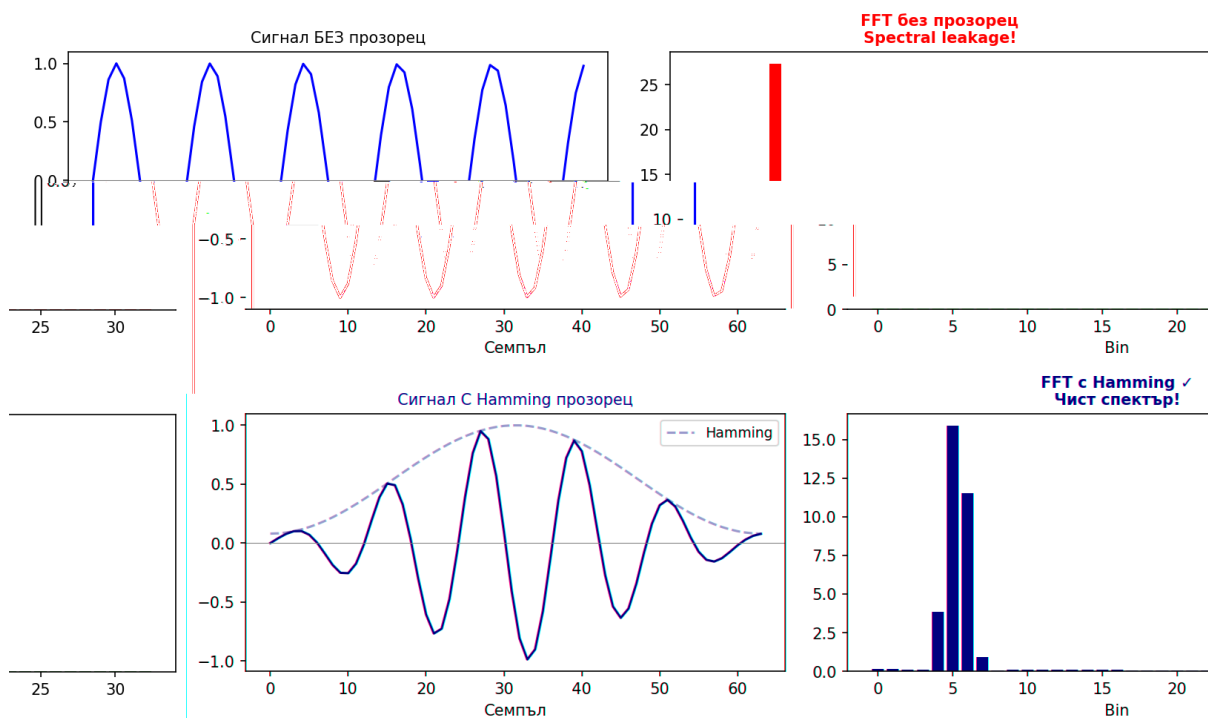
```
import dsp
fir = dsp.FIRFilter([0.2, 0.2, 0.2, 0.2, 0.2]) # 5-tap moving average
fir.run(input_buf, output_buf)
```

FFT спектър



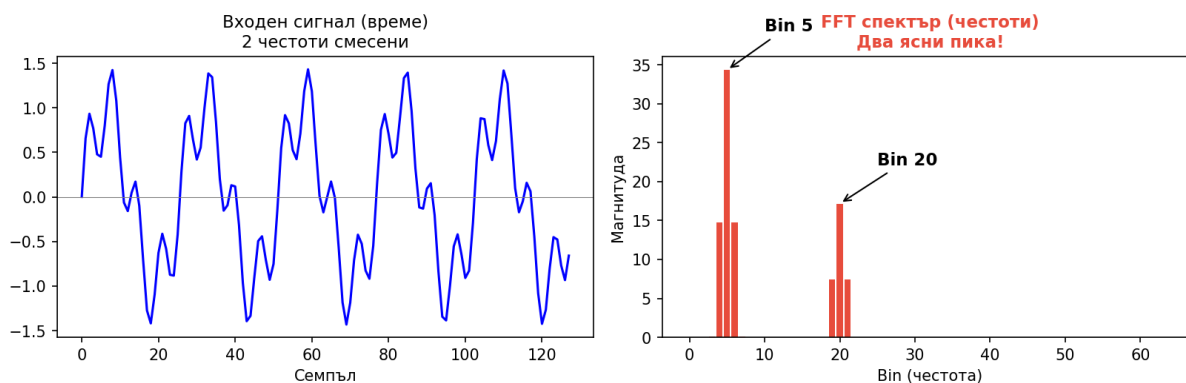
FFT pipeline

Hamming прозорец — защо е нужен?



Hamming window

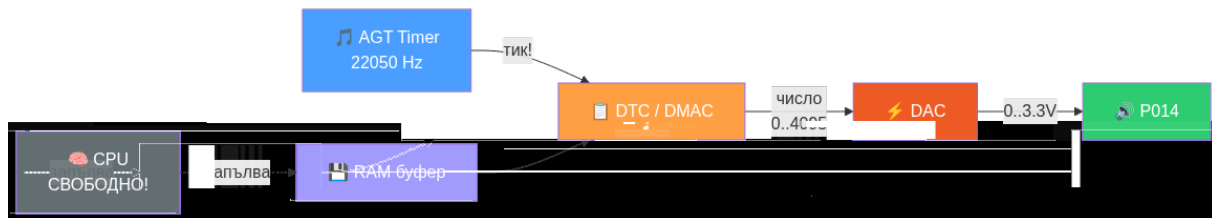
FFT — от време към честота



FFT спектър на реален сигнал

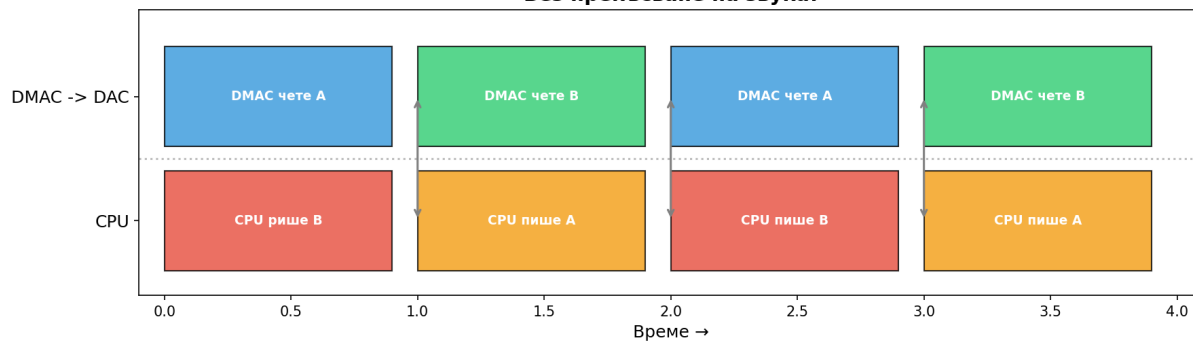
```
fft = dsp.FFT(128)
fft.run(buf, out)    # out[i] = magnitude на i-тия честотен bin
```

Hardware audio pipeline — DTC/DMAC



Hardware DAC pipeline — AGT + DTC + CPU

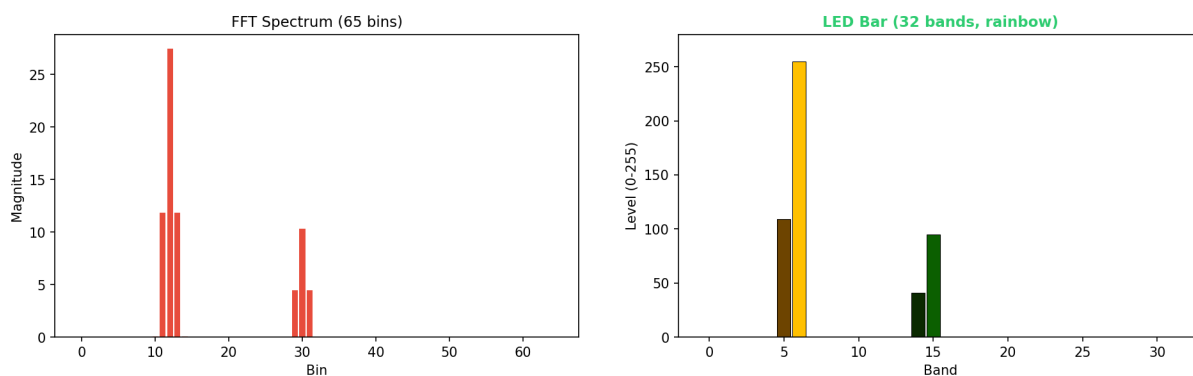
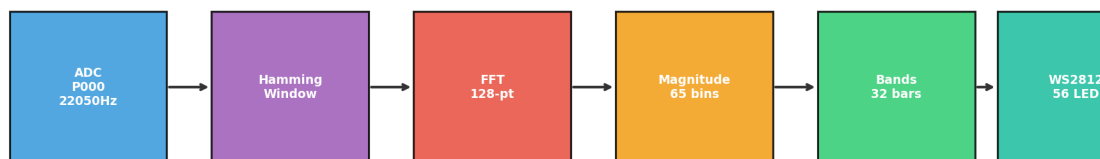
Double Buffering (Ping-Pong) Без прекъсване на звука!



Double buffer — ping-pong

Спектрален анализатор с LED лента

Spectrum Analyzer - DSP Pipeline



Spectrum Analyzer — pipeline

```

from machine import AudioADC
import dsp

adc = AudioADC("P000", fs=22050, frame=128)
adc.start()
fft = dsp.FFT(128)
while True:
    if adc.ready():
        adc.read_f32(buf)
        fft.run(buf, mag)
        # → mag[i] = сила на честота i * (22050 / 128) Hz

```

Дванадесетурочен маршрут

#	Урок	Ключова концепция	Файл в dsp_for_kids/
0	Първият звук	DAC, array('H'), MID=2048	vk_ra4m2_signal_processing_course_kids_bg.md
1	DC, AC и средната точка	DC offset, амплитуда	същият файл
2	Вълновите форми	sine, square, triangle, saw	същият файл
3	Генериране с числа	wavetable, math.sin, sample rate	същият файл
4	Signed/Unsigned, clipping, aliasing	clamp12(), Найкуист	същият файл
5	Измерване: RMS и Power	dsp.rms(), dsp.power()	същият файл
6	Смесване	адитивен синтез, headroom	същият файл
7	FIR филтър	moving average, нискочестотен	същият файл
8	IIR biquad	рекурсивен филтър	същият файл
9	FFT спектър	честотен анализ, Hamming window	същият файл
10	Audiomixer	block playback, без underrun	същият файл
11	Спектрален анализатор	AudioADC + FFT + WS2812	dsp_for_kids/ → spectrum_analyzer.py

Част XI: Цифрова обработка на аудио сигнали (DSP)

Ключови термини: FFT — алгоритъм: 128 семпла (време) → 64 бина (честота). ping-pong буфер — двоен буфер: DTC пълни единия, CPU обработва другия. CMSIS-DSP — ARM библиотека с оптимизирани DSP функции. sample rate — семпли/сек (22050 Hz). Nyquist — максимална честота = $F_s/2$. magnitude — енергия на честотен бин: $\sqrt{Re^2 + Im^2}$. Q15 — fixed-point формат за бързо аудио.

Тази част обхваща пълния аудио pipeline: от аналогов вход (Storm ADC с DTC ping-pong буфер) през цифров анализ (FFT с CMSIS-DSP, MSGEQ7 спектрална визуализация) до цифров синтез (wavetable DAC с RetroSynth).

Глава 53. Storm ADC — DTC-управлявано аудио семплиране [Напреднал]

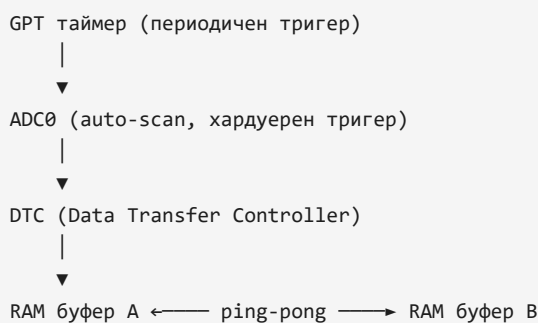
[Tip] Аналогия: ADC.read() е като да изпратите служител да носи по един лист хартия от архива. Storm ADC е конвейерна лента — GPT таймерът дава такт, DTC носи данните автоматично, CPU само взима готовите пакети. Без участие на Python при всеки семпл.

Метод	Кой взима семпла	Гарантиран timing	Аудио?
ADC.read() в Python loop	Python (CPU)	[X] GC може да закъснее	[X]
Storm ADC (GPT+ADC+DTC)	DTC хардуер	[OK] Прецизен до μs	[OK] 22050 Hz

Какъв е проблемът

Стандартният ADC.read() в MicroPython чете **една стойност** по заявка. За аудио анализ ни трябва непрекъснат поток от семпли с точно фиксирана честота (напр. 22050 Hz). Ако Python цикълът се забави с 1 ms заради garbage collector, се губят 22 семпла — и спектърът се изкривява безвъзвратно.

Решението е **Storm ADC** — хардуерна верига, която работи изцяло без участие на CPU:

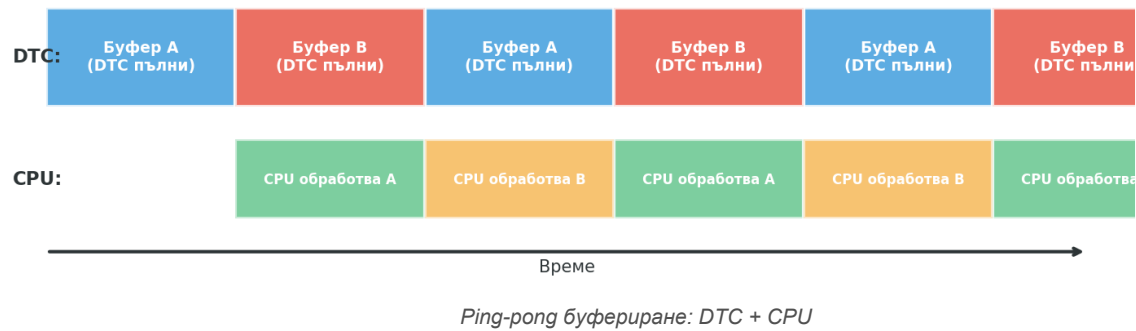


Архитектура: GPT → ADC → DTC → ping-pong

GPT таймер генерира периодичен TRGA сигнал с честота F_s . На всеки фронт ADC0 стартира конверсия на избрания канал. Когато конверсията завърши, DTC автоматично копира резултата от ADDR (ADC data register) в RAM.

DTC работи в **repeat-block** режим: след frame конверсии превключва към втория буфер и продължава без прекъсване. Така CPU може да обработва буфер A, докато DTC пълни буфер B — и обратно.

Ping-pong буфериране: DTC пълни единия, CPU обработва другия — без загуба на семпли



Регистрова конфигурация (RA4M2)

GPT0 (таймер за тригер):

Регистър	Стойност	Описание
GTCR.MD	0 (Saw-wave)	Нарастващ брояч с нулиране при период
GTPR	PCLKD / Fs - 1	Период = честота на дискретизация
GTIOR	TRGA enable	Тригер при GTCNT == 0
GTSSR	Software start	Стартиране от код

При PCLKD = 50 MHz и Fs = 22050 Hz: $GTPR = 50000000 / 22050 - 1 = 2267$.

ADC0 (аналогово-цифров преобразувател):

Регистър	Стойност	Описание
ADCSR.ADCS	0b10 (Group scan)	Групово сканиране
ADCSR.TRGE	1	Хардуерен тригер разрешен
ADSTRGR.TRSA	GPT0 TRGA	Източник на тригер
ADANSA0	bit за избрания канал	Кой AN вход да се конвертира
ADCER.ADRFMT	0 (Right-aligned)	12-бит дясно подравнени данни

DTC (Data Transfer Controller):

Параметър	Стойност	Описание
Source	&ADC0.ADDR[ch]	ADC data register (16-bit)
Destination	buf_a[] / buf_b[]	RAM ping-pong буфери
Transfer mode	Repeat-block	Автоматично зацикляне
Block size	frame (128 семпла)	Размер на FFT прозореца
Data width	16-bit	Съответства на ADC резолюцията
Repeat area	Destination	Адресът на destination се нулира при край

MicroPython интерфейс — клас AudioADC

```

from machine import AudioADC

adc = AudioADC("P000", fs=22050, frame=128)
adc.start()          # Стартира GPT + ADC + DTC.

while not adc.ready(): # Проверява дали DTC е запълнил текущия буфер.
    pass

adc.read_f32(buf)     # Копира 128 семпла като float32, нормализирани [-1.0 .. +1.0].
                     # DC offset (2048 при 12-bit) се изважда автоматично.

adc.stop()           # Спира GPT таймера, ADC и DTC.

```

„`read_f32(buf)`“ извършва следната конверсия в C (zero-alloc):

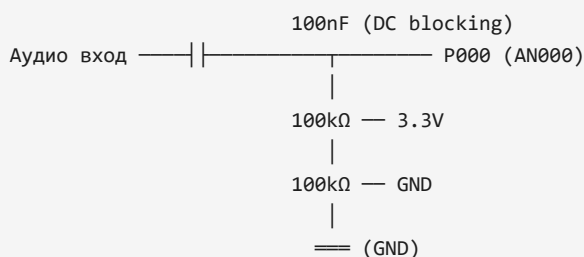
```

for (int i = 0; i < frame; i++) {
    buf[i] = ((float32_t)raw[i] - 2048.0f) / 2048.0f; // [-1.0 .. +1.0]
}

```

Стойността 2048 съответства на DC bias точката ($V_{cc}/2 = 1.65V$ при 3.3V захранване).

Входна схема (AC coupling)



- **100nF кондензатор** блокира DC компонента от аудио източника.
- **Резистивен делител 100kΩ/100kΩ** задава DC bias = $V_{cc}/2 = 1.65V$.
- ADC вижда сигнал в обхвата [0V .. 3.3V] с център 1.65V.
- Максимална амплитуда без клипване: $\pm 1.65V$ peak.

Защо е важен ping-pong буферът

Без ping-pong има два лоши варианта:

1. **Спираме ADC → четем → пускаме ADC:** губим семпли по време на четене + FFT (~2 ms). При 22050 Hz това са ~44 изгубени семпла.
2. **Четем от активния буфер:** DTC може да презапише данните докато ги четем (race condition).

Ping-pong решава и двата проблема: DTC винаги пише в единия буфер, CPU винаги чете от другия. Няма race condition, няма загуба на данни.

Файлове

- `ra/ra_storm_adc.c` — хардуерен драйвер (GPT + ADC + DTC регистрова конфигурация)
- `ra/ra_storm_adc.h` — дефиниции на `storm_adc_config_t` структурата

- `machine_audioadc.c` — MicroPython обвивка (AudioADC клас)
- `examples/hw_noise_generator_plan.py` — дизайн документ (само коментари, без runnable код): архитектура на хардуерен генератор на шум чрез TSN→ADC12→ADBUF→DMAC→RAM ring buffer при 0% CPU

Резюме

- Storm ADC е **хардуерна верига без CPU участие**: GPT тригерира ADC, DTC копира в RAM.
- Ping-pong буферът гарантира **нула загубени семпли и нула race conditions**.
- AudioADC класът предоставя прост Python интерфейс: `start()`, `ready()`, `read_f32()`, `stop()`.
- Конверсията от 12-bit unsigned към float32 [-1.0..+1.0] е вградена в `read_f32()`.

Самопроверка

1. Каква е ролята на GPT таймера в Storm ADC веригата?
2. Защо DTC работи в repeat-block режим, а не в normal режим?
3. Какво е значението на стойността 2048 при конверсията в `read_f32()`?
4. Какво би се случило, ако резистивният делител на входа е несиметричен (напр. 47kΩ + 100kΩ)?

Упражнения

1. Изчислете стойността на GPT период регистъра за $F_s = 44100$ Hz при $PCLKD = 50$ MHz.
Подсказка: `period = PCLKD / F_s - 1 = 50_000_000 / 44100 - 1 ≈ 1133` — резултатът се записва в GPT GTPR регистъра.
2. Модифицирайте входната схема за микрофонен елемент с ниско изходно напрежение (~20 mV): какъв усилвател бихте добавили?
Подсказка: нужен е предусилвател с усилване ~40 dB (100×) за довеждане на 20 mV до ~1.6V около DC bias точката.
3. Напишете скрипт, който мери реалната честота на дискретизация чрез `time.ticks_us()` и `adc.ready()`.
*Подсказка: запазете `t0 = time.ticks_us()` при стартиране и разделете `frame_count FRAME_LEN` на изминалото μs.**

Ако не работи

- Симптом: `adc.ready()` никога не връща True. Проверка: уверете се, че `adc.start()` е извикан и че GPT таймерът работи.
- Симптом: данните от `read_f32()` са постоянно ~0.0. Проверка: проверете DC bias на входа — трябва да е ~1.65V. Без резистивен делител ADC може да чете 0V.
- Симптом: спектърът показва силен DC компонент. Проверка: кондензаторът от 100nF може да е пропуснат или повреден.

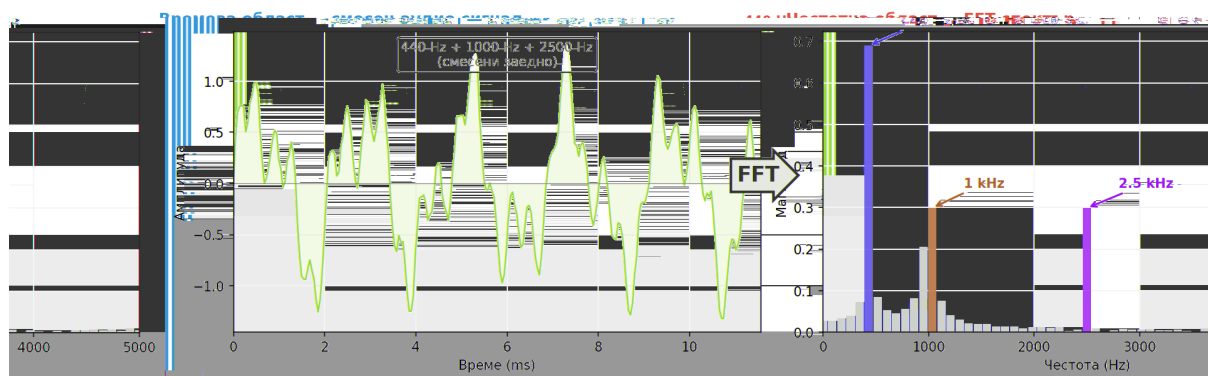
Нататък

След тази глава преминете към „FFT — честотен анализ“, за да видите как суровите PCM семпли от Storm ADC се превръщат в спектрална информация.

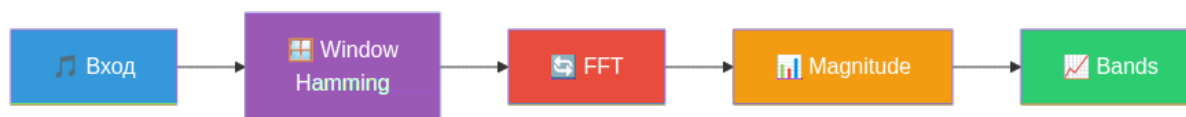
Глава 54. FFT — честотен анализ с CMSIS-DSP [Напреднал]

[Тір] Аналогия: FFT е като разложение на бяла светлина чрез призма — вместо "бял сигнал" виждате точно кои цветове (честоти) го съставят и с каква интензивност. Даваш 128 числа (времева ос) → получаваш 64 числа (честотна ос).

FFT: от времева област към честотен спектър (CMSIS-DSP)



FFT: от времева към честотна област



FFT pipeline

Какъв е проблемът

Storm ADC ни дава 128 семпла от аудио сигнала (time-domain). Но за спектрален анализатор имаме нужда от **честотна** информация: колко енергия има на 100 Hz, колко на 1 kHz, колко на 10 kHz. Трансформацията от time-domain към frequency-domain се нарича **FFT** (Fast Fourier Transform).

Горната графика показва точно тази трансформация: смесен сигнал от 440 Hz + 1 kHz + 2.5 kHz, разложен до ясни пикове в честотната област.

CMSIS-DSP: защо не пишем FFT сами

FFT алгоритъмът е математически сложен и изисква внимателна оптимизация. ARM предоставя библиотеката **CMSIS-DSP**, която включва:

- `arm_rfft_fast_f32()` — оптимизирана Real FFT за float32
- Предварително изчислени twiddle factors (sin/cos таблици)
- Използване на хардуерната FPU на Cortex-M33

RA4M2 (Cortex-M33 @ 100 MHz) изпълнява 128-точкова RFFT за **~35 μs** — повече от достатъчно за реално време.

DSP pipeline: 4 стъпки

```

PCM буфер (128 float32)
|
▼ fft.window_apply(in, out)
Hamming прозорец
|
▼ fft.run(in, out)      ← arm_rfft_fast_f32()
FFT комплексен резултат
|
▼ fft.magnitude(cplx, mag) ← arm_cmplx_mag_f32()
Магнитуди (65 реални стойности)
|
▼ fft.bands(mag, out)
N ленти (0-255 байтове)

```

Стъпка 1: Hamming прозорец

Без прозорец FFT "мисли", че сигналът е безкраен и периодичен — което води до **spectral leakage** (разливане на енергия между бинове). Hamming прозорецът плавно затихва краищата на прозореца:

$$w[n] = 0.54 - 0.46 * \cos(2\pi * n / (N-1))$$

Прозорецът се изчислява **веднъж** при създаване на FFT обекта и се съхранява в RAM. При всеки кадър `window_apply()` просто умножава:

```

for (int i = 0; i < fft_len; i++) {
    out[i] = in[i] * self->window[i];
}

```

Стъпка 2: RFFT (Real FFT)

`arm_rfft_fast_f32()` приема 128 реални стойности и произвежда 128 стойности, пакетирани като 64 комплексни двойки (Re, Im). Специален случай: bin 0 (DC) и bin 64 (Nyquist) са чисто реални и се пакетират заедно в първата двойка.

Вход: [x0, x1, x2, ..., x127] (128 float32)
Изход: [DC, Nyq, Re1, Im1, Re2, Im2, ... Re63, Im63] (128 float32)

Bin spacing = $F_s / \text{FFT_LEN} = 22050 / 128 = 172.27 \text{ Hz}$ на бин.

Bin	Честота	Пример
0	0 Hz (DC)	Постоянна компонента
1	172 Hz	Ниски басове
6	1034 Hz	Среден диапазон
29	5000 Hz	Високи средни
58	10000 Hz	Високи честоти
64	11025 Hz	Nyquist (горна граница)

Стъпка 3: Магнитуди

Комплексните стойности (Re, Im) се превръщат в реални магнитуди:

```
mag[k] = sqrt(Re[k]2 + Im[k]2)
```

CMSIS-DSP предоставя `arm_cmplx_mag_f32()`, която извлича магнитудата на всяка двойка. Резултатът е масив от 65 стойности (bin 0 до bin 64).

Стъпка 4: Лентови магнитуди (bands)

65 бина са твърде много за визуализация на 32 LED. Затова ги групираме в **N ленти** (bands) с логаритмично разпределение. Ниските честоти получават по-малко бинове (1-2), високите — повече (10-20). Това съответства на човешкото слухово възприятие.

Границите се изчисляват **веднъж** с `set_bands(n, db_floor)`:

```
float32_t ratio = powf((float32_t)half, 1.0f / (float32_t)n);
for (b = 0; b < n; ++b) {
    band_lo[b] = (uint16_t)(powf(ratio, b) + 0.5f);
    band_hi[b] = (uint16_t)(powf(ratio, b + 1) + 0.5f);
    if (band_hi[b] <= band_lo[b]) band_hi[b] = band_lo[b] + 1;
}
```

За всяка лента `bands()` намира **максималната** магнитуда, конвертира в dBFS и мащабира до 0-255:

```
for (b = 0; b < n_bands; ++b) {
    // Намиране на максимума в обхвата [band_lo .. band_hi]
    float32_t peak = 0.0f;
    for (k = band_lo[b]; k < band_hi[b]; ++k) {
        if (mag[k] > peak) peak = mag[k];
    }
    // Конверсия в dBFS: 20*log10(peak)
    float32_t db = 20.0f * log10f(peak + 1e-12f);
    // Мащабиране: db_floor..-0 → 0..255
    int32_t val = (int32_t)((db - db_floor) * db_scale);
    out[b] = (val < 0) ? 0 : (val > 255) ? 255 : (uint8_t)val;
}
```

`db_floor` контролира чувствителността:

db_floor	Значение	Използване
-40	По-малко чувствителен	Силни аудио източници
-60	Нормално (по подразбиране)	Линеен вход
-80	Много чувствителен	Микрофон, тих сигнал

MicroPython интерфейс — модул dsp

```

import dsp
from array import array

fft = dsp.FFT(128)           # Създава FFT обект за 128 семпла.
fft.set_bands(32, -60)      # 32 ленти, шумов праг -60 dBFS.

frame = array('f', [0.0] * 128) # PCM вход.
fbuf = array('f', [0.0] * 128)  # FFT работен буфер.
mag = array('f', [0.0] * 65)    # Магнитуди (N/2 + 1).
out = bytearray(32)            # 32 ленти × 1 байт.

# В главния цикъл:
fft.window_apply(frame, fbuf)   # Hamming прозорец.
fft.run(fbuf, fbuf)            # RFFT in-place.
fft.magnitude(fbuf, mag)       # Комплексни → магнитуди.
fft.bands(mag, out)            # 65 бина → 32 ленти (0-255).

```

Всички буфери са pre-allocated. В главния цикъл **няма** GC allocation — само записване в съществуващи буфери.

Връзка FFT параметри ↔ реална резолюция

FFT_LEN	Fs (Hz)	Bin spacing	Мин. честота	Макс. честота	Латентност
64	22050	344.5 Hz	344 Hz	11025 Hz	2.9 ms
128	22050	172.3 Hz	172 Hz	11025 Hz	5.8 ms
256	22050	86.1 Hz	86 Hz	11025 Hz	11.6 ms
512	22050	43.1 Hz	43 Hz	11025 Hz	23.2 ms

По-голям FFT_LEN = по-добра честотна резолюция, но по-голяма латентност и повече RAM.

Файлове

- moddsp.c — С имплементация на FFT модула (window, run, magnitude, bands)
- CMSIS-DSP библиотека (предоставена от ARM, вградена в порта)
- examples/fft_print_test.py — конзолен FFT тест: Timer(6) генерира 500 Hz квадратна вълна на P302, AudioADC чете P001, 128-pt FFT с 8-frame осредняване, ASCII хоризонтален спектър на всички 64 бина

Резюме

- FFT превръща 128 PCM семпла в 65 честотни бина за **~35 µs** на Cortex-M33.
- Hamming прозорецът намалява spectral leakage.
- set_bands() предизчислява логаритмични граници ВЕДНЪЖ, bands() ги ползва без powf().
- Всичко работи с pre-allocated буфери — нула GC натоварване.

Самопроверка

1. Защо е необходим Hamming прозорец преди FFT?
2. Колко честотни бина произвежда 128-точкова RFFT?
3. Какво означава "bin spacing = 172 Hz" на практика?
4. Защо се използва логаритмично, а не линейно разпределение на лентите?

Упражнения

1. Изчислете bin spacing за $\text{FFT_LEN}=256$ и $F_s=44100$ Hz.

Подсказка: $\text{bin_spacing} = F_s / \text{FFT_LEN} = 44100 / 256 \approx 172$ Hz — двойно по-добра резолюция от 128-точкова FFT.

2. Напишете скрипт, който измерва времето за изпълнение на всяка от 4-те DSP стъпки поотделно.

Подсказка: `t=time_ticks_us(); fft.window_apply(frame, fbuf); print(time_ticks_diff(time_ticks_us(),t))` за всяка стъпка.

3. Експериментирайте с `db_floor`: сравнете визуализацията при -40, -60 и -80 dBFS.

Подсказка: `fft.set_bands(32, -80)` прави анализатора по-чувствителен — тихи сигнали стават видими, но и шумът расте.

Ако не работи

- Симптом: всички ленти показват максимална стойност. Проверка: входният сигнал може да е прекалено силен (клипване). Намалете нивото или използвайте `db_floor=-40`.
- Симптом: само първите 1-2 ленти светят. Проверка: входният сигнал може да е чист синус на ниска честота — това е очаквано поведение.
- Симптом: спектърът изглежда "размазан". Проверка: уверете се, че `window_apply()` се извиква преди `run()`.

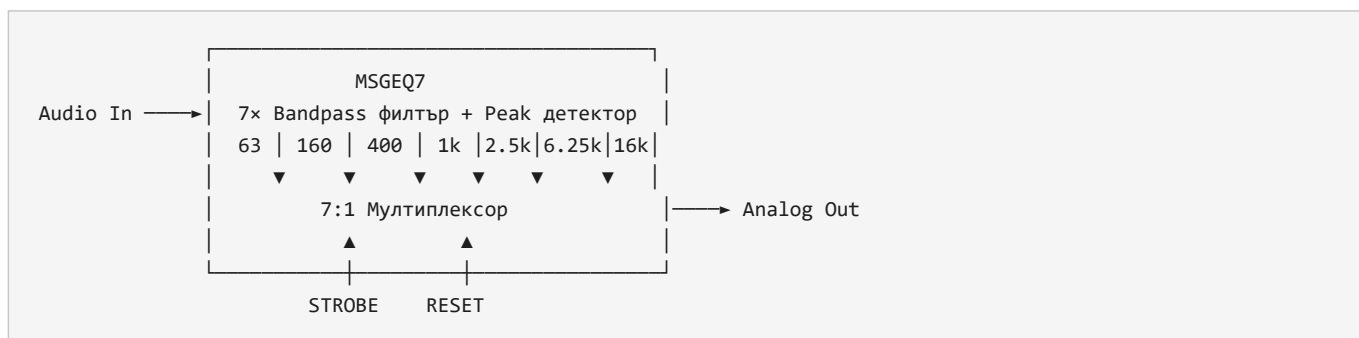
Нататък

След тази глава преминете към „MSGEQ7 емуляция“, за да видите как FFT може да замести специализиран аналогов чип за аудио спектрален анализ.

Глава 55. MSGEQ7 емуляция — 7-лентов спектрален анализатор [Напреднал]

Какъв е проблемът

MSGEQ7 е популярен аналогов чип (Mixed Signal Integration), който разделя аудио сигнала на 7 честотни ленти с хардуерни bandpass филтри и peak детектори. Той е основата на повечето LED спектрални анализатори в Arduino общността.

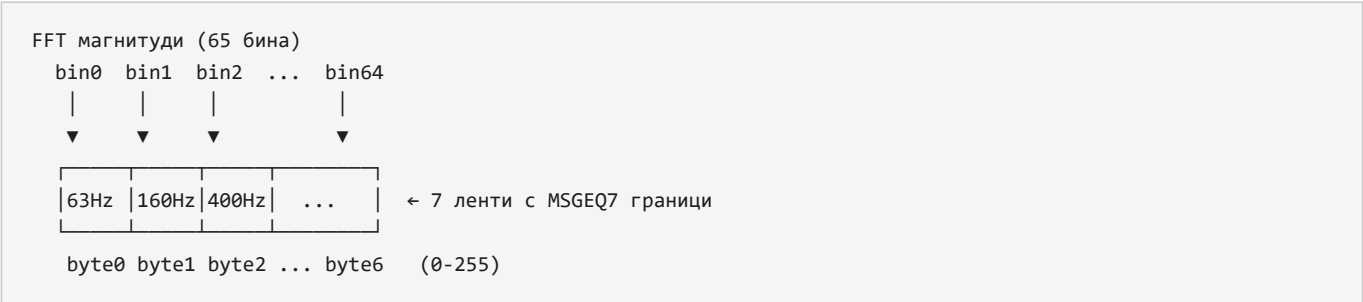


Проблемът с MSGEQ7:

Недостатък	Описание
Фиксирани 7 ленти	Не може да се промени резолюцията
Допълнителен чип	Изисква допълнителна платка и окабеляване
Нисък динамичен обхват	~40 dB (вътрешни реак детектори)
Зависимост от clock	Честотите зависят от външен RC или crystal
Шумен аналогов изход	Стъпаловидна DC стойност + switch noise

Решението: FFT-базирана емуляция

Вместо аналогов чип, използваме **цифрова емуляция**: FFT анализът от [Глава 54](#) вече ни дава пълния спектър. Трябва само да групираме FFT биновете в 7 ленти с центрови честоти, съответстващи на MSGEQ7.



Метод `set_bands_msgeq7(fs, db_floor=-60)`

Вместо логаритмично разпределение (`set_bands`), `set_bands_msgeq7` изчислява **геометрични средни** между съседни MSGEQ7 центрови честоти като граници:

Центрове:	63	160	400	1000	2500	6250	16000
	↓	↓	↓	↓	↓	↓	
Граници:	39	100	253	632	1581	3953	10000 Nyquist

Граница между две съседни ленти = $\text{sqrt}(\text{fc}[i] * \text{fc}[i+1])$:

Граница	Изчисление	Честота
Долна на 63 Hz	$63 / 1.6$	≈ 39 Hz
63–160 Hz	$\sqrt{63 \times 160}$	≈ 100 Hz
160–400 Hz	$\sqrt{160 \times 400}$	≈ 253 Hz
400–1000 Hz	$\sqrt{400 \times 1000}$	≈ 632 Hz
1000–2500 Hz	$\sqrt{1000 \times 2500}$	≈ 1581 Hz
2500–6250 Hz	$\sqrt{2500 \times 6250}$	≈ 3953 Hz
6250–16000 Hz	$\sqrt{6250 \times 16000}$	≈ 10000 Hz
Горна на 16 kHz	Nyquist	11025 Hz

Тези граници се превръщат в **FFT бин индекси** чрез деление на bin spacing:

```
bin = round(freq / bin_width)
bin_width = Fs / FFT_LEN = 22050 / 128 = 172.27 Hz
```

С имплементация в `moddsp.c`:

```
static const float32_t msgeq7_fc[7] = {
    63.0f, 160.0f, 400.0f, 1000.0f, 2500.0f, 6250.0f, 16000.0f
};

for (uint8_t b = 0; b < 7; ++b) {
    float32_t f_lo = (b == 0)
        ? msgeq7_fc[0] / 1.6f
        : sqrtf(msgeq7_fc[b-1] * msgeq7_fc[b]);
    float32_t f_hi = (b == 6)
        ? nyquist
        : sqrtf(msgeq7_fc[b] * msgeq7_fc[b+1]);

    uint32_t blo = (uint32_t)(f_lo / bin_w + 0.5f);
    uint32_t bhi = (uint32_t)(f_hi / bin_w + 0.5f);
    // ... clamp & store
}
```

След `set_bands_msgeq7()` се използва **същият** `bands()` метод — той вече знае да работи с 7 ленти.

Бинова карта при $F_s=22050$, $FFT_LEN=128$

Лента	Център	f_low	f_high	bin_lo	bin_hi	Бинове
0	63 Hz	39 Hz	100 Hz	1	1	1
1	160 Hz	100 Hz	253 Hz	1	1	1
2	400 Hz	253 Hz	632 Hz	2	4	3
3	1 kHz	632 Hz	1581 Hz	4	9	6
4	2.5 kHz	1581 Hz	3953 Hz	9	23	14
5	6.25 kHz	3953 Hz	10000 Hz	23	58	35
6	16 kHz	10000 Hz	11025 Hz	58	64	6

Забележка: При $FFT_LEN=128$ лентите за 63 Hz и 160 Hz имат само по 1 бин — ограничение на резолюцията. За по-добра резолюция на ниските честоти е необходим $FFT_LEN \geq 512$.

Визуализация: 7 × 8 LED bar graph

В MSGEQ7 режим 56 WS2812 светодиода се организират като матрица 7 колони × 8 реда. Всяка колона съответства на честотна лента. Височината на светещата колона показва нивото.

![7-лентов спектрален анализатор – LED визуализация](img/55_spectrum_7band.png)

Render функция (zero-alloc):

LEDS_PER_BAND = 8

def render_msgeq7(pb, n_bands):

for b in range(n_bands):

level = pb[b]

lit = (level * LEDS_PER_BAND + 127) >> 8 # 0..8 запалени

for j in range(LEDS_PER_BAND):

ofs = (b * LEDS_PER_BAND + j) * 3

```

if j < lit:
bv = (level * BRIGHTNESS) >> 8
if j < 3: rgb_to_buf(0, bv, 0, pixel_buf, ofs) # Зелено
elif j < 6: rgb_to_buf(bv, bv, 0, pixel_buf, ofs) # Жълто
else:     rgb_to_buf(bv, 0, 0, pixel_buf, ofs) # Червено
else:
pixel_buf[ofs] = pixel_buf[ofs+1] = pixel_buf[ofs+2] = 0

```

Сравнение: истински MSGEQ7 vs цифрова емуляция

Параметър	Хардуерен MSGEQ7	FFT емуляция (RA4M2)
Брой ленти	7 (фиксиран)	7 (конфигуруеми)
Динамичен обхват	~40 dB	>90 dB (float32 FFT)
Допълнителен хардуер	MSGEQ7 + clock + capacitors	Нищо (само ADC + LED)
Гъвкавост	Фиксиран	7 или 32 ленти, сменяем
Цена	~\$2 за чип + PCB	\$0 (вградено в софтуера)
Strobe/Reset протокол	Необходим (7 ADC четения)	Не е необходим
Латентност	~1 ms (аналогов)	~6 ms (FFT 128 точки)
CPU натоварване	0% (отделен чип)	~5% (FFT + bands)

Проект от общността: 14-Channel Analyzer (donnersm)

Проектът [14ChannelAnalyzerV2.0](<https://github.com/donnersm/14ChannelAnalyzerV2.0>) от Mark Donners демонст...

MicroPython пример – пълен MSGEQ7 анализатор

```

from machine import Pin, AudioADC, WS2812, Timer
from array import array
import dsp, time

```

Конфигурация

N_PIXELS = 56 # $7 \times 8 = 56$ LED.

N_MSGEQ7 = 7

LEDS_PER_BAND = 8

BRIGHTNESS = 60

DECAY = 12

Хардуер

```
vcc = Pin("P500", Pin.OUT, value=1)
time.sleep_ms(50)
strip = WS2812(pixel_count=N_PIXELS, pin=Pin("P112"), channels=3)
adc = AudioADC("P000", fs=22050, frame=128)
```

DSP

```
fft = dsp.FFT(128)
fft.set_bands_msgeq7(22050, -60)    # MSGEQ7 эмуляция!
```

Буферы

```
frame_buf = array('f', [0.0] * 128)
fft_buf  = array('f', [0.0] * 128)
mag_buf  = array('f', [0.0] * 65)
band_buf = bytearray(N_MSGEQ7)
peak_buf = bytearray(N_MSGEQ7)
pixel_buf = bytearray(N_PIXELS * 3)

adc.start()
while True:
    while not adc.ready():
        pass
    adc.read_f32(frame_buf)
    fft.window_apply(frame_buf, fft_buf)
    fft.run(fft_buf, fft_buf)
    fft.magnitude(fft_buf, mag_buf)
    fft.bands(mag_buf, band_buf)      # 7 ленти!
```

Peak hold + render + WS2812 ...

Файлове

- `moddsp.c` – `set\_bands\_msgeq7()` имплементация (C)
- `examples/spectrum\_analyzer.py` – пълен анализатор с 5 визуални режима
- `examples/spectrum\_analyzer\_adc\_test.py` – тестова/диагностична версия: AudioADC P001, 128-pt FFT, 32-ban...
- `examples/spectrum\_msgeq7\_test.py` – MSGEQ7 7-band тест: AudioADC P001, FFT, 7 MSGEQ7 ленти (63 Hz–16 kHz...

Резюме

- MSGEQ7 е аналогов 7-лентов спектрален анализатор, който може да се замени с FFT + софтуерно групиране на ...
- `set\_bands\_msgeq7(fs)` изчислява граници чрез геометрични средни между 7-те MSGEQ7 центрови честоти.
- След конфигурацията, `bands()` работи с 7-байтов изход – без промяна на API.
- 56 WS2812 (7×8) дават класически bar graph с TriBar цветова схема.
- Цифровата емуляция превъзхожда хардуерния MSGEQ7 по динамичен обхват, гъвкавост и цена.

Самопроверка

1. Защо границите между MSGEQ7 лентите се изчисляват като геометрична средна, а не като аритметична?
2. Колко FFT бина покрива лентата за 6.25 kHz при FFT_LEN=128 и Fs=22050?
3. Какво е предимството на цифровата емуляция пред хардуерния MSGEQ7 чип?
4. Защо ниските ленти (63 Hz, 160 Hz) имат само по 1 бин при FFT_LEN=128?

Упражнения

1. Добавете 8-ма лента с център 20 kHz: модифицирайте масива `msgeq7\_fc[]` и стойността `7` в `moddsp.c`.
Подсказка: добавете `20000.0f` в `msgeq7\_fc[]` и сменете `#define N\_MSGEQ7\_BANDS 7` на `8` в `moddsp.c`.
2. Сравнете визуално `set\_bands(7)` и `set\_bands\_msgeq7(22050)` – коя версия дава по-естествен спектър?
*Подсказка: `set\_bands\_msgeq7()` поставя лентите на 63/160/400/1k/2.5k/6.25k/16k Hz – по-близо до музика...
3. Експериментирайте с FFT_LEN=256: подобрява ли се разделянето на лентите за 63 Hz и 160 Hz?
*Подсказка: `AudioADC("P000", fs=22050, frame=256)` и `dsp.FFT(256)` – bin spacing намалява от 172 до 86...

Ако не работи

- Симптом: само горните ленти (2.5k, 6.25k, 16k) светят. Проверка: нормално е за говор или музика с малко б...
- Симптом: лентата за 63 Hz не реагира на бас. Проверка: при FFT_LEN=128 тази лента има само 1 бин (172 Hz)...
- Симптом: нивата са твърде ниски. Проверка: опитайте `db\_floor=-80` за по-висока чувствителност.

Нататък

С тези три глави – Storm ADC, FFT и MSGEQ7 емуляция – имате пълната DSP верига от аналогов сигнал до визуал...

Глава 56. DSP Synthesis – Петро синтезатор архитектура `[Напреднал]`

Глави 56–58 покриваха аудио ****входа****: ADC → FFT → визуализация. Тази глава обхваща обратната посока – ****из...**

Архитектура на wavetable синтеза

Принципът е прост: един период на вълната се записва в lookup таблица (`array('H', ...)`), а DTC циклично п...

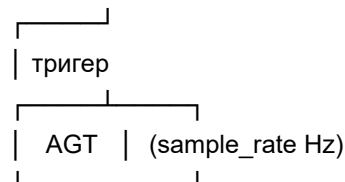
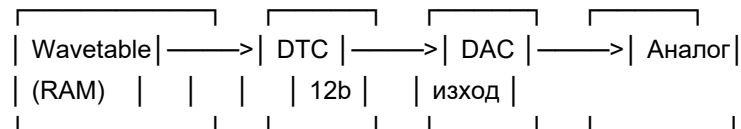
`f_out = sample_rate / table_len`

$\text{sample_rate} = f_{\text{out}} \times \text{table_len}$

Например за нота A4 (440 Hz) с таблица от 128 семпла:

$\text{sample_rate} = 440 \times 128 = 56\,320 \text{ Hz}$

AGT таймерът генерира прекъсване на 56 320 Hz, DTC прехвърля по 1 семпъл от таблицата към DAC регистъра при...



Вълнови форми

`retro\_synth.py` предизчислява 6 вълнови таблици при създаване на обекта (веднъж, без GC натоварване в горе...

****Square (50% duty cycle)****

```
def make_square(length, amp, duty_num=1, duty_den=2):
```

```
threshold = (length * duty_num) // duty_den
```

```
for i in range(length):
```

```
buf[i] = MID + amp if i < threshold else MID - amp
```

Резултат: правоъгълна вълна с рязко превключване между +amp и -amp. Богат на нечетни хармоници (1f, 3f, 5f....

****Pulse 25% и Pulse 12.5%****

Същата функция `make\_square()` с `duty\_num=1, duty\_den=4` (25%) или `duty\_den=8` (12.5%). По-тесният pulse ...

****Triangle****

```
def make_triangle(length, amp):
```

```
half = length // 2
```

```
for i in range(length):
```

```
if i < half: # нарастваща фаза
```

```
buf[i] = bottom + slope * i
```

```
else: # намаляваща фаза
```

```
buf[i] = top - slope * (i - half)
```

Линеен наклон нагоре, после надолу. Съдържа само нечетни хармоници, но с бързо намаляваща амплитуда ($1/n^2$)....

****Sawtooth****

```
def make_saw(length, amp):
for i in range(length):
buff[i] = start + (span * i) // (length - 1)
```

Линейно нарастване от -amp до +amp, после рязко падане. Съдържа ****всички**** хармоници (четни + нечетни). Най...

****LFSR шум (noise burst)****

```
def make_noise_burst(length, amp):
lfsr = 0xACE1 # 16-bit seed
for i in range(length):
bit = ((lfsr >> 0) ^ (lfsr >> 2) ^ (lfsr >> 3) ^ (lfsr >> 5)) & 1
lfsr = (lfsr >> 1) | (bit << 15)
level = amp * (length - i) // length # линейно затихване
buff[i] = MID + level if (lfsr & 1) else MID - level
```

16-bit LFSR (Linear Feedback Shift Register) генерира псевдослучайна последователност. Полиномът $x^{16} + x^...$

За разлика от периодичните вълни, шумовият буфер се изсвирва еднократно (mode=`NORMAL`), защото цикличното ...

Нотна система и MIDI конверсия

Модулът поддържа три формата на вход:

Формат	Пример	Как работи
Числена честота	`440` или `440.0`	Директно, без конверсия
Честота като текст	`"440Hz"`	Парсва числото преди "Hz"
Научна нотация	`"A4"`, `"C#5"`, `"Eb3"`	MIDI → честота

Конверсията от нота към честота минава през MIDI номер:

$MIDI = (октава + 1) \times 12 + полутонов_индекс$

$f = 440 \times 2^{((MIDI - 69) / 12)}$

Полутоново транспониране (за арпеджа и sweep):

```
def semitone_freq(base_freq, semitones):
return base_freq * 2.0 * (semitones / 12.0)
```

12 полутона = 1 октава = удвояване на честотата. Темперираната скала е логаритмична – всеки полутон е $\times 2^{1/12}$ (...)

Sweep алгоритъм

Sweep-ът е линейна интерполация на честотата от start до end за определено време:

```
def play_sweep(self, start_note, end_note, duration_ms, steps=24):
    start_hz = note_to_freq(start_note)
    end_hz = note_to_freq(end_note)
    step_delay = duration_ms // steps
    for i in range(steps):
        freq = start_hz + (end_hz - start_hz) * i / (steps - 1)
        self.start(freq, wave) # DTC рестарт с нов sample_rate
        time.sleep_ms(step_delay) # изчакваме 1 стъпка
```

Всяка стъпка **рестартира** DTC с нов sample rate (нова честота), запазвайки същата wavetable. Резултатът е...

SFX	Start	End	Стъпки	Вълна	Ефект
Jump	A3 (220 Hz)	F5 (698 Hz)	18	square	Sweep нагоре
Laser	A6 (1760 Hz)	A3 (220 Hz)	28	saw	Sweep надолу

Арпеджо архитектура

Арпеджото е повтаряща се последователност от ноти, дефинирана чрез полутонови отмествания от основната:

```
def play_arpeggio(self, root_note, semitones=(0, 4, 7), step_ms=60, cycles=4):
    root_hz = note_to_freq(root_note)
    for _ in range(cycles):
        for s in semitones:
            self.play_note(semitone_freq(root_hz, s), step_ms)
```

По подразбиране `(0, 4, 7)` е мажорен акорд:

- 0 полутона = основна нота (root)
- 4 полутона = голяма терца
- 7 полутона = чиста квинта

Добавянето на `12` (октава) създава пълен октавен арпеджо: `(0, 4, 7, 12)`.

DTC vs DMAC за аудио

Параметър	DTC	DMAC
Макс. repeat	256 семпла	1024+ (ISR re-arm)
Circular mode	Хардуерен (безкраен)	ISR рестарт
CPU натоварване	0% ~0.01% (ISR)	
Каналите	Споделен	8 независими
Предпочитан за	Wavetable loop (≤ 256)	Noise burst (> 256)

RetroSynth автоматично опитва DMAC за noise burst (2048 семпла > 256 DTC лимит), с fallback на DTC ако DMA...

```
def play_noise_burst(self, sample_rate=12000):
    try:
        self._play_once(sample_rate, DAC.TRANSFER_DMAL)
    except OSError:
        self._play_once(sample_rate, DAC.TRANSFER_DTC)
```

Смесване на сигнали – четири подхода

Смесването (mixing) на аудио сигнали е фундаментална операция. VK_RA4M2 предлага четири различни подхода, в...

****Подход 1: Pre-computed (офлайн) смесване****

Сумираме вълните математически в буфера ****преди**** DTC стартиране. CPU не участва по време на playback.

audiomixer_demo.py — триъгълник + синус в 1 буфер

```
for i in range(TBL):
    phase = i / TBL
    tri = 4.0 * abs(phase - 0.5) - 1.0      # Триъгълник: -1..+1
    sine = math.sin(2 * math.pi * 20 * phase)  # Синус: 20 цикъла
    mix_buff[i] = clamp12(MID + int(AMP*0.7*tri + 500*sine)) # Сума + clamp
```

Правилото за предотвратяване на клипване: ****сумата на амплитудите не трябва да надвишава MID**** (2048):

AMP × 0.7 × tri	+	SINE_AMP × sine	≤ 2048
------------------------	----------	------------------------	---------------

Ако сумата > 2048, `clamp12()` отрязва – звукът се изкривява (hard clipping).

```
| Плюс | Минус |
|-----|-----|
| 0% CPU по време на playback | Вълновата форма е статична |
| Прост код | Не може да се променя в реално време |
| DTC CIRCULAR – безкраен | Трябва да знаем всички сигнали предварително |
```

****Подход 2: Dual DAC (хардуерно паралелно)****

Два DAC канала (DA0 + DA1) генерират независими сигнали едновременно. Смесването е ****електрическо**** – чрез ...

dac_dual_dtc_dmac.py — един буфер, два канала

```
dac0.write_timed(buf, SR, mode=DAC.CIRCULAR, transfer=DAC.TRANSFER_DTC)
dac1.write_timed(buf, SR, mode=DAC.CIRCULAR, transfer=DAC.TRANSFER_DMAC)
```

Схема за пасивно смесване с резистори:

DA0 (P014) —[10kΩ]—
|— Изход (смесен сигнал)
DA1 (P015) —[10kΩ]—

Двата канала могат да имат различни буфери, различни SR, различни DTC/DMAC:

Бас на DA0 (ниска честота) + мелодия на DA1 (висока честота)

```
dac0.write_timed(bass_buf, bass_sr, mode=DAC.CIRCULAR, transfer=DAC.TRANSFER_DTC)
dac1.write_timed(melody_buf, melody_sr, mode=DAC.CIRCULAR, transfer=DAC.TRANSFER_DTC)
```

```
| Плюс | Минус |
|-----|-----|
| Истински паралелен изход | Само 2 канала (RA4M2 ограничение) |
| Независими sample rate | Нужна е външна схема за смесване |
| 0% CPU | Двата сигнала не "знаят" един за друг |
```

****Подход 3: Real-time AudioMixer (C модул, полифоничен)****

Модулът `audiomixer` (`modaudiomixer.c`) реализира ****истински real-time полифоничен миксер**** с до 16 гласа....

```
import audiomixer
from array import array
```

Създаваме миксер: 4 гласа, 22050 Hz, DA0

```
mixer = audiomixer.Mixer(4, 22050, pin=Pin("P014"))
```

Подготвяме два семпла (signed 16-bit)

```
tone_a = array('h', [...]) # 440 Hz синус
```

```
tone_b = array('h', [...]) # 880 Hz синус
```

Пускаме два гласа ЕДНОВРЕМЕННО

```
mixer.voice[0].play(tone_a, repeat=True) # Глас 0: 440 Hz loop  
mixer.voice[1].play(tone_b, repeat=True) # Глас 1: 880 Hz loop
```

Контролираме нивата в реално време

`mixer.voice[0].level = 0.7` # Глас 0 → 70% volume

`mixer.voice[1].level = 0.3` # Глас 1 → 30% volume

Алгоритъмът на ``audiomixer_render_buffer()`` за всеки изходен семпъл:

```
int32 mix = 0                ← Акумулатор (32-bit, избягва overflow)
for voice in active_voices:
    sample = read_sample(voice)    ← S8/U8/S16/U16 → int16
    sample = sample × level_q15 / 32767  ← Per-voice volume (Q15 fixed-point)
    mix += sample                ← АДИТИВНО СУМИРАНЕ
    clamp(mix, -32768, 32767)      ← Hard clipping при overflow
    buf[out] = (mix + 32768) >> 4    ← int16 → uint12 DAC стойност
```

****Защо Q15?*** Fixed-point аритметика е ~10× по-бърза от float на Cortex-M33 без FPU coprocessor. Q15 формат...

****Защо int32 акумулатор?*** При 4 гласа с level=1.0: max sum = 4 × 32767 = 131068. Това надхвърля int16 (327...

****Как предотвратяваме клипване?*** Намаляваме level на всеки глас:

4 гласа: $1.0/4 = 0.25$ на глас $\rightarrow$ сума ≤ 1.0 (без клипване)

for v in mixer.voice:

v.level = 0.25

Или: 2 гласа по 0.5 = 1.0 (граничен случай)

```
mixer.voice[0].level = 0.5
```

```
mixer.voice[1].level = 0.5
```

Правило: ****сумата на level стойностите на всички едновременно активни гласове трябва да е ≤ 1.0 **** за гарант...

Double buffering осигурява непрекъснат поток:

```
DMAC играе буфер A —————|
| callback: render_buffer(B)
DMAC играе буфер B —————|
| callback: render_buffer(A)
DMAC играе буфер A —————| ... (безкрайно)
```

```
| Плюс | Минус |
|-----|-----|
| До 16 гласа | ~1-5% CPU (render в ISR callback) |
| Real-time level control | По-сложен код |
| Supports loop + one-shot | Един DAC канал |
| S8/U8/S16/U16 формати | Фиксиран sample rate |
```

****Подход 4: Time-division (последователно)****

RetroSynth свири ноти ****една след друга**** на един DAC канал. Не е истинско смесване, но е най-простият подх...

```
synth.play_note("C4", 200) # До4, 200 ms
```

```
synth.play_note("E4", 200) # Ми4, 200 ms (НЕ едновременно!)
```

```
synth.play_note("G4", 200) # Сол4, 200 ms
```

Плюс	Минус
-----	-----
Най-прост код	Монофоничен (без акорди)
0% CPU при DTC	Паузи между нотите
1 DAC канал	Не може 2 тона едновременно

****Сравнение на четирите подхода****

Подход	Гласове	CPU	Real-time	Код
Pre-computed	1 (static mix)	0%	Не	`audiomixer_demo.py`
Dual DAC	2 (паралелни)	0%	Частично	`dac_dual_dtc_dmac.py`
AudioMixer	1-16 (полифоничен)	1-5%	Да	`modaudiomixer.c`
Time-division	1 (монофоничен)	0%	Не	`retro_synth.py`

Пълен SFX pipeline

Един SFX ефект (например `coin()`) минава през следните стъпки:

Python: `synth.coin()`

```

├─ play_note("B5", 70ms, "pulse25")
│   └─ _table("pulse25") → pre-computed array('H', 128)
│   └─ note_rate("B5") → 494 Hz × 128 = 63 232 Hz
│   └─ dac.write_timed(table, 63232, CIRCULAR, DTC)
│       └─ AGT таймер: 63 232 Hz
│       └─ DTC: table[0..127] → DAC12 (repeat)
│           └─ DAC12 → аналогов изход (P014)
│   └─ time.sleep_ms(70)
│   └─ dac.stop()
├─ play_note("E6", 90ms, "pulse25")
├─ note_rate("E6") → 1319 Hz × 128 = 168 832 Hz
└─ ... (същата верига)

```

Времедиаграма:

t=0ms t=70ms t=90ms t=180ms

├─ B5 ───┤	gap ───┤	├─ E6 ───┤
494 Hz	20ms	1319 Hz
pulse25	тиш.	pulse25